TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN — ĐHQG-HCM
KHOA MẠNG MÁY TÍNH & TRUYỀN THÔNG

SỔ TAY THỰC THI & TÁC CHIẾN ĐỒ ÁN LABGUARD

HỆ THỐNG GIÁM SÁT MẠNG VÀ QUẢN LÝ SỰ CỐ PHÒNG MÁY TẬP TRUNG
Môn học: NT106 — Lập trình mạng căn bản | Học kỳ 2, Năm học 2025-2026
Phiên bản tài liệu: Version 2.0 (Toàn diện — Chuẩn hóa Tiếng Việt)

📌 THÔNG TIN HỒ SƠ DỰ ÁN LABGUARD V2

• Nhóm thực hiện: 03 Sinh viên (Sinh viên A: Agent & Lập trình mạng; Sinh viên B: Central Server & CSDL; Sinh viên C: WinForms Dashboard & UI Client).
• Quy mô kế hoạch: 60 nhiệm vụ chuẩn WBS (20 task/người), tổng tải 211.5 giờ, tải cam kết MVP 164.5 giờ, quỹ dự trữ 15.5 giờ.
• Công nghệ chủ đạo: C# .NET 9, Raw TCP Sockets, TLS (SslStream), Big-Endian Framing, SQL Server 2022, Optimistic Concurrency Control (OCC) rowversion, WinForms PerMonitorV2 High-DPI.
• Trọng tâm tài liệu: Giải thích trực giác và bản chất kỹ thuật trước khi code, hỗ trợ tối đa cho sinh viên năm 2 lần đầu làm hệ phân tán.

PHẦN 0: BẢN ĐỒ ĐỊNH HƯỚNG CHO SINH VIÊN NĂM 2

0.1. Vị trí đồ án trong chương trình đào tạo UIT: Chào mừng các bạn sinh viên đến với đồ án môn học NT106 — Lập trình mạng căn bản. Đồ án LabGuard được thiết kế với mục tiêu đưa các bạn tiếp cận với một hệ thống mạng phân tán thực thụ (real-world distributed system) thay vì chỉ xây dựng các ứng dụng web CRUD đơn giản vốn đã quá quen thuộc.

0.2. Ba khối kiến trúc cốt lõi: Hệ thống LabGuard bao gồm 3 khối kiến trúc chạy độc lập trên các máy tính khác nhau trong mạng LAN phòng máy:
1. Khối Agent (Máy trạm): Dịch vụ nền chạy ngầm trên từng máy tính trong phòng thực hành, định kỳ thu thập chỉ số phần cứng (CPU, RAM, Mạng, Danh sách tiến trình) và gửi về máy chủ.
2. Khối Central Server (Máy chủ trung tâm): Tiếp nhận đồng thời kết nối từ hàng chục máy trạm qua kênh TCP/TLS, khử trùng lặp dữ liệu, đánh giá quy tắc cảnh báo và quản lý CSDL tập trung.
3. Khối Dashboard (Máy trạm quản trị WinForms): Ứng dụng giao diện đồ họa dành cho cán bộ quản trị phòng máy theo dõi trực quan sơ đồ phòng máy theo thời gian thực và xử lý sự cố.

0.3. Phân vai trách nhiệm 3 sinh viên: Mỗi thành viên trong nhóm 3 người đảm nhận một phân hệ chuyên biệt nhưng phụ thuộc lẫn nhau một cách chặt chẽ qua các hợp đồng giao thức (Interface Contracts):
• Sinh viên A (Agent & Lập trình mạng TCP/TLS): Làm chủ tầng socket nhị phân, giải mã khung dữ liệu (FrameCodec), chứng chỉ số bảo mật TLS, cơ chế nhịp tim (heartbeat) và bộ đệm số liệu máy trạm.
• Sinh viên B (Central Server & Cơ sở dữ liệu): Làm chủ máy chủ socket đa luồng, kênh đệm có chặn (Bounded Channel), khử trùng lặp dữ liệu (idempotent ingestion), lược đồ 8 bảng CSDL, thuật toán khóa lạc quan (OCC rowversion) và bảo mật PBKDF2.
• Sinh viên C (WinForms Dashboard & UI Client): Làm chủ kiến trúc giao diện điều khiển, điều phối luồng UI an toàn (Control.Invoke marshal), quản lý bộ nhớ đệm (cache), khớp cặp RPC request/response và ứng dụng Collector dồn kênh (P1).

> **📌 3 NGUYÊN TẮC VÀNG VẬN HÀNH DỰ ÁN**
> 1. KHÔNG commit dòng code nào mà bản thân chưa hiểu: Trong buổi vấn đáp, thầy cô sẽ hỏi sâu về bản chất luồng dữ liệu, tại sao lại dùng hàm này. Nếu dùng AI sinh code mà không hiểu sẽ bị trừ điểm rất nặng.
> 2. KHÔNG tự ý sửa đổi giao thức (Protocol Drift): Mọi thay đổi về tên trường JSON hay mã bản tin phải được cả 3 người họp và thống nhất bằng văn bản.
> 3. TUÂN THỦ kỷ luật cổng chất lượng M4: Chỉ được làm tính năng mở rộng Collector (P1) khi hệ thống MVP cốt lõi đã chạy mượt mà 100%.

PHẦN 1: KIẾN TRÚC HỆ THỐNG NHÌN QUA LUỒNG DỮ LIỆU

1.1. Luồng dữ liệu xuyên suốt (End-to-End Data Pipeline): Để hiểu rõ cách thức hoạt động của LabGuard, sinh viên cần theo dõi luồng đi của một gói tin số liệu từ khi sinh ra tại máy trạm cho đến khi hiển thị trên màn hình quản trị:

Bước 1 (Thu thập tại Agent): Mỗi chu kỳ 5 giây, Agent đọc chỉ số CPU/RAM qua PerformanceCounter hoặc /proc/stat, tạo gói tin METRIC_BATCH.

Bước 2 (Gửi qua Socket có Framing): Agent bọc JSON vào khung dữ liệu có tiền tố 4 byte độ dài Big-Endian, mã hóa qua SslStream gửi đi.

Bước 3 (Tiếp nhận tại Central Ingress): Central Server đọc đủ 4 byte độ dài, trích xuất byte JSON, đẩy vào IngressChannel có giới hạn 1000 phần tử.

Bước 4 (Khử trùng lặp & Lưu trữ): BatchIngestProcessor kiểm tra bảng ProcessedBatches bằng bộ khóa (DeviceId, BootId, BatchId). Nếu hợp lệ, lưu vào MetricSnapshots và phản hồi BATCH_ACK.

Bước 5 (Đánh giá quy tắc Hysteresis): RuleEvaluator kiểm tra nếu CPU >= 90% liên tiếp 3 mẫu, kích hoạt IncidentManager tạo sự cố mới trong bảng Incidents.

Bước 6 (Phát sóng sự kiện tới Dashboard): Central phát sóng bản tin INCIDENT_CREATED hoặc METRIC_UPDATE tới tất cả các ứng dụng WinForms đang kết nối.

Bước 7 (Hiển thị an toàn trên UI Thread): WinForms Client nhận gói tin trên luồng mạng nền, dùng Control.Invoke chuyển việc vẽ lại giao diện sang luồng chính (STA Thread).

1.2. Bản chất TCP Byte Stream và bài toán Framing: Giao thức TCP là giao thức 'hướng dòng byte' (Byte Stream), nghĩa là dữ liệu gửi qua socket giống như dòng nước chảy liên tục trong đường ống, hoàn toàn KHÔNG có khái niệm ranh giới gói tin như UDP. Khi gửi chuỗi 'HELLO', bên nhận có thể đọc được 2 byte 'HE' trong lần gọi read đầu tiên (Partial Read), hoặc nếu gửi liên tiếp 2 gói tin, bên nhận có thể đọc được cả 2 gói cùng một lúc dính chùm vào nhau (Frame Coalescing).
Do đó, LabGuard bắt buộc phải cài đặt FrameCodec với quy ước: 4 byte đầu tiên là số nguyên không dấu 32-bit (UInt32 Big-Endian) biểu thị chính xác độ dài N của chuỗi UTF-8 JSON theo sau. Phía nhận phải dùng vòng lặp ReadExactlyBytesAsync đọc tích lũy đến khi đủ đúng N byte mới chuyển sang bộ giải mã JSON.

> **📌 CẢNH BÁO BẢO MẬT & TRÀN BỘ NHỚ (OOM PROTECTION)**
> Trong FrameCodec, nếu 4 byte độ dài giải mã ra giá trị N > 262,144 bytes (256 KiB), Server phải ngay lập tức ném ngoại lệ InvalidDataException và đóng kết nối ngay lập tức! Điều này nhằm ngăn chặn kẻ tấn công gửi gói tin giả mạo độ dài 2GB làm máy chủ cấp phát mảng byte khổng lồ dẫn đến cạn kiệt RAM (Out Of Memory Attack).

PHẦN 2: HỢP ĐỒNG DỮ LIỆU & QUY ƯỚC WIRE PROTOCOL

2.1. Cấu trúc Outer Envelope 6 trường bất biến: Mọi gói tin trao đổi trong hệ thống LabGuard (giữa Agent - Central, Central - Admin, Collector - Central) đều phải tuân thủ nghiêm ngặt cấu trúc Outer Envelope gồm đúng 6 trường bất biến định dạng UTF-8 JSON:

• **version (int):** Phiên bản giao thức, hiện tại cố định là 1.

• **messageType (string):** Tên loại bản tin viết HOA nối gạch dưới (ví dụ: HEARTBEAT, METRIC_BATCH, INCIDENT_ACK).

• **requestId (string):** Mã định danh duy nhất của yêu cầu dạng UUIDv4, dùng để khớp cặp câu hỏi/câu trả lời.

• **deviceId (string):** Mã định danh máy trạm (ví dụ: LAB01-PC05), để trống nếu là kết nối của Admin.

• **sentAtUtc (string):** Dấu mốc thời gian theo chuẩn ISO-8601 UTC (ví dụ: 2026-09-13T14:30:00.000Z).

• **payload (object):** Khối dữ liệu nội dung chi tiết đặc thù của từng loại bản tin.

2.2. Phân loại 69 bản tin trong Wire Protocol: Giao thức LabGuard bao gồm chính xác 69 loại bản tin trong danh mục chuẩn, được phân thành 3 phạm vi rõ ràng:
1. Phạm vi P0-MVP BẮT BUỘC (34 bản tin): Đảm bảo vận hành đầy đủ hệ thống giám sát 3 khối: Bắt tay kết nối, Gửi nhịp tim, Đẩy lô số liệu, Báo nhận BATCH_ACK, Đăng nhập Admin, Đồng bộ Snapshot (SNAPSHOT_BEGIN, SNAPSHOT_PAGE, SNAPSHOT_END), Xử lý sự cố OCC (INCIDENT_ACK, START, RESOLVE, ASSIGN), Báo cáo sức khỏe HEALTH_UPDATE.
2. Phạm vi P0 TÙY CHỌN (25 bản tin): Mở rộng quản lý: Đăng ký tài khoản có mã mời (SIGN_UP, CREATE_INVITE), Truy vấn chi tiết máy trạm và sự cố (GET_DEVICE, GET_INCIDENT).
3. Phạm vi P1 CÓ ĐIỀU KIỆN (10 bản tin): Dành cho tính năng mở rộng nút gom Collector nhiều phòng máy (COLLECTOR_REGISTER, COLLECTOR_FORWARD).

2.3. Khóa lạc quan (OCC) và xử lý xung đột tranh chấp: Khi có 2 cán bộ quản trị cùng mở ứng dụng WinForms và cùng bấm nút nhận (ACK) trên một sự cố, hệ thống sử dụng cơ chế Khóa lạc quan (Optimistic Concurrency Control - OCC) qua cột rowversion trong SQL Server:
1. WinForms Client gửi bản tin INCIDENT_ACK kèm theo expectedRowVersion hiện đang hiển thị trên màn hình.
2. Central Server thực thi lệnh: UPDATE Incidents SET WorkflowState = 'ACKED' WHERE IncidentId = @id AND RowVersion = @expectedRowVersion.
3. Nếu hàng bị sửa bằng 0 (chứng minh đã có người khác sửa trước làm thay đổi RowVersion), Server trả về ngay mã lỗi CONCURRENCY_CONFLICT.
4. WinForms của người đến sau nhận lỗi, hiển thị thông báo thân thiện và tự động tải lại dữ liệu mới nhất.

PHẦN 3: MA TRẬN TRÁCH NHIỆM & QUY TRÌNH CỘNG TÁC GIT

3.1. Phân định trách nhiệm RACI: Mỗi thành viên trong nhóm chịu trách nhiệm làm chủ (Owner) 20 nhiệm vụ và làm người dự phòng (Backup Owner) hoặc người kiểm tra (Reviewer) cho các nhiệm vụ của đồng đội. Phân bổ này đảm bảo không có bất kỳ nút thắt cổ chai đơn lẻ nào (No Single Point of Failure) trong nhóm.

3.2. Quy trình nhánh và Pull Request chuẩn: Nhóm áp dụng quy trình Git Flow rút gọn nhưng cực kỳ nghiêm ngặt:
• Nhánh main là nhánh sản phẩm, được bật chế độ bảo vệ (Branch Protection), tuyệt đối cấm đẩy mã nguồn trực tiếp (cấm push to main) hoặc force push.
• Mỗi nhiệm vụ trong 60 tasks phải được tạo trên một nhánh tính năng riêng biệt mang đúng mã Task ID, ví dụ: feature/LG-A-002-frame-codec hoặc feature/LG-B-001-schema-migration.

• Khi viết xong và test pass cục bộ, thành viên tạo Pull Request (PR). Bắt buộc phải có ít nhất 1 thành viên khác (được phân công là Reviewer trong WBS) duyệt mã nguồn và bấm merge.
• Quy ước commit message: [Task_ID] Động_từ: Nội_dung_ngắn_gọn (ví dụ: [LG-A-002] Feat: Implement ReadExactlyBytesAsync and OOM bounds).

> 📌 **BẢO MẬT MÃ NGUỒN TRÊN GIT**
> Tuyệt đối KHÔNG commit khóa riêng (Private Key .key/.pfx), tệp cơ sở dữ liệu (.mdf/.ldf), chuỗi kết nối chứa mật khẩu thật hoặc tệp rác biên dịch (.vs, bin, obj) lên Git! Tệp .gitignore phải được cấu hình kỹ lưỡng ngay từ ngày đầu tiên.

PHẦN 4: KẾ HOẠCH 5 TUẦN, CÁC MỐC KIỂM SOÁT (GATE M1-M7) & KỶ LUẬT CẮT PHẠM VI

4.1. Lộ trình 5 tuần và 7 cổng chất lượng: Đồ án được chia thành 5 tuần tác chiến với 7 cổng kiểm soát chất lượng (Quality Gates) kỹ thuật. Việc vượt qua các cổng này là điều kiện tiên quyết để bước sang giai đoạn tiếp theo:

Cổng M1 (Ngày 3, Tuần 1): Đóng băng hợp đồng giao thức (Contract & Envelope Freeze). Chốt xong 16 fixtures JSON và 69 catalog messages.

Cổng M2 (Ngày 7, Tuần 1): Khung gói tin nhị phân (TCP Framing & Baseline Wire). FrameCodec vượt qua 10,000 khung ngẫu nhiên.

Cổng M3 (Ngày 14, Tuần 2): Đường ống thu thập & Nhịp tim sống (Ingestion & Liveness). Khử trùng lặp lô thành công và phát hiện máy offline sau 20s.

Cổng M4 (Ngày 21, Tuần 3) — GATE SINH TỬ: Nghiệm thu toàn diện hệ thống MVP 3 khối và họp quyết định cắt phạm vi P1 (Scope Cut Gate).

Cổng M5 (Ngày 28, Tuần 4): Triển khai Collector P1 HOẶC Tinh chỉnh ổn định mạng LAN (Hardening).

Cổng M6 (Ngày 32, Tuần 5): Kiểm thử thực địa mạng LAN 3 máy vật lý & Quay video Demo 12 phút dự phòng.

Cổng M7 (Ngày 35, Tuần 5): Đóng gói phát hành chính thức & Sẵn sàng 90 câu hỏi vấn đáp (Viva Readiness).

4.2. Kỷ luật cắt phạm vi tại Cổng M4 (Scope Reduction Rule): Cổng M4 cuối Tuần 3 là mốc quyết định thành bại của đồ án. Nhóm chỉ được phép làm tính năng mở rộng Collector (P1) ở Tuần 4 nếu thỏa mãn đồng thời cả 3 điều kiện:
1. Không còn bất kỳ lỗi P0 nào tồn đọng trong mã nguồn.
2. Bài kiểm tra chịu tải đồng thời MVP 3 khối chạy đạt 100% không văng ngoại lệ.
3. Cả 3 thành viên đều còn dư ít nhất 15 giờ công dự phòng.
Nếu không thỏa mãn 3 điều kiện trên, nhóm kích hoạt ngay quyết định DEFER P1: Hoãn 9 tasks P1 (LG-A-013..015, LG-B-013..015, LG-C-013..015), cắt giảm chính xác 47.0 giờ công. Khi đó, tổng tải cam kết của

nhóm giảm xuống 164.5 giờ, tương đương mức trung bình 10.9 giờ/người/tuần — hoàn toàn nằm dưới trần công suất 12 giờ/tuần của sinh viên UIT!

PHẦN 5: SỔ TAY CHI TIẾT 20 NHIỆM VỤ CỦA SINH VIÊN A (AGENT & MẠNG TCP/TLS)

Sinh viên A chịu trách nhiệm về phân hệ Agent trên máy trạm, giao thức mạng cốt lõi, bộ giải mã khung (FrameCodec), bảo mật TLS và thu thập số liệu phần cứng. Dưới đây là hướng dẫn chi tiết cho toàn bộ 20 nhiệm vụ từ LG-A-001 đến LG-A-020:

Nhiệm vụ 01: [LG-A-001] — Chốt wire contract và protocol fixtures

Mã & Tên nhiệm vụ: [LG-A-001] Chốt wire contract và protocol fixtures	**Tuần & Ưu tiên:** Tuần W1 (P0) | P0
Khối lượng công việc: Cơ bản: 2.0h | Dự phòng: 0.5h | Tổng: 2.5h	**Phân loại tải:** P0 Core
Trách nhiệm thực hiện: Phụ trách: SV A | Dự phòng: SV B | Duyệt: SV B	**Phân hệ & Thư mục:** Shared.Protocol (Networking)
Sản phẩm bàn giao: CONTRACT_MATRIX.md & protocol fixtures	**Hợp đồng giao diện:** Outer envelope 6 fields, Big-Endian uint32 length prefix, UTF-8 JSON

1. Mục đích kỹ thuật & Tại sao tồn tại: Trước khi viết bất kỳ dòng code mạng nào, 3 thành viên phải thống nhất chính xác cấu trúc gói tin (wire envelope). Nếu không có bản đặc tả và các fixture JSON chuẩn, phía Agent gửi một kiểu, Central nhận một kiểu sẽ gây lỗi Serialization hoặc gãy kết nối ngay từ byte đầu tiên.

2. Kiến thức nền tảng cần nắm vững: Định dạng JSON (RFC 8259), cấu trúc trường dữ liệu, quy chuẩn thời gian UTC ISO-8601, mã băm UUIDv4.

3. Dữ liệu đầu vào & Đầu ra kỹ thuật: Đầu vào: Đặc tả giao thức 03_PROTOCOL_SPEC.md và bảng danh mục 69 bản tin trong CONTRACT_MATRIX.md.
Đầu ra: Tài liệu CONTRACT_MATRIX.md đã chốt và 16 tệp fixture JSON mẫu đại diện cho các thông điệp cốt lõi.

4. Các bước triển khai chi tiết:
1. Xác định 6 trường bất biến trong Outer Envelope: version (int=1), messageType (string), requestId (UUIDv4), deviceId (string), sentAtUtc (ISO-8601 UTC), payload (object).
2. Tạo thư mục fixtures và viết 16 mẫu JSON chuẩn (HEARTBEAT, METRIC_BATCH, LOGIN, INCIDENT_ACK...).
3. Viết script kiểm tra cú pháp JSON hợp lệ và đảm bảo không có khoảng trắng thừa hay sai tên trường.
4. Họp nhanh với B và C để cả 3 người cùng review và đồng ý với envelope chuẩn.

> 📌 **BẪY LỖI CẦN TRÁNH TRONG LG-A-001**
> Tự ý đổi chữ hoa chữ thường (ví dụ: messageType thành MessageType hoặc messagetype) làm bộ giải mã JSON của người khác không parse được.

5. Tiêu chuẩn hoàn thành 11 mục (Definition of Done — DoD):
• [ ] 1. Hợp đồng giao thức & Wire Envelope chuẩn UTF-8 JSON
• [ ] 2. Lập trình bất đồng bộ Async/Await không chặn luồng mạng
• [ ] 3. Bắt ngoại lệ mạng an toàn (EOF, SocketException, Timeout)
• [ ] 4. Giải phóng tài nguyên sạch sẽ (Dispose / Using / CancellationToken)
• [ ] 5. An toàn dữ liệu (Khử trùng lặp lô / Khóa lạc quan OCC / Transaction)
• [ ] 6. Unit Test / Test Harness chạy đạt 100% không có cảnh báo
• [ ] 7. Tích hợp chéo thành công với module của đồng đội liên quan
• [ ] 8. Kiểm thử thực tế trên mạng LAN nhiều máy tính
• [ ] 9. Đồng bộ tài liệu thiết kế & mã nguồn khớp hoàn toàn
• [ ] 10. Thu thập đầy đủ hồ sơ minh chứng (Log, Screenshot, Pcap)
• [ ] 11. Sẵn sàng trả lời trôi chảy câu hỏi vấn đáp (Viva) liên quan

6. Kịch bản kiểm thử & Hồ sơ minh chứng:
• Mã bài test: TEST-A01
• Phương pháp kiểm thử: Schema validation tool và JSON parser kiểm tra fixture mẫu
• Tiêu chuẩn nghiệm thu đạt: Tất cả wire catalog có fixture JSON hợp lệ, không sai lệch tên trường
• Hồ sơ minh chứng lưu tại: CONTRACT_MATRIX.md, fixture JSON files

7. Tình huống lỗi tiềm ẩn & Kế hoạch ứng phó: Nếu lệch tên trường thì Central hoặc Admin không deserialize được; rollback về CONTRACT_MATRIX freeze

8. Câu hỏi vấn đáp (Viva) & Chiến lược trả lời:
• Chủ đề câu hỏi: Framing nhị phân và ranh giới thông điệp TCP
• Câu trả lời kỳ vọng: TCP là stream-oriented không bảo toàn ranh giới; cần 4 byte length-prefix để bên nhận biết chính xác kích thước frame
• Câu hỏi mở rộng / Bẫy của giảng viên: Tại sao dùng Big-Endian mà không dùng Little-Endian mặc định của x86?

[HƯỚNG DẪN ĐỀ XUẤT]: *Tham khảo mã nguồn mẫu chuẩn và tài liệu kỹ thuật tại thư mục Networking. Khi hiện thực, luôn đặt an toàn luồng và xử lý hủy tác vụ (CancellationToken) lên hàng đầu.*

Nhiệm vụ 02: [LG-A-002] — FrameCodec và xử lý partial reads / message framing

Mã & Tên nhiệm vụ: [LG-A-002] FrameCodec và xử lý partial reads / message framing	**Tuần & Ưu tiên:** Tuần W1 (P0) | P0

Khối lượng công việc: Cơ bản: 3.5h | Dự phòng: 0.5h | Tổng: 4.0h	**Phân loại tải:** P0 Core
Trách nhiệm thực hiện: Phụ trách: SV A | Dự phòng: SV B | Duyệt: SV B	**Phân hệ & Thư mục:** Shared.Protocol (Networking)
Sản phẩm bàn giao: FrameCodec.cs & Unit tests	**Hợp đồng giao diện:** FrameCodec.ReadExactlyBytesAsync, MaxFrameSize = 262144 bytes

1. Mục đích kỹ thuật & Tại sao tồn tại: Giao thức TCP là 'Byte Stream' (dòng byte liên tục), không có khái niệm 'ranh giới gói tin'. Dữ liệu gửi đi có thể bị chia nhỏ (Partial Read) hoặc dính chùm vào nhau (Frame Coalescing). Phải có FrameCodec với 4 byte độ dài Big-Endian đi trước để bên nhận biết chính xác khi nào đọc đủ một bản tin hoàn chỉnh.

2. Kiến thức nền tảng cần nắm vững: Khái niệm TCP Stream vs Datagram, Big-Endian vs Little-Endian (Network Byte Order), cơ chế đệm Stream.ReadAsync trong C#.

3. Dữ liệu đầu vào & Đầu ra kỹ thuật: Đầu vào: Luồng mạng NetworkStream nhị phân nhận từ Socket TCP.
Đầu ra: Các mảng byte JSON payload đã tách rời sạch sẽ; ném ngoại lệ an toàn nếu frame vượt 256 KiB hoặc đứt kết nối (EOF).

4. Các bước triển khai chi tiết:
1. Hiện thực hàm ReadExactlyBytesAsync(Stream stream, int count) dùng vòng lặp while (read < count) tích lũy byte.
2. Nếu ReadAsync trả về 0 byte giữa chừng, ném ngay EndOfStreamException (xử lý ngắt kết nối an toàn).
3. Đọc 4 byte header, dùng BinaryPrimitives.ReadUInt32BigEndian để giải mã độ dài payload N.
4. Kiểm tra điều kiện bảo mật: nếu N > 262,144 bytes (256 KiB), ném InvalidDataException và đóng kết nối ngay (chống tấn công làm tràn RAM OOM).
5. Đọc đúng N byte payload và trả về byte[] hoàn chỉnh.

> **📌 BẪY LỖI CẦN TRÁNH TRONG LG-A-002**
> Nghĩ rằng gọi stream.ReadAsync(buffer, 0, count) một lần là đọc đủ count byte. Trong mạng thật, một lệnh read có thể chỉ trả về 2 byte thay vì 4 byte header!

5. Tiêu chuẩn hoàn thành 11 mục (Definition of Done — DoD):
- [ ] 1. Hợp đồng giao thức & Wire Envelope chuẩn UTF-8 JSON
- [ ] 2. Lập trình bất đồng bộ Async/Await không chặn luồng mạng
- [ ] 3. Bắt ngoại lệ mạng an toàn (EOF, SocketException, Timeout)
- [ ] 4. Giải phóng tài nguyên sạch sẽ (Dispose / Using / CancellationToken)
- [ ] 5. An toàn dữ liệu (Khử trùng lặp lô / Khóa lạc quan OCC / Transaction)
- [ ] 6. Unit Test / Test Harness chạy đạt 100% không có cảnh báo
- [ ] 7. Tích hợp chéo thành công với module của đồng đội liên quan
- [ ] 8. Kiểm thử thực tế trên mạng LAN nhiều máy tính

• [ ] 9. Đồng bộ tài liệu thiết kế & mã nguồn khớp hoàn toàn
• [ ] 10. Thu thập đầy đủ hồ sơ minh chứng (Log, Screenshot, Pcap)
• [ ] 11. Sẵn sàng trả lời trôi chảy câu hỏi vấn đáp (Viva) liên quan

6. Kịch bản kiểm thử & Hồ sơ minh chứng:
• Mã bài test: TEST-A02
• Phương pháp kiểm thử: Unit test đọc 1-byte chunking, 3 frame liền mạch và frame vượt 256 KiB
• Tiêu chuẩn nghiệm thu đạt: Pass 5/5 kịch bản unit test; không nuốt ngoại lệ stream; không cấp phát buffer khi frame size quá lớn
• Hồ sơ minh chứng lưu tại: scratch/test_frame_codec/Program.cs, test_frame_codec_run.log

7. Tình huống lỗi tiềm ẩn & Kế hoạch ứng phó: Đọc thiếu byte dẫn đến parse JSON hỏng; ném EndOfStreamException và đóng kết nối an toàn

8. Câu hỏi vấn đáp (Viva) & Chiến lược trả lời:
• Chủ đề câu hỏi: Hiện tượng Partial Read và cách xử lý trong socket TCP
• Câu trả lời kỳ vọng: Stream.ReadAsync có thể trả về ít byte hơn yêu cầu; phải lặp vòng lặp tích lũy byte cho đến khi đủ count
• Câu hỏi mở rộng / Bẫy của giảng viên: Tại sao không dùng ReadAsync trả về 0 để tiếp tục đợi dữ liệu?

[HƯỚNG DẪN ĐỀ XUẤT]: *Tham khảo mã nguồn mẫu chuẩn và tài liệu kỹ thuật tại thư mục Networking. Khi hiện thực, luôn đặt an toàn luồng và xử lý hủy tác vụ (CancellationToken) lên hàng đầu.*

Nhiệm vụ 03: [LG-A-003] — Agent TCP connection và writer queue

Mã & Tên nhiệm vụ: [LG-A-003] Agent TCP connection và writer queue	**Tuần & Ưu tiên:** Tuần W1 (P0) | P0
Khối lượng công việc: Cơ bản: 2.5h | Dự phòng: 0.5h | Tổng: 3.0h	**Phân loại tải:** P0 Core
Trách nhiệm thực hiện: Phụ trách: SV A | Dự phòng: SV B | Duyệt: SV B	**Phân hệ & Thư mục:** Agent.Core (Networking)
Sản phẩm bàn giao: AgentClient.cs & WriterLoop	**Hợp đồng giao diện:** TcpClient, NetworkStream, Channel<byte[]> Bounded(100)

1. Mục đích kỹ thuật & Tại sao tồn tại: Trong .NET, NetworkStream chỉ an toàn khi có tối đa 1 tác vụ đọc và 1 tác vụ ghi đồng thời. Nếu luồng Heartbeat và luồng Telemetry cùng lúc gọi stream.WriteAsync, các byte của 2 bản tin sẽ bị trộn lẫn vào nhau, làm hỏng toàn bộ cấu trúc framing của phía nhận.

2. Kiến thức nền tảng cần nắm vững: Mô hình Single Writer Pattern, lớp System.Threading.Channels.Channel trong .NET, SemaphoreSlim / Concurrency.

3. Dữ liệu đầu vào & Đầu ra kỹ thuật: Đầu vào: Các gói tin nhị phân (byte[]) được tạo từ luồng thu thập dữ liệu hoặc gửi nhịp tim.
Đầu ra: Kênh gửi dữ liệu tuần tự, ghi từng khung dữ liệu duy nhất vào NetworkStream.

4. Các bước triển khai chi tiết:

1. Tạo một Channel<byte[]> có giới hạn kích thước (BoundedChannelOptions(100)).
2. Viết vòng lặp nền WriterLoopAsync làm nhiệm vụ duy nhất: đọc từng gói tin từ Channel và ghi vào NetworkStream.
3. Khi các luồng khác muốn gửi tin, chỉ cần gọi writerChannel.Writer.WriteAsync(frame).
4. Thiết lập chính sách khi hàng đợi đầy (Backpressure): ghi log cảnh báo và xử lý drop tin không quan trọng nếu mạng bị nghẽn kéo dài.

> 📌 **BẪY LỖI CẦN TRÁNH TRONG LG-A-003**
> Dùng lock (stream) bao quanh hàm bất đồng bộ WriteAsync (gây lỗi compiler hoặc deadlock) hoặc tạo nhiều Task độc lập tự do ghi vào stream.

5. Tiêu chuẩn hoàn thành 11 mục (Definition of Done — DoD):

- [ ] 1. Hợp đồng giao thức & Wire Envelope chuẩn UTF-8 JSON
- [ ] 2. Lập trình bất đồng bộ Async/Await không chặn luồng mạng
- [ ] 3. Bắt ngoại lệ mạng an toàn (EOF, SocketException, Timeout)
- [ ] 4. Giải phóng tài nguyên sạch sẽ (Dispose / Using / CancellationToken)
- [ ] 5. An toàn dữ liệu (Khử trùng lặp lô / Khóa lạc quan OCC / Transaction)
- [ ] 6. Unit Test / Test Harness chạy đạt 100% không có cảnh báo
- [ ] 7. Tích hợp chéo thành công với module của đồng đội liên quan
- [ ] 8. Kiểm thử thực tế trên mạng LAN nhiều máy tính
- [ ] 9. Đồng bộ tài liệu thiết kế & mã nguồn khớp hoàn toàn
- [ ] 10. Thu thập đầy đủ hồ sơ minh chứng (Log, Screenshot, Pcap)
- [ ] 11. Sẵn sàng trả lời trôi chảy câu hỏi vấn đáp (Viva) liên quan

6. Kịch bản kiểm thử & Hồ sơ minh chứng:

- Mã bài test: TEST-A03
- Phương pháp kiểm thử: Chạy song song 10 thread ghi vào writer channel tới Mini Test Server
- Tiêu chuẩn nghiệm thu đạt: Dữ liệu trên socket được ghi tuần tự, không bị đan xen byte giữa các frame
- Hồ sơ minh chứng lưu tại: Trace log socket client, Mini Test Server output

7. Tình huống lỗi tiềm ẩn & Kế hoạch ứng phó: Nhiều luồng ghi đồng thời vào NetworkStream làm hỏng framing; Channel bảo đảm đúng 1 luồng ghi duy nhất

8. Câu hỏi vấn đáp (Viva) & Chiến lược trả lời:

- Chủ đề câu hỏi: Single Writer pattern trên TCP socket đa luồng
- Câu trả lời kỳ vọng: NetworkStream không an toàn đa luồng cho các thao tác ghi đồng thời; Channel làm hàng đợi trung chuyển đảm bảo 1 consumer ghi tuần tự
- Câu hỏi mở rộng / Bẫy của giảng viên: Nếu hàng đợi channel bị đầy thì xử lý thế nào để không crash ứng dụng?

[HƯỚNG DẪN ĐỀ XUẤT]: *Tham khảo mã nguồn mẫu chuẩn và tài liệu kỹ thuật tại thư mục Networking. Khi hiện thực, luôn đặt an toàn luồng và xử lý hủy tác vụ (CancellationToken) lên hàng đầu.*

Nhiệm vụ 04: [LG-A-004] — TLS client và local identity enrollment

Mã & Tên nhiệm vụ: [LG-A-004] TLS client và local identity enrollment	**Tuần & Ưu tiên:** Tuần W1 (P0) | P0
Khối lượng công việc: Cơ bản: 2.0h | Dự phòng: 0.5h | Tổng: 2.5h	**Phân loại tải:** P0 Core
Trách nhiệm thực hiện: Phụ trách: SV A | Dự phòng: SV B | Duyệt: SV B	**Phân hệ & Thư mục:** Agent.Security (Security)
Sản phẩm bàn giao: TlsClientHandler.cs & IdentityStore	**Hợp đồng giao diện:** SslStream, RemoteCertificateValidationCallback, Local DeviceId store

1. Mục đích kỹ thuật & Tại sao tồn tại: Dữ liệu giám sát phòng máy và điều khiển sự cố nếu truyền dạng plaintext (không mã hóa) sẽ dễ dàng bị sinh viên trong phòng lab bắt gói tin bằng Wireshark hoặc can thiệp giả mạo. TLS 1.3 bảo đảm tính bí mật và toàn vẹn của kết nối.

2. Kiến thức nền tảng cần nắm vững: Mã hóa khóa công khai/bất đối xứng, Chứng chỉ số X.509, lớp SslStream và hàm ủy quyền RemoteCertificateValidationCallback.

3. Dữ liệu đầu vào & Đầu ra kỹ thuật: Đầu vào: Socket TCP đã kết nối tới Central Server, cấu hình mã băm chứng chỉ mong đợi (Certificate Thumbprint).
Đầu ra: Kênh truyền an toàn được bọc bởi SslStream với giao thức TLS 1.3.

4. Các bước triển khai chi tiết:
1. Bọc NetworkStream bên trong SslStream: new SslStream(networkStream, false, ValidateServerCertificate).
2. Gọi AuthenticateAsClientAsync(serverName) để tiến hành bắt tay (Handshake) TLS.
3. Trong hàm kiểm tra chứng chỉ: so khớp Certificate.GetCertHashString() với Thumbprint cấu hình sẵn (Certificate Pinning) để chấp nhận cert tự ký trong lab mà vẫn chặn giả mạo (MITM).
4. Đọc/ghi dữ liệu thông qua SslStream thay vì NetworkStream thô.

> **📌 BẪY LỖI CẦN TRÁNH TRONG LG-A-004**
> Viết return true vô điều kiện trong hàm callback xác thực chứng chỉ, biến TLS thành 'bình phong' và hoàn toàn mất khả năng chống tấn công mạo danh server.

5. Tiêu chuẩn hoàn thành 11 mục (Definition of Done — DoD):
- [ ] 1. Hợp đồng giao thức & Wire Envelope chuẩn UTF-8 JSON
- [ ] 2. Lập trình bất đồng bộ Async/Await không chặn luồng mạng
- [ ] 3. Bắt ngoại lệ mạng an toàn (EOF, SocketException, Timeout)
- [ ] 4. Giải phóng tài nguyên sạch sẽ (Dispose / Using / CancellationToken)
- [ ] 5. An toàn dữ liệu (Khử trùng lặp lô / Khóa lạc quan OCC / Transaction)
- [ ] 6. Unit Test / Test Harness chạy đạt 100% không có cảnh báo

• [ ] 7. Tích hợp chéo thành công với module của đồng đội liên quan
• [ ] 8. Kiểm thử thực tế trên mạng LAN nhiều máy tính
• [ ] 9. Đồng bộ tài liệu thiết kế & mã nguồn khớp hoàn toàn
• [ ] 10. Thu thập đầy đủ hồ sơ minh chứng (Log, Screenshot, Pcap)
• [ ] 11. Sẵn sàng trả lời trôi chảy câu hỏi vấn đáp (Viva) liên quan

6. Kịch bản kiểm thử & Hồ sơ minh chứng:
• Mã bài test: TEST-A04
• Phương pháp kiểm thử: Kết nối TLS tới server có cert tự ký; thử nghiệm cert giả mạo để kiểm tra từ chối
• Tiêu chuẩn nghiệm thu đạt: Bắt tay TLS thành công với cert hợp lệ; ngắt kết nối ngay khi cert không khớp thumbprint
• Hồ sơ minh chứng lưu tại: Wireshark packet dump mã hóa TLS, client error log

7. Tình huống lỗi tiềm ẩn & Kế hoạch ứng phó: MITM attack; từ chối kết nối nếu chứng chỉ không xác minh được tính toàn vẹn

8. Câu hỏi vấn đáp (Viva) & Chiến lược trả lời:
• Chủ đề câu hỏi: Bảo mật kênh truyền với SslStream trong .NET
• Câu trả lời kỳ vọng: SslStream bọc ngoài NetworkStream, mã hóa payload và bắt tay bảo mật; callback kiểm tra thumbprint ngăn chặn tấn công giả mạo
• Câu hỏi mở rộng / Bẫy của giảng viên: Tại sao không bỏ qua kiểm tra chứng chỉ bằng return true trong môi trường thật?

[HƯỚNG DẪN ĐỀ XUẤT]: *Tham khảo mã nguồn mẫu chuẩn và tài liệu kỹ thuật tại thư mục Security. Khi hiện thực, luôn đặt an toàn luồng và xử lý hủy tác vụ (CancellationToken) lên hàng đầu.*

Nhiệm vụ 05: [LG-A-005] — Thu thập CPU, RAM, NIC, Uptime hệ thống

Mã & Tên nhiệm vụ: [LG-A-005] Thu thập CPU, RAM, NIC, Uptime hệ thống	**Tuần & Ưu tiên:** Tuần W1 (P0) | P0
Khối lượng công việc: Cơ bản: 2.5h | Dự phòng: 0.5h | Tổng: 3.0h	**Phân loại tải:** P0 Core
Trách nhiệm thực hiện: Phụ trách: SV A | Dự phòng: SV C | Duyệt: SV C	**Phân hệ & Thư mục:** Agent.Metrics (System)
Sản phẩm bàn giao: WindowsMetricsCollector.cs	**Hợp đồng giao diện:** IMetricsCollector: ReadCpuPercent(), ReadMemoryBytes(), ReadNicBytes()

1. Mục đích kỹ thuật & Tại sao tồn tại: Mục tiêu cốt lõi của Agent là biết máy tính phòng lab đang khỏe hay yếu. Cần thu thập thông số CPU, RAM, lưu lượng mạng và thời gian bật máy (uptime) mà không tiêu tốn tài nguyên của máy trạm.

2. Kiến thức nền tảng cần nắm vững: Lớp System.Diagnostics.PerformanceCounter, System.Diagnostics.Process, System.Net.NetworkInformation.NetworkInterface.

3. Dữ liệu đầu vào & Đầu ra kỹ thuật: Đầu vào: Hệ điều hành Windows máy trạm phòng máy.
Đầu ra: Đối tượng dữ liệu Metric: cpuPercent, usedRamBytes, totalRamBytes, networkBytesSent, networkBytesRecv, uptimeSeconds.

4. Các bước triển khai chi tiết:
1. Khởi tạo PerformanceCounter cho CPU ('Processor', '% Processor Time', '_Total').
2. Lấy thông tin RAM vật lý qua API GC.GetGCMemoryInfo() hoặc P/Invoke GlobalMemoryStatusEx.
3. Đọc lưu lượng card mạng qua NetworkInterface.GetIPv4Statistics().
4. Đảm bảo thời gian đọc không vượt quá 100ms; nếu đọc lỗi thì trả về null an toàn kèm cờ lỗi, không crash Agent.

> **📌 BẪY LỖI CẦN TRÁNH TRONG LG-A-005**
> Lần gọi đầu tiên của CPU PerformanceCounter luôn trả về 0% (vì nó cần 2 mốc thời gian để tính delta). Phải gọi khởi động 1 lần lúc startup.

5. Tiêu chuẩn hoàn thành 11 mục (Definition of Done — DoD):
• [ ] 1. Hợp đồng giao thức & Wire Envelope chuẩn UTF-8 JSON
• [ ] 2. Lập trình bất đồng bộ Async/Await không chặn luồng mạng
• [ ] 3. Bắt ngoại lệ mạng an toàn (EOF, SocketException, Timeout)
• [ ] 4. Giải phóng tài nguyên sạch sẽ (Dispose / Using / CancellationToken)
• [ ] 5. An toàn dữ liệu (Khử trùng lặp lô / Khóa lạc quan OCC / Transaction)
• [ ] 6. Unit Test / Test Harness chạy đạt 100% không có cảnh báo
• [ ] 7. Tích hợp chéo thành công với module của đồng đội liên quan
• [ ] 8. Kiểm thử thực tế trên mạng LAN nhiều máy tính
• [ ] 9. Đồng bộ tài liệu thiết kế & mã nguồn khớp hoàn toàn
• [ ] 10. Thu thập đầy đủ hồ sơ minh chứng (Log, Screenshot, Pcap)
• [ ] 11. Sẵn sàng trả lời trôi chảy câu hỏi vấn đáp (Viva) liên quan

6. Kịch bản kiểm thử & Hồ sơ minh chứng:
• Mã bài test: TEST-A05
• Phương pháp kiểm thử: Chạy trên Windows 11 thật và máy ảo tải giả lập; đối chiếu số liệu với Task Manager
• Tiêu chuẩn nghiệm thu đạt: Chỉ số CPU/RAM đo liên tục mỗi 5s sai lệch <= 5% so với Task Manager, thời gian thu thập < 100ms
• Hồ sơ minh chứng lưu tại: Raw metrics CSV log so sánh với Task Manager

7. Tình huống lỗi tiềm ẩn & Kế hoạch ứng phó: Lỗi truy cập counter hiệu năng; fallback sang WMI hoặc trả giá trị an toàn kèm cờ lỗi

8. Câu hỏi vấn đáp (Viva) & Chiến lược trả lời:
• Chủ đề câu hỏi: Kỹ thuật thu thập chỉ số hiệu năng hệ điều hành Windows
• Câu trả lời kỳ vọng: Dùng PerformanceCounter lấy mẫu delta giữa 2 thời điểm; cần chạy trong background thread để tránh giật giao diện

• Câu hỏi mở rộng / Bẫy của giảng viên: Tại sao lần đọc đầu tiên của CPU PerformanceCounter luôn trả về 0?

[HƯỚNG DẪN ĐỀ XUẤT]: *Tham khảo mã nguồn mẫu chuẩn và tài liệu kỹ thuật tại thư mục System. Khi hiện thực, luôn đặt an toàn luồng và xử lý hủy tác vụ (CancellationToken) lên hàng đầu.*

Nhiệm vụ 06: [LG-A-006] — Heartbeat và batch sender

Mã & Tên nhiệm vụ: [LG-A-006] Heartbeat và batch sender	**Tuần & Ưu tiên:** Tuần W1 (P0) | P0
Khối lượng công việc: Cơ bản: 2.5h | Dự phòng: 0.5h | Tổng: 3.0h	**Phân loại tải:** P0 Core
Trách nhiệm thực hiện: Phụ trách: SV A | Dự phòng: SV B | Duyệt: SV B	**Phân hệ & Thư mục:** Agent.Transport (Networking)
Sản phẩm bàn giao: BatchScheduler.cs & SenderQueue	**Hợp đồng giao diện:** HEARTBEAT chu kỳ 5s, METRIC_BATCH (batchId, bootId, metrics)

1. Mục đích kỹ thuật & Tại sao tồn tại: Nếu mỗi giây gửi 1 gói tin nhỏ thì socket sẽ bị quá tải bởi header mạng. Vì vậy số liệu telemetry được gom mẻ (batch) gửi định kỳ. Tuy nhiên, nếu chờ gom mẻ thì server không biết máy còn sống hay không, do đó nhịp tim (Heartbeat) nhỏ phải được gửi tách rời độc lập.

2. Kiến thức nền tảng cần nắm vững: Bộ định thời PeriodicTimer hoặc Task.Delay, gom mẻ dữ liệu (Batching), vòng đời UUID BootId.

3. Dữ liệu đầu vào & Đầu ra kỹ thuật: Đầu vào: Dữ liệu metric thu thập từ A-005 và bộ đếm thời gian hệ thống.
Đầu ra: Gói tin HEARTBEAT gửi mỗi 5s và gói tin METRIC_BATCH gửi mỗi 10s lên Central Server.

4. Các bước triển khai chi tiết:
1. Sinh một mã BootId ngẫu nhiên (UUIDv4) mỗi khi tiến trình Agent khởi động (đại diện cho một phiên bật máy).
2. Thiết lập vòng lặp nhịp tim gửi HEARTBEAT với uptimeSeconds đều đặn mỗi 5 giây.
3. Thiết lập vòng lặp telemetry gom dữ liệu gửi METRIC_BATCH với mã batchId tự tăng.
4. Đưa gói tin vào hàng đợi chờ xác nhận; khi nhận BATCH_ACK từ Central Server thì xóa mẻ tin khỏi bộ nhớ.

> 📌 **BẪY LỖI CẦN TRÁNH TRONG LG-A-006**
> Gom chung Heartbeat vào trong Telemetry Batch. Khi máy tải nặng hoặc nghẽn batch, nhịp tim bị trễ khiến Central Server hiểu nhầm máy bị sập nguồn.

5. Tiêu chuẩn hoàn thành 11 mục (Definition of Done — DoD):
• [ ] 1. Hợp đồng giao thức & Wire Envelope chuẩn UTF-8 JSON
• [ ] 2. Lập trình bất đồng bộ Async/Await không chặn luồng mạng
• [ ] 3. Bắt ngoại lệ mạng an toàn (EOF, SocketException, Timeout)
• [ ] 4. Giải phóng tài nguyên sạch sẽ (Dispose / Using / CancellationToken)
• [ ] 5. An toàn dữ liệu (Khử trùng lặp lô / Khóa lạc quan OCC / Transaction)
• [ ] 6. Unit Test / Test Harness chạy đạt 100% không có cảnh báo
• [ ] 7. Tích hợp chéo thành công với module của đồng đội liên quan
• [ ] 8. Kiểm thử thực tế trên mạng LAN nhiều máy tính
• [ ] 9. Đồng bộ tài liệu thiết kế & mã nguồn khớp hoàn toàn
• [ ] 10. Thu thập đầy đủ hồ sơ minh chứng (Log, Screenshot, Pcap)
• [ ] 11. Sẵn sàng trả lời trôi chảy câu hỏi vấn đáp (Viva) liên quan

6. Kịch bản kiểm thử & Hồ sơ minh chứng:
• Mã bài test: TEST-A06
• Phương pháp kiểm thử: Gửi liên tục 100 batch tới Mini Server; đo đạc độ trễ nhận ACK
• Tiêu chuẩn nghiệm thu đạt: Heartbeat gửi đúng chu kỳ 5s ± 200ms; nhận BATCH_ACK và xóa mẻ khỏi queue bộ nhớ
• Hồ sơ minh chứng lưu tại: Trace log trao đổi bản tin, time delta log

7. Tình huống lỗi tiềm ẩn & Kế hoạch ứng phó: Không nhận được ACK sau timeout 5s; mẻ tin được giữ lại hàng đợi replay để retry

8. Câu hỏi vấn đáp (Viva) & Chiến lược trả lời:
• Chủ đề câu hỏi: Cơ chế phân tách luồng Heartbeat và Telemetry Batch
• Câu trả lời kỳ vọng: Heartbeat là bản tin nhỏ ưu tiên cao để duy trì liveness; telemetry batch là dữ liệu lớn gom mẻ để giảm tải socket
• Câu hỏi mở rộng / Bẫy của giảng viên: Nếu gom mẻ quá lâu thì có ảnh hưởng gì tới tính thời gian thực của cảnh báo?

[HƯỚNG DẪN ĐỀ XUẤT]: *Tham khảo mã nguồn mẫu chuẩn và tài liệu kỹ thuật tại thư mục Networking. Khi hiện thực, luôn đặt an toàn luồng và xử lý hủy tác vụ (CancellationToken) lên hàng đầu.*

Nhiệm vụ 07: [LG-A-007] — Local TCP service probe theo allowlist

Mã & Tên nhiệm vụ: [LG-A-007] Local TCP service probe theo allowlist	**Tuần & Ưu tiên:** Tuần W2 (P0) | P0
Khối lượng công việc: Cơ bản: 2.0h | Dự phòng: 0.5h | Tổng: 2.5h	**Phân loại tải:** P0 Core
Trách nhiệm thực hiện: Phụ trách: SV A | Dự phòng: SV C | Duyệt: SV C	**Phân hệ & Thư mục:** Agent.Service (Networking)
Sản phẩm bàn giao: ServiceProbeWorker.cs	**Hợp đồng giao diện:** SERVICE_STATUS (probes: [{port, serviceName, isUp, responseMs}])

1. Mục đích kỹ thuật & Tại sao tồn tại: Máy tính có thể bật nhưng phần mềm phục vụ thực hành (như Web Server cổng 80, SQL Server cổng 1433, App chuyên dụng cổng 8080) có thể bị treo hoặc chưa bật. Agent cần thăm dò cổng TCP nội bộ để phát hiện dịch vụ chết.

2. Kiến thức nền tảng cần nắm vững: Lớp TcpClient.ConnectAsync, kỹ thuật ngắt timeout bằng CancellationTokenSource, danh sách cổng cho phép (Allowlist).

3. Dữ liệu đầu vào & Đầu ra kỹ thuật: Đầu vào: Danh sách allowlist tối đa 5 cổng TCP cục bộ (ví dụ: [80, 1433, 8080]).
Đầu ra: Bản tin SERVICE_STATUS chứa mảng kết quả: {port, serviceName, isUp, responseTimeMs}.

4. Các bước triển khai chi tiết:
1. Đọc danh sách cổng cho phép từ file cấu hình (tối đa 5 cổng để không biến Agent thành công cụ quét port quá tải).
2. Duyệt qua từng cổng, dùng TcpClient.ConnectAsync("127.0.0.1", port) với CancellationToken giới hạn đúng 2000ms.
3. Nếu kết nối thành công: isUp = true, đo thời gian phản hồi (ms).
4. Nếu timeout hoặc bị từ chối (Connection Refused): isUp = false, đóng socket sạch sẽ.
5. Đóng gói kết quả vào payload SERVICE_STATUS gửi lên Central Server.

> **📌 BẪY LỖI CẦN TRÁNH TRONG LG-A-007**
> Không đặt timeout 2 giây khiến ConnectAsync bị treo mặc định tới 20-30 giây của hệ điều hành, làm tắc nghẽn toàn bộ luồng làm việc của Agent.

5. Tiêu chuẩn hoàn thành 11 mục (Definition of Done — DoD):
• [ ] 1. Hợp đồng giao thức & Wire Envelope chuẩn UTF-8 JSON
• [ ] 2. Lập trình bất đồng bộ Async/Await không chặn luồng mạng
• [ ] 3. Bắt ngoại lệ mạng an toàn (EOF, SocketException, Timeout)
• [ ] 4. Giải phóng tài nguyên sạch sẽ (Dispose / Using / CancellationToken)
• [ ] 5. An toàn dữ liệu (Khử trùng lặp lô / Khóa lạc quan OCC / Transaction)
• [ ] 6. Unit Test / Test Harness chạy đạt 100% không có cảnh báo
• [ ] 7. Tích hợp chéo thành công với module của đồng đội liên quan
• [ ] 8. Kiểm thử thực tế trên mạng LAN nhiều máy tính
• [ ] 9. Đồng bộ tài liệu thiết kế & mã nguồn khớp hoàn toàn
• [ ] 10. Thu thập đầy đủ hồ sơ minh chứng (Log, Screenshot, Pcap)
• [ ] 11. Sẵn sàng trả lời trôi chảy câu hỏi vấn đáp (Viva) liên quan

6. Kịch bản kiểm thử & Hồ sơ minh chứng:
• Mã bài test: TEST-A07
• Phương pháp kiểm thử: Mô phỏng mở và đóng port 80/8080 cục bộ; ghi nhận trạng thái probe
• Tiêu chuẩn nghiệm thu đạt: Phát hiện port mở trả isUp=true < 50ms; port đóng/bị lọc trả isUp=false sau

đúng timeout 2s
• Hồ sơ minh chứng lưu tại: Service probe status log, timing benchmark

7. Tình huống lỗi tiềm ẩn & Kế hoạch ứng phó: Socket connect bị treo vô hạn; sử dụng CancellationTokenSource với timeout 2000ms để hủy kết nối

8. Câu hỏi vấn đáp (Viva) & Chiến lược trả lời:
• Chủ đề câu hỏi: Thăm dò cổng TCP và xử lý timeout bất đồng bộ
• Câu trả lời kỳ vọng: Dùng TcpClient.ConnectAsync kèm CancellationToken hoặc Task.WhenAny với Task.Delay để ép timeout 2s, tránh nghẽn thread
• Câu hỏi mở rộng / Bẫy của giảng viên: Sự khác biệt giữa cổng đóng (RST) và cổng bị tường lửa drop (Timeout)?

[HƯỚNG DẪN ĐỀ XUẤT]: *Tham khảo mã nguồn mẫu chuẩn và tài liệu kỹ thuật tại thư mục Networking. Khi hiện thực, luôn đặt an toàn luồng và xử lý hủy tác vụ (CancellationToken) lên hàng đầu.*

Nhiệm vụ 08: [LG-A-008] — Agent Simulator 10/20/50 socket độc lập

Mã & Tên nhiệm vụ: [LG-A-008] Agent Simulator 10/20/50 socket độc lập	**Tuần & Ưu tiên:** Tuần W2 (P0) | P0
Khối lượng công việc: Cơ bản: 4.0h | Dự phòng: 0.5h | Tổng: 4.5h	**Phân loại tải:** Test/Harness
Trách nhiệm thực hiện: Phụ trách: SV A | Dự phòng: SV B | Duyệt: SV B	**Phân hệ & Thư mục:** Agent.Simulator (Testing)
Sản phẩm bàn giao: LabGuard.Simulator.exe	**Hợp đồng giao diện:** Giả lập N Agent clients, mỗi client có DeviceId, BootId, IP và metrics profile riêng

1. Mục đích kỹ thuật & Tại sao tồn tại: Trong giai đoạn phát triển, nhóm không thể lúc nào cũng có sẵn 20-50 máy tính thật trong phòng lab để thử tải. Cần một công cụ giả lập (Simulator) tạo ra 20-50 socket TCP độc lập chạy song song trên một máy tính để kiểm tra độ chịu tải của Central Server.

2. Kiến thức nền tảng cần nắm vững: Lập trình bất đồng bộ đa luồng (Task.WhenAll), quản lý kết nối socket số lượng lớn, sinh số ngẫu nhiên mô phỏng tải.

3. Dữ liệu đầu vào & Đầu ra kỹ thuật: Đầu vào: Số lượng máy giả lập N (10, 20, 50), địa chỉ Central Server.
Đầu ra: Ứng dụng console LabGuard.Simulator.exe tạo N client kết nối đồng thời, gửi nhịp tim và metric độc lập.

4. Các bước triển khai chi tiết:
1. Khởi tạo mảng N đối tượng Agent client độc lập, mỗi client có DeviceId (PC-01 .. PC-20) và BootId riêng.
2. Mỗi simulated agent chạy vòng lặp riêng: gửi bắt tay, gửi heartbeat mỗi 5s và gửi metric mỗi 10s có kèm độ trễ ngẫu nhiên (jitter) để tránh đồng loạt gửi cùng một mili-giây.

3. Hiển thị bảng điều khiển console tóm tắt số kết nối đang sống, số batch đã gửi và số ACK nhận được.
4. Hỗ trợ phím bấm giả lập máy mất kết nối hoặc giả lập máy bị quá tải CPU (>=90%) để kiểm tra Rule Engine của B.

> 📌 **BẪY LỖI CẦN TRÁNH TRONG LG-A-008**
> Dùng chung một kết nối Socket hoặc chung một đối tượng Random giữa các máy giả lập, làm mất tính chân thực của môi trường mạng phân tán.

5. Tiêu chuẩn hoàn thành 11 mục (Definition of Done — DoD):
• [ ] 1. Hợp đồng giao thức & Wire Envelope chuẩn UTF-8 JSON
• [ ] 2. Lập trình bất đồng bộ Async/Await không chặn luồng mạng
• [ ] 3. Bắt ngoại lệ mạng an toàn (EOF, SocketException, Timeout)
• [ ] 4. Giải phóng tài nguyên sạch sẽ (Dispose / Using / CancellationToken)
• [ ] 5. An toàn dữ liệu (Khử trùng lặp lô / Khóa lạc quan OCC / Transaction)
• [ ] 6. Unit Test / Test Harness chạy đạt 100% không có cảnh báo
• [ ] 7. Tích hợp chéo thành công với module của đồng đội liên quan
• [ ] 8. Kiểm thử thực tế trên mạng LAN nhiều máy tính
• [ ] 9. Đồng bộ tài liệu thiết kế & mã nguồn khớp hoàn toàn
• [ ] 10. Thu thập đầy đủ hồ sơ minh chứng (Log, Screenshot, Pcap)
• [ ] 11. Sẵn sàng trả lời trôi chảy câu hỏi vấn đáp (Viva) liên quan

6. Kịch bản kiểm thử & Hồ sơ minh chứng:
• Mã bài test: TEST-A08
• Phương pháp kiểm thử: Chạy 20 simulated agent kết nối tới Central Server liên tục 30 phút
• Tiêu chuẩn nghiệm thu đạt: 20 socket duy trì kết nối độc lập; gửi heartbeat 5s và metric 10s không bị đan xen hay deadlock
• Hồ sơ minh chứng lưu tại: Simulator process monitor, Central connection count screenshot

7. Tình huống lỗi tiềm ẩn & Kế hoạch ứng phó: Tràn socket hoặc cạn kiệt port ephemeral; tái sử dụng kết nối và quản lý vòng đời socket cẩn thận

8. Câu hỏi vấn đáp (Viva) & Chiến lược trả lời:
• Chủ đề câu hỏi: Mô hình giả lập tải mạng nhiều node trên môi trường thử nghiệm
• Câu trả lời kỳ vọng: Mỗi simulated node là một state machine độc lập chạy trên Task riêng; mô phỏng đúng hành vi delay và jitter thực tế
• Câu hỏi mở rộng / Bẫy của giảng viên: Giả lập nhiều socket trên 1 máy khác gì so với nhiều máy thật trong mạng LAN?

[HƯỚNG DẪN ĐỀ XUẤT]: *Tham khảo mã nguồn mẫu chuẩn và tài liệu kỹ thuật tại thư mục Testing. Khi hiện thực, luôn đặt an toàn luồng và xử lý hủy tác vụ (CancellationToken) lên hàng đầu.*

Nhiệm vụ 09: [LG-A-009] — Reconnect backoff và metric replay retry

Mã & Tên nhiệm vụ: [LG-A-009] Reconnect backoff và metric replay retry	**Tuần & Ưu tiên:** Tuần W2 (P0) | P0
Khối lượng công việc: Cơ bản: 2.5h | Dự phòng: 0.5h | Tổng: 3.0h	**Phân loại tải:** P0 Core
Trách nhiệm thực hiện: Phụ trách: SV A | Dự phòng: SV B | Duyệt: SV B	**Phân hệ & Thư mục:** Agent.Transport (Networking)
Sản phẩm bàn giao: ReplayQueue.cs & BackoffPolicy.cs	**Hợp đồng giao diện:** Backoff: 1s, 2s, 4s, 8s, max 30s + jitter 10%; payload.isReplay = true

1. Mục đích kỹ thuật & Tại sao tồn tại: Mạng phòng thực hành thường xuyên bị chập chờn hoặc rút dây cáp mạng. Nếu Agent ngắt kết nối rồi bỏ cuộc thì hệ thống sẽ mất giám sát. Tuy nhiên nếu mọi máy cùng lao vào kết nối lại cùng 1 giây (Reconnect Storm), server sẽ bị nghẽn cứng. Cần thuật toán Exponential Backoff kèm Jitter.

2. Kiến thức nền tảng cần nắm vững: Thuật toán Exponential Backoff, cơ chế thêm độ lệch ngẫu nhiên Jitter (RFC 6598), hàng đợi Replay trong bộ nhớ.

3. Dữ liệu đầu vào & Đầu ra kỹ thuật: Đầu vào: Sự kiện đứt socket khi đang gửi hoặc đọc dữ liệu.
Đầu ra: Cơ chế tự kết nối lại an toàn và gửi bù các mẻ telemetry chưa được ACK với cờ isReplay = true.

4. Các bước triển khai chi tiết:
1. Bắt ngoại lệ mất kết nối (SocketException, IOException).
2. Tính thời gian chờ: delay = Min(30s, baseDelay * 2^attempt) + RandomJitter(-10%, +10%).
3. Trong lúc mất mạng, lưu tạm tối đa 50 mẻ telemetry vào hàng đợi RAM cục bộ (không lưu heartbeat vì heartbeat cũ vô giá trị).
4. Khi kết nối lại thành công: gửi lại các mẻ telemetry cũ với cờ isReplay = true để Central Server biết và không kích hoạt cảnh báo sai mốc thời gian.

> 📌 **BẪY LỖI CẦN TRÁNH TRONG LG-A-009**
> Replay lại toàn bộ các gói tin HEARTBEAT cũ sau khi có mạng lại. Việc này là vô nghĩa và làm ngập lụt server bằng dữ liệu rác.

5. Tiêu chuẩn hoàn thành 11 mục (Definition of Done — DoD):
- [ ] 1. Hợp đồng giao thức & Wire Envelope chuẩn UTF-8 JSON
- [ ] 2. Lập trình bất đồng bộ Async/Await không chặn luồng mạng
- [ ] 3. Bắt ngoại lệ mạng an toàn (EOF, SocketException, Timeout)
- [ ] 4. Giải phóng tài nguyên sạch sẽ (Dispose / Using / CancellationToken)
- [ ] 5. An toàn dữ liệu (Khử trùng lặp lô / Khóa lạc quan OCC / Transaction)
- [ ] 6. Unit Test / Test Harness chạy đạt 100% không có cảnh báo
- [ ] 7. Tích hợp chéo thành công với module của đồng đội liên quan
- [ ] 8. Kiểm thử thực tế trên mạng LAN nhiều máy tính

• [ ] 9. Đồng bộ tài liệu thiết kế & mã nguồn khớp hoàn toàn
• [ ] 10. Thu thập đầy đủ hồ sơ minh chứng (Log, Screenshot, Pcap)
• [ ] 11. Sẵn sàng trả lời trôi chảy câu hỏi vấn đáp (Viva) liên quan

6. Kịch bản kiểm thử & Hồ sơ minh chứng:
• Mã bài test: TEST-A09
• Phương pháp kiểm thử: Ngắt mạng 30s rồi khôi phục; kiểm tra số mẻ replay được gửi lên Central
• Tiêu chuẩn nghiệm thu đạt: Tự động kết nối lại theo backoff; replay các mẻ telemetry chưa ACK với isReplay=true; không replay heartbeat
• Hồ sơ minh chứng lưu tại: Replay queue dump log, reconnect timeline capture

7. Tình huống lỗi tiềm ẩn & Kế hoạch ứng phó: Reconnect storm khi server restart; Jitter phân tán thời điểm kết nối giúp giảm xung đột

8. Câu hỏi vấn đáp (Viva) & Chiến lược trả lời:
• Chủ đề câu hỏi: Thuật toán Exponential Backoff kèm Jitter và hàng đợi Replay
• Câu trả lời kỳ vọng: Nhân đôi thời gian chờ sau mỗi lần thử kèm độ lệch ngẫu nhiên để tránh hiện tượng mọi client đồng loạt kết nối lại cùng lúc
• Câu hỏi mở rộng / Bẫy của giảng viên: Tại sao không được replay bản tin Heartbeat khi kết nối lại?

[HƯỚNG DẪN ĐỀ XUẤT]: *Tham khảo mã nguồn mẫu chuẩn và tài liệu kỹ thuật tại thư mục Networking. Khi hiện thực, luôn đặt an toàn luồng và xử lý hủy tác vụ (CancellationToken) lên hàng đầu.*

Nhiệm vụ 10: [LG-A-010] — Failure injection và counters

Mã & Tên nhiệm vụ: [LG-A-010] Failure injection và counters	**Tuần & Ưu tiên:** Tuần W2 (P0) | P0
Khối lượng công việc: Cơ bản: 3.0h | Dự phòng: 0.5h | Tổng: 3.5h	**Phân loại tải:** Test/Harness
Trách nhiệm thực hiện: Phụ trách: SV A | Dự phòng: SV C | Duyệt: SV C	**Phân hệ & Thư mục:** Agent.Testing (Testing)
Sản phẩm bàn giao: NetworkFailureInjector.cs & DiagnosticsCounters	**Hợp đồng giao diện:** IFailureInjector: InjectDrop(rate), InjectDelay(ms), InjectDisconnect()

1. Mục đích kỹ thuật & Tại sao tồn tại: Để chứng minh phần mềm có độ tin cậy cao, không thể chỉ test trường hợp mạng hoàn hảo ('Happy Path'). Phải chủ động tiêm lỗi (Chaos/Failure Injection) như cố tình cắt ngang frame, gửi gói tin rác, giả lập nghẽn mạng để đo lường khả năng tự phục hồi.

2. Kiến thức nền tảng cần nắm vững: Kỹ thuật kiểm thử độ bền (Chaos Engineering), bộ đếm số liệu (Counters / Metrics).

3. Dữ liệu đầu vào & Đầu ra kỹ thuật: Đầu vào: Lệnh kích hoạt từ console hoặc phím tắt trong chế độ Debug.
Đầu ra: Báo cáo số lỗi đã bắt, số lần ngắt kết nối an toàn, bộ đếm không bị rò rỉ bộ nhớ.

4. Các bước triển khai chi tiết:

1. Thêm cờ cấu hình --inject-faults vào Agent hoặc Simulator.
2. Hỗ trợ các kịch bản: ngắt kết nối đột ngột ngay sau khi gửi 2 byte header, gửi frame vượt 256 KiB, gửi chuỗi JSON hỏng cú pháp.
3. Ghi nhận nhật ký: Agent và Central Server phải xử lý ngoại lệ trơn tru, không crash tiến trình, đóng socket lỗi và giải phóng buffer.
4. Xuất log đo kiểm phục vụ hồ sơ nghiệm thu.

> 📌 **BẪY LỖI CẦN TRÁNH TRONG LG-A-010**
> Nuốt ngoại lệ bằng khối catch { } rỗng khiến lỗi bị giấu nhẹm, gây rò rỉ socket và làm sập ứng dụng sau vài giờ chạy.

5. Tiêu chuẩn hoàn thành 11 mục (Definition of Done — DoD):

• [ ] 1. Hợp đồng giao thức & Wire Envelope chuẩn UTF-8 JSON
• [ ] 2. Lập trình bất đồng bộ Async/Await không chặn luồng mạng
• [ ] 3. Bắt ngoại lệ mạng an toàn (EOF, SocketException, Timeout)
• [ ] 4. Giải phóng tài nguyên sạch sẽ (Dispose / Using / CancellationToken)
• [ ] 5. An toàn dữ liệu (Khử trùng lặp lô / Khóa lạc quan OCC / Transaction)
• [ ] 6. Unit Test / Test Harness chạy đạt 100% không có cảnh báo
• [ ] 7. Tích hợp chéo thành công với module của đồng đội liên quan
• [ ] 8. Kiểm thử thực tế trên mạng LAN nhiều máy tính
• [ ] 9. Đồng bộ tài liệu thiết kế & mã nguồn khớp hoàn toàn
• [ ] 10. Thu thập đầy đủ hồ sơ minh chứng (Log, Screenshot, Pcap)
• [ ] 11. Sẵn sàng trả lời trôi chảy câu hỏi vấn đáp (Viva) liên quan

6. Kịch bản kiểm thử & Hồ sơ minh chứng:

• Mã bài test: TEST-A10
• Phương pháp kiểm thử: Inject ngắt kết nối giữa chừng lúc đang gửi frame lớn; theo dõi phản ứng của Agent
• Tiêu chuẩn nghiệm thu đạt: Agent phát hiện lỗi socket, kích hoạt backoff kết nối lại, đếm đúng số sự kiện ngắt và không crash tiến trình
• Hồ sơ minh chứng lưu tại: Failure injection log, counter report

7. Tình huống lỗi tiềm ẩn & Kế hoạch ứng phó: Ngoại lệ SocketException không được bắt; bọc try-catch toàn diện tại tầng I/O và ghi nhận counter

8. Câu hỏi vấn đáp (Viva) & Chiến lược trả lời:

• Chủ đề câu hỏi: Kiểm thử khả năng phục hồi lỗi mạng (Chaos / Resilience Testing)
• Câu trả lời kỳ vọng: Chủ động tạo lỗi giả định giúp xác minh mã nguồn tự khôi phục đúng thiết kế trước khi đưa vào môi trường thật
• Câu hỏi mở rộng / Bẫy của giảng viên: Làm thế nào để phân biệt mất kết nối do cáp mạng rút hay do server chủ động đóng?

[HƯỚNG DẪN ĐỀ XUẤT]: *Tham khảo mã nguồn mẫu chuẩn và tài liệu kỹ thuật tại thư mục Testing. Khi hiện thực, luôn đặt an toàn luồng và xử lý hủy tác vụ (CancellationToken) lên hàng đầu.*

Nhiệm vụ 11: [LG-A-011] — Log xoay vòng và evidence hỗ trợ debug

Mã & Tên nhiệm vụ: [LG-A-011] Log xoay vòng và evidence hỗ trợ debug	**Tuần & Ưu tiên:** Tuần W3 (P0) | P0
Khối lượng công việc: Cơ bản: 1.5h | Dự phòng: 0.5h | Tổng: 2.0h	**Phân loại tải:** P0 Core
Trách nhiệm thực hiện: Phụ trách: SV A | Dự phòng: SV B | Duyệt: SV B	**Phân hệ & Thư mục:** Agent.Diagnostics (System)
Sản phẩm bàn giao: RollingLogger.cs	**Hợp đồng giao diện:** MaxFileSize = 10MB, MaxFiles = 5, Async file append

1. Mục đích kỹ thuật & Tại sao tồn tại: Khi chạy thực tế trong phòng lab, nếu file log ghi liên tục không giới hạn sẽ làm đầy ổ cứng của máy trạm. Cần cơ chế xoay vòng file log (Rolling File Log) tự động xóa hoặc nén file cũ khi vượt quá dung lượng cho phép.

2. Kiến thức nền tảng cần nắm vững: Ghi nhật ký ứng dụng (Logging), xoay vòng tệp tin (File Rotation), kích thước tối đa và số lượng tệp lưu giữ.

3. Dữ liệu đầu vào & Đầu ra kỹ thuật: Đầu vào: Các sự kiện hoạt động của Agent (kết nối, ngắt kết nối, gửi mẻ, lỗi socket).
Đầu ra: Thư mục log chứa tối đa 3 file log, mỗi file tối đa 5 MB (tự động ghi đè file cũ nhất khi đầy).

4. Các bước triển khai chi tiết:
1. Cấu hình thư viện logging (như Serilog hoặc viết RollingFileSink đơn giản).
2. Thiết lập kích thước tối đa 5 MB cho mỗi tệp log; số tệp tối đa lưu trữ là 3.
3. Định dạng log chuẩn: [Timestamp UTC] [Level] [DeviceId] Message.
4. Đảm bảo thao tác ghi log không làm chậm luồng mạng chính (dùng background log channel).

> 📌 **BẪY LỖI CẦN TRÁNH TRONG LG-A-011**
> Mở file log dạng độc quyền (exclusive lock) khiến nhiều tiến trình không thể đọc log để hiển thị hoặc gây lỗi khi tiến trình cố gắng ghi đồng thời.

5. Tiêu chuẩn hoàn thành 11 mục (Definition of Done — DoD):
• [ ] 1. Hợp đồng giao thức & Wire Envelope chuẩn UTF-8 JSON
• [ ] 2. Lập trình bất đồng bộ Async/Await không chặn luồng mạng
• [ ] 3. Bắt ngoại lệ mạng an toàn (EOF, SocketException, Timeout)
• [ ] 4. Giải phóng tài nguyên sạch sẽ (Dispose / Using / CancellationToken)
• [ ] 5. An toàn dữ liệu (Khử trùng lặp lô / Khóa lạc quan OCC / Transaction)

• [ ] 6. Unit Test / Test Harness chạy đạt 100% không có cảnh báo
• [ ] 7. Tích hợp chéo thành công với module của đồng đội liên quan
• [ ] 8. Kiểm thử thực tế trên mạng LAN nhiều máy tính
• [ ] 9. Đồng bộ tài liệu thiết kế & mã nguồn khớp hoàn toàn
• [ ] 10. Thu thập đầy đủ hồ sơ minh chứng (Log, Screenshot, Pcap)
• [ ] 11. Sẵn sàng trả lời trôi chảy câu hỏi vấn đáp (Viva) liên quan

6. Kịch bản kiểm thử & Hồ sơ minh chứng:
• Mã bài test: TEST-A11
• Phương pháp kiểm thử: Bơm liên tục 50MB log kiểm tra xoay vòng file
• Tiêu chuẩn nghiệm thu đạt: Tổng dung lượng thư mục log luôn <= 50MB; file cũ tự xóa khi vượt quá 5 file
• Hồ sơ minh chứng lưu tại: Thư mục log mẫu chứa các file xoay vòng

7. Tình huống lỗi tiềm ẩn & Kế hoạch ứng phó: Nghẽn I/O đĩa làm chậm luồng mạng; ghi log bất đồng bộ qua hàng đợi riêng biệt

8. Câu hỏi vấn đáp (Viva) & Chiến lược trả lời:
• Chủ đề câu hỏi: Chiến lược lưu log xoay vòng trong ứng dụng giám sát
• Câu trả lời kỳ vọng: Giới hạn dung lượng và số file để tránh làm cạn kiệt ổ cứng máy trạm trong các đợt phát sinh lỗi liên tục
• Câu hỏi mở rộng / Bẫy của giảng viên: Tại sao định dạng JSON log lại thuận tiện cho phân tích tự động?

[HƯỚNG DẪN ĐỀ XUẤT]: *Tham khảo mã nguồn mẫu chuẩn và tài liệu kỹ thuật tại thư mục System. Khi hiện thực, luôn đặt an toàn luồng và xử lý hủy tác vụ (CancellationToken) lên hàng đầu.*

Nhiệm vụ 12: [LG-A-012] — MVP Agent end-to-end trên hai máy thật LAN

Mã & Tên nhiệm vụ: [LG-A-012] MVP Agent end-to-end trên hai máy thật LAN	**Tuần & Ưu tiên:** Tuần W3 (P0) | P0
Khối lượng công việc: Cơ bản: 3.5h | Dự phòng: 0.5h | Tổng: 4.0h	**Phân loại tải:** Tích hợp
Trách nhiệm thực hiện: Phụ trách: SV A | Dự phòng: SV B | Duyệt: SV B	**Phân hệ & Thư mục:** Agent.Integration (Integration)
Sản phẩm bàn giao: Script cài đặt & Báo cáo đo kiểm LAN	**Hợp đồng giao diện:** TCP/TLS qua địa chỉ IP LAN thật (192.168.x.x)

1. Mục đích kỹ thuật & Tại sao tồn tại: Chạy localhost (127.0.0.1) bỏ qua toàn bộ các yếu tố thực tế như tường lửa (Windows Defender Firewall), trễ mạng LAN, phân mảnh gói tin và mất gói. Đây là mốc kiểm chứng thực tế quan trọng nhất của Lane A trước khi nghiệm thu MVP tại Tuần 3.

2. Kiến thức nền tảng cần nắm vững: Mạng LAN, cấp phát địa chỉ IP qua DHCP, cấu hình mở cổng Inbound/Outbound trên Windows Firewall, lệnh ping và tracert.

3. Dữ liệu đầu vào & Đầu ra kỹ thuật: Đầu vào: 2 máy tính vật lý thật kết nối cùng mạng Wi-Fi hoặc Switch phòng máy UIT.
Đầu ra: Báo cáo đo kiểm LAN thật, ảnh chụp màn hình Central Server nhận telemetry từ 2 máy thật liên tục 1 giờ.

4. Các bước triển khai chi tiết:
1. Cài đặt Agent lên 2 máy tính vật lý trong phòng máy (hoặc 2 laptop cá nhân kết nối chung mạng hotspot).
2. Cấu hình địa chỉ IP LAN của máy Central Server (máy sinh viên B).
3. Mở cổng TCP 5000 trên Windows Firewall của máy B: netsh advfirewall firewall add rule ...
4. Chạy Agent liên tục 1 giờ: quan sát cả 2 máy hiển thị trạng thái Online trên Dashboard của sinh viên C.
5. Thu thập log kết nối và bằng chứng chụp màn hình.

> 📌 **BẪY LỖI CẦN TRÁNH TRONG LG-A-012**
> Quên mở port trên Windows Firewall của máy Server khiến Agent bên máy trạm báo lỗi Connection Refused hoặc Timed out.

5. Tiêu chuẩn hoàn thành 11 mục (Definition of Done — DoD):
• [ ] 1. Hợp đồng giao thức & Wire Envelope chuẩn UTF-8 JSON
• [ ] 2. Lập trình bất đồng bộ Async/Await không chặn luồng mạng
• [ ] 3. Bắt ngoại lệ mạng an toàn (EOF, SocketException, Timeout)
• [ ] 4. Giải phóng tài nguyên sạch sẽ (Dispose / Using / CancellationToken)
• [ ] 5. An toàn dữ liệu (Khử trùng lặp lô / Khóa lạc quan OCC / Transaction)
• [ ] 6. Unit Test / Test Harness chạy đạt 100% không có cảnh báo
• [ ] 7. Tích hợp chéo thành công với module của đồng đội liên quan
• [ ] 8. Kiểm thử thực tế trên mạng LAN nhiều máy tính
• [ ] 9. Đồng bộ tài liệu thiết kế & mã nguồn khớp hoàn toàn
• [ ] 10. Thu thập đầy đủ hồ sơ minh chứng (Log, Screenshot, Pcap)
• [ ] 11. Sẵn sàng trả lời trôi chảy câu hỏi vấn đáp (Viva) liên quan

6. Kịch bản kiểm thử & Hồ sơ minh chứng:
• Mã bài test: TEST-A12
• Phương pháp kiểm thử: Chạy đồng thời 2 Agent máy thật liên tục 1 giờ trong giờ thực hành
• Tiêu chuẩn nghiệm thu đạt: Cả 2 máy hiển thị trạng thái Online trên Dashboard của sinh viên C; dữ liệu metric cập nhật đều đặn
• Hồ sơ minh chứng lưu tại: Ảnh chụp màn hình 2 máy thật gửi tin, log kết nối LAN

7. Tình huống lỗi tiềm ẩn & Kế hoạch ứng phó: Windows Defender Firewall chặn port; thêm rule cho phép inbound/outbound TCP port tương ứng

8. Câu hỏi vấn đáp (Viva) & Chiến lược trả lời:
• Chủ đề câu hỏi: Thực tế kiểm thử mạng LAN phòng máy so với chạy localhost
• Câu trả lời kỳ vọng: Localhost bỏ qua tường lửa, trễ mạng và phân mảnh gói tin thực tế; test LAN kiểm chứng đúng giao thức mạng

• Câu hỏi mở rộng / Bẫy của giảng viên: Làm thế nào xác định địa chỉ IP máy server khi phòng máy dùng DHCP cấp động?

[HƯỚNG DẪN ĐỀ XUẤT]: *Tham khảo mã nguồn mẫu chuẩn và tài liệu kỹ thuật tại thư mục Integration. Khi hiện thực, luôn đặt an toàn luồng và xử lý hủy tác vụ (CancellationToken) lên hàng đầu.*

Nhiệm vụ 13: [LG-A-013] — Gateway least-connections admission

Mã & Tên nhiệm vụ: [LG-A-013] Gateway least-connections admission	**Tuần & Ưu tiên:** Tuần W4 (P1) | P1
Khối lượng công việc: Cơ bản: 6.0h | Dự phòng: 0.5h | Tổng: 6.5h	**Phân loại tải:** P1 Extension
Trách nhiệm thực hiện: Phụ trách: SV A | Dự phòng: SV C | Duyệt: SV C	**Phân hệ & Thư mục:** P1.Gateway (Networking)
Sản phẩm bàn giao: LabGuard.Gateway.exe & LoadBalancerCore.cs	**Hợp đồng giao diện:** TCP proxy L4, thuật toán Least-Connections trên danh sách backend Collector

1. Mục đích kỹ thuật & Tại sao tồn tại: Ở tầng P1 (sau khi MVP đạt), nếu số lượng máy trạm tăng lên hàng trăm máy, một Central Server không thể gánh nổi toàn bộ kết nối. Cần một Gateway đóng vai trò Reverse Proxy tiếp nhận kết nối và điều phối (Admission) vào Collector ít tải nhất.

2. Kiến thức nền tảng cần nắm vững: Khái niệm Reverse Proxy, thuật toán cân bằng tải Ít kết nối nhất (Least-Connections Load Balancing), bắt tay chuyển tiếp socket.

3. Dữ liệu đầu vào & Đầu ra kỹ thuật: Đầu vào: Kết nối TCP từ Agent mới và thông tin số lượng kết nối hiện tại của các node Collector.
Đầu ra: Quyết định điều phối: chuyển tiếp kết nối vào Collector có ít client nhất.

4. Các bước triển khai chi tiết:
1. Gateway lắng nghe kết nối client trên cổng công khai 5000.
2. Duy trì bảng theo dõi trạng thái các node Collector: {collectorId, address, port, activeConnections, isHealthy}.
3. Khi có kết nối mới: chọn node Collector có isHealthy = true và activeConnections nhỏ nhất.
4. Mở kết nối nội bộ tới Collector đó và tăng bộ đếm active connections.

> 📌 **BẪY LỖI CẦN TRÁNH TRONG LG-A-013**
> Chọn Collector dựa trên số thứ tự tĩnh (Round-Robin) mà không kiểm tra tình trạng sức khỏe thực tế, dẫn đến việc chuyển client vào một Collector vừa bị chết.

5. Tiêu chuẩn hoàn thành 11 mục (Definition of Done — DoD):
• [ ] 1. Hợp đồng giao thức & Wire Envelope chuẩn UTF-8 JSON
• [ ] 2. Lập trình bất đồng bộ Async/Await không chặn luồng mạng

• [ ] 3. Bắt ngoại lệ mạng an toàn (EOF, SocketException, Timeout)
• [ ] 4. Giải phóng tài nguyên sạch sẽ (Dispose / Using / CancellationToken)
• [ ] 5. An toàn dữ liệu (Khử trùng lặp lô / Khóa lạc quan OCC / Transaction)
• [ ] 6. Unit Test / Test Harness chạy đạt 100% không có cảnh báo
• [ ] 7. Tích hợp chéo thành công với module của đồng đội liên quan
• [ ] 8. Kiểm thử thực tế trên mạng LAN nhiều máy tính
• [ ] 9. Đồng bộ tài liệu thiết kế & mã nguồn khớp hoàn toàn
• [ ] 10. Thu thập đầy đủ hồ sơ minh chứng (Log, Screenshot, Pcap)
• [ ] 11. Sẵn sàng trả lời trôi chảy câu hỏi vấn đáp (Viva) liên quan

6. Kịch bản kiểm thử & Hồ sơ minh chứng:
• Mã bài test: TEST-A13
• Phương pháp kiểm thử: Mở lần lượt 10 kết nối mới qua Gateway; kiểm tra số kết nối phân bổ về 2 Collector
• Tiêu chuẩn nghiệm thu đạt: Phân bổ đều 5-5 giữa 2 Collector; chuyển kết nối mới về node ít tải hơn khi có sự chênh lệch
• Hồ sơ minh chứng lưu tại: Gateway connection metrics log, bảng phân bổ tải

7. Tình huống lỗi tiềm ẩn & Kế hoạch ứng phó: Collector bị đầy kết nối; từ chối kết nối hoặc giữ trong hàng đợi chờ với timeout quy định

8. Câu hỏi vấn đáp (Viva) & Chiến lược trả lời:
• Chủ đề câu hỏi: Thuật toán cân bằng tải Least-Connections tầng giao vận (L4)
• Câu trả lời kỳ vọng: Theo dõi số lượng kết nối đang hoạt động trên từng backend để định tuyến kết nối mới vào node rảnh nhất
• Câu hỏi mở rộng / Bẫy của giảng viên: Tại sao L4 proxy không cần parse nội dung JSON bên trong packet?

[HƯỚNG DẪN ĐỀ XUẤT]: *Tham khảo mã nguồn mẫu chuẩn và tài liệu kỹ thuật tại thư mục Networking. Khi hiện thực, luôn đặt an toàn luồng và xử lý hủy tác vụ (CancellationToken) lên hàng đầu.*

Nhiệm vụ 14: [LG-A-014] — Gateway duplex relay và health polling

Mã & Tên nhiệm vụ: [LG-A-014] Gateway duplex relay và health polling	**Tuần & Ưu tiên:** Tuần W4 (P1) | P1
Khối lượng công việc: Cơ bản: 6.0h | Dự phòng: 0.5h | Tổng: 6.5h	**Phân loại tải:** P1 Extension
Trách nhiệm thực hiện: Phụ trách: SV A | Dự phòng: SV B | Duyệt: SV B	**Phân hệ & Thư mục:** P1.Gateway (Networking)
Sản phẩm bàn giao: DuplexRelay.cs & HealthPoller.cs	**Hợp đồng giao diện:** Stream.CopyToAsync hai chiều, Health Probe TCP chu kỳ 2s (3 fail / 2 success)

1. Mục đích kỹ thuật & Tại sao tồn tại: Sau khi chọn được Collector, Gateway phải làm nhiệm vụ 'cầu nối' chuyển tiếp dữ liệu 2 chiều (Duplex Relay): đọc byte từ Agent thì đẩy sang Collector, đọc phản hồi từ Collector thì trả về Agent, đồng thời định kỳ thăm dò sức khỏe (Health Polling) của Collector.

2. Kiến thức nền tảng cần nắm vững: Chuyển tiếp dòng dữ liệu TCP (Stream.CopyToAsync), bắt tay relay 2 chiều, định thời thăm dò (Health Probing).

3. Dữ liệu đầu vào & Đầu ra kỹ thuật: Đầu vào: Dòng dữ liệu 2 chiều giữa client ngoài và Collector bên trong.
Đầu ra: Chuyển tiếp dữ liệu thông suốt; phát hiện ngay khi Collector sập nguồn để đóng kết nối và giảm tải bộ đếm.

4. Các bước triển khai chi tiết:
1. Sử dụng 2 Task chạy song song: clientStream.CopyToAsync(collectorStream) và collectorStream.CopyToAsync(clientStream).
2. Sử dụng Task.WhenAny để khi 1 trong 2 đầu ngắt kết nối thì đóng đầu còn lại an toàn.
3. Chạy luồng nền định kỳ 5 giây gửi bản tin nội bộ HEALTH_PROBE tới từng Collector; nếu không trả lời quá 2 chu kỳ thì đánh dấu node bị Unhealthy.
4. Cập nhật số kết nối active khi phiên kết thúc.

> **📌 BẪY LỖI CẦN TRÁNH TRONG LG-A-014**
> Không đóng kết nối phía client khi phía Collector bị crash, khiến socket client bị treo ở trạng thái nửa mở (Half-Open) gây rò rỉ tài nguyên.

5. Tiêu chuẩn hoàn thành 11 mục (Definition of Done — DoD):
• [ ] 1. Hợp đồng giao thức & Wire Envelope chuẩn UTF-8 JSON
• [ ] 2. Lập trình bất đồng bộ Async/Await không chặn luồng mạng
• [ ] 3. Bắt ngoại lệ mạng an toàn (EOF, SocketException, Timeout)
• [ ] 4. Giải phóng tài nguyên sạch sẽ (Dispose / Using / CancellationToken)
• [ ] 5. An toàn dữ liệu (Khử trùng lặp lô / Khóa lạc quan OCC / Transaction)
• [ ] 6. Unit Test / Test Harness chạy đạt 100% không có cảnh báo
• [ ] 7. Tích hợp chéo thành công với module của đồng đội liên quan
• [ ] 8. Kiểm thử thực tế trên mạng LAN nhiều máy tính
• [ ] 9. Đồng bộ tài liệu thiết kế & mã nguồn khớp hoàn toàn
• [ ] 10. Thu thập đầy đủ hồ sơ minh chứng (Log, Screenshot, Pcap)
• [ ] 11. Sẵn sàng trả lời trôi chảy câu hỏi vấn đáp (Viva) liên quan

6. Kịch bản kiểm thử & Hồ sơ minh chứng:
• Mã bài test: TEST-A14
• Phương pháp kiểm thử: Tắt đột ngột 1 Collector; đo thời gian Gateway phát hiện và dừng chuyển tiếp
• Tiêu chuẩn nghiệm thu đạt: Phát hiện Collector chết sau 3 lần probe lỗi liên tiếp; đóng socket kết nối phía Agent để kích hoạt reconnect
• Hồ sơ minh chứng lưu tại: Duplex relay throughput log, health probe timeline

7. Tình huống lỗi tiềm ẩn & Kế hoạch ứng phó: Half-open socket gây rò rỉ file descriptor; đóng đồng thời cả 2 đầu socket khi 1 bên ngắt kết nối

8. Câu hỏi vấn đáp (Viva) & Chiến lược trả lời:
• Chủ đề câu hỏi: Chuyển tiếp byte hai chiều bất đồng bộ và xử lý half-closed socket
• Câu trả lời kỳ vọng: Dùng 2 Task CopyToAsync đồng thời; khi 1 Task hoàn thành (do ngắt kết nối), lập tức hủy Task còn lại và đóng socket
• Câu hỏi mở rộng / Bẫy của giảng viên: Tại sao probe health check cần ngưỡng 3 fail và 2 success thay vì 1 lần?

[HƯỚNG DẪN ĐỀ XUẤT]: *Tham khảo mã nguồn mẫu chuẩn và tài liệu kỹ thuật tại thư mục Networking. Khi hiện thực, luôn đặt an toàn luồng và xử lý hủy tác vụ (CancellationToken) lên hàng đầu.*

Nhiệm vụ 15: [LG-A-015] — LB integration và collector kill failover proof

Mã & Tên nhiệm vụ: [LG-A-015] LB integration và collector kill failover proof	**Tuần & Ưu tiên:** Tuần W5 (P1) | P1
Khối lượng công việc: Cơ bản: 3.0h | Dự phòng: 0.5h | Tổng: 3.5h	**Phân loại tải:** P1 Extension
Trách nhiệm thực hiện: Phụ trách: SV A | Dự phòng: SV C | Duyệt: SV C	**Phân hệ & Thư mục:** P1.Testing (Testing)
Sản phẩm bàn giao: Báo cáo failover & Raw metrics capture	**Hợp đồng giao diện:** Gateway + 2 Collector instances + Central Server

1. Mục đích kỹ thuật & Tại sao tồn tại: Mục đích lớn nhất của cụm mở rộng P1 là khả năng chịu lỗi (Fault Tolerance). Cần chứng minh thực nghiệm: khi một node Collector đang phục vụ client bị tắt đột ngột (Kill Collector), hệ thống tự động phát hiện và các Agent tự động chuyển sang Collector còn lại mà không mất dữ liệu.

2. Kiến thức nền tảng cần nắm vững: Cơ chế chịu lỗi (Failover), quy trình kiểm thử khả năng sống sót của cụm máy chủ.

3. Dữ liệu đầu vào & Đầu ra kỹ thuật: Đầu vào: Hệ thống đang chạy với 2 instance Collector và nhiều Agent kết nối qua Gateway.
Đầu ra: Bằng chứng video/log: kill 1 Collector, Gateway chuyển sang node sống, Agent kết nối lại thành công.

4. Các bước triển khai chi tiết:
1. Khởi động Gateway và 2 instance Collector (Node 1 port 5001, Node 2 port 5002) cùng kết nối về Central Server.
2. Bật 10 simulated agent qua Gateway; xác nhận các kết nối được chia đều cho cả 2 node.
3. Dùng lệnh taskkill /F cưỡng bức dừng tiến trình Collector Node 1.

4. Ghi nhận nhật ký: Gateway phát hiện probe fail, đánh dấu Node 1 chết; các Agent bị ngắt kích hoạt Reconnect Backoff và được Gateway chuyển tiếp sang Node 2; hệ thống tiếp tục hoạt động bình thường.

> 📌 **BẪY LỖI CẦN TRÁNH TRONG LG-A-015**
> Không chờ Agent kết nối lại đủ số lượng mà vội kết luận failover thất bại; thời gian failover p95 thiết kế là <= 45s.

5. Tiêu chuẩn hoàn thành 11 mục (Definition of Done — DoD):
• [ ] 1. Hợp đồng giao thức & Wire Envelope chuẩn UTF-8 JSON
• [ ] 2. Lập trình bất đồng bộ Async/Await không chặn luồng mạng
• [ ] 3. Bắt ngoại lệ mạng an toàn (EOF, SocketException, Timeout)
• [ ] 4. Giải phóng tài nguyên sạch sẽ (Dispose / Using / CancellationToken)
• [ ] 5. An toàn dữ liệu (Khử trùng lặp lô / Khóa lạc quan OCC / Transaction)
• [ ] 6. Unit Test / Test Harness chạy đạt 100% không có cảnh báo
• [ ] 7. Tích hợp chéo thành công với module của đồng đội liên quan
• [ ] 8. Kiểm thử thực tế trên mạng LAN nhiều máy tính
• [ ] 9. Đồng bộ tài liệu thiết kế & mã nguồn khớp hoàn toàn
• [ ] 10. Thu thập đầy đủ hồ sơ minh chứng (Log, Screenshot, Pcap)
• [ ] 11. Sẵn sàng trả lời trôi chảy câu hỏi vấn đáp (Viva) liên quan

6. Kịch bản kiểm thử & Hồ sơ minh chứng:
• Mã bài test: TEST-A15
• Phương pháp kiểm thử: Kill tiến trình Collector 01 khi có 20 agent đang kết nối; đo thời gian tái kết nối p95
• Tiêu chuẩn nghiệm thu đạt: Thời gian tái kết nối p95 <= 45s; Agent chuyển sang Collector 02; không thất thoát dữ liệu do có replay queue
• Hồ sơ minh chứng lưu tại: Failover latency histogram, video quay lại quá trình kill tiến trình

7. Tình huống lỗi tiềm ẩn & Kế hoạch ứng phó: Mất dữ liệu in-flight; Agent replay lại mẻ tin chưa có ACK, Central khử trùng lặp qua ProcessedBatches

8. Câu hỏi vấn đáp (Viva) & Chiến lược trả lời:
• Chủ đề câu hỏi: Hiện tượng Failover trong hệ thống phân tán và bảo toàn dữ liệu
• Câu trả lời kỳ vọng: Hệ thống chấp nhận gián đoạn tạm thời nhưng bảo toàn dữ liệu nhờ kết hợp hàng đợi replay phía client và dedup phía server
• Câu hỏi mở rộng / Bẫy của giảng viên: Tại sao không cần zero-downtime cho đồ án quy mô phòng máy?

[HƯỚNG DẪN ĐỀ XUẤT]: *Tham khảo mã nguồn mẫu chuẩn và tài liệu kỹ thuật tại thư mục Testing. Khi hiện thực, luôn đặt an toàn luồng và xử lý hủy tác vụ (CancellationToken) lên hàng đầu.*

Nhiệm vụ 16: [LG-A-016] — Internet deployment chuẩn và fallback

Mã & Tên nhiệm vụ: [LG-A-016] Internet deployment chuẩn và fallback	**Tuần & Ưu tiên:** Tuần W5 (P0) | P0/Release
Khối lượng công việc: Cơ bản: 1.5h | Dự phòng: 0.5h | Tổng: 2.0h	**Phân loại tải:** P0 Core
Trách nhiệm thực hiện: Phụ trách: SV A | Dự phòng: SV B | Duyệt: SV B	**Phân hệ & Thư mục:** Release.Deploy (System)
Sản phẩm bàn giao: Topology.md & Runbook triển khai Internet	**Hợp đồng giao diện:** TLS connection qua Public Domain / DNS / DDNS / Tunnel

1. Mục đích kỹ thuật & Tại sao tồn tại: Khi triển khai demo qua mạng Internet thật, máy tính sinh viên thường nằm sau nhiều lớp NAT của nhà mạng (CGNAT) khiến client bên ngoài không thể kết nối trực tiếp vào địa chỉ IP nội bộ của Central Server. Cần phương án triển khai chuẩn kèm giải pháp dự phòng (Tunneling / Hotspot).

2. Kiến thức nền tảng cần nắm vững: Khái niệm NAT, CGNAT, Port Forwarding, công cụ tạo đường hầm TCP an toàn (như Cloudflare Tunnel, ngrok, Tailscale).

3. Dữ liệu đầu vào & Đầu ra kỹ thuật: Đầu vào: Central Server chạy trên máy của B, Agent chạy từ xa qua mạng 4G/Internet.
Đầu ra: Kịch bản kết nối Internet hoạt động ổn định; tài liệu phương án dự phòng chuyển về mạng LAN nếu Internet phòng thi bị chặn.

4. Các bước triển khai chi tiết:
1. Thiết lập đường hầm TCP cho cổng 5000 bằng công cụ tunneling hoặc cấu hình mở cổng router nếu có IP tĩnh.
2. Cấu hình Agent kết nối qua domain/port công khai của đường hầm.
3. Kiểm tra bắt tay TLS 1.3 qua Internet.
4. Chuẩn bị sẵn kịch bản Fallback: nếu Wi-Fi phòng lab trường UIT chặn cổng hoặc mất mạng Internet, lập tức bật mạng phát Wi-Fi từ điện thoại di động (Hotspot) để chuyển về LAN cục bộ trong 2 phút.

> 📌 **BẪY LỖI CẦN TRÁNH TRONG LG-A-016**
> Phụ thuộc 100% vào mạng Internet phòng thi mà không chuẩn bị phương án dự phòng LAN cục bộ, dẫn đến việc không demo được khi mạng trường gặp sự cố.

5. Tiêu chuẩn hoàn thành 11 mục (Definition of Done — DoD):
- [ ] 1. Hợp đồng giao thức & Wire Envelope chuẩn UTF-8 JSON
- [ ] 2. Lập trình bất đồng bộ Async/Await không chặn luồng mạng
- [ ] 3. Bắt ngoại lệ mạng an toàn (EOF, SocketException, Timeout)
- [ ] 4. Giải phóng tài nguyên sạch sẽ (Dispose / Using / CancellationToken)
- [ ] 5. An toàn dữ liệu (Khử trùng lặp lô / Khóa lạc quan OCC / Transaction)
- [ ] 6. Unit Test / Test Harness chạy đạt 100% không có cảnh báo

• [ ] 7. Tích hợp chéo thành công với module của đồng đội liên quan
• [ ] 8. Kiểm thử thực tế trên mạng LAN nhiều máy tính
• [ ] 9. Đồng bộ tài liệu thiết kế & mã nguồn khớp hoàn toàn
• [ ] 10. Thu thập đầy đủ hồ sơ minh chứng (Log, Screenshot, Pcap)
• [ ] 11. Sẵn sàng trả lời trôi chảy câu hỏi vấn đáp (Viva) liên quan

6. Kịch bản kiểm thử & Hồ sơ minh chứng:
• Mã bài test: TEST-A16
• Phương pháp kiểm thử: Kết nối Agent từ mạng 4G điện thoại vào Central Server đặt tại mạng nhà/trường
• Tiêu chuẩn nghiệm thu đạt: Bắt tay TLS thành công qua Internet; heartbeat duy trì ổn định không rớt gói bất thường
• Hồ sơ minh chứng lưu tại: Tracert log, sơ đồ topology mạng, ảnh chụp kết nối 4G

7. Tình huống lỗi tiềm ẩn & Kế hoạch ứng phó: CGNAT nhà mạng chặn Port Forwarding; fallback sang giải pháp Tunnel bảo mật

8. Câu hỏi vấn đáp (Viva) & Chiến lược trả lời:
• Chủ đề câu hỏi: Kỹ thuật định tuyến kết nối từ Internet qua NAT và Firewall
• Câu trả lời kỳ vọng: Trình bày sự khác biệt giữa Port Forwarding router thông thường và Tunnel bảo mật khi gặp hạ tầng CGNAT
• Câu hỏi mở rộng / Bẫy của giảng viên: Nguy cơ bảo mật khi mở port máy chủ ra môi trường Internet công cộng?

[HƯỚNG DẪN ĐỀ XUẤT]: *Tham khảo mã nguồn mẫu chuẩn và tài liệu kỹ thuật tại thư mục System. Khi hiện thực, luôn đặt an toàn luồng và xử lý hủy tác vụ (CancellationToken) lên hàng đầu.*

Nhiệm vụ 17: [LG-A-017] — Load test và reconnect storm

Mã & Tên nhiệm vụ: [LG-A-017] Load test và reconnect storm	**Tuần & Ưu tiên:** Tuần W5 (P0) | P0/Release
Khối lượng công việc: Cơ bản: 3.0h | Dự phòng: 0.5h | Tổng: 3.5h	**Phân loại tải:** Test/Harness
Trách nhiệm thực hiện: Phụ trách: SV A | Dự phòng: SV B | Duyệt: SV B	**Phân hệ & Thư mục:** Release.Testing (Testing)
Sản phẩm bàn giao: LoadStormReport.md & System counters	**Hợp đồng giao diện:** 50 concurrent TCP sockets, heartbeat 5s, telemetry 10s

1. Mục đích kỹ thuật & Tại sao tồn tại: Trước khi đóng gói xuất xưởng, cần thực hiện bài kiểm tra tải tối đa (Stress Test) và kịch bản bão kết nối (Reconnect Storm): mô phỏng tình huống toàn bộ 50 máy trạm cùng khởi động lại sau giờ nghỉ trưa và đồng loạt kết nối tới máy chủ.

2. Kiến thức nền tảng cần nắm vững: Kiểm thử tải (Load Testing), bão kết nối (Connection Storm), phân tích biểu đồ CPU và RAM.

3. Dữ liệu đầu vào & Đầu ra kỹ thuật: Đầu vào: Ứng dụng Simulator kích hoạt 50 socket client đồng thời.
Đầu ra: Báo cáo đo kiểm tải: tỷ lệ thành công kết nối, thời gian ổn định, Central Server không bị crash.

4. Các bước triển khai chi tiết:
1. Cấu hình Simulator tạo 50 kết nối cùng lúc trong vòng 5 giây.
2. Quan sát biểu đồ tài nguyên của Central Server: CPU không bị treo 100%, bộ nhớ RAM không tăng đột biến không giới hạn.
3. Kiểm tra hàng đợi Ingress Channel của Central Server có kích hoạt Backpressure an toàn hay không.
4. Ghi nhận thời gian để toàn bộ 50 máy ổn định trạng thái Online trên Dashboard.

> **📌 BẪY LỖI CẦN TRÁNH TRONG LG-A-017**
> Bật 50 tiến trình Simulator riêng biệt thay vì dùng 1 tiến trình quản lý 50 socket Task, gây cạn kiệt tài nguyên của chính máy chạy kiểm thử.

5. Tiêu chuẩn hoàn thành 11 mục (Definition of Done — DoD):
• [ ] 1. Hợp đồng giao thức & Wire Envelope chuẩn UTF-8 JSON
• [ ] 2. Lập trình bất đồng bộ Async/Await không chặn luồng mạng
• [ ] 3. Bắt ngoại lệ mạng an toàn (EOF, SocketException, Timeout)
• [ ] 4. Giải phóng tài nguyên sạch sẽ (Dispose / Using / CancellationToken)
• [ ] 5. An toàn dữ liệu (Khử trùng lặp lô / Khóa lạc quan OCC / Transaction)
• [ ] 6. Unit Test / Test Harness chạy đạt 100% không có cảnh báo
• [ ] 7. Tích hợp chéo thành công với module của đồng đội liên quan
• [ ] 8. Kiểm thử thực tế trên mạng LAN nhiều máy tính
• [ ] 9. Đồng bộ tài liệu thiết kế & mã nguồn khớp hoàn toàn
• [ ] 10. Thu thập đầy đủ hồ sơ minh chứng (Log, Screenshot, Pcap)
• [ ] 11. Sẵn sàng trả lời trôi chảy câu hỏi vấn đáp (Viva) liên quan

6. Kịch bản kiểm thử & Hồ sơ minh chứng:
• Mã bài test: TEST-A17
• Phương pháp kiểm thử: Khởi động lại Central trong khi 50 agent đang chạy; quan sát biểu đồ CPU và RAM
• Tiêu chuẩn nghiệm thu đạt: Toàn bộ 50 agent kết nối lại thành công trong vòng 60s; server không bị tràn bộ nhớ hay treo CPU
• Hồ sơ minh chứng lưu tại: Biểu đồ tải CPU/RAM server, reconnect storm log

7. Tình huống lỗi tiềm ẩn & Kế hoạch ứng phó: Bão kết nối làm cạn ThreadPool; ThreadPool tự điều hòa nhờ cấu trúc Bounded Queue và async I/O

8. Câu hỏi vấn đáp (Viva) & Chiến lược trả lời:
• Chủ đề câu hỏi: Hiện tượng Reconnect Storm và cách phòng ngừa

• Câu trả lời kỳ vọng: Khi hàng loạt client mất kết nối và cùng thử lại đồng thời sẽ gây quá tải máy chủ; giải pháp là Exponential Backoff kèm Jitter ngẫu nhiên
• Câu hỏi mở rộng / Bẫy của giảng viên: Jitter ngẫu nhiên giúp phân bổ đồ thị tải như thế nào?

[HƯỚNG DẪN ĐỀ XUẤT]: *Tham khảo mã nguồn mẫu chuẩn và tài liệu kỹ thuật tại thư mục Testing. Khi hiện thực, luôn đặt an toàn luồng và xử lý hủy tác vụ (CancellationToken) lên hàng đầu.*

Nhiệm vụ 18: [LG-A-018] — Thu proof LAN Internet cho release candidate

Mã & Tên nhiệm vụ: [LG-A-018] Thu proof LAN Internet cho release candidate	**Tuần & Ưu tiên:** Tuần W5 (P0) | P0/Release
Khối lượng công việc: Cơ bản: 2.5h | Dự phòng: 0.5h | Tổng: 3.0h	**Phân loại tải:** Tích hợp
Trách nhiệm thực hiện: Phụ trách: SV A | Dự phòng: SV C | Duyệt: SV C	**Phân hệ & Thư mục:** Release.Evidence (Integration)
Sản phẩm bàn giao: Bộ minh chứng thực nghiệm & Capture file (.pcapng)	**Hợp đồng giao diện:** Toàn bộ hệ thống LABGUARD chạy thực tế

1. Mục đích kỹ thuật & Tại sao tồn tại: Hội đồng chấm thi và giảng viên hướng dẫn chỉ tin tưởng vào các bằng chứng xác thực (Evidence) đã được kiểm chứng độc lập. Cần thu thập đầy đủ bộ ảnh chụp, log tệp tin, packet capture Wireshark cho phiên bản Release Candidate.

2. Kiến thức nền tảng cần nắm vững: Phần mềm bắt gói tin Wireshark, chụp màn hình hệ thống kèm đồng hồ hệ thống, xuất log định dạng CSV/TXT.

3. Dữ liệu đầu vào & Đầu ra kỹ thuật: Đầu vào: Hệ thống hoàn chỉnh đang chạy kiểm thử trên môi trường mạng thật.
Đầu ra: Bộ thư mục bằng chứng kiểm thử hoàn chỉnh cho Lane A sẵn sàng bàn giao cho người duyệt.

4. Các bước triển khai chi tiết:
1. Bật Wireshark bắt gói tin trên card mạng: chứng minh gói tin được bọc TLS mã hóa, không thấy plaintext payload.
2. Chụp ảnh màn hình 2 máy thật trong mạng LAN đang gửi heartbeat và metric đều đặn.
3. Lưu tệp log của Agent thể hiện các pha kết nối, ngắt mạng, backoff và reconnect thành công.
4. Đóng gói bằng chứng và đối soát với cột Evidence trong WBS.

> 📌 **BẪY LỖI CẦN TRÁNH TRONG LG-A-018**
> Chụp ảnh màn hình bị mờ hoặc cắt xén mất thanh tiêu đề, không hiển thị được thời gian thực của máy tính làm bằng chứng mất tính thuyết phục.

5. Tiêu chuẩn hoàn thành 11 mục (Definition of Done — DoD):
• [ ] 1. Hợp đồng giao thức & Wire Envelope chuẩn UTF-8 JSON

• [ ] 2. Lập trình bất đồng bộ Async/Await không chặn luồng mạng
• [ ] 3. Bắt ngoại lệ mạng an toàn (EOF, SocketException, Timeout)
• [ ] 4. Giải phóng tài nguyên sạch sẽ (Dispose / Using / CancellationToken)
• [ ] 5. An toàn dữ liệu (Khử trùng lặp lô / Khóa lạc quan OCC / Transaction)
• [ ] 6. Unit Test / Test Harness chạy đạt 100% không có cảnh báo
• [ ] 7. Tích hợp chéo thành công với module của đồng đội liên quan
• [ ] 8. Kiểm thử thực tế trên mạng LAN nhiều máy tính
• [ ] 9. Đồng bộ tài liệu thiết kế & mã nguồn khớp hoàn toàn
• [ ] 10. Thu thập đầy đủ hồ sơ minh chứng (Log, Screenshot, Pcap)
• [ ] 11. Sẵn sàng trả lời trôi chảy câu hỏi vấn đáp (Viva) liên quan

6. Kịch bản kiểm thử & Hồ sơ minh chứng:
• Mã bài test: TEST-A18
• Phương pháp kiểm thử: Đối soát chéo dữ liệu packet capture với log hệ thống của Central Server
• Tiêu chuẩn nghiệm thu đạt: Bằng chứng rõ ràng, thể hiện đúng mốc thời gian, IP thật, gói tin TLS mã hóa và kịch bản chuyển đổi trạng thái
• Hồ sơ minh chứng lưu tại: File .pcapng, ảnh chụp màn hình Wireshark follow stream

7. Tình huống lỗi tiềm ẩn & Kế hoạch ứng phó: File pcap thiếu thông tin; bắt gói tin trực tiếp trên card mạng vật lý có lọc IP rõ ràng

8. Câu hỏi vấn đáp (Viva) & Chiến lược trả lời:
• Chủ đề câu hỏi: Phương pháp thu thập và phân tích gói tin mạng bằng Wireshark
• Câu trả lời kỳ vọng: Đặt bộ lọc capture filter và display filter phù hợp để ghi lại quá trình bắt tay 3 bước TCP, TLS handshake và frame dữ liệu
• Câu hỏi mở rộng / Bẫy của giảng viên: Làm thế nào chứng minh dữ liệu telemetry đã thực sự được mã hóa trên đường truyền?

[HƯỚNG DẪN ĐỀ XUẤT]: *Tham khảo mã nguồn mẫu chuẩn và tài liệu kỹ thuật tại thư mục Integration. Khi hiện thực, luôn đặt an toàn luồng và xử lý hủy tác vụ (CancellationToken) lên hàng đầu.*

Nhiệm vụ 19: [LG-A-019] — Release scripts và package checksum

Mã & Tên nhiệm vụ: [LG-A-019] Release scripts và package checksum	**Tuần & Ưu tiên:** Tuần W5 (P0) | P0/Release
Khối lượng công việc: Cơ bản: 1.5h | Dự phòng: 0.0h | Tổng: 1.5h	**Phân loại tải:** P0 Core
Trách nhiệm thực hiện: Phụ trách: SV A | Dự phòng: SV B | Duyệt: SV B	**Phân hệ & Thư mục:** Release.Packaging (System)
Sản phẩm bàn giao: PackageRelease.ps1 & SHA256SUMS.txt	**Hợp đồng giao diện:** .NET SDK CLI publish -c Release, SHA-256 hash algorithm

1. Mục đích kỹ thuật & Tại sao tồn tại: Để đảm bảo gói phần mềm nộp cho hội đồng là nguyên bản, không bị lỗi thiếu file runtime hoặc bị chỉnh sửa tùy tiện, cần kịch bản đóng gói tự động xuất bản (Publish Release) và tạo mã băm toàn vẹn SHA-256 cho từng tệp tin.

2. Kiến thức nền tảng cần nắm vững: Lệnh dotnet publish -c Release, mã băm mật mã học SHA-256, kịch bản PowerShell tự động hóa.

3. Dữ liệu đầu vào & Đầu ra kỹ thuật: Đầu vào: Mã nguồn hoàn chỉnh của các dự án trong solution. Đầu ra: Gói cài đặt ZIP chạy được ngay và tệp bảng mã băm MANIFEST_SHA256.txt.

4. Các bước triển khai chi tiết:
1. Viết script PowerShell biên dịch toàn bộ solution với cấu hình Release tối ưu: dotnet publish -c Release -r win-x64 --self-contained false.
2. Thu thập các tệp thực thi (.exe), cấu hình (.json), tài liệu (.pdf) vào một thư mục staging sạch.
3. Tạo tệp MANIFEST_SHA256.txt chứa mã băm SHA-256 và kích thước của từng file.
4. Nén thành tệp ZIP duy nhất; kiểm tra giải nén trên một máy tính sạch khác để đảm bảo chạy được ngay.

> 📌 **BẪY LỖI CẦN TRÁNH TRONG LG-A-019**
> Đóng gói cả thư mục bin/Debug chứa các file tạm khổng lồ hoặc quên kèm theo các file thư viện DLL phụ thuộc.

5. Tiêu chuẩn hoàn thành 11 mục (Definition of Done — DoD):
• [ ] 1. Hợp đồng giao thức & Wire Envelope chuẩn UTF-8 JSON
• [ ] 2. Lập trình bất đồng bộ Async/Await không chặn luồng mạng
• [ ] 3. Bắt ngoại lệ mạng an toàn (EOF, SocketException, Timeout)
• [ ] 4. Giải phóng tài nguyên sạch sẽ (Dispose / Using / CancellationToken)
• [ ] 5. An toàn dữ liệu (Khử trùng lặp lô / Khóa lạc quan OCC / Transaction)
• [ ] 6. Unit Test / Test Harness chạy đạt 100% không có cảnh báo
• [ ] 7. Tích hợp chéo thành công với module của đồng đội liên quan
• [ ] 8. Kiểm thử thực tế trên mạng LAN nhiều máy tính
• [ ] 9. Đồng bộ tài liệu thiết kế & mã nguồn khớp hoàn toàn
• [ ] 10. Thu thập đầy đủ hồ sơ minh chứng (Log, Screenshot, Pcap)
• [ ] 11. Sẵn sàng trả lời trôi chảy câu hỏi vấn đáp (Viva) liên quan

6. Kịch bản kiểm thử & Hồ sơ minh chứng:
• Mã bài test: TEST-A19
• Phương pháp kiểm thử: Chạy script trên máy tính môi trường sạch; giải nén và đối chiếu checksum
• Tiêu chuẩn nghiệm thu đạt: Tạo gói ZIP hoàn chỉnh chạy được ngay; 100% file có mã băm SHA-256 khớp tuyệt đối
• Hồ sơ minh chứng lưu tại: PackageRelease.ps1, MANIFEST_SHA256.txt

7. Tình huống lỗi tiềm ẩn & Kế hoạch ứng phó: Thiếu runtime dependency; xuất bản ở chế độ self-contained hoặc quy định rõ .NET Runtime yêu cầu

8. Câu hỏi vấn đáp (Viva) & Chiến lược trả lời:
• Chủ đề câu hỏi: Quy trình đóng gói phần mềm và kiểm tra tính toàn vẹn (Integrity Check)
• Câu trả lời kỳ vọng: Dùng mã băm mật mã học SHA-256 để đảm bảo gói cài đặt không bị hỏng hóc hoặc can thiệp trên đường truyền
• Câu hỏi mở rộng / Bẫy của giảng viên: Sự khác biệt giữa mã băm toàn vẹn (Checksum) và chữ ký số (Digital Signature)?

[HƯỚNG DẪN ĐỀ XUẤT]: *Tham khảo mã nguồn mẫu chuẩn và tài liệu kỹ thuật tại thư mục System. Khi hiện thực, luôn đặt an toàn luồng và xử lý hủy tác vụ (CancellationToken) lên hàng đầu.*

Nhiệm vụ 20: [LG-A-020] — Viva A và bàn giao backup

Mã & Tên nhiệm vụ: [LG-A-020] Viva A và bàn giao backup	**Tuần & Ưu tiên:** Tuần W5 (P0) | P0/Viva
Khối lượng công việc: Cơ bản: 5.0h | Dự phòng: 0.0h | Tổng: 5.0h	**Phân loại tải:** Viva
Trách nhiệm thực hiện: Phụ trách: SV A | Dự phòng: SV B | Duyệt: SV B	**Phân hệ & Thư mục:** Viva (Viva)
Sản phẩm bàn giao: HANDBOOK_A.md & Viva rehearsal log	**Hợp đồng giao diện:** 30 câu hỏi viva chuyên sâu phần Agent, Networking & Gateway

1. Mục đích kỹ thuật & Tại sao tồn tại: Bước cuối cùng quyết định điểm số cá nhân trong đồ án là buổi vấn đáp (Viva). Sinh viên A phải tự tin giải thích lưu loát các cơ chế mạng, vẽ được sơ đồ luồng dữ liệu và chỉ ra đúng dòng code mình đã viết trong repo mà không cần nhìn tài liệu.

2. Kiến thức nền tảng cần nắm vững: Toàn bộ 30 câu hỏi vấn đáp chuyên sâu phần Agent, Socket TCP, TLS, Framing và Simulator.

3. Dữ liệu đầu vào & Đầu ra kỹ thuật: Đầu vào: Bộ câu hỏi viva trong HANDBOOK_A.md và bảng theo dõi trong Excel 09_Viva_Tracker.
Đầu ra: Biên bản diễn tập vấn đáp nội bộ với 2 thành viên B và C chấm điểm chéo đạt yêu cầu.

4. Các bước triển khai chi tiết:
1. Tự luyện tập trả lời 30 câu hỏi vấn đáp trong handbook; tự chấm điểm và ghi nhận điểm yếu.
2. Tổ chức buổi bảo vệ thử (Mock Viva): thành viên B và C đóng vai giảng viên phản biện, đặt câu hỏi bất kỳ và yêu cầu chỉ code.
3. Rèn luyện phản xạ: nêu định nghĩa -> giải thích cơ chế -> giải thích tại sao LabGuard lại làm vậy -> mở đúng file code minh họa.
4. Đóng gói tài liệu dự phòng (USB, in sẵn đề cương) sẵn sàng cho ngày bảo vệ chính thức.

> **📌 BẪY LỖI CẦN TRÁNH TRONG LG-A-020**
> Học vẹt định nghĩa mà không hiểu bản chất luồng dữ liệu; khi bị giảng viên yêu cầu mở file code minh họa

> thì lúng túng không biết code nằm ở đâu.

5. Tiêu chuẩn hoàn thành 11 mục (Definition of Done — DoD):

• [ ] 1. Hợp đồng giao thức & Wire Envelope chuẩn UTF-8 JSON
• [ ] 2. Lập trình bất đồng bộ Async/Await không chặn luồng mạng
• [ ] 3. Bắt ngoại lệ mạng an toàn (EOF, SocketException, Timeout)
• [ ] 4. Giải phóng tài nguyên sạch sẽ (Dispose / Using / CancellationToken)
• [ ] 5. An toàn dữ liệu (Khử trùng lặp lô / Khóa lạc quan OCC / Transaction)
• [ ] 6. Unit Test / Test Harness chạy đạt 100% không có cảnh báo
• [ ] 7. Tích hợp chéo thành công với module của đồng đội liên quan
• [ ] 8. Kiểm thử thực tế trên mạng LAN nhiều máy tính
• [ ] 9. Đồng bộ tài liệu thiết kế & mã nguồn khớp hoàn toàn
• [ ] 10. Thu thập đầy đủ hồ sơ minh chứng (Log, Screenshot, Pcap)
• [ ] 11. Sẵn sàng trả lời trôi chảy câu hỏi vấn đáp (Viva) liên quan

6. Kịch bản kiểm thử & Hồ sơ minh chứng:

• Mã bài test: TEST-A20
• Phương pháp kiểm thử: Vấn đáp chéo giả định với thành viên B và C chấm điểm độc lập
• Tiêu chuẩn nghiệm thu đạt: Giải thích lưu loát các cơ chế mạng, vẽ được sơ đồ luồng dữ liệu và chỉ ra đúng dòng code trong repo
• Hồ sơ minh chứng lưu tại: HANDBOOK_A.md, biên bản diễn tập vấn đáp nội bộ

7. Tình huống lỗi tiềm ẩn & Kế hoạch ứng phó: Lúng túng khi bị hỏi sâu về socket; ôn tập kỹ bảng thuật ngữ và các kịch bản kiểm thử trong handbook

8. Câu hỏi vấn đáp (Viva) & Chiến lược trả lời:

• Chủ đề câu hỏi: Tổng quan kiến trúc phân tầng mạng và ranh giới trách nhiệm của Lane A
• Câu trả lời kỳ vọng: Trình bày mạch lạc vai trò của Agent thu thập dữ liệu, cơ chế framing, đảm bảo an toàn kết nối và phân tải P1
• Câu hỏi mở rộng / Bẫy của giảng viên: Điểm tâm đắc nhất và bài học lớn nhất rút ra từ đồ án môn học?

[HƯỚNG DẪN ĐỀ XUẤT]: *Tham khảo mã nguồn mẫu chuẩn và tài liệu kỹ thuật tại thư mục Viva. Khi hiện thực, luôn đặt an toàn luồng và xử lý hủy tác vụ (CancellationToken) lên hàng đầu.*

PHẦN 6: SỔ TAY CHI TIẾT 20 NHIỆM VỤ CỦA SINH VIÊN B (SERVER & CƠ SỞ DỮ LIỆU)

Sinh viên B chịu trách nhiệm về Central Server Daemon, kênh đệm có chặn (Bounded Ingress), khử trùng lặp dữ liệu, cơ chế nhịp thở liveness, bộ đánh giá quy tắc Hysteresis, 8 bảng CSDL, khóa lạc quan OCC và bảo mật PBKDF2. Dưới đây là chi tiết 20 nhiệm vụ từ LG-B-001 đến LG-B-020:

Nhiệm vụ 01: [LG-B-001] — Schema migrations và seed an toàn

Mã & Tên nhiệm vụ: [LG-B-001] Schema migrations và seed an toàn	**Tuần & Ưu tiên:** Tuần W1 (P0) | P0
Khối lượng công việc: Cơ bản: 2.5h | Dự phòng: 0.5h | Tổng: 3.0h	**Phân loại tải:** P0 Core
Trách nhiệm thực hiện: Phụ trách: SV B | Dự phòng: SV C | Duyệt: SV C	**Phân hệ & Thư mục:** Central.Database (Storage)
Sản phẩm bàn giao: V1__Initial_Schema.sql & Seed.sql	**Hợp đồng giao diện:** SQL Server 2022 schema, 8 tables, rowversion column for OCC

1. Mục đích kỹ thuật & Tại sao tồn tại: Hệ thống giám sát phòng máy cần cơ sở dữ liệu quan hệ mạnh mẽ để lưu trữ trạng thái máy trạm, số liệu đo lường và vòng đời sự cố. Nếu schema thiết kế sai hoặc thiếu ràng buộc toàn vẹn ngay từ đầu, dữ liệu sẽ bị phân mảnh, trùng lặp và không thể hỗ trợ khóa lạc quan (OCC).

2. Kiến thức nền tảng cần nắm vững: SQL DDL/DML, khóa chính/ngoại, chỉ mục có điều kiện (Filtered Unique Index), kiểu dữ liệu nhị phân rowversion trong SQL Server 2022.

3. Dữ liệu đầu vào & Đầu ra kỹ thuật: Đầu vào: Đặc tả kiến trúc cơ sở dữ liệu trong 03_PROTOCOL_SPEC.md và CONTRACT_MATRIX.md.
Đầu ra: Tệp migration V1__Initial_Schema.sql tạo 8 bảng chuẩn và Seed.sql chèn tài khoản Admin khởi tạo.

4. Các bước triển khai chi tiết:
1. Tạo 8 bảng: Devices, DeviceSessions, MetricSnapshots, Processes, Incidents, IncidentEvents, ProcessedBatches, ProcessedCommands.
2. Bổ sung cột RowVersion rowversion NOT NULL vào bảng Incidents để phục vụ Optimistic Concurrency Control.
3. Thêm Filtered Unique Index trên Incidents: CREATE UNIQUE NONCLUSTERED INDEX IX_Unique_Active_Incident ON Incidents(DeviceId, RuleCode) WHERE WorkflowState != 'RESOLVED'.
4. Soạn thảo script Seed.sql chèn tài khoản khởi tạo (bootstrap admin) với mật khẩu băm PBKDF2 chuẩn.
5. Chạy thử trên SQL Server 2022 LocalDB/Express và kiểm tra tính toàn vẹn khóa ngoại.

> **📌 BẪY LỖI CẦN TRÁNH TRONG LG-B-001**
> Quên Filtered Index khiến hệ thống tạo ra nhiều sự cố trùng lặp cho cùng một máy trạm khi ngưỡng

> CPU/RAM liên tục vượt mức cảnh báo.

5. Tiêu chuẩn hoàn thành 11 mục (Definition of Done — DoD):
• [ ] 1. Hợp đồng giao thức & Wire Envelope chuẩn UTF-8 JSON
• [ ] 2. Lập trình bất đồng bộ Async/Await không chặn luồng mạng
• [ ] 3. Bắt ngoại lệ mạng an toàn (EOF, SocketException, Timeout)
• [ ] 4. Giải phóng tài nguyên sạch sẽ (Dispose / Using / CancellationToken)
• [ ] 5. An toàn dữ liệu (Khử trùng lặp lô / Khóa lạc quan OCC / Transaction)
• [ ] 6. Unit Test / Test Harness chạy đạt 100% không có cảnh báo
• [ ] 7. Tích hợp chéo thành công với module của đồng đội liên quan
• [ ] 8. Kiểm thử thực tế trên mạng LAN nhiều máy tính
• [ ] 9. Đồng bộ tài liệu thiết kế & mã nguồn khớp hoàn toàn
• [ ] 10. Thu thập đầy đủ hồ sơ minh chứng (Log, Screenshot, Pcap)
• [ ] 11. Sẵn sàng trả lời trôi chảy câu hỏi vấn đáp (Viva) liên quan

6. Kịch bản kiểm thử & Hồ sơ minh chứng:
• Mã bài test: TEST-B01
• Phương pháp kiểm thử: Chạy script migration trên instance SQL Server sạch
• Tiêu chuẩn nghiệm thu đạt: Tạo đầy đủ 8 bảng không lỗi; ràng buộc khóa ngoại và chỉ mục toàn vẹn
• Hồ sơ minh chứng lưu tại: Script SQL migration, sơ đồ quan hệ thực thể ERD

7. Tình huống lỗi tiềm ẩn & Kế hoạch ứng phó: Lỗi xung đột kiểu dữ liệu hoặc cú pháp; dùng migration script có version kiểm soát

8. Câu hỏi vấn đáp (Viva) & Chiến lược trả lời:
• Chủ đề câu hỏi: Kiểu dữ liệu rowversion trong SQL Server và cơ chế OCC
• Câu trả lời kỳ vọng: rowversion là kiểu dữ liệu đặc thù của SQL Server tự động tăng khi dòng thay đổi; client coi là token mờ 8-byte kiểm tra tranh chấp
• Câu hỏi mở rộng / Bẫy của giảng viên: Tại sao không dùng cột timestamp datetime để kiểm tra tranh chấp dữ liệu?

[HƯỚNG DẪN ĐỀ XUẤT]: *Tham khảo mã nguồn mẫu chuẩn và tài liệu kỹ thuật tại thư mục Storage. Khi hiện thực, luôn đặt an toàn luồng và xử lý hủy tác vụ (CancellationToken) lên hàng đầu.*

Nhiệm vụ 02: [LG-B-002] — SQL repositories và transaction boundary

Mã & Tên nhiệm vụ: [LG-B-002] SQL repositories và transaction boundary	**Tuần & Ưu tiên:** Tuần W1 (P0) | P0
Khối lượng công việc: Cơ bản: 2.5h | Dự phòng: 0.5h | Tổng: 3.0h	**Phân loại tải:** P0 Core
Trách nhiệm thực hiện: Phụ trách: SV B | Dự phòng: SV C | Duyệt: SV C	**Phân hệ & Thư mục:** Central.Database (Storage)

Sản phẩm bàn giao: IncidentRepository.cs & MetricRepository.cs	**Hợp đồng giao diện:** IDbTransaction, atomic incident creation with audit history

1. Mục đích kỹ thuật & Tại sao tồn tại: Tầng ứng dụng không được viết câu truy vấn SQL rải rác. Phải đóng gói qua Repository Pattern và quản lý ranh giới giao dịch (IDbTransaction) để đảm bảo khi tạo sự cố thì bản ghi Incident và IncidentEvents được ghi nhận nguyên tử (Atomicity).

2. Kiến thức nền tảng cần nắm vững: Mẫu thiết kế Repository, ADO.NET / Dapper, giao dịch ACID (IDbTransaction), nguyên tắc mở/đóng kết nối SqlConnection an toàn.

3. Dữ liệu đầu vào & Đầu ra kỹ thuật: Đầu vào: Kết nối CSDL từ chuỗi kết nối (connection string) và các đối tượng DTO sự cố/số liệu.
Đầu ra: Lớp IncidentRepository.cs và MetricRepository.cs hỗ trợ gọi bất đồng bộ với CancellationToken.

4. Các bước triển khai chi tiết:
1. Viết interface IIncidentRepository và IMetricRepository với các phương thức Async.
2. Hiện thực hàm CreateIncidentWithEventAsync bọc trong SqlConnection.BeginTransactionAsync().
3. Sử dụng tham số hóa hoàn toàn (SqlParameter) để triệt tiêu lỗ hổng SQL Injection.
4. Đảm bảo khối try/catch thực hiện transaction.RollbackAsync() nếu xảy ra bất kỳ ngoại lệ nào.
5. Viết unit test giả lập lỗi ghi bảng con để chứng minh bảng cha không bị lưu rác (rollback thành công).

> 📌 **BẪY LỖI CẦN TRÁNH TRONG LG-B-002**
> Không bọc trong Transaction, dẫn đến việc bản ghi Incident được tạo thành công nhưng IncidentEvents bị lỗi mạng/khóa bảng, làm mất lịch sử kiểm toán (audit trail).

5. Tiêu chuẩn hoàn thành 11 mục (Definition of Done — DoD):
• [ ] 1. Hợp đồng giao thức & Wire Envelope chuẩn UTF-8 JSON
• [ ] 2. Lập trình bất đồng bộ Async/Await không chặn luồng mạng
• [ ] 3. Bắt ngoại lệ mạng an toàn (EOF, SocketException, Timeout)
• [ ] 4. Giải phóng tài nguyên sạch sẽ (Dispose / Using / CancellationToken)
• [ ] 5. An toàn dữ liệu (Khử trùng lặp lô / Khóa lạc quan OCC / Transaction)
• [ ] 6. Unit Test / Test Harness chạy đạt 100% không có cảnh báo
• [ ] 7. Tích hợp chéo thành công với module của đồng đội liên quan
• [ ] 8. Kiểm thử thực tế trên mạng LAN nhiều máy tính
• [ ] 9. Đồng bộ tài liệu thiết kế & mã nguồn khớp hoàn toàn
• [ ] 10. Thu thập đầy đủ hồ sơ minh chứng (Log, Screenshot, Pcap)
• [ ] 11. Sẵn sàng trả lời trôi chảy câu hỏi vấn đáp (Viva) liên quan

6. Kịch bản kiểm thử & Hồ sơ minh chứng:
• Mã bài test: TEST-B02
• Phương pháp kiểm thử: Unit test cố tình throw exception giữa giao dịch; kiểm tra rollback CSDL
• Tiêu chuẩn nghiệm thu đạt: Giao dịch rollback hoàn toàn, không để lại dữ liệu rác (orphan records)
• Hồ sơ minh chứng lưu tại: Test log kiểm chứng rollback thành công

7. Tình huống lỗi tiềm ẩn & Kế hoạch ứng phó: Deadlock cơ sở dữ liệu; sắp xếp thứ tự cập nhật bảng nhất quán và dùng command timeout phù hợp

8. Câu hỏi vấn đáp (Viva) & Chiến lược trả lời:
• Chủ đề câu hỏi: Tính nguyên tử (Atomicity) và ranh giới giao dịch trong CSDL
• Câu trả lời kỳ vọng: Mọi thao tác ghi liên quan (mở sự cố, ghi lịch sử, cập nhật thiết bị) phải nằm trong 1 Transaction để đảm bảo tính nhất quán
• Câu hỏi mở rộng / Bẫy của giảng viên: Sự khác biệt giữa giao dịch ở tầng CSDL và khóa trong bộ nhớ RAM?

[HƯỚNG DẪN ĐỀ XUẤT]: *Tham khảo mã nguồn mẫu chuẩn và tài liệu kỹ thuật tại thư mục Storage. Khi hiện thực, luôn đặt an toàn luồng và xử lý hủy tác vụ (CancellationToken) lên hàng đầu.*

Nhiệm vụ 03: [LG-B-003] — Central accept TLS và bounded ingress

Mã & Tên nhiệm vụ: [LG-B-003] Central accept TLS và bounded ingress	**Tuần & Ưu tiên:** Tuần W1 (P0) | P0
Khối lượng công việc: Cơ bản: 3.0h | Dự phòng: 0.5h | Tổng: 3.5h	**Phân loại tải:** P0 Core
Trách nhiệm thực hiện: Phụ trách: SV B | Dự phòng: SV A | Duyệt: SV A	**Phân hệ & Thư mục:** Central.Server (Networking)
Sản phẩm bàn giao: CentralServerDaemon.cs & IngressChannel.cs	**Hợp đồng giao diện:** TcpListener, SslStream (Server cert), BoundedChannelOptions(1000)

1. Mục đích kỹ thuật & Tại sao tồn tại: Central Server phải lắng nghe hàng chục máy trạm kết nối đồng thời mà không làm cạn kiệt luồng (thread starvation) hoặc tràn bộ nhớ khi có bão lưu lượng. Phải dùng Socket bất đồng bộ kèm kênh đệm có giới hạn (Bounded Channel).

2. Kiến thức nền tảng cần nắm vững: TcpListener trong .NET, giao thức mã hóa TLS (SslStream), System.Threading.Channels.Channel với BoundedChannelOptions.

3. Dữ liệu đầu vào & Đầu ra kỹ thuật: Đầu vào: Cổng mạng TCP (mặc định 5000), chứng chỉ số X.509 cho TLS và giới hạn hàng đợi Capacity = 1000.
Đầu ra: CentralServerDaemon.cs lắng nghe kết nối và IngressChannel.cs tiếp nhận gói tin vào hàng đợi xử lý.

4. Các bước triển khai chi tiết:
1. Khởi tạo TcpListener trên IPAddress.Any và Port chỉ định, bắt đầu vòng lặp AcceptTcpClientAsync.
2. Thiết lập SslStream xác thực chứng chỉ máy chủ (Server Authentication) an toàn.
3. Khởi tạo System.Threading.Channels.Channel.CreateBounded<WireEnvelope>(options) với FullMode = Wait hoặc DropOldest có ghi log.
4. Với mỗi kết nối client được chấp nhận, bàn giao luồng đọc FrameCodec cho Task độc lập đẩy vào

Ingress Channel.
5. Đảm bảo cơ chế Graceful Shutdown khi nhận tín hiệu hủy CancellationToken.

> **📌 BẪY LỖI CẦN TRÁNH TRONG LG-B-003**
> Dùng Unbounded Channel (hàng đợi không giới hạn) khiến Server bị OutOfMemory khi 40 Agent cùng gửi dồn dập trong lúc DB đang bị nghẽn.

5. Tiêu chuẩn hoàn thành 11 mục (Definition of Done — DoD):
• [ ] 1. Hợp đồng giao thức & Wire Envelope chuẩn UTF-8 JSON
• [ ] 2. Lập trình bất đồng bộ Async/Await không chặn luồng mạng
• [ ] 3. Bắt ngoại lệ mạng an toàn (EOF, SocketException, Timeout)
• [ ] 4. Giải phóng tài nguyên sạch sẽ (Dispose / Using / CancellationToken)
• [ ] 5. An toàn dữ liệu (Khử trùng lặp lô / Khóa lạc quan OCC / Transaction)
• [ ] 6. Unit Test / Test Harness chạy đạt 100% không có cảnh báo
• [ ] 7. Tích hợp chéo thành công với module của đồng đội liên quan
• [ ] 8. Kiểm thử thực tế trên mạng LAN nhiều máy tính
• [ ] 9. Đồng bộ tài liệu thiết kế & mã nguồn khớp hoàn toàn
• [ ] 10. Thu thập đầy đủ hồ sơ minh chứng (Log, Screenshot, Pcap)
• [ ] 11. Sẵn sàng trả lời trôi chảy câu hỏi vấn đáp (Viva) liên quan

6. Kịch bản kiểm thử & Hồ sơ minh chứng:
• Mã bài test: TEST-B03
• Phương pháp kiểm thử: Bơm đồng thời 50 kết nối TLS thử nghiệm kiểm tra phân bổ luồng ThreadPool
• Tiêu chuẩn nghiệm thu đạt: Chấp nhận kết nối bất đồng bộ không tạo 1 thread riêng cho mỗi socket; kênh ingress kích hoạt backpressure khi đầy
• Hồ sơ minh chứng lưu tại: Server socket listen log, ThreadPool performance counters

7. Tình huống lỗi tiềm ẩn & Kế hoạch ứng phó: Tràn bộ nhớ do quá nhiều kết nối; giới hạn số kết nối tối đa và áp dụng Bounded Channel

8. Câu hỏi vấn đáp (Viva) & Chiến lược trả lời:
• Chủ đề câu hỏi: Mô hình I/O bất đồng bộ (Non-blocking I/O) trên máy chủ mạng
• Câu trả lời kỳ vọng: Dùng async/await tận dụng I/O Completion Ports của hệ điều hành, giải phóng thread về pool khi chờ dữ liệu socket
• Câu hỏi mở rộng / Bẫy của giảng viên: Tại sao không nên tạo mới một Thread cho mỗi kết nối client?

[HƯỚNG DẪN ĐỀ XUẤT]: *Tham khảo mã nguồn mẫu chuẩn và tài liệu kỹ thuật tại thư mục Networking. Khi hiện thực, luôn đặt an toàn luồng và xử lý hủy tác vụ (CancellationToken) lên hàng đầu.*

Nhiệm vụ 04: [LG-B-004] — Session registry, server auth PBKDF2 và RBAC

Mã & Tên nhiệm vụ: [LG-B-004] Session registry, server auth PBKDF2 và RBAC	**Tuần & Ưu tiên:** Tuần W1 (P0) | P0

Khối lượng công việc: Cơ bản: 3.0h | Dự phòng: 0.5h | Tổng: 3.5h	**Phân loại tải:** P0 Core
Trách nhiệm thực hiện: Phụ trách: SV B | Dự phòng: SV C | Duyệt: SV C	**Phân hệ & Thư mục:** Central.Auth (Security)
Sản phẩm bàn giao: AuthService.cs & SessionStore.cs	**Hợp đồng giao diện:** PBKDF2-HMAC-SHA256 (600k iter, 16B salt, 32B key, FixedTimeEquals), RBAC: Admin, Operator, Viewer

1. Mục đích kỹ thuật & Tại sao tồn tại: Quản trị viên và người vận hành cần xác thực an toàn trước khi xem số liệu hoặc xử lý sự cố. Mật khẩu phải được băm bằng thuật toán hiện đại chống vét cạn GPU, và phân quyền (RBAC) phải phân định rõ quyền của từng nhóm vai trò.

2. Kiến thức nền tảng cần nắm vững: Thuật toán băm PBKDF2-HMAC-SHA256, so sánh byte thời gian hằng số (FixedTimeEquals chống Timing Attack), mô hình RBAC (Admin, Operator, Viewer).

3. Dữ liệu đầu vào & Đầu ra kỹ thuật: Đầu vào: Thông tin đăng nhập từ bản tin LOGIN (username, password) và danh sách vai trò hệ thống.
Đầu ra: AuthService.cs xử lý băm mật khẩu/xác thực và SessionStore.cs quản lý phiên làm việc trong bộ nhớ.

4. Các bước triển khai chi tiết:
1. Hiện thực hàm HashPassword(string password): tạo ngẫu nhiên salt 16 byte (RandomNumberGenerator), băm PBKDF2 600,000 vòng với HMAC-SHA256 cho ra subkey 32 byte.
2. Hiện thực VerifyPassword bằng CryptographicOperations.FixedTimeEquals để ngăn chặn kẻ tấn công dò độ dài chuỗi qua thời gian phản hồi.
3. Lưu trữ chuỗi phiên token ngẫu nhiên (UUIDv4) kèm vai trò tương ứng trong ConcurrentDictionary.
4. Kiểm tra phân quyền chặt chẽ: Viewer chỉ đọc; Operator được phép thao tác sự cố (ACK, START, RESOLVE); Admin toàn quyền quản lý người dùng và cấu hình.
5. Xử lý kịch bản SIGN_UP yêu cầu inviteCode hợp lệ từ Admin trước khi tạo tài khoản.

> **📌 BẪY LỖI CẦN TRÁNH TRONG LG-B-004**
> Dùng hàm so sánh chuỗi thông thường (string == string) gây lỗ hổng Timing Attack, hoặc lưu mật khẩu dạng MD5/SHA1 đã lỗi thời.

5. Tiêu chuẩn hoàn thành 11 mục (Definition of Done — DoD):
- [ ] 1. Hợp đồng giao thức & Wire Envelope chuẩn UTF-8 JSON
- [ ] 2. Lập trình bất đồng bộ Async/Await không chặn luồng mạng
- [ ] 3. Bắt ngoại lệ mạng an toàn (EOF, SocketException, Timeout)
- [ ] 4. Giải phóng tài nguyên sạch sẽ (Dispose / Using / CancellationToken)
- [ ] 5. An toàn dữ liệu (Khử trùng lặp lô / Khóa lạc quan OCC / Transaction)
- [ ] 6. Unit Test / Test Harness chạy đạt 100% không có cảnh báo
- [ ] 7. Tích hợp chéo thành công với module của đồng đội liên quan
- [ ] 8. Kiểm thử thực tế trên mạng LAN nhiều máy tính

• [ ] 9. Đồng bộ tài liệu thiết kế & mã nguồn khớp hoàn toàn
• [ ] 10. Thu thập đầy đủ hồ sơ minh chứng (Log, Screenshot, Pcap)
• [ ] 11. Sẵn sàng trả lời trôi chảy câu hỏi vấn đáp (Viva) liên quan

6. Kịch bản kiểm thử & Hồ sơ minh chứng:
• Mã bài test: TEST-B04
• Phương pháp kiểm thử: Kiểm tra đăng nhập đúng/sai pass; thử nghiệm bypass quyền từ client gửi trực tiếp command
• Tiêu chuẩn nghiệm thu đạt: Mật khẩu băm an toàn kèm salt; server chặn đứng mọi lệnh không đủ quyền dù client có cố tình gửi
• Hồ sơ minh chứng lưu tại: Bảng dữ liệu Users đã băm mật khẩu, auth security audit log

7. Tình huống lỗi tiềm ẩn & Kế hoạch ứng phó: Timing attack khi so sánh chuỗi hash; sử dụng CryptographicOperations.FixedTimeEquals

8. Câu hỏi vấn đáp (Viva) & Chiến lược trả lời:
• Chủ đề câu hỏi: Thuật toán băm mật khẩu PBKDF2 và kiểm soát quyền bắt buộc phía server
• Câu trả lời kỳ vọng: PBKDF2 làm chậm quá trình dò vét brute-force; kiểm tra quyền phía server là chốt chặn an ninh bắt buộc vì UI client có thể bị can thiệp
• Câu hỏi mở rộng / Bẫy của giảng viên: Tại sao việc ẩn nút bấm trên WinForms không được xem là giải pháp an ninh?

[HƯỚNG DẪN ĐỀ XUẤT]: *Tham khảo mã nguồn mẫu chuẩn và tài liệu kỹ thuật tại thư mục Security. Khi hiện thực, luôn đặt an toàn luồng và xử lý hủy tác vụ (CancellationToken) lên hàng đầu.*

Nhiệm vụ 05: [LG-B-005] — Metric ingestion, dedup ProcessedBatches, commit ACK

Mã & Tên nhiệm vụ: [LG-B-005] Metric ingestion, dedup ProcessedBatches, commit ACK	**Tuần & Ưu tiên:** Tuần W2 (P0) | P0
Khối lượng công việc: Cơ bản: 2.5h | Dự phòng: 0.5h | Tổng: 3.0h	**Phân loại tải:** P0 Core
Trách nhiệm thực hiện: Phụ trách: SV B | Dự phòng: SV A | Duyệt: SV A	**Phân hệ & Thư mục:** Central.Ingest (Storage)
Sản phẩm bàn giao: BatchIngestProcessor.cs	**Hợp đồng giao diện:** Table ProcessedBatches (DeviceId, BootId, BatchId, ReceivedUtc), BATCH_ACK

1. Mục đích kỹ thuật & Tại sao tồn tại: Do môi trường mạng có thể chập chờn, Agent có thể gửi lại (retry) một lô số liệu mà Central đã nhận nhưng ACK bị thất lạc. Central phải có cơ chế khử trùng lặp (Idempotent Ingestion) để không tính toán sai số liệu phòng máy.

2. Kiến thức nền tảng cần nắm vững: Tính khả chuyển (Idempotency), khóa tổng hợp (Composite Key), quy trình commit bản tin trong giao dịch CSDL.

3. Dữ liệu đầu vào & Đầu ra kỹ thuật: Đầu vào: Gói tin METRIC_BATCH từ Agent chứa DeviceId, BootId, BatchId và danh sách số liệu CPU/RAM/Network.
Đầu ra: BatchIngestProcessor.cs lưu số liệu sạch vào CSDL và gửi phản hồi BATCH_ACK về cho Agent.

4. Các bước triển khai chi tiết:
1. Kiểm tra sự tồn tại của bộ khóa (DeviceId, BootId, BatchId) trong bảng ProcessedBatches.
2. Nếu đã tồn tại: Bỏ qua bước ghi dữ liệu, gửi ngay BATCH_ACK xác nhận thành công về cho Agent (khử trùng lặp an toàn).
3. Nếu chưa tồn tại: Mở Transaction, ghi nhận lô số liệu vào MetricSnapshots và Processes, đồng thời chèn bản ghi vào ProcessedBatches.
4. Commit Transaction và trả về gói tin BATCH_ACK mang đúng batchId.
5. Đẩy thông báo số liệu mới vào hàng đợi cho bộ đánh giá quy tắc (Rule Engine).

> 📌 **BẪY LỖI CẦN TRÁNH TRONG LG-B-005**
> Ném lỗi ngoại lệ hoặc từ chối gửi BATCH_ACK khi gặp gói tin trùng lặp, khiến Agent tiếp tục thử lại vô tận và nghẽn mạng.

5. Tiêu chuẩn hoàn thành 11 mục (Definition of Done — DoD):
• [ ] 1. Hợp đồng giao thức & Wire Envelope chuẩn UTF-8 JSON
• [ ] 2. Lập trình bất đồng bộ Async/Await không chặn luồng mạng
• [ ] 3. Bắt ngoại lệ mạng an toàn (EOF, SocketException, Timeout)
• [ ] 4. Giải phóng tài nguyên sạch sẽ (Dispose / Using / CancellationToken)
• [ ] 5. An toàn dữ liệu (Khử trùng lặp lô / Khóa lạc quan OCC / Transaction)
• [ ] 6. Unit Test / Test Harness chạy đạt 100% không có cảnh báo
• [ ] 7. Tích hợp chéo thành công với module của đồng đội liên quan
• [ ] 8. Kiểm thử thực tế trên mạng LAN nhiều máy tính
• [ ] 9. Đồng bộ tài liệu thiết kế & mã nguồn khớp hoàn toàn
• [ ] 10. Thu thập đầy đủ hồ sơ minh chứng (Log, Screenshot, Pcap)
• [ ] 11. Sẵn sàng trả lời trôi chảy câu hỏi vấn đáp (Viva) liên quan

6. Kịch bản kiểm thử & Hồ sơ minh chứng:
• Mã bài test: TEST-B05
• Phương pháp kiểm thử: Gửi 10 mẻ tin trùng lặp liên tiếp; kiểm tra số dòng ghi vào bảng telemetry
• Tiêu chuẩn nghiệm thu đạt: Chỉ ghi vào bảng telemetry 1 lần duy nhất; luôn gửi lại BATCH_ACK xác nhận cho client
• Hồ sơ minh chứng lưu tại: Bảng ProcessedBatches SQL Server, log xử lý mẻ tin

7. Tình huống lỗi tiềm ẩn & Kế hoạch ứng phó: Client gửi lặp gây sai lệch thống kê; khóa chính bảng ProcessedBatches ngăn chặn triệt để

8. Câu hỏi vấn đáp (Viva) & Chiến lược trả lời:
• Chủ đề câu hỏi: Cơ chế khử trùng lặp bền vững (Deduplication) và Idempotency
• Câu trả lời kỳ vọng: Dùng bảng cơ sở dữ liệu có khóa chính tự nhiên để chặn trùng lặp qua các đợt

restart, không phụ thuộc vào bộ nhớ RAM tạm
• Câu hỏi mở rộng / Bẫy của giảng viên: Tại sao khi phát hiện batch trùng vẫn phải gửi lại BATCH_ACK?

[HƯỚNG DẪN ĐỀ XUẤT]: *Tham khảo mã nguồn mẫu chuẩn và tài liệu kỹ thuật tại thư mục Storage. Khi hiện thực, luôn đặt an toàn luồng và xử lý hủy tác vụ (CancellationToken) lên hàng đầu.*

Nhiệm vụ 06: [LG-B-006] — Liveness registry và monotonic deadline evaluator

Mã & Tên nhiệm vụ: [LG-B-006] Liveness registry và monotonic deadline evaluator	**Tuần & Ưu tiên:** Tuần W2 (P0) | P0
Khối lượng công việc: Cơ bản: 2.5h | Dự phòng: 0.5h | Tổng: 3.0h	**Phân loại tải:** P0 Core
Trách nhiệm thực hiện: Phụ trách: SV B | Dự phòng: SV A | Duyệt: SV A	**Phân hệ & Thư mục:** Central.Rule (Networking)
Sản phẩm bàn giao: LivenessRegistry.cs	**Hợp đồng giao diện:** Liveness threshold: Missed 3 intervals (15s) + 5s grace = 20s offline deadline

1. Mục đích kỹ thuật & Tại sao tồn tại: Máy trạm trong phòng máy có thể bị tắt đột ngột, rút dây mạng hoặc treo máy mà không kịp gửi thông báo chia tay. Central phải định kỳ quét nhịp tim để phát hiện và đánh dấu máy trạm mất kết nối (OFFLINE).

2. Kiến thức nền tảng cần nắm vững: Đồng hồ đơn điệu (Monotonic Clock) tránh lỗi đổi giờ hệ thống, cơ chế kỳ hạn (Deadline), luồng hẹn giờ định kỳ (PeriodicTimer / BackgroundService).

3. Dữ liệu đầu vào & Đầu ra kỹ thuật: Đầu vào: Bản tin HEARTBEAT từ Agent cập nhật dấu mốc thời gian UTC và bảng trạng thái máy trạm.
Đầu ra: LivenessRegistry.cs theo dõi thời hạn sống và kích hoạt cảnh báo OFFLINE khi quá hạn.

4. Các bước triển khai chi tiết:
1. Duy trì bảng ánh xạ ConcurrentDictionary<string, DateTime> ghi nhận LastSeenUtc của từng máy trạm.
2. Khi nhận HEARTBEAT, cập nhật LastSeenUtc và chuyển trạng thái máy thành ONLINE nếu trước đó đang OFFLINE.
3. Khởi tạo vòng lặp kiểm tra định kỳ mỗi 5 giây (sử dụng PeriodicTimer).
4. Áp dụng công thức hạn chót: Nếu thời gian trôi qua vượt quá 3 khoảng nhịp tim (15 giây) + 5 giây dung sai (grace period) = 20 giây -> đánh dấu OFFLINE.
5. Phát sinh sự cố DEVICE_OFFLINE và thông báo cho Dashboard qua luồng sự kiện.

> **📌 BẪY LỖI CẦN TRÁNH TRONG LG-B-006**
> Dùng DateTime.Now thay vì DateTime.UtcNow hoặc đồng hồ đơn điệu, dẫn đến việc đổi giờ mùa hè hoặc chỉnh giờ máy chủ làm hệ thống báo sập hàng loạt máy trạm giả tạo.

5. Tiêu chuẩn hoàn thành 11 mục (Definition of Done — DoD):
• [ ] 1. Hợp đồng giao thức & Wire Envelope chuẩn UTF-8 JSON
• [ ] 2. Lập trình bất đồng bộ Async/Await không chặn luồng mạng
• [ ] 3. Bắt ngoại lệ mạng an toàn (EOF, SocketException, Timeout)
• [ ] 4. Giải phóng tài nguyên sạch sẽ (Dispose / Using / CancellationToken)
• [ ] 5. An toàn dữ liệu (Khử trùng lặp lô / Khóa lạc quan OCC / Transaction)
• [ ] 6. Unit Test / Test Harness chạy đạt 100% không có cảnh báo
• [ ] 7. Tích hợp chéo thành công với module của đồng đội liên quan
• [ ] 8. Kiểm thử thực tế trên mạng LAN nhiều máy tính
• [ ] 9. Đồng bộ tài liệu thiết kế & mã nguồn khớp hoàn toàn
• [ ] 10. Thu thập đầy đủ hồ sơ minh chứng (Log, Screenshot, Pcap)
• [ ] 11. Sẵn sàng trả lời trôi chảy câu hỏi vấn đáp (Viva) liên quan

6. Kịch bản kiểm thử & Hồ sơ minh chứng:
• Mã bài test: TEST-B06
• Phương pháp kiểm thử: Ngắt heartbeat của một Agent thử nghiệm; đo thời gian server công bố offline
• Tiêu chuẩn nghiệm thu đạt: Chuyển trạng thái máy sang Offline sau đúng 20s ± 500ms; phát sự kiện DEVICE_UPDATE tới Admin
• Hồ sơ minh chứng lưu tại: Liveness deadline event log, timeline đối chiếu

7. Tình huống lỗi tiềm ẩn & Kế hoạch ứng phó: Đồng hồ máy trạm bị sai lệch giờ; server chỉ căn cứ vào thời điểm nhận trên đồng hồ Stopwatch nội bộ

8. Câu hỏi vấn đáp (Viva) & Chiến lược trả lời:
• Chủ đề câu hỏi: Đánh giá trạng thái hoạt động (Liveness) bằng đồng hồ đơn điệu máy chủ
• Câu trả lời kỳ vọng: Không tin tưởng mốc thời gian của client gửi lên; dùng thời gian thực trôi qua trên server để tránh lỗi lệch giờ (Clock Drift)
• Câu hỏi mở rộng / Bẫy của giảng viên: Tại sao cần khoảng thời gian ân hạn (Grace Period) 5 giây?

[HƯỚNG DẪN ĐỀ XUẤT]: *Tham khảo mã nguồn mẫu chuẩn và tài liệu kỹ thuật tại thư mục Networking. Khi hiện thực, luôn đặt an toàn luồng và xử lý hủy tác vụ (CancellationToken) lên hàng đầu.*

Nhiệm vụ 07: [LG-B-007] — Rule evaluator: hysteresis và counters

Mã & Tên nhiệm vụ: [LG-B-007] Rule evaluator: hysteresis và counters	**Tuần & Ưu tiên:** Tuần W2 (P0) | P0
Khối lượng công việc: Cơ bản: 2.5h | Dự phòng: 0.5h | Tổng: 3.0h	**Phân loại tải:** P0 Core
Trách nhiệm thực hiện: Phụ trách: SV B | Dự phòng: SV C | Duyệt: SV C	**Phân hệ & Thư mục:** Central.Rule (RuleEngine)
Sản phẩm bàn giao: RuleEvaluator.cs	**Hợp đồng giao diện:** Trigger: CPU >= 90% (streak 3), Recovery: CPU <= 75% (streak 3); SERVICE_DOWN 2 fail / 2 success

1. Mục đích kỹ thuật & Tại sao tồn tại: Nếu số liệu CPU dao động quanh ngưỡng (ví dụ 89% rồi 91%), việc tạo và đóng sự cố liên tục sẽ làm giao diện bị giật (flapping / alert storm). Bộ đánh giá quy tắc cần cơ chế trễ (Hysteresis) và đếm chuỗi liên tiếp để chỉ cảnh báo khi bất thường thực sự xảy ra.

2. Kiến thức nền tảng cần nắm vững: Kỹ thuật chống rung tín hiệu (Hysteresis), máy trạng thái hữu hạn (FSM), bộ đếm chuỗi liên tiếp (Consecutive Counter).

3. Dữ liệu đầu vào & Đầu ra kỹ thuật: Đầu vào: Dòng dữ liệu số liệu đo lường đã được chuẩn hóa từ BatchIngestProcessor.
Đầu ra: Lớp RuleEvaluator.cs phát hiện trạng thái sự cố và phát sinh yêu cầu mở/hồi phục sự cố.

4. Các bước triển khai chi tiết:
1. Định nghĩa quy tắc CPU: Kích hoạt khi CPU >= 90% liên tiếp 3 mẫu đo; Hồi phục khi CPU <= 75% liên tiếp 3 mẫu đo.
2. Định nghĩa quy tắc SERVICE_DOWN: Kích hoạt khi dịch vụ giám sát dừng 2 lần liên tiếp; Hồi phục khi dịch vụ chạy lại 2 lần liên tiếp.
3. Duy trì bộ đếm streak trong bộ nhớ theo cặp (DeviceId, RuleCode).
4. Khi thỏa điều kiện kích hoạt, gọi IncidentManager để tạo sự cố mới nếu chưa có sự cố nào đang mở.
5. Khi thỏa điều kiện hồi phục, cập nhật trạng thái ConditionState = RECOVERED mà không làm đổi người đang phụ trách.

> 📌 **BẪY LỖI CẦN TRÁNH TRONG LG-B-007**
> Tạo sự cố ngay khi CPU chạm ngưỡng 90% ở mẫu đầu tiên, khiến người quản trị bị quá tải bởi các cảnh báo tức thời vô hại.

5. Tiêu chuẩn hoàn thành 11 mục (Definition of Done — DoD):
• [ ] 1. Hợp đồng giao thức & Wire Envelope chuẩn UTF-8 JSON
• [ ] 2. Lập trình bất đồng bộ Async/Await không chặn luồng mạng
• [ ] 3. Bắt ngoại lệ mạng an toàn (EOF, SocketException, Timeout)
• [ ] 4. Giải phóng tài nguyên sạch sẽ (Dispose / Using / CancellationToken)
• [ ] 5. An toàn dữ liệu (Khử trùng lặp lô / Khóa lạc quan OCC / Transaction)
• [ ] 6. Unit Test / Test Harness chạy đạt 100% không có cảnh báo
• [ ] 7. Tích hợp chéo thành công với module của đồng đội liên quan
• [ ] 8. Kiểm thử thực tế trên mạng LAN nhiều máy tính
• [ ] 9. Đồng bộ tài liệu thiết kế & mã nguồn khớp hoàn toàn
• [ ] 10. Thu thập đầy đủ hồ sơ minh chứng (Log, Screenshot, Pcap)
• [ ] 11. Sẵn sàng trả lời trôi chảy câu hỏi vấn đáp (Viva) liên quan

6. Kịch bản kiểm thử & Hồ sơ minh chứng:
• Mã bài test: TEST-B07
• Phương pháp kiểm thử: Bơm chuỗi telemetry biến thiên qua test harness (89%, 91%, 92%, 90%, 80%, 74%, 73%, 72%)

• Tiêu chuẩn nghiệm thu đạt: Bật sự cố sau đúng 3 mẫu vượt ngưỡng; không tắt cảnh báo ở mức 80%; chỉ phục hồi sau đủ 3 mẫu <= 75%
• Hồ sơ minh chứng lưu tại: Test log kịch bản streak biến thiên, biểu đồ trạng thái

7. Tình huống lỗi tiềm ẩn & Kế hoạch ứng phó: Cảnh báo chớp nháy liên tục (flapping/alert fatigue); thuật toán hysteresis giải quyết triệt để

8. Câu hỏi vấn đáp (Viva) & Chiến lược trả lời:
• Chủ đề câu hỏi: Cơ chế trễ (Hysteresis) và đếm chuỗi liên tiếp (Streak) trong giám sát
• Câu trả lời kỳ vọng: Tạo khoảng cách giữa ngưỡng kích hoạt và ngưỡng phục hồi kèm điều kiện duy trì mẫu để loại bỏ các đỉnh tải tức thời
• Câu hỏi mở rộng / Bẫy của giảng viên: Tại sao không kích hoạt cảnh báo ngay khi CPU chạm ngưỡng 90% một lần duy nhất?

[HƯỚNG DẪN ĐỀ XUẤT]: *Tham khảo mã nguồn mẫu chuẩn và tài liệu kỹ thuật tại thư mục RuleEngine. Khi hiện thực, luôn đặt an toàn luồng và xử lý hủy tác vụ (CancellationToken) lên hàng đầu.*

Nhiệm vụ 08: [LG-B-008] — Incident persistence và unique active constraint

Mã & Tên nhiệm vụ: [LG-B-008] Incident persistence và unique active constraint	**Tuần & Ưu tiên:** Tuần W2 (P0) | P0
Khối lượng công việc: Cơ bản: 2.5h | Dự phòng: 0.5h | Tổng: 3.0h	**Phân loại tải:** P0 Core
Trách nhiệm thực hiện: Phụ trách: SV B | Dự phòng: SV C | Duyệt: SV C	**Phân hệ & Thư mục:** Central.Incident (Storage)
Sản phẩm bàn giao: IncidentManager.cs	**Hợp đồng giao diện:** Filtered Unique Index: WHERE WorkflowState != 'RESOLVED'; ConditionState (ACTIVE/RECOVERED)

1. Mục đích kỹ thuật & Tại sao tồn tại: Sự cố trong phòng máy phải phản ánh đúng thực tế vận hành: một máy trạm không thể có 2 sự cố mở cho cùng một lỗi. CSDL phải đảm bảo tính duy nhất của sự cố đang hoạt động và ghi nhận đầy đủ dòng thời gian.

2. Kiến thức nền tảng cần nắm vững: Mô hình trạng thái sự cố (NEW -> ACKED -> IN_PROGRESS -> RESOLVED), chỉ mục lọc duy nhất (Filtered Unique Index), quan hệ 1-n giữa Incident và IncidentEvents.

3. Dữ liệu đầu vào & Đầu ra kỹ thuật: Đầu vào: Tín hiệu kích hoạt sự cố từ RuleEvaluator hoặc LivenessRegistry.
Đầu ra: IncidentManager.cs quản lý vòng đời sự cố và lưu vết trạng thái vào CSDL.

4. Các bước triển khai chi tiết:
1. Kiểm tra xem máy trạm đã có sự cố cùng RuleCode ở trạng thái khác RESOLVED chưa.
2. Tận dụng ràng buộc Filtered Unique Index ở mức CSDL để bảo vệ chống trùng lặp ngay cả khi có 2 luồng cùng ghi.

3. Khi tạo sự cố, ghi nhận trạng thái ban đầu WorkflowState = OPEN/NEW và ConditionState = ACTIVE.
4. Thêm bản ghi tương ứng vào bảng IncidentEvents ghi rõ thời điểm và nguyên nhân kích hoạt.
5. Phát sóng bản tin INCIDENT_CREATED hoặc cập nhật đến tất cả các client Admin đang kết nối.

> 📌 **BẪY LỖI CẦN TRÁNH TRONG LG-B-008**
> Chỉ kiểm tra điều kiện tồn tại bằng mã code C# mà không có chỉ mục duy nhất trong CSDL, gây lỗi ghi trùng khi có nhiều luồng xử lý song song.

5. Tiêu chuẩn hoàn thành 11 mục (Definition of Done — DoD):
• [ ] 1. Hợp đồng giao thức & Wire Envelope chuẩn UTF-8 JSON
• [ ] 2. Lập trình bất đồng bộ Async/Await không chặn luồng mạng
• [ ] 3. Bắt ngoại lệ mạng an toàn (EOF, SocketException, Timeout)
• [ ] 4. Giải phóng tài nguyên sạch sẽ (Dispose / Using / CancellationToken)
• [ ] 5. An toàn dữ liệu (Khử trùng lặp lô / Khóa lạc quan OCC / Transaction)
• [ ] 6. Unit Test / Test Harness chạy đạt 100% không có cảnh báo
• [ ] 7. Tích hợp chéo thành công với module của đồng đội liên quan
• [ ] 8. Kiểm thử thực tế trên mạng LAN nhiều máy tính
• [ ] 9. Đồng bộ tài liệu thiết kế & mã nguồn khớp hoàn toàn
• [ ] 10. Thu thập đầy đủ hồ sơ minh chứng (Log, Screenshot, Pcap)
• [ ] 11. Sẵn sàng trả lời trôi chảy câu hỏi vấn đáp (Viva) liên quan

6. Kịch bản kiểm thử & Hồ sơ minh chứng:
• Mã bài test: TEST-B08
• Phương pháp kiểm thử: Cố tình kích hoạt mở sự cố 2 lần liên tiếp trên cùng thiết bị và rule
• Tiêu chuẩn nghiệm thu đạt: Chỉ tạo đúng 1 dòng sự cố; lần kích hoạt sau chỉ cập nhật lịch sử và giữ nguyên ticket đang mở
• Hồ sơ minh chứng lưu tại: Script định nghĩa Filtered Index, log test chống duplicate ticket

7. Tình huống lỗi tiềm ẩn & Kế hoạch ứng phó: Trùng lặp sự cố làm phân tán nhân lực xử lý; ràng buộc CSDL bảo đảm toàn vẹn dữ liệu ở mức cứng

8. Câu hỏi vấn đáp (Viva) & Chiến lược trả lời:
• Chủ đề câu hỏi: Phân tách ConditionState vs WorkflowState và chỉ mục duy nhất có điều kiện
• Câu trả lời kỳ vọng: ConditionState phản ánh tình trạng kỹ thuật thực tế của máy, WorkflowState phản ánh tiến trình xử lý của con người
• Câu hỏi mở rộng / Bẫy của giảng viên: Nếu sự cố tự phục hồi kỹ thuật trước khi kỹ thuật viên xác nhận thì trạng thái thế nào?

[HƯỚNG DẪN ĐỀ XUẤT]: *Tham khảo mã nguồn mẫu chuẩn và tài liệu kỹ thuật tại thư mục Storage. Khi hiện thực, luôn đặt an toàn luồng và xử lý hủy tác vụ (CancellationToken) lên hàng đầu.*

Nhiệm vụ 09: [LG-B-009] — Commands ACK/START/RESOLVE + rowversion OCC

Mã & Tên nhiệm vụ: [LG-B-009] Commands ACK/START/RESOLVE + rowversion OCC	**Tuần & Ưu tiên:** Tuần W3 (P0) | P0
Khối lượng công việc: Cơ bản: 2.5h | Dự phòng: 0.5h | Tổng: 3.0h	**Phân loại tải:** P0 Core
Trách nhiệm thực hiện: Phụ trách: SV B | Dự phòng: SV C | Duyệt: SV C	**Phân hệ & Thư mục:** Central.Command (Storage)
Sản phẩm bàn giao: IncidentCommandHandler.cs	**Hợp đồng giao diện:** Optimistic Concurrency Control (OCC) via rowversion, Table ProcessedCommands

1. Mục đích kỹ thuật & Tại sao tồn tại: Khi có nhiều cán bộ quản trị (Admin/Operator) cùng mở Dashboard, họ có thể đồng thời bấm nhận (ACK) hoặc giải quyết (RESOLVE) cùng một sự cố. Phải dùng khóa lạc quan (OCC) để ngăn chặn việc người này ghi đè lên thao tác của người khác mà không hay biết.

2. Kiến thức nền tảng cần nắm vững: Khóa lạc quan (Optimistic Concurrency Control), cột rowversion/TIMESTAMP của SQL Server, ngoại lệ DbUpdateConcurrencyException.

3. Dữ liệu đầu vào & Đầu ra kỹ thuật: Đầu vào: Các bản tin lệnh INCIDENT_ACK, INCIDENT_START, INCIDENT_RESOLVE mang kèm incidentId và expectedRowVersion.
Đầu ra: IncidentCommandHandler.cs xử lý chuyển trạng thái an toàn hoặc trả về lỗi xung đột CONCURRENCY_CONFLICT.

4. Các bước triển khai chi tiết:
1. Trích xuất incidentId và expectedRowVersion từ payload của bản tin gửi lên.
2. Thực thi câu lệnh UPDATE Incidents SET WorkflowState = @newState, AssignedTo = @user WHERE IncidentId = @id AND RowVersion = @expectedRowVersion.
3. Kiểm tra số dòng bị ảnh hưởng (rows affected): nếu bằng 0, chứng minh sự cố đã bị sửa bởi người khác giữa chừng.
4. Nếu bằng 0: Trả về bản tin lỗi có mã CONCURRENCY_CONFLICT kèm trạng thái mới nhất để UI của client tự động tải lại.
5. Nếu thành công: Ghi sự kiện vào IncidentEvents, cập nhật bảng ProcessedCommands và gửi phản hồi thành công.

> **📌 BẪY LỖI CẦN TRÁNH TRONG LG-B-009**
> Bỏ qua việc kiểm tra RowVersion trong mệnh đề WHERE, dẫn đến hiện tượng Lost Update (thao tác của người bấm sau đè bẹp thao tác người trước).

5. Tiêu chuẩn hoàn thành 11 mục (Definition of Done — DoD):
• [ ] 1. Hợp đồng giao thức & Wire Envelope chuẩn UTF-8 JSON
• [ ] 2. Lập trình bất đồng bộ Async/Await không chặn luồng mạng

• [ ] 3. Bắt ngoại lệ mạng an toàn (EOF, SocketException, Timeout)
• [ ] 4. Giải phóng tài nguyên sạch sẽ (Dispose / Using / CancellationToken)
• [ ] 5. An toàn dữ liệu (Khử trùng lặp lô / Khóa lạc quan OCC / Transaction)
• [ ] 6. Unit Test / Test Harness chạy đạt 100% không có cảnh báo
• [ ] 7. Tích hợp chéo thành công với module của đồng đội liên quan
• [ ] 8. Kiểm thử thực tế trên mạng LAN nhiều máy tính
• [ ] 9. Đồng bộ tài liệu thiết kế & mã nguồn khớp hoàn toàn
• [ ] 10. Thu thập đầy đủ hồ sơ minh chứng (Log, Screenshot, Pcap)
• [ ] 11. Sẵn sàng trả lời trôi chảy câu hỏi vấn đáp (Viva) liên quan

6. Kịch bản kiểm thử & Hồ sơ minh chứng:
• Mã bài test: TEST-B09
• Phương pháp kiểm thử: Mô phỏng 2 Admin cùng gửi lệnh ACK trên cùng 1 sự cố tại cùng 1 mốc thời gian
• Tiêu chuẩn nghiệm thu đạt: 1 Admin cập nhật thành công, 1 Admin nhận mã lỗi xung đột 409 Conflict kèm dữ liệu mới nhất
• Hồ sơ minh chứng lưu tại: Command handler log, test report xử lý tranh chấp 2 Admin

7. Tình huống lỗi tiềm ẩn & Kế hoạch ứng phó: Ghi đè mất dữ liệu (Lost Update); OCC bằng rowversion phát hiện ngay lập tức mà không cần lock bảng

8. Câu hỏi vấn đáp (Viva) & Chiến lược trả lời:
• Chủ đề câu hỏi: Kiểm soát tương tranh lạc quan (OCC) bằng kiểu rowversion
• Câu trả lời kỳ vọng: Đính kèm rowversion khi gửi lệnh cập nhật; câu lệnh SQL kiểm tra WHERE rowversion = @expected, nếu số dòng ảnh hưởng = 0 nghĩa là đã bị sửa đổi
• Câu hỏi mở rộng / Bẫy của giảng viên: Tại sao OCC phù hợp hơn Pessimistic Locking trong hệ thống giám sát nhiều người xem?

[HƯỚNG DẪN ĐỀ XUẤT]: *Tham khảo mã nguồn mẫu chuẩn và tài liệu kỹ thuật tại thư mục Storage. Khi hiện thực, luôn đặt an toàn luồng và xử lý hủy tác vụ (CancellationToken) lên hàng đầu.*

Nhiệm vụ 10: [LG-B-010] — Restart/DB fault handling và Central health

Mã & Tên nhiệm vụ: [LG-B-010] Restart/DB fault handling và Central health	**Tuần & Ưu tiên:** Tuần W3 (P0) | P0
Khối lượng công việc: Cơ bản: 3.0h | Dự phòng: 0.5h | Tổng: 3.5h	**Phân loại tải:** Test/Harness
Trách nhiệm thực hiện: Phụ trách: SV B | Dự phòng: SV A | Duyệt: SV A	**Phân hệ & Thư mục:** Central.Health (Testing)
Sản phẩm bàn giao: HealthMonitor.cs & DbFaultRecovery.cs	**Hợp đồng giao diện:** HEALTH_UPDATE (activeConnections, dbConnected, queueLag, uptime)

1. Mục đích kỹ thuật & Tại sao tồn tại: Trong vận hành thật, SQL Server có thể bị khởi động lại, đứt mạng tạm thời hoặc đầy bộ nhớ. Central Server không được sập hoàn toàn mà phải biết tạm giữ kết nối, báo cáo sức khỏe (Health Check) và tự kết nối lại CSDL khi có thể.

2. Kiến thức nền tảng cần nắm vững: Mẫu phục hồi Circuit Breaker / Retry Policy, bản tin HEALTH_UPDATE, giám sát tài nguyên tiến trình qua System.Diagnostics.Process.

3. Dữ liệu đầu vào & Đầu ra kỹ thuật: Đầu vào: Trạng thái kết nối SQL Server, độ dài hàng đợi IngressChannel và số lượng kết nối máy trạm hiện hữu.
Đầu ra: HealthMonitor.cs phát sóng sức khỏe hệ thống và DbFaultRecovery.cs tự động khôi phục kết nối CSDL.

4. Các bước triển khai chi tiết:
1. Xây dựng dịch vụ nền định kỳ 10 giây đo lường: activeConnections, dbConnected, queueLag, memoryUsed, uptime.
2. Phát sóng bản tin HEALTH_UPDATE đến các Dashboard quản trị để người trực phòng máy nắm được tình trạng máy chủ.
3. Khi câu lệnh DB ném ngoại lệ SqlException do mất mạng, không để crash tiến trình mà bật cờ DbDegraded.
4. Thử kết nối lại CSDL với chính sách gia tăng thời gian chờ (Exponential Backoff: 1s, 2s, 4s, tối đa 30s).
5. Khi CSDL online trở lại, giải tỏa hàng đợi đệm và cập nhật lại trạng thái bình thường.

> 📌 **BẪY LỖI CẦN TRÁNH TRONG LG-B-010**
> Để ngoại lệ mất kết nối DB văng ra khỏi phạm vi bắt lỗi, làm sập hoàn toàn tiến trình Central Server đang phục vụ toàn bộ phòng máy.

5. Tiêu chuẩn hoàn thành 11 mục (Definition of Done — DoD):
• [ ] 1. Hợp đồng giao thức & Wire Envelope chuẩn UTF-8 JSON
• [ ] 2. Lập trình bất đồng bộ Async/Await không chặn luồng mạng
• [ ] 3. Bắt ngoại lệ mạng an toàn (EOF, SocketException, Timeout)
• [ ] 4. Giải phóng tài nguyên sạch sẽ (Dispose / Using / CancellationToken)
• [ ] 5. An toàn dữ liệu (Khử trùng lặp lô / Khóa lạc quan OCC / Transaction)
• [ ] 6. Unit Test / Test Harness chạy đạt 100% không có cảnh báo
• [ ] 7. Tích hợp chéo thành công với module của đồng đội liên quan
• [ ] 8. Kiểm thử thực tế trên mạng LAN nhiều máy tính
• [ ] 9. Đồng bộ tài liệu thiết kế & mã nguồn khớp hoàn toàn
• [ ] 10. Thu thập đầy đủ hồ sơ minh chứng (Log, Screenshot, Pcap)
• [ ] 11. Sẵn sàng trả lời trôi chảy câu hỏi vấn đáp (Viva) liên quan

6. Kịch bản kiểm thử & Hồ sơ minh chứng:
• Mã bài test: TEST-B10
• Phương pháp kiểm thử: Tạm dừng dịch vụ SQL Server 10s trong lúc client đang kết nối; sau đó bật lại
• Tiêu chuẩn nghiệm thu đạt: Central Server tự động kết nối lại CSDL thành công; không bị crash tiến

trình; khôi phục ghi dữ liệu bình thường
• Hồ sơ minh chứng lưu tại: DB reconnect test trace, Health update broadcast log

7. Tình huống lỗi tiềm ẩn & Kế hoạch ứng phó: Ngoại lệ mất kết nối DB làm chết server mạng; bọc retry logic có giới hạn và thông báo trạng thái NotReady

8. Câu hỏi vấn đáp (Viva) & Chiến lược trả lời:
• Chủ đề câu hỏi: Xử lý lỗi đứt gãy kết nối CSDL trong ứng dụng máy chủ thường trực
• Câu trả lời kỳ vọng: Dùng Connection Resiliency / Retry Policy; hàng đợi tạm thời lưu trữ tin nhắn trong RAM ngắn hạn chờ DB phục hồi
• Câu hỏi mở rộng / Bẫy của giảng viên: Nếu CSDL chết quá lâu thì xử lý hàng đợi RAM thế nào để không sập server?

[HƯỚNG DẪN ĐỀ XUẤT]: *Tham khảo mã nguồn mẫu chuẩn và tài liệu kỹ thuật tại thư mục Testing. Khi hiện thực, luôn đặt an toàn luồng và xử lý hủy tác vụ (CancellationToken) lên hàng đầu.*

Nhiệm vụ 11: [LG-B-011] — Retention, query/report và restore plan

Mã & Tên nhiệm vụ: [LG-B-011] Retention, query/report và restore plan	**Tuần & Ưu tiên:** Tuần W3 (P0) | P0
Khối lượng công việc: Cơ bản: 1.5h | Dự phòng: 0.5h | Tổng: 2.0h	**Phân loại tải:** P0 Core
Trách nhiệm thực hiện: Phụ trách: SV B | Dự phòng: SV C | Duyệt: SV C	**Phân hệ & Thư mục:** Central.Retention (Storage)
Sản phẩm bàn giao: RetentionWorker.cs & QueryService.cs	**Hợp đồng giao diện:** Purge query: ReceivedUtc < 48h ago, QUERY_INCIDENT_HISTORY, QUERY_METRICS_HISTORY

1. Mục đích kỹ thuật & Tại sao tồn tại: Sau nhiều ngày vận hành, hàng triệu bản ghi số liệu đo lường sẽ làm đầy ổ cứng và làm chậm tốc độ truy vấn của Dashboard. Cần có chính sách dọn dẹp dữ liệu cũ (Retention Policy) và các hàm truy vấn phân trang hiệu quả.

2. Kiến thức nền tảng cần nắm vững: Chiến lược lưu trữ dữ liệu theo thời gian (Data Retention), câu lệnh phân trang OFFSET/FETCH trong SQL Server, tác vụ chạy ngầm định kỳ.

3. Dữ liệu đầu vào & Đầu ra kỹ thuật: Đầu vào: Cấu hình thời gian lưu trữ (mặc định 48 giờ cho số liệu chi tiết) và các bản tin yêu cầu lịch sử từ Dashboard.
Đầu ra: RetentionWorker.cs tự động xóa số liệu hết hạn và QueryService.cs trả về lịch sử sự cố/số liệu.

4. Các bước triển khai chi tiết:
1. Viết thủ tục lưu trữ hoặc câu lệnh DELETE có giới hạn theo lô (Batch Delete 1000 dòng) để tránh khóa bảng quá lâu.
2. Thiết lập RetentionWorker chạy vào ban đêm (hoặc mỗi giờ một lần) xóa các bản ghi trong MetricSnapshots có ReceivedUtc < 48 giờ trước.

3. Hiện thực hàm xử lý bản tin QUERY_INCIDENT_HISTORY hỗ trợ lọc theo thời gian, thiết bị, trạng thái và phân trang.
4. Hiện thực hàm QUERY_METRICS_HISTORY trích xuất biểu đồ biến thiên tài nguyên của máy trạm phục vụ báo cáo.
5. Đảm bảo các chỉ mục trên cột ReceivedUtc và IncidentId hoạt động tối ưu không bị quét toàn bảng (Table Scan).

> **📌 BẪY LỖI CẦN TRÁNH TRONG LG-B-011**
> Chạy lệnh DELETE FROM MetricSnapshots WHERE ... một lần cho hàng triệu dòng, gây tràn Transaction Log của SQL Server và làm treo toàn bộ hệ thống.

5. Tiêu chuẩn hoàn thành 11 mục (Definition of Done — DoD):
• [ ] 1. Hợp đồng giao thức & Wire Envelope chuẩn UTF-8 JSON
• [ ] 2. Lập trình bất đồng bộ Async/Await không chặn luồng mạng
• [ ] 3. Bắt ngoại lệ mạng an toàn (EOF, SocketException, Timeout)
• [ ] 4. Giải phóng tài nguyên sạch sẽ (Dispose / Using / CancellationToken)
• [ ] 5. An toàn dữ liệu (Khử trùng lặp lô / Khóa lạc quan OCC / Transaction)
• [ ] 6. Unit Test / Test Harness chạy đạt 100% không có cảnh báo
• [ ] 7. Tích hợp chéo thành công với module của đồng đội liên quan
• [ ] 8. Kiểm thử thực tế trên mạng LAN nhiều máy tính
• [ ] 9. Đồng bộ tài liệu thiết kế & mã nguồn khớp hoàn toàn
• [ ] 10. Thu thập đầy đủ hồ sơ minh chứng (Log, Screenshot, Pcap)
• [ ] 11. Sẵn sàng trả lời trôi chảy câu hỏi vấn đáp (Viva) liên quan

6. Kịch bản kiểm thử & Hồ sơ minh chứng:
• Mã bài test: TEST-B11
• Phương pháp kiểm thử: Chèn dữ liệu giả lập 3 ngày trước; chạy job retention; kiểm tra số dòng còn lại
• Tiêu chuẩn nghiệm thu đạt: Xóa sạch dữ liệu thô quá 48h; giữ nguyên 100% dữ liệu trong 48h và toàn bộ lịch sử Incident
• Hồ sơ minh chứng lưu tại: Retention execution log, số lượng record trước và sau dọn dẹp

7. Tình huống lỗi tiềm ẩn & Kế hoạch ứng phó: Lệnh DELETE lớn gây khóa bảng (Table Lock); xóa theo từng batch nhỏ 5000 dòng để không ảnh hưởng ghi mới

8. Câu hỏi vấn đáp (Viva) & Chiến lược trả lời:
• Chủ đề câu hỏi: Chiến lược lưu trữ phân tầng và dọn dẹp dữ liệu (Data Retention)
• Câu trả lời kỳ vọng: Dữ liệu thô phát sinh liên tục làm phình to CSDL nhanh chóng; cần dọn dẹp định kỳ và chỉ lưu trữ dài hạn sự cố tóm tắt
• Câu hỏi mở rộng / Bẫy của giảng viên: Tại sao xóa dữ liệu theo ReceivedUtc an toàn hơn theo sentAtUtc của client?

[HƯỚNG DẪN ĐỀ XUẤT]: *Tham khảo mã nguồn mẫu chuẩn và tài liệu kỹ thuật tại thư mục Storage. Khi hiện thực, luôn đặt an toàn luồng và xử lý hủy tác vụ (CancellationToken) lên hàng đầu.*

Nhiệm vụ 12: [LG-B-012] — MVP concurrency và Incident acceptance

Mã & Tên nhiệm vụ: [LG-B-012] MVP concurrency và Incident acceptance	**Tuần & Ưu tiên:** Tuần W3 (P0) | P0
Khối lượng công việc: Cơ bản: 3.5h | Dự phòng: 0.5h | Tổng: 4.0h	**Phân loại tải:** Tích hợp
Trách nhiệm thực hiện: Phụ trách: SV B | Dự phòng: SV C | Duyệt: SV C	**Phân hệ & Thư mục:** Central.Review (Integration)
Sản phẩm bàn giao: ConcurrencyAcceptanceReport.md	**Hợp đồng giao diện:** Hệ thống MVP tích hợp đầy đủ 3 khối

1. Mục đích kỹ thuật & Tại sao tồn tại: Trước khi kết thúc tuần 3 (cổng M4), toàn bộ hệ thống MVP phải được kiểm thử chịu tải đồng thời từ cả 3 phía: Agent gửi dồn dập, Server xử lý khử trùng lặp và Admin thao tác sự cố. Cần báo cáo nghiệm thu thực tế để chứng minh kiến trúc cốt lõi đã vững vàng.

2. Kiến thức nền tảng cần nắm vững: Kiểm thử chịu tải đồng thời (Concurrency Testing), công cụ đo lường tài nguyên, tiêu chí nghiệm thu cổng chất lượng M4.

3. Dữ liệu đầu vào & Đầu ra kỹ thuật: Đầu vào: Bộ giả lập Agent gửi đồng thời 10 kết nối với 50 lô số liệu và 2 tài khoản Admin cùng thao tác.
Đầu ra: Báo cáo ConcurrencyAcceptanceReport.md chứng minh hệ thống đạt 100% tiêu chí chịu tải không lỗi dữ liệu.

4. Các bước triển khai chi tiết:
1. Phối hợp cùng A và C dựng kịch bản thử nghiệm tích hợp 3 khối trên môi trường mạng nội bộ.
2. Chạy kịch bản 10 máy trạm gửi đồng thời nhịp tim và số liệu, kiểm tra hàng đợi IngressChannel không bị tràn.
3. Kích hoạt kịch bản 2 Admin cùng tranh chấp xử lý 1 sự cố để kiểm chứng phản hồi CONCURRENCY_CONFLICT.
4. Kiểm tra CSDL không xuất hiện bản ghi mồ côi hoặc vi phạm ràng buộc toàn vẹn.
5. Lập biên bản nghiệm thu kỹ thuật xác nhận MVP hoàn thành để làm cơ sở họp xét duyệt cổng M4.

> **📌 BẪY LỖI CẦN TRÁNH TRONG LG-B-012**
> Chỉ kiểm tra đơn lẻ từng tính năng trên máy cá nhân mà không chạy kiểm thử đồng thời qua mạng, dẫn đến vỡ trận khi tích hợp thật.

5. Tiêu chuẩn hoàn thành 11 mục (Definition of Done — DoD):
- [ ] 1. Hợp đồng giao thức & Wire Envelope chuẩn UTF-8 JSON
- [ ] 2. Lập trình bất đồng bộ Async/Await không chặn luồng mạng
- [ ] 3. Bắt ngoại lệ mạng an toàn (EOF, SocketException, Timeout)
- [ ] 4. Giải phóng tài nguyên sạch sẽ (Dispose / Using / CancellationToken)

• [ ] 5. An toàn dữ liệu (Khử trùng lặp lô / Khóa lạc quan OCC / Transaction)
• [ ] 6. Unit Test / Test Harness chạy đạt 100% không có cảnh báo
• [ ] 7. Tích hợp chéo thành công với module của đồng đội liên quan
• [ ] 8. Kiểm thử thực tế trên mạng LAN nhiều máy tính
• [ ] 9. Đồng bộ tài liệu thiết kế & mã nguồn khớp hoàn toàn
• [ ] 10. Thu thập đầy đủ hồ sơ minh chứng (Log, Screenshot, Pcap)
• [ ] 11. Sẵn sàng trả lời trôi chảy câu hỏi vấn đáp (Viva) liên quan

6. Kịch bản kiểm thử & Hồ sơ minh chứng:
• Mã bài test: TEST-B12
• Phương pháp kiểm thử: Chạy kịch bản 33 test scenarios từ Master Plan; kiểm tra log lỗi CSDL và mạng
• Tiêu chuẩn nghiệm thu đạt: Không có lỗi nghiêm trọng Sev1/Sev2; số liệu hiển thị trên Admin nhất quán 100% với CSDL
• Hồ sơ minh chứng lưu tại: ConcurrencyAcceptanceReport.md, trace đối soát dữ liệu

7. Tình huống lỗi tiềm ẩn & Kế hoạch ứng phó: Phát hiện lỗi race condition; cô lập module lỗi và sửa dứt điểm trước khi quyết định mở rộng P1

8. Câu hỏi vấn đáp (Viva) & Chiến lược trả lời:
• Chủ đề câu hỏi: Đảm bảo tính nhất quán dữ liệu trong môi trường nhiều tác nhân đồng thời
• Câu trả lời kỳ vọng: Kết hợp phân tầng: hàng đợi mạng tuần tự, transaction CSDL, kiểm tra khóa chính dedup và OCC trên đối tượng nghiệp vụ
• Câu hỏi mở rộng / Bẫy của giảng viên: Những rủi ro chính khi chuyển từ kiểm thử đơn luồng sang đa luồng?

[HƯỚNG DẪN ĐỀ XUẤT]: *Tham khảo mã nguồn mẫu chuẩn và tài liệu kỹ thuật tại thư mục Integration. Khi hiện thực, luôn đặt an toàn luồng và xử lý hủy tác vụ (CancellationToken) lên hàng đầu.*

Nhiệm vụ 13: [LG-B-013] — Internal Collector listener và dispatch

Mã & Tên nhiệm vụ: [LG-B-013] Internal Collector listener và dispatch	**Tuần & Ưu tiên:** Tuần W4 (P1) | P1
Khối lượng công việc: Cơ bản: 6.0h | Dự phòng: 0.5h | Tổng: 6.5h	**Phân loại tải:** P1 Extension
Trách nhiệm thực hiện: Phụ trách: SV B | Dự phòng: SV C | Duyệt: SV C	**Phân hệ & Thư mục:** P1.Internal (Networking)
Sản phẩm bàn giao: InternalCollectorDispatcher.cs	**Hợp đồng giao diện:** COLLECTOR_REGISTER, COLLECTOR_FORWARD (multiplexed stream qua 1 socket)

1. Mục đích kỹ thuật & Tại sao tồn tại: Để mở rộng hệ thống lên quy mô nhiều phòng máy (tính năng mở rộng P1), Central Server cần lắng nghe các nút gom nội bộ (Internal Collector) gửi dòng dữ liệu đã được gộp thay vì nhận từng kết nối lẻ.

2. Kiến thức nền tảng cần nắm vững: Giao thức dồn kênh (Multiplexing Stream), phân định luồng nội bộ (Internal vs Edge Listener), bản tin COLLECTOR_REGISTER.

3. Dữ liệu đầu vào & Đầu ra kỹ thuật: Đầu vào: Cổng kết nối nội bộ riêng biệt và các gói tin gom COLLECTOR_FORWARD mang nhiều thông điệp con.
Đầu ra: InternalCollectorDispatcher.cs tiếp nhận và bóc tách các thông điệp từ Collector chuyển về Central.

4. Các bước triển khai chi tiết:
1. Mở thêm một cổng lắng nghe chuyên biệt dành riêng cho các Collector nội bộ.
2. Xử lý bản tin xác thực COLLECTOR_REGISTER để đăng ký danh tính và dải phòng máy mà Collector đại diện.
3. Hiện thực bộ giải mã bóc tách luồng dữ liệu gộp (Multiplexed Frame) thành từng WireEnvelope riêng biệt.
4. Đưa các thông điệp đã bóc tách vào IngressChannel chung của hệ thống với cờ nhận diện nguồn từ Collector.
5. Cung cấp cơ chế dự phòng tắt tính năng nhẹ nhàng nếu cổng M4 quyết định hoãn (defer) P1.

> 📌 **BẪY LỖI CẦN TRÁNH TRONG LG-B-013**
> Dùng chung cổng của Agent cho Collector mà không có cơ chế xác thực riêng, tạo nguy cơ bảo mật và khó quản lý băng thông.

5. Tiêu chuẩn hoàn thành 11 mục (Definition of Done — DoD):
• [ ] 1. Hợp đồng giao thức & Wire Envelope chuẩn UTF-8 JSON
• [ ] 2. Lập trình bất đồng bộ Async/Await không chặn luồng mạng
• [ ] 3. Bắt ngoại lệ mạng an toàn (EOF, SocketException, Timeout)
• [ ] 4. Giải phóng tài nguyên sạch sẽ (Dispose / Using / CancellationToken)
• [ ] 5. An toàn dữ liệu (Khử trùng lặp lô / Khóa lạc quan OCC / Transaction)
• [ ] 6. Unit Test / Test Harness chạy đạt 100% không có cảnh báo
• [ ] 7. Tích hợp chéo thành công với module của đồng đội liên quan
• [ ] 8. Kiểm thử thực tế trên mạng LAN nhiều máy tính
• [ ] 9. Đồng bộ tài liệu thiết kế & mã nguồn khớp hoàn toàn
• [ ] 10. Thu thập đầy đủ hồ sơ minh chứng (Log, Screenshot, Pcap)
• [ ] 11. Sẵn sàng trả lời trôi chảy câu hỏi vấn đáp (Viva) liên quan

6. Kịch bản kiểm thử & Hồ sơ minh chứng:
• Mã bài test: TEST-B13
• Phương pháp kiểm thử: Kết nối đồng thời 2 Collector vào Central; đẩy dữ liệu từ 20 agent qua Collector
• Tiêu chuẩn nghiệm thu đạt: Central nhận diện đúng danh tính từng Collector; giải mã và xử lý trơn tru luồng tin ghép
• Hồ sơ minh chứng lưu tại: Internal listener connection log, dispatch throughput counters

7. Tình huống lỗi tiềm ẩn & Kế hoạch ứng phó: Collector giả mạo kết nối vào cổng nội bộ; xác thực token bí mật chia sẻ (Pre-shared Secret) khi bắt tay

8. Câu hỏi vấn đáp (Viva) & Chiến lược trả lời:
• Chủ đề câu hỏi: Mô hình dồn luồng (Multiplexing) dữ liệu từ Collector về Central
• Câu trả lời kỳ vọng: Nhiều Agent kết nối tới Collector, Collector gộp dữ liệu đẩy qua một kết nối socket duy nhất về Central để tối ưu tài nguyên
• Câu hỏi mở rộng / Bẫy của giảng viên: Làm thế nào Central định tuyến phản hồi ACK về đúng Agent ban đầu?

[HƯỚNG DẪN ĐỀ XUẤT]: *Tham khảo mã nguồn mẫu chuẩn và tài liệu kỹ thuật tại thư mục Networking. Khi hiện thực, luôn đặt an toàn luồng và xử lý hủy tác vụ (CancellationToken) lên hàng đầu.*

Nhiệm vụ 14: [LG-B-014] — Session generation qua Collector

Mã & Tên nhiệm vụ: [LG-B-014] Session generation qua Collector	**Tuần & Ưu tiên:** Tuần W4 (P1) | P1
Khối lượng công việc: Cơ bản: 5.0h | Dự phòng: 0.5h | Tổng: 5.5h	**Phân loại tải:** P1 Extension
Trách nhiệm thực hiện: Phụ trách: SV B | Dự phòng: SV C | Duyệt: SV C	**Phân hệ & Thư mục:** P1.Session (Architecture)
Sản phẩm bàn giao: SessionGenerationManager.cs	**Hợp đồng giao diện:** Header sessionGeneration (monotonically increasing integer per device)

1. Mục đích kỹ thuật & Tại sao tồn tại: Khi Collector trung gian bị khởi động lại hoặc chuyển đổi dự phòng, các thông tin định tuyến cũ có thể bị lẫn với thông tin mới. Server cần trường thế hệ phiên (Session Generation) tăng dần để loại bỏ các gói tin lỗi thời.

2. Kiến thức nền tảng cần nắm vững: Số thế hệ phiên (Epoch / Generation Number), thứ tự thông điệp trong hệ phân tán, xử lý gói tin đến muộn.

3. Dữ liệu đầu vào & Đầu ra kỹ thuật: Đầu vào: Trường sessionGeneration trong tiêu đề gói tin chuyển tiếp từ Collector.
Đầu ra: SessionGenerationManager.cs kiểm tra và từ chối các gói tin mang thế hệ cũ hơn thế hệ hiện tại.

4. Các bước triển khai chi tiết:
1. Bổ sung trường sessionGeneration kiểu số nguyên tăng dần vào bảng theo dõi phiên máy trạm.
2. Khi Collector đăng ký lại sau sự cố, Central cấp phát một sessionGeneration mới tăng 1 đơn vị.
3. Trong mọi gói tin gửi qua Collector, kiểm tra số thế hệ: nếu nhỏ hơn thế hệ hiện tại của máy trạm thì loại bỏ ngay.
4. Ghi log cảnh báo khi phát hiện gói tin đến trễ hoặc rớt từ phiên làm việc trước.
5. Đảm bảo tính nhất quán trạng thái máy trạm không bị đảo lộn do gói tin cũ đến muộn.

> **📌 BẪY LỖI CẦN TRÁNH TRONG LG-B-014**
> Không kiểm tra số thế hệ, khiến gói tin nhịp tim cũ từ phiên trước làm sống lại một máy trạm vừa được ghi

> nhận là đã tắt.

5. Tiêu chuẩn hoàn thành 11 mục (Definition of Done — DoD):
• [ ] 1. Hợp đồng giao thức & Wire Envelope chuẩn UTF-8 JSON
• [ ] 2. Lập trình bất đồng bộ Async/Await không chặn luồng mạng
• [ ] 3. Bắt ngoại lệ mạng an toàn (EOF, SocketException, Timeout)
• [ ] 4. Giải phóng tài nguyên sạch sẽ (Dispose / Using / CancellationToken)
• [ ] 5. An toàn dữ liệu (Khử trùng lặp lô / Khóa lạc quan OCC / Transaction)
• [ ] 6. Unit Test / Test Harness chạy đạt 100% không có cảnh báo
• [ ] 7. Tích hợp chéo thành công với module của đồng đội liên quan
• [ ] 8. Kiểm thử thực tế trên mạng LAN nhiều máy tính
• [ ] 9. Đồng bộ tài liệu thiết kế & mã nguồn khớp hoàn toàn
• [ ] 10. Thu thập đầy đủ hồ sơ minh chứng (Log, Screenshot, Pcap)
• [ ] 11. Sẵn sàng trả lời trôi chảy câu hỏi vấn đáp (Viva) liên quan

6. Kịch bản kiểm thử & Hồ sơ minh chứng:
• Mã bài test: TEST-B14
• Phương pháp kiểm thử: Mô phỏng gửi xen kẽ các bản tin thế hệ cũ (gen 1) sau khi đã thiết lập thế hệ mới (gen 2)
• Tiêu chuẩn nghiệm thu đạt: Central tự động hủy bỏ các bản tin có thế hệ thấp hơn thế hệ hiện tại mà không gây xáo trộn trạng thái
• Hồ sơ minh chứng lưu tại: Session fencing log, test report loại bỏ bản tin cũ

7. Tình huống lỗi tiềm ẩn & Kế hoạch ứng phó: Gói tin cũ bị trễ mạng đến sau làm sai lệch trạng thái mới (Out-of-order packets); session generation ngăn chặn triệt để

8. Câu hỏi vấn đáp (Viva) & Chiến lược trả lời:
• Chủ đề câu hỏi: Kỹ thuật Session Generation / Fencing Token trong hệ thống mạng phân tán
• Câu trả lời kỳ vọng: Gán số thứ tự thế hệ tăng dần; mọi thao tác từ thế hệ cũ hơn đều bị từ chối để chống lỗi ghi đè dữ liệu quá hạn
• Câu hỏi mở rộng / Bẫy của giảng viên: Mối quan hệ giữa Session Generation và khóa khử trùng lặp Dedup Key?

[HƯỚNG DẪN ĐỀ XUẤT]: *Tham khảo mã nguồn mẫu chuẩn và tài liệu kỹ thuật tại thư mục Architecture. Khi hiện thực, luôn đặt an toàn luồng và xử lý hủy tác vụ (CancellationToken) lên hàng đầu.*

Nhiệm vụ 15: [LG-B-015] — Collector loss recovery và offline suppression

Mã & Tên nhiệm vụ: [LG-B-015] Collector loss recovery và offline suppression	**Tuần & Ưu tiên:** Tuần W4 (P1) | P1
Khối lượng công việc: Cơ bản: 5.0h | Dự phòng: 0.5h | Tổng: 5.5h	**Phân loại tải:** P1 Extension

Trách nhiệm thực hiện: Phụ trách: SV B | Dự phòng: SV A | Duyệt: SV A	**Phân hệ & Thư mục:** P1.Recovery (RuleEngine)
Sản phẩm bàn giao: CollectorFailoverHandler.cs	**Hợp đồng giao diện:** Grace period 45s cho các Agent thuộc node Collector vừa mất kết nối

1. Mục đích kỹ thuật & Tại sao tồn tại: Nếu một Collector đại diện cho cả phòng máy bị rớt mạng, hàng chục máy trạm sẽ đồng loạt ngưng gửi tín hiệu. Central không được vội vàng báo sập toàn bộ phòng máy gây hoảng loạn, mà phải áp dụng thời gian chờ (Grace Period) để Collector tự hồi phục.

2. Kiến thức nền tảng cần nắm vững: Thời gian ân hạn (Grace Period), ức chế cảnh báo dây chuyền (Alert Suppression), phát hiện sự cố cấp cụm.

3. Dữ liệu đầu vào & Đầu ra kỹ thuật: Đầu vào: Sự kiện ngắt kết nối Socket của Collector và danh sách các máy trạm thuộc phạm vi quản lý của nút đó.
Đầu ra: CollectorFailoverHandler.cs kích hoạt cơ chế chờ 45 giây trước khi phát sinh cảnh báo OFFLINE hàng loạt.

4. Các bước triển khai chi tiết:
1. Khi phát hiện kết nối Collector bị đứt, ghi nhận thời điểm đứt và gán trạng thái UNCERTAIN cho các máy trạm thuộc Collector đó.
2. Thiết lập bộ đếm thời gian ân hạn 45 giây, tạm dừng kích hoạt sự cố DEVICE_OFFLINE cho từng máy riêng lẻ.
3. Nếu Collector kết nối lại trong vòng 45 giây: Đồng bộ lại trạng thái và hủy bỏ lệnh báo sập hàng loạt.
4. Nếu quá 45 giây vẫn không thấy Collector: Báo động sự cố COLLECTOR_DOWN cấp độ cao và đánh dấu các máy trạm là OFFLINE.
5. Báo cáo trực quan trạng thái ân hạn này lên giao diện của C.

> 📌 **BẪY LỖI CẦN TRÁNH TRONG LG-B-015**
> Bắn ra 40 cảnh báo OFFLINE cùng một giây ngay khi đứt cáp Collector, gây tràn ngập màn hình Dashboard và nghẽn bộ nhớ gửi thông báo.

5. Tiêu chuẩn hoàn thành 11 mục (Definition of Done — DoD):
• [ ] 1. Hợp đồng giao thức & Wire Envelope chuẩn UTF-8 JSON
• [ ] 2. Lập trình bất đồng bộ Async/Await không chặn luồng mạng
• [ ] 3. Bắt ngoại lệ mạng an toàn (EOF, SocketException, Timeout)
• [ ] 4. Giải phóng tài nguyên sạch sẽ (Dispose / Using / CancellationToken)
• [ ] 5. An toàn dữ liệu (Khử trùng lặp lô / Khóa lạc quan OCC / Transaction)
• [ ] 6. Unit Test / Test Harness chạy đạt 100% không có cảnh báo
• [ ] 7. Tích hợp chéo thành công với module của đồng đội liên quan
• [ ] 8. Kiểm thử thực tế trên mạng LAN nhiều máy tính
• [ ] 9. Đồng bộ tài liệu thiết kế & mã nguồn khớp hoàn toàn
• [ ] 10. Thu thập đầy đủ hồ sơ minh chứng (Log, Screenshot, Pcap)
• [ ] 11. Sẵn sàng trả lời trôi chảy câu hỏi vấn đáp (Viva) liên quan

6. Kịch bản kiểm thử & Hồ sơ minh chứng:
• Mã bài test: TEST-B15
• Phương pháp kiểm thử: Ngắt kết nối Collector 01; quan sát danh sách sự cố trên Admin trong 45s đầu
• Tiêu chuẩn nghiệm thu đạt: Không kích hoạt bão cảnh báo Offline hàng loạt; chỉ báo trạng thái chuyển vùng; nếu quá 45s chưa kết nối lại mới báo Offline
• Hồ sơ minh chứng lưu tại: Collector loss trace log, suppression timeline capture

7. Tình huống lỗi tiềm ẩn & Kế hoạch ứng phó: Bão cảnh báo giả làm tê liệt đội ngũ hỗ trợ; trì hoãn thông minh (Suppression) giúp hệ thống ổn định

8. Câu hỏi vấn đáp (Viva) & Chiến lược trả lời:
• Chủ đề câu hỏi: Kỹ thuật hoãn cảnh báo (Alert Suppression) khi xảy ra sự cố hạ tầng tập trung
• Câu trả lời kỳ vọng: Khi một node trung chuyển gặp sự cố, nguyên nhân là ở hạ tầng mạng chứ không phải toàn bộ máy con bị hỏng; cần hoãn báo lỗi máy trạm
• Câu hỏi mở rộng / Bẫy của giảng viên: Làm thế nào phân biệt giữa 1 Collector chết và đứt cáp toàn bộ phòng máy?

[HƯỚNG DẪN ĐỀ XUẤT]: *Tham khảo mã nguồn mẫu chuẩn và tài liệu kỹ thuật tại thư mục RuleEngine. Khi hiện thực, luôn đặt an toàn luồng và xử lý hủy tác vụ (CancellationToken) lên hàng đầu.*

Nhiệm vụ 16: [LG-B-016] — Tune SQL index và ingest budgets

Mã & Tên nhiệm vụ: [LG-B-016] Tune SQL index và ingest budgets	**Tuần & Ưu tiên:** Tuần W4 (P0) | P0/Stability
Khối lượng công việc: Cơ bản: 3.0h | Dự phòng: 0.5h | Tổng: 3.5h	**Phân loại tải:** Test/Harness
Trách nhiệm thực hiện: Phụ trách: SV B | Dự phòng: SV A | Duyệt: SV A	**Phân hệ & Thư mục:** Central.Tuning (Testing)
Sản phẩm bàn giao: DbPerformanceAudit.md & OptimizedIndexes.sql	**Hợp đồng giao diện:** Ingest budget: latency < 50ms cho batch 50 records

1. Mục đích kỹ thuật & Tại sao tồn tại: Để hệ thống vận hành êm ái khi phòng máy đầy tải, các câu truy vấn nạp số liệu và quét cảnh báo phải có thời gian thực thi dưới 50ms. Cần tinh chỉnh chỉ mục (Indexes) và ngân sách thời gian để không làm nghẽn CSDL.

2. Kiến thức nền tảng cần nắm vững: Tối ưu hóa chỉ mục SQL (Clustered vs Non-Clustered Index), kế hoạch thực thi truy vấn (Execution Plan), ngân sách hiệu năng (Performance Budget).

3. Dữ liệu đầu vào & Đầu ra kỹ thuật: Đầu vào: Dữ liệu đo đạc thực tế khi nạp 50 bản ghi/giây và kế hoạch truy vấn từ SQL Server Management Studio (SSMS).
Đầu ra: Báo cáo DbPerformanceAudit.md và tệp script OptimizedIndexes.sql bổ sung chỉ mục tối ưu.

4. Các bước triển khai chi tiết:
1. Phân tích kế hoạch thực thi của các câu truy vấn trọng yếu: Ingest, Liveness Check, Incident Query.

2. Thêm chỉ mục bao phủ (Covering Index) chứa cột Include để loại bỏ thao tác Key Lookup tốn kém.
3. Kiểm tra ngân sách: Thao tác ghi nhận một lô 50 bản ghi phải hoàn thành trong dưới 50ms.
4. Tinh chỉnh cấu hình kích thước cấp phát tệp dữ liệu MDF và nhật ký LDF để tránh phân mảnh đĩa.
5. Viết tài liệu đánh giá hiệu năng đính kèm ảnh chụp Execution Plan làm bằng chứng nghiệm thu.

> **📌 BẪY LỖI CẦN TRÁNH TRONG LG-B-016**
> Tạo quá nhiều chỉ mục dư thừa trên các bảng thường xuyên ghi chép, làm chậm tốc độ INSERT và phình to dung lượng ổ cứng.

5. Tiêu chuẩn hoàn thành 11 mục (Definition of Done — DoD):
• [ ] 1. Hợp đồng giao thức & Wire Envelope chuẩn UTF-8 JSON
• [ ] 2. Lập trình bất đồng bộ Async/Await không chặn luồng mạng
• [ ] 3. Bắt ngoại lệ mạng an toàn (EOF, SocketException, Timeout)
• [ ] 4. Giải phóng tài nguyên sạch sẽ (Dispose / Using / CancellationToken)
• [ ] 5. An toàn dữ liệu (Khử trùng lặp lô / Khóa lạc quan OCC / Transaction)
• [ ] 6. Unit Test / Test Harness chạy đạt 100% không có cảnh báo
• [ ] 7. Tích hợp chéo thành công với module của đồng đội liên quan
• [ ] 8. Kiểm thử thực tế trên mạng LAN nhiều máy tính
• [ ] 9. Đồng bộ tài liệu thiết kế & mã nguồn khớp hoàn toàn
• [ ] 10. Thu thập đầy đủ hồ sơ minh chứng (Log, Screenshot, Pcap)
• [ ] 11. Sẵn sàng trả lời trôi chảy câu hỏi vấn đáp (Viva) liên quan

6. Kịch bản kiểm thử & Hồ sơ minh chứng:
• Mã bài test: TEST-B16
• Phương pháp kiểm thử: Đo đạc execution plan và thời gian insert liên tục 10,000 dòng telemetry
• Tiêu chuẩn nghiệm thu đạt: Thời gian ghi mẻ đạt p95 < 30ms; không xảy ra hiện tượng thiếu bộ đệm hay lock escalation
• Hồ sơ minh chứng lưu tại: SQL Server Execution Plan, báo cáo đo đạc hiệu năng

7. Tình huống lỗi tiềm ẩn & Kế hoạch ứng phó: Chỉ mục quá nhiều làm chậm thao tác ghi; cân bằng giữa tốc độ ghi của Ingest và tốc độ đọc của Admin

8. Câu hỏi vấn đáp (Viva) & Chiến lược trả lời:
• Chủ đề câu hỏi: Chiến lược tối ưu hóa chỉ mục CSDL cho hệ thống chuyên ghi (Write-heavy)
• Câu trả lời kỳ vọng: Tránh đánh index trên các cột có độ biến thiên cao không cần thiết; dùng Clustered Index phù hợp và duy trì bảo trì chỉ mục
• Câu hỏi mở rộng / Bẫy của giảng viên: Tại sao khóa chính tự tăng (Identity) thường tối ưu cho bảng ghi liên tục hơn GUID ngẫu nhiên?

[HƯỚNG DẪN ĐỀ XUẤT]: *Tham khảo mã nguồn mẫu chuẩn và tài liệu kỹ thuật tại thư mục Testing. Khi hiện thực, luôn đặt an toàn luồng và xử lý hủy tác vụ (CancellationToken) lên hàng đầu.*

Nhiệm vụ 17: [LG-B-017] — Parser/auth/incident regression

Mã & Tên nhiệm vụ: [LG-B-017] Parser/auth/incident regression	**Tuần & Ưu tiên:** Tuần W4 (P0) | P0/Stability
Khối lượng công việc: Cơ bản: 3.0h | Dự phòng: 0.5h | Tổng: 3.5h	**Phân loại tải:** Test/Harness
Trách nhiệm thực hiện: Phụ trách: SV B | Dự phòng: SV C | Duyệt: SV C	**Phân hệ & Thư mục:** Central.Regression (Testing)
Sản phẩm bàn giao: RegressionSuiteResults.md	**Hợp đồng giao diện:** Test suite tự động bao phủ các kịch bản quan trọng

1. Mục đích kỹ thuật & Tại sao tồn tại: Trước khi đóng gói phát hành, phải đảm bảo các bản sửa lỗi không làm hỏng những tính năng đã chạy tốt từ các tuần trước. Cần có bộ kiểm thử hồi quy (Regression Test Suite) tự động chạy lại toàn bộ các ca kiểm thử cốt lõi.

2. Kiến thức nền tảng cần nắm vững: Kiểm thử hồi quy tự động, xUnit / NUnit trong C#, mô phỏng luồng dữ liệu liên tục.

3. Dữ liệu đầu vào & Đầu ra kỹ thuật: Đầu vào: Các ca kiểm thử từ Tuần 1 đến Tuần 4 bao gồm: giải mã gói tin, xác thực PBKDF2, tính toán chuỗi quy tắc, xử lý xung đột OCC.
Đầu ra: Báo cáo RegressionSuiteResults.md ghi nhận 100% ca kiểm thử chạy đạt.

4. Các bước triển khai chi tiết:
1. Gom toàn bộ các bài kiểm thử đơn vị và tích hợp vào một kịch bản chạy tự động (dotnet test).
2. Kiểm tra lại việc bảo mật mật khẩu PBKDF2 với các trường hợp mật khẩu đúng, sai, chuỗi rỗng.
3. Kiểm tra lại tính nguyên tử của giao dịch sự cố khi giả lập đứt kết nối giữa chừng.
4. Chạy lại bài kiểm tra khử trùng lặp lô số liệu với 100 gói tin lặp liên tiếp.
5. Xuất kết quả kiểm thử ra tệp báo cáo chuẩn để làm hồ sơ minh chứng nghiệm thu đồ án.

> **📌 BẪY LỖI CẦN TRÁNH TRONG LG-B-017**
> Sửa code tối ưu ở Tuần 4 nhưng không chạy lại bài test bảo mật của Tuần 1, làm mất đi tính năng chống timing-attack mà không phát hiện ra.

5. Tiêu chuẩn hoàn thành 11 mục (Definition of Done — DoD):
- [ ] 1. Hợp đồng giao thức & Wire Envelope chuẩn UTF-8 JSON
- [ ] 2. Lập trình bất đồng bộ Async/Await không chặn luồng mạng
- [ ] 3. Bắt ngoại lệ mạng an toàn (EOF, SocketException, Timeout)
- [ ] 4. Giải phóng tài nguyên sạch sẽ (Dispose / Using / CancellationToken)
- [ ] 5. An toàn dữ liệu (Khử trùng lặp lô / Khóa lạc quan OCC / Transaction)
- [ ] 6. Unit Test / Test Harness chạy đạt 100% không có cảnh báo
- [ ] 7. Tích hợp chéo thành công với module của đồng đội liên quan
- [ ] 8. Kiểm thử thực tế trên mạng LAN nhiều máy tính
- [ ] 9. Đồng bộ tài liệu thiết kế & mã nguồn khớp hoàn toàn

• [ ] 10. Thu thập đầy đủ hồ sơ minh chứng (Log, Screenshot, Pcap)
• [ ] 11. Sẵn sàng trả lời trôi chảy câu hỏi vấn đáp (Viva) liên quan

6. Kịch bản kiểm thử & Hồ sơ minh chứng:
• Mã bài test: TEST-B17
• Phương pháp kiểm thử: Thực thi toàn bộ kịch bản kiểm thử tích hợp
• Tiêu chuẩn nghiệm thu đạt: 100% kịch bản kiểm thử pass; không phát sinh lỗi hồi quy sau các chỉnh sửa hiệu năng
• Hồ sơ minh chứng lưu tại: Báo cáo kết quả kiểm thử hồi quy, test run console log

7. Tình huống lỗi tiềm ẩn & Kế hoạch ứng phó: Sửa chức năng này làm hỏng chức năng khác; bộ test tự động phát hiện sớm sai sót

8. Câu hỏi vấn đáp (Viva) & Chiến lược trả lời:
• Chủ đề câu hỏi: Tầm quan trọng của kiểm thử hồi quy (Regression Testing) trước khi đóng băng mã nguồn
• Câu trả lời kỳ vọng: Đảm bảo các thay đổi tối ưu mã hoặc sửa lỗi không vô tình phá vỡ các hợp đồng giao tiếp và tính toàn vẹn đã được kiểm chứng
• Câu hỏi mở rộng / Bẫy của giảng viên: Làm thế nào thiết kế kiểm thử để chạy độc lập không phụ thuộc vào thứ tự thực thi?

[HƯỚNG DẪN ĐỀ XUẤT]: *Tham khảo mã nguồn mẫu chuẩn và tài liệu kỹ thuật tại thư mục Testing. Khi hiện thực, luôn đặt an toàn luồng và xử lý hủy tác vụ (CancellationToken) lên hàng đầu.*

Nhiệm vụ 18: [LG-B-018] — DB restore và crash consistency proof

Mã & Tên nhiệm vụ: [LG-B-018] DB restore và crash consistency proof	**Tuần & Ưu tiên:** Tuần W5 (P0) | P0/Release
Khối lượng công việc: Cơ bản: 2.0h | Dự phòng: 0.5h | Tổng: 2.5h	**Phân loại tải:** P0 Core
Trách nhiệm thực hiện: Phụ trách: SV B | Dự phòng: SV A | Duyệt: SV A	**Phân hệ & Thư mục:** Release.DbProof (Storage)
Sản phẩm bàn giao: DbRestoreProof.md & Backup.bak	**Hợp đồng giao diện:** SQL Server Full Backup & Restore, DBCC CHECKDB

1. Mục đích kỹ thuật & Tại sao tồn tại: Hệ thống giám sát không được phép mất dữ liệu khi máy chủ gặp sự cố mất điện hoặc lỗi phần cứng đột ngột. Cần có quy trình sao lưu (Backup) định kỳ và bằng chứng chứng minh CSDL có thể phục hồi toàn vẹn (Crash Consistency).

2. Kiến thức nền tảng cần nắm vững: Lệnh sao lưu và phục hồi CSDL (BACKUP/RESTORE DATABASE), kiểm tra toàn vẹn vật lý (DBCC CHECKDB), mức độ phục hồi (Full / Simple Recovery Model).

3. Dữ liệu đầu vào & Đầu ra kỹ thuật: Đầu vào: CSDL LabGuard đang hoạt động và kịch bản giả lập ngắt điện đột ngột.

Đầu ra: Bản sao lưu an toàn Backup.bak và tài liệu DbRestoreProof.md kèm kết quả chạy lệnh DBCC CHECKDB không lỗi.

4. Các bước triển khai chi tiết:

1. Soạn thảo lệnh BACKUP DATABASE LabGuard TO DISK = '...' WITH INIT, COMPRESSION.
2. Tiến hành giả lập sự cố: tắt nóng tiến trình SQL Server hoặc ngắt ứng dụng giữa lúc đang ghi số liệu.
3. Khởi động lại dịch vụ và thực thi câu lệnh DBCC CHECKDB ('LabGuard') WITH NO_INFOMSGS kiểm tra tính toàn vẹn trang dữ liệu.
4. Thực hiện kịch bản phục hồi cơ sở dữ liệu từ tệp Backup.bak sang một CSDL tạm thời để kiểm tra dữ liệu.
5. Ghi lại toàn bộ log phục hồi và ảnh chụp màn hình vào báo cáo chứng minh khả năng phục hồi sau thảm họa.

> **📌 BẪY LỖI CẦN TRÁNH TRONG LG-B-018**
> Chỉ tạo file backup mà không bao giờ chạy thử lệnh RESTORE, đến khi có sự cố thật mới phát hiện file backup bị lỗi hoặc thiếu file nhật ký giao dịch.

5. Tiêu chuẩn hoàn thành 11 mục (Definition of Done — DoD):

- [ ] 1. Hợp đồng giao thức & Wire Envelope chuẩn UTF-8 JSON
- [ ] 2. Lập trình bất đồng bộ Async/Await không chặn luồng mạng
- [ ] 3. Bắt ngoại lệ mạng an toàn (EOF, SocketException, Timeout)
- [ ] 4. Giải phóng tài nguyên sạch sẽ (Dispose / Using / CancellationToken)
- [ ] 5. An toàn dữ liệu (Khử trùng lặp lô / Khóa lạc quan OCC / Transaction)
- [ ] 6. Unit Test / Test Harness chạy đạt 100% không có cảnh báo
- [ ] 7. Tích hợp chéo thành công với module của đồng đội liên quan
- [ ] 8. Kiểm thử thực tế trên mạng LAN nhiều máy tính
- [ ] 9. Đồng bộ tài liệu thiết kế & mã nguồn khớp hoàn toàn
- [ ] 10. Thu thập đầy đủ hồ sơ minh chứng (Log, Screenshot, Pcap)
- [ ] 11. Sẵn sàng trả lời trôi chảy câu hỏi vấn đáp (Viva) liên quan

6. Kịch bản kiểm thử & Hồ sơ minh chứng:

- Mã bài test: TEST-B18
- Phương pháp kiểm thử: Backup CSDL; kill tiến trình SQL khi đang ghi; restore từ bản backup và kiểm tra toàn vẹn
- Tiêu chuẩn nghiệm thu đạt: DBCC CHECKDB trả về 0 lỗi; dữ liệu sự cố và người dùng được phục hồi nguyên vẹn
- Hồ sơ minh chứng lưu tại: File backup CSDL .bak, output lệnh DBCC CHECKDB

7. Tình huống lỗi tiềm ẩn & Kế hoạch ứng phó: File cơ sở dữ liệu bị hỏng vật lý; cấu hình chế độ ghi đồng bộ và kiểm tra trang lưu trữ

8. Câu hỏi vấn đáp (Viva) & Chiến lược trả lời:

- Chủ đề câu hỏi: Cơ chế Write-Ahead Logging (WAL) và tính bền vững (Durability) trong CSDL
- Câu trả lời kỳ vọng: Mọi thay đổi phải được ghi vào Transaction Log trên đĩa trước khi ghi vào file dữ liệu

chính để bảo đảm phục hồi khi mất điện
• Câu hỏi mở rộng / Bẫy của giảng viên: Sự khác biệt giữa sao lưu toàn phần (Full Backup) và sao lưu nhật ký giao dịch (Log Backup)?

[HƯỚNG DẪN ĐỀ XUẤT]: *Tham khảo mã nguồn mẫu chuẩn và tài liệu kỹ thuật tại thư mục Storage. Khi hiện thực, luôn đặt an toàn luồng và xử lý hủy tác vụ (CancellationToken) lên hàng đầu.*

Nhiệm vụ 19: [LG-B-019] — Audit tài liệu architecture/protocol/database

Mã & Tên nhiệm vụ: [LG-B-019] Audit tài liệu architecture/protocol/database	**Tuần & Ưu tiên:** Tuần W5 (P0) | P0/Docs
Khối lượng công việc: Cơ bản: 2.5h | Dự phòng: 0.0h | Tổng: 2.5h	**Phân loại tải:** Tích hợp
Trách nhiệm thực hiện: Phụ trách: SV B | Dự phòng: SV A | Duyệt: SV A	**Phân hệ & Thư mục:** Release.Audit (Integration)
Sản phẩm bàn giao: AuditChecklist_B.md & Đồng bộ hồ sơ thiết kế	**Hợp đồng giao diện:** Toàn bộ tài liệu thiết kế và mã nguồn Central/CSDL

1. Mục đích kỹ thuật & Tại sao tồn tại: Mã nguồn và tài liệu thiết kế phải hoàn toàn khớp nhau trước khi nộp đồ án. Bất kỳ sự sai lệch nào về tên bảng, tên trường hay danh mục mã lỗi giữa code và tài liệu đều sẽ bị hội đồng trừ điểm rất nặng.

2. Kiến thức nền tảng cần nắm vững: Kỹ năng rà soát tài liệu kỹ thuật, đối chiếu mã nguồn với lược đồ CSDL và bảng đặc tả giao thức.

3. Dữ liệu đầu vào & Đầu ra kỹ thuật: Đầu vào: Mã nguồn C# phần Central Server, các tệp SQL migration và các văn bản thiết kế (03_PROTOCOL_SPEC.md, CONTRACT_MATRIX.md).
Đầu ra: Tài liệu AuditChecklist_B.md xác nhận 100% trường dữ liệu, mã lỗi và luồng xử lý đã được đồng bộ hoàn toàn.

4. Các bước triển khai chi tiết:
1. Đối chiếu danh sách cột của 8 bảng CSDL trong code DTO với file schema V1__Initial_Schema.sql.
2. Kiểm tra danh mục 69 bản tin xem phía Server đã xử lý đầy đủ các bản tin thuộc phạm vi P0 cốt lõi chưa.
3. Kiểm tra các mã lỗi trả về (ERROR_RESPONSE, CONCURRENCY_CONFLICT, AUTH_FAILED) có đúng chuẩn chuỗi đã cam kết không.
4. Rà soát lại các giá trị cấu hình mặc định (cổng 5000, dung sai 20s liveness, chu kỳ dọn dẹp 48h).
5. Ký duyệt mục kiểm tra của B và phối hợp cùng A, C để chốt biên bản nghiệm thu tài liệu toàn dự án.

> **📌 BẪY LỖI CẦN TRÁNH TRONG LG-B-019**
> Đổi tên cột trong CSDL trong lúc code nhưng không cập nhật lại tài liệu đặc tả, khiến người chấm thi không thể đối chiếu.

5. Tiêu chuẩn hoàn thành 11 mục (Definition of Done — DoD):
• [ ] 1. Hợp đồng giao thức & Wire Envelope chuẩn UTF-8 JSON
• [ ] 2. Lập trình bất đồng bộ Async/Await không chặn luồng mạng
• [ ] 3. Bắt ngoại lệ mạng an toàn (EOF, SocketException, Timeout)
• [ ] 4. Giải phóng tài nguyên sạch sẽ (Dispose / Using / CancellationToken)
• [ ] 5. An toàn dữ liệu (Khử trùng lặp lô / Khóa lạc quan OCC / Transaction)
• [ ] 6. Unit Test / Test Harness chạy đạt 100% không có cảnh báo
• [ ] 7. Tích hợp chéo thành công với module của đồng đội liên quan
• [ ] 8. Kiểm thử thực tế trên mạng LAN nhiều máy tính
• [ ] 9. Đồng bộ tài liệu thiết kế & mã nguồn khớp hoàn toàn
• [ ] 10. Thu thập đầy đủ hồ sơ minh chứng (Log, Screenshot, Pcap)
• [ ] 11. Sẵn sàng trả lời trôi chảy câu hỏi vấn đáp (Viva) liên quan

6. Kịch bản kiểm thử & Hồ sơ minh chứng:
• Mã bài test: TEST-B19
• Phương pháp kiểm thử: Rà soát từng bảng CSDL, từng trường DTO và so khớp với file CONTRACT_MATRIX.md
• Tiêu chuẩn nghiệm thu đạt: 100% tên trường, kiểu dữ liệu và mô tả kỹ thuật thống nhất tuyệt đối giữa tài liệu và mã nguồn
• Hồ sơ minh chứng lưu tại: Bảng đối soát kiến trúc hoàn chỉnh, biên bản review

7. Tình huống lỗi tiềm ẩn & Kế hoạch ứng phó: Tài liệu không đồng bộ với code thực tế; cập nhật trực tiếp tài liệu theo đúng hiện trạng mã nguồn chuẩn

8. Câu hỏi vấn đáp (Viva) & Chiến lược trả lời:
• Chủ đề câu hỏi: Tầm quan trọng của tính đồng bộ giữa tài liệu kiến trúc và mã nguồn
• Câu trả lời kỳ vọng: Tài liệu chuẩn xác giúp bảo trì hệ thống hiệu quả và là căn cứ pháp lý kỹ thuật khi nghiệm thu đồ án với hội đồng
• Câu hỏi mở rộng / Bẫy của giảng viên: Làm thế nào duy trì tính cập nhật của tài liệu khi dự án thay đổi liên tục?

[HƯỚNG DẪN ĐỀ XUẤT]: *Tham khảo mã nguồn mẫu chuẩn và tài liệu kỹ thuật tại thư mục Integration. Khi hiện thực, luôn đặt an toàn luồng và xử lý hủy tác vụ (CancellationToken) lên hàng đầu.*

Nhiệm vụ 20: [LG-B-020] — Viva B và bàn giao backup

Mã & Tên nhiệm vụ: [LG-B-020] Viva B và bàn giao backup	**Tuần & Ưu tiên:** Tuần W5 (P0) | P0/Viva
Khối lượng công việc: Cơ bản: 5.0h | Dự phòng: 0.0h | Tổng: 5.0h	**Phân loại tải:** Viva
Trách nhiệm thực hiện: Phụ trách: SV B | Dự phòng: SV C | Duyệt: SV C	**Phân hệ & Thư mục:** Viva (Viva)

Sản phẩm bàn giao: HANDBOOK_B.md & Viva rehearsal log	**Hợp đồng giao diện:** 30 câu hỏi viva chuyên sâu phần Server, Database & Rule Engine

1. Mục đích kỹ thuật & Tại sao tồn tại: Buổi vấn đáp đồ án yêu cầu sinh viên B phải bảo vệ được các quyết định kỹ thuật sâu về CSDL, an toàn giao dịch, kiểm soát tranh chấp và tối ưu hóa hệ thống. Cần luyện tập nhuần nhuyễn 30 câu hỏi để đạt điểm tối đa.

2. Kiến thức nền tảng cần nắm vững: Toàn bộ kiến thức lý thuyết và thực hành của phần Server, CSDL, giao thức mạng và cơ chế chống xung đột.

3. Dữ liệu đầu vào & Đầu ra kỹ thuật: Đầu vào: Bộ 30 câu hỏi vấn đáp chuyên sâu dành cho vai B trong sổ tay HANDBOOK_B.md.
Đầu ra: Hồ sơ chuẩn bị vấn đáp hoàn chỉnh kèm nhật ký diễn tập trả lời trôi chảy (Viva rehearsal log).

4. Các bước triển khai chi tiết:
1. Ôn tập kỹ nguyên lý khóa lạc quan (OCC) và giải thích tại sao dùng rowversion thay vì khóa bi quan (SELECT FOR UPDATE).
2. Nắm chắc thuật toán băm PBKDF2, giải thích ý nghĩa của salt, số vòng lặp và hàm FixedTimeEquals.
3. Chuẩn bị sẵn đoạn code minh chứng cho cơ chế khử trùng lặp (Idempotent Ingestion) qua bảng ProcessedBatches.
4. Diễn tập trả lời các câu hỏi tình huống: 'Điều gì xảy ra nếu mạng chập chờn làm gửi lại gói tin?', 'Tại sao cần Filtered Index?'.
5. Đóng gói toàn bộ mã nguồn, dữ liệu mẫu và file backup vào USB dự phòng trước ngày bảo vệ.

> **📌 BẪY LỖI CẦN TRÁNH TRONG LG-B-020**
> Học vẹt câu trả lời mà không chỉ được vị trí dòng code cụ thể trong project chứng minh cho câu trả lời của mình.

5. Tiêu chuẩn hoàn thành 11 mục (Definition of Done — DoD):
• [ ] 1. Hợp đồng giao thức & Wire Envelope chuẩn UTF-8 JSON
• [ ] 2. Lập trình bất đồng bộ Async/Await không chặn luồng mạng
• [ ] 3. Bắt ngoại lệ mạng an toàn (EOF, SocketException, Timeout)
• [ ] 4. Giải phóng tài nguyên sạch sẽ (Dispose / Using / CancellationToken)
• [ ] 5. An toàn dữ liệu (Khử trùng lặp lô / Khóa lạc quan OCC / Transaction)
• [ ] 6. Unit Test / Test Harness chạy đạt 100% không có cảnh báo
• [ ] 7. Tích hợp chéo thành công với module của đồng đội liên quan
• [ ] 8. Kiểm thử thực tế trên mạng LAN nhiều máy tính
• [ ] 9. Đồng bộ tài liệu thiết kế & mã nguồn khớp hoàn toàn
• [ ] 10. Thu thập đầy đủ hồ sơ minh chứng (Log, Screenshot, Pcap)
• [ ] 11. Sẵn sàng trả lời trôi chảy câu hỏi vấn đáp (Viva) liên quan

6. Kịch bản kiểm thử & Hồ sơ minh chứng:
• Mã bài test: TEST-B20

• Phương pháp kiểm thử: Vấn đáp chéo giả định với thành viên A và C chấm điểm độc lập
• Tiêu chuẩn nghiệm thu đạt: Giải thích xuất sắc các khái niệm giao dịch, khóa, bất đồng bộ và tự tin thao tác SQL trực tiếp
• Hồ sơ minh chứng lưu tại: HANDBOOK_B.md, biên bản diễn tập vấn đáp nội bộ

7. Tình huống lỗi tiềm ẩn & Kế hoạch ứng phó: Nhầm lẫn giữa cơ chế khóa CSDL và khóa trong bộ nhớ; làm rõ ranh giới trong handbook

8. Câu hỏi vấn đáp (Viva) & Chiến lược trả lời:
• Chủ đề câu hỏi: Tổng quan kiến trúc máy chủ xử lý dữ liệu tập trung của Lane B
• Câu trả lời kỳ vọng: Trình bày toàn diện luồng nhận dữ liệu, xác thực an ninh, xử lý quy tắc sự cố và bảo toàn dữ liệu bằng CSDL quan hệ
• Câu hỏi mở rộng / Bẫy của giảng viên: Thử thách kỹ thuật lớn nhất gặp phải khi xây dựng Central Server?

[HƯỚNG DẪN ĐỀ XUẤT]: *Tham khảo mã nguồn mẫu chuẩn và tài liệu kỹ thuật tại thư mục Viva. Khi hiện thực, luôn đặt an toàn luồng và xử lý hủy tác vụ (CancellationToken) lên hàng đầu.*

PHẦN 7: SỔ TAY CHI TIẾT 20 NHIỆM VỤ CỦA SINH VIÊN C (DASHBOARD & UI CLIENT)

Sinh viên C chịu trách nhiệm về ứng dụng WinForms Admin Dashboard, khớp cặp RPC Request/Response, đồng bộ Snapshot và sự kiện, điều phối luồng UI an toàn (Control.Invoke marshal), lưới phòng máy High-DPI và ứng dụng Collector dồn kênh (P1). Dưới đây là chi tiết 20 nhiệm vụ từ LG-C-001 đến LG-C-020:

Nhiệm vụ 01: [LG-C-001] — WinForms UI shell wireframe và mock DTO

Mã & Tên nhiệm vụ: [LG-C-001] WinForms UI shell wireframe và mock DTO	**Tuần & Ưu tiên:** Tuần W1 (P0) | P0
Khối lượng công việc: Cơ bản: 2.5h | Dự phòng: 0.5h | Tổng: 3.0h	**Phân loại tải:** P0 Core
Trách nhiệm thực hiện: Phụ trách: SV C | Dự phòng: SV B | Duyệt: SV B	**Phân hệ & Thư mục:** Admin.UI (UI)
Sản phẩm bàn giao: MainForm.cs & NavigationFrame	**Hợp đồng giao diện:** WinForms native controls, UI layout standards, Mock view models

1. Mục đích kỹ thuật & Tại sao tồn tại: Người quản trị phòng máy cần một giao diện trực quan, rõ ràng để theo dõi hàng chục máy tính cùng lúc. Cần dựng khung xương WinForms (Shell Wireframe) với mô hình dữ liệu giả lập (Mock DTO) trước khi kết nối mạng thật để kiểm tra luồng trải nghiệm người dùng.

2. Kiến thức nền tảng cần nắm vững: Lập trình WinForms cơ bản, bố cục giao diện (TableLayoutPanel, SplitContainer, FlowLayoutPanel), mô hình phân tách dữ liệu và hiển thị (DTO/View Model).

3. Dữ liệu đầu vào & Đầu ra kỹ thuật: Đầu vào: Bản thiết kế giao diện từ đề cương và danh mục các trường thông tin máy trạm/sự cố.
Đầu ra: Cửa sổ chính MainForm.cs có thanh điều hướng bên trái và khung nội dung linh hoạt (Navigation Frame).

4. Các bước triển khai chi tiết:
1. Thiết lập dự án WinForms trên .NET 9, cấu hình High-DPI PerMonitorV2 trong Program.cs.
2. Tạo MainForm với bố cục thanh điều hướng (Dashboard, Incidents, Reports, Settings) và vùng hiển thị chính.
3. Viết các lớp Mock DTO (MockDevice, MockIncident) tạo dữ liệu giả lập 20 máy trạm trong phòng máy.
4. Đảm bảo giao diện co giãn hợp lý khi thay đổi kích thước cửa sổ (Anchor, Dock).
5. Kiểm tra tính mượt mà khi chuyển đổi giữa các màn hình chức năng.

> **📌 BẪY LỖI CẦN TRÁNH TRONG LG-C-001**
> Kéo thả tọa độ cố định mà không dùng Dock/Anchor, khiến giao diện bị vỡ nát hoặc mất nút bấm khi phóng to thu nhỏ cửa sổ trên các màn hình có độ phân giải khác nhau.

5. Tiêu chuẩn hoàn thành 11 mục (Definition of Done — DoD):
• [ ] 1. Hợp đồng giao thức & Wire Envelope chuẩn UTF-8 JSON
• [ ] 2. Lập trình bất đồng bộ Async/Await không chặn luồng mạng
• [ ] 3. Bắt ngoại lệ mạng an toàn (EOF, SocketException, Timeout)
• [ ] 4. Giải phóng tài nguyên sạch sẽ (Dispose / Using / CancellationToken)
• [ ] 5. An toàn dữ liệu (Khử trùng lặp lô / Khóa lạc quan OCC / Transaction)
• [ ] 6. Unit Test / Test Harness chạy đạt 100% không có cảnh báo
• [ ] 7. Tích hợp chéo thành công với module của đồng đội liên quan
• [ ] 8. Kiểm thử thực tế trên mạng LAN nhiều máy tính
• [ ] 9. Đồng bộ tài liệu thiết kế & mã nguồn khớp hoàn toàn
• [ ] 10. Thu thập đầy đủ hồ sơ minh chứng (Log, Screenshot, Pcap)
• [ ] 11. Sẵn sàng trả lời trôi chảy câu hỏi vấn đáp (Viva) liên quan

6. Kịch bản kiểm thử & Hồ sơ minh chứng:
• Mã bài test: TEST-C01
• Phương pháp kiểm thử: Chạy ứng dụng WinForms, kiểm tra điều hướng giữa các màn hình
• Tiêu chuẩn nghiệm thu đạt: Giao diện hiển thị rõ ràng, không vỡ layout khi co giãn cửa sổ, các menu chuyển đổi mượt mà
• Hồ sơ minh chứng lưu tại: Ảnh chụp màn hình các form giao diện, file mã nguồn layout

7. Tình huống lỗi tiềm ẩn & Kế hoạch ứng phó: Giao diện bị giật (flickering); bật thuộc tính DoubleBuffered trên các UserControl

8. Câu hỏi vấn đáp (Viva) & Chiến lược trả lời:
• Chủ đề câu hỏi: Thiết kế giao diện WinForms chuẩn mực và trải nghiệm người dùng
• Câu trả lời kỳ vọng: Sử dụng tông màu sáng chuyên nghiệp, bố cục phân cấp rõ ràng và hỗ trợ đầy đủ phím tắt điều hướng
• Câu hỏi mở rộng / Bẫy của giảng viên: Tại sao đồ án lựa chọn công nghệ Windows Forms thay vì WPF hay Web?

[HƯỚNG DẪN ĐỀ XUẤT]: *Tham khảo mã nguồn mẫu chuẩn và tài liệu kỹ thuật tại thư mục UI. Khi hiện thực, luôn đặt an toàn luồng và xử lý hủy tác vụ (CancellationToken) lên hàng đầu.*

Nhiệm vụ 02: [LG-C-002] — Admin TCP/TLS RPC client connection

Mã & Tên nhiệm vụ: [LG-C-002] Admin TCP/TLS RPC client connection	**Tuần & Ưu tiên:** Tuần W1 (P0) | P0
Khối lượng công việc: Cơ bản: 3.0h | Dự phòng: 0.5h | Tổng: 3.5h	**Phân loại tải:** P0 Core
Trách nhiệm thực hiện: Phụ trách: SV C | Dự phòng: SV B | Duyệt: SV B	**Phân hệ & Thư mục:** Admin.Network (Networking)
Sản phẩm bàn giao: AdminRpcClient.cs	**Hợp đồng giao diện:** TcpClient, SslStream, FrameCodec, Duplex channel

1. Mục đích kỹ thuật & Tại sao tồn tại: Ứng dụng quản trị WinForms cần một kênh liên lạc hai chiều tin cậy tới Central Server để gửi lệnh điều khiển và nhận sự kiện thời gian thực. Kênh này phải được mã hóa bằng TLS qua SslStream và dùng chung bộ mã hóa khung (FrameCodec).

2. Kiến thức nền tảng cần nắm vững: Lớp TcpClient, SslStream (Client Authentication / RemoteCertificateValidationCallback), mô hình đọc/ghi hai chiều bất đồng bộ.

3. Dữ liệu đầu vào & Đầu ra kỹ thuật: Đầu vào: Địa chỉ IP máy chủ Central, cổng kết nối (mặc định 5000) và chứng chỉ bảo mật máy chủ.
Đầu ra: AdminRpcClient.cs quản lý vòng đời kết nối TCP/TLS và truyền nhận các gói tin WireEnvelope.

4. Các bước triển khai chi tiết:
1. Sử dụng TcpClient.ConnectAsync để thiết lập kết nối socket tới Central Server.
2. Khởi tạo SslStream bọc lấy NetworkStream, cấu hình kiểm tra chứng chỉ máy chủ an toàn.
3. Tích hợp FrameCodec để đóng gói độ dài khung 4 byte Big-Endian và giải mã dòng byte nhận được.
4. Duy trì vòng lặp đọc ngầm liên tục để đón các gói tin sự kiện phát sóng từ máy chủ.
5. Hiện thực các sự kiện ConnectionLost, Connected để thông báo lên giao diện.

> 📌 **BẪY LỖI CẦN TRÁNH TRONG LG-C-002**
> Tạo kết nối mới cho mỗi lần gửi lệnh thay vì duy trì một kết nối song công (Duplex Persistent Connection) lâu dài, gây lãng phí tài nguyên bắt tay TLS.

5. Tiêu chuẩn hoàn thành 11 mục (Definition of Done — DoD):
• [ ] 1. Hợp đồng giao thức & Wire Envelope chuẩn UTF-8 JSON
• [ ] 2. Lập trình bất đồng bộ Async/Await không chặn luồng mạng
• [ ] 3. Bắt ngoại lệ mạng an toàn (EOF, SocketException, Timeout)
• [ ] 4. Giải phóng tài nguyên sạch sẽ (Dispose / Using / CancellationToken)
• [ ] 5. An toàn dữ liệu (Khử trùng lặp lô / Khóa lạc quan OCC / Transaction)
• [ ] 6. Unit Test / Test Harness chạy đạt 100% không có cảnh báo
• [ ] 7. Tích hợp chéo thành công với module của đồng đội liên quan
• [ ] 8. Kiểm thử thực tế trên mạng LAN nhiều máy tính
• [ ] 9. Đồng bộ tài liệu thiết kế & mã nguồn khớp hoàn toàn
• [ ] 10. Thu thập đầy đủ hồ sơ minh chứng (Log, Screenshot, Pcap)
• [ ] 11. Sẵn sàng trả lời trôi chảy câu hỏi vấn đáp (Viva) liên quan

6. Kịch bản kiểm thử & Hồ sơ minh chứng:
• Mã bài test: TEST-C02
• Phương pháp kiểm thử: Kết nối tới Central Server, gửi bản tin echo và kiểm tra nhận phản hồi
• Tiêu chuẩn nghiệm thu đạt: Bắt tay TLS thành công; duy trì kết nối duplex ổn định; tự động phát hiện khi kết nối đứt
• Hồ sơ minh chứng lưu tại: Network communication trace log, Wireshark dump

7. Tình huống lỗi tiềm ẩn & Kế hoạch ứng phó: Đứt cáp mạng không nhận được thông báo; gửi định kỳ heartbeat ping tầng ứng dụng để dò kênh truyền

8. Câu hỏi vấn đáp (Viva) & Chiến lược trả lời:
• Chủ đề câu hỏi: Kênh truyền hai chiều (Duplex Connection) trên nền tảng Socket TCP
• Câu trả lời kỳ vọng: Một kết nối socket duy nhất được sử dụng đồng thời cho cả việc gửi request và nhận các event bất đồng bộ do server chủ động đẩy về
• Câu hỏi mở rộng / Bẫy của giảng viên: Làm thế nào phân biệt giữa bản tin phản hồi của Request và bản tin Event đẩy về?

[HƯỚNG DẪN ĐỀ XUẤT]: *Tham khảo mã nguồn mẫu chuẩn và tài liệu kỹ thuật tại thư mục Networking. Khi hiện thực, luôn đặt an toàn luồng và xử lý hủy tác vụ (CancellationToken) lên hàng đầu.*

Nhiệm vụ 03: [LG-C-003] — Login/Signup WinForms UI & token storage client

Mã & Tên nhiệm vụ: [LG-C-003] Login/Signup WinForms UI & token storage client	**Tuần & Ưu tiên:** Tuần W1 (P0) | P0
Khối lượng công việc: Cơ bản: 2.5h | Dự phòng: 0.5h | Tổng: 3.0h	**Phân loại tải:** P0 Core
Trách nhiệm thực hiện: Phụ trách: SV C | Dự phòng: SV B | Duyệt: SV B	**Phân hệ & Thư mục:** Admin.Auth (UI)
Sản phẩm bàn giao: LoginForm.cs & SessionStorage.cs	**Hợp đồng giao diện:** LOGIN (username, password), LOGIN_RESULT (token, role), Client RAM token store

1. Mục đích kỹ thuật & Tại sao tồn tại: Trước khi được phép xem màn hình giám sát, người quản trị phải đăng nhập danh tính. Ứng dụng phải có màn hình đăng nhập đẹp mắt, kiểm tra tính hợp lệ của trường thông tin và lưu trữ token an toàn trong bộ nhớ RAM.

2. Kiến thức nền tảng cần nắm vững: Thiết kế biểu mẫu đăng nhập (LoginForm), xử lý chuỗi mật khẩu trong WinForms (PasswordChar / SecureString), lưu trữ phiên an toàn.

3. Dữ liệu đầu vào & Đầu ra kỹ thuật: Đầu vào: Tên đăng nhập và mật khẩu do người dùng nhập trên giao diện.
Đầu ra: LoginForm.cs tương tác người dùng và SessionStorage.cs quản lý token/vai trò trong suốt phiên chạy.

4. Các bước triển khai chi tiết:
1. Thiết kế biểu mẫu LoginForm với ô nhập Username, Password (ẩn ký tự), nút Đăng nhập và Đăng ký (nếu có mã mời).
2. Gửi bản tin LOGIN (username, password) qua AdminRpcClient và chờ phản hồi LOGIN_RESULT.
3. Khi đăng nhập thành công, lưu chuỗi token và thông tin vai trò (Role: Admin/Operator/Viewer) vào SessionStorage.

4. Đóng LoginForm và mở màn hình chính MainForm với quyền hạn tương ứng.
5. Xử lý kịch bản đăng ký tài khoản mới (SIGN_UP) yêu cầu nhập mã mời inviteCode từ quản trị viên.

> 📌 **BẪY LỖI CẦN TRÁNH TRONG LG-C-003**
> Lưu token hoặc mật khẩu thô vào file cấu hình trên ổ cứng thay vì giữ trong bộ nhớ RAM, tạo lỗ hổng bảo mật nghiêm trọng trên máy trạm.

5. Tiêu chuẩn hoàn thành 11 mục (Definition of Done — DoD):
• [ ] 1. Hợp đồng giao thức & Wire Envelope chuẩn UTF-8 JSON
• [ ] 2. Lập trình bất đồng bộ Async/Await không chặn luồng mạng
• [ ] 3. Bắt ngoại lệ mạng an toàn (EOF, SocketException, Timeout)
• [ ] 4. Giải phóng tài nguyên sạch sẽ (Dispose / Using / CancellationToken)
• [ ] 5. An toàn dữ liệu (Khử trùng lặp lô / Khóa lạc quan OCC / Transaction)
• [ ] 6. Unit Test / Test Harness chạy đạt 100% không có cảnh báo
• [ ] 7. Tích hợp chéo thành công với module của đồng đội liên quan
• [ ] 8. Kiểm thử thực tế trên mạng LAN nhiều máy tính
• [ ] 9. Đồng bộ tài liệu thiết kế & mã nguồn khớp hoàn toàn
• [ ] 10. Thu thập đầy đủ hồ sơ minh chứng (Log, Screenshot, Pcap)
• [ ] 11. Sẵn sàng trả lời trôi chảy câu hỏi vấn đáp (Viva) liên quan

6. Kịch bản kiểm thử & Hồ sơ minh chứng:
• Mã bài test: TEST-C03
• Phương pháp kiểm thử: Thực hiện đăng nhập thành công và đăng nhập sai mật khẩu; kiểm tra lưu token
• Tiêu chuẩn nghiệm thu đạt: Đăng nhập đúng mở màn hình chính; đăng nhập sai báo lỗi thân thiện; token được giữ trong RAM, không lưu đĩa dạng plain text
• Hồ sơ minh chứng lưu tại: Ảnh chụp màn hình form đăng nhập, code review session store

7. Tình huống lỗi tiềm ẩn & Kế hoạch ứng phó: Lưu mật khẩu plain text trên client; chỉ truyền mật khẩu qua TLS và không lưu mật khẩu vào bộ nhớ sau khi đăng nhập

8. Câu hỏi vấn đáp (Viva) & Chiến lược trả lời:
• Chủ đề câu hỏi: Quản lý vòng đời phiên làm việc (Session Lifecycle) phía máy khách
• Câu trả lời kỳ vọng: Lưu session token trong bộ nhớ tạm (RAM) của ứng dụng; tự động hủy bỏ khi người dùng nhấn Logout hoặc tắt ứng dụng
• Câu hỏi mở rộng / Bẫy của giảng viên: Nguy cơ rò rỉ token nếu lưu trực tiếp vào file cấu hình trên đĩa cứng?

[HƯỚNG DẪN ĐỀ XUẤT]: *Tham khảo mã nguồn mẫu chuẩn và tài liệu kỹ thuật tại thư mục UI. Khi hiện thực, luôn đặt an toàn luồng và xử lý hủy tác vụ (CancellationToken) lên hàng đầu.*

Nhiệm vụ 04: [LG-C-004] — Khớp cặp Request/Response correlation và timeout

Mã & Tên nhiệm vụ: [LG-C-004] Khớp cặp Request/Response correlation và timeout	**Tuần & Ưu tiên:** Tuần W1 (P0) | P0
Khối lượng công việc: Cơ bản: 3.0h | Dự phòng: 0.5h | Tổng: 3.5h	**Phân loại tải:** Test/Harness
Trách nhiệm thực hiện: Phụ trách: SV C | Dự phòng: SV B | Duyệt: SV B	**Phân hệ & Thư mục:** Admin.RPC (Testing)
Sản phẩm bàn giao: RpcCorrelationManager.cs	**Hợp đồng giao diện:** ConcurrentDictionary<Guid, TaskCompletionSource<WireEnvelope>>, timeout 5000ms

1. Mục đích kỹ thuật & Tại sao tồn tại: Khi gửi một câu hỏi (Request) lên Server qua một kết nối socket duy nhất đang có nhiều gói tin khác chạy qua, ứng dụng phải biết chính xác gói tin trả lời (Response) nào thuộc về câu hỏi nào và phải báo lỗi nếu Server quá lâu không hồi đáp.

2. Kiến thức nền tảng cần nắm vững: Mẫu khớp cặp yêu cầu/phản hồi (Request/Response Correlation), lớp TaskCompletionSource<T> trong .NET, ConcurrentDictionary, CancellationTokenSource với Timeout.

3. Dữ liệu đầu vào & Đầu ra kỹ thuật: Đầu vào: Các gói tin lệnh cần gửi và mã định danh duy nhất requestId (UUIDv4).
Đầu ra: RpcCorrelationManager.cs ánh xạ các yêu cầu đang chờ phản hồi và xử lý quá hạn 5000ms an toàn.

4. Các bước triển khai chi tiết:
1. Với mỗi yêu cầu gửi đi, tạo một đối tượng TaskCompletionSource<WireEnvelope> và lưu vào ConcurrentDictionary với khóa là requestId.
2. Thiết lập bộ đếm thời gian quá hạn 5000ms; nếu sau 5 giây không có hồi đáp, hủy Task và ném TimeoutException.
3. Khi luồng đọc mạng nhận được gói tin trả lời, trích xuất requestId trong Envelope, tìm kiếm trong Dictionary và kích hoạt tcs.SetResult(envelope).
4. Xóa đối tượng ra khỏi Dictionary ngay sau khi khớp cặp thành công để tránh rò rỉ bộ nhớ (Memory Leak).
5. Đảm bảo giao diện người dùng không bị treo đơ trong suốt 5 giây chờ đợi.

> **📌 BẪY LỖI CẦN TRÁNH TRONG LG-C-004**
> Quên xóa requestId ra khỏi Dictionary sau khi hoàn thành hoặc quá hạn, khiến bộ nhớ RAM của ứng dụng phình to liên tục theo thời gian.

5. Tiêu chuẩn hoàn thành 11 mục (Definition of Done — DoD):
• [ ] 1. Hợp đồng giao thức & Wire Envelope chuẩn UTF-8 JSON

• [ ] 2. Lập trình bất đồng bộ Async/Await không chặn luồng mạng
• [ ] 3. Bắt ngoại lệ mạng an toàn (EOF, SocketException, Timeout)
• [ ] 4. Giải phóng tài nguyên sạch sẽ (Dispose / Using / CancellationToken)
• [ ] 5. An toàn dữ liệu (Khử trùng lặp lô / Khóa lạc quan OCC / Transaction)
• [ ] 6. Unit Test / Test Harness chạy đạt 100% không có cảnh báo
• [ ] 7. Tích hợp chéo thành công với module của đồng đội liên quan
• [ ] 8. Kiểm thử thực tế trên mạng LAN nhiều máy tính
• [ ] 9. Đồng bộ tài liệu thiết kế & mã nguồn khớp hoàn toàn
• [ ] 10. Thu thập đầy đủ hồ sơ minh chứng (Log, Screenshot, Pcap)
• [ ] 11. Sẵn sàng trả lời trôi chảy câu hỏi vấn đáp (Viva) liên quan

6. Kịch bản kiểm thử & Hồ sơ minh chứng:
• Mã bài test: TEST-C04
• Phương pháp kiểm thử: Gửi đồng thời 20 request qua kênh RPC tới mock server; kiểm tra khớp đúng kết quả
• Tiêu chuẩn nghiệm thu đạt: 100% response trả về đúng luồng gọi tương ứng; tự động hủy và ném TimeoutException sau 5s nếu server không phản hồi
• Hồ sơ minh chứng lưu tại: RPC correlation benchmark log, unit test suite

7. Tình huống lỗi tiềm ẩn & Kế hoạch ứng phó: Rò rỉ bộ nhớ do các TaskCompletionSource không bao giờ hoàn thành; dùng Timer hoặc CancellationToken để tự dọn dẹp

8. Câu hỏi vấn đáp (Viva) & Chiến lược trả lời:
• Chủ đề câu hỏi: Mô hình Request-Response trên kết nối Socket TCP hai chiều (RPC Correlation)
• Câu trả lời kỳ vọng: Mỗi request được gán một requestId (UUID); client lưu TaskCompletionSource vào từ điển; khi nhận response sẽ lấy ra hoàn thành Task
• Câu hỏi mở rộng / Bẫy của giảng viên: Tại sao không thể dùng cơ chế tuần tự Read/Write đơn giản trên socket hai chiều?

[HƯỚNG DẪN ĐỀ XUẤT]: *Tham khảo mã nguồn mẫu chuẩn và tài liệu kỹ thuật tại thư mục Testing. Khi hiện thực, luôn đặt an toàn luồng và xử lý hủy tác vụ (CancellationToken) lên hàng đầu.*

Nhiệm vụ 05: [LG-C-005] — Bộ nhận Snapshot và khởi tạo cache Admin

Mã & Tên nhiệm vụ: [LG-C-005] Bộ nhận Snapshot và khởi tạo cache Admin	**Tuần & Ưu tiên:** Tuần W2 (P0) | P0
Khối lượng công việc: Cơ bản: 2.5h | Dự phòng: 0.5h | Tổng: 3.0h	**Phân loại tải:** P0 Core
Trách nhiệm thực hiện: Phụ trách: SV C | Dự phòng: SV B | Duyệt: SV B	**Phân hệ & Thư mục:** Admin.Cache (Storage)
Sản phẩm bàn giao: SnapshotReceiver.cs & LocalCacheStore.cs	**Hợp đồng giao diện:** SNAPSHOT_BEGIN, SNAPSHOT_PAGE, SNAPSHOT_END protocol flow

1. Mục đích kỹ thuật & Tại sao tồn tại: Khi ứng dụng Admin vừa kết nối tới Server, nó chưa hề có dữ liệu gì về 40 máy trạm. Server sẽ gửi một luồng ảnh chụp nhanh gồm nhiều trang dữ liệu để khởi tạo bộ nhớ đệm (Cache) ban đầu cho máy trạm quản trị.

2. Kiến thức nền tảng cần nắm vững: Quy trình phân trang dữ liệu (Pagination Stream), các bản tin SNAPSHOT_BEGIN, SNAPSHOT_PAGE, SNAPSHOT_END, bộ nhớ đệm trong bộ nhớ (In-Memory Cache).

3. Dữ liệu đầu vào & Đầu ra kỹ thuật: Đầu vào: Chuỗi các gói tin Snapshot truyền từ Central Server qua kết nối mạng.
Đầu ra: SnapshotReceiver.cs tiếp nhận, lắp ghép các trang dữ liệu và LocalCacheStore.cs lưu trữ toàn bộ trạng thái phòng máy.

4. Các bước triển khai chi tiết:
1. Bắt bản tin SNAPSHOT_BEGIN để chuẩn bị vùng nhớ đệm tạm thời và xóa bỏ trạng thái cũ.
2. Đón nhận lần lượt các gói tin SNAPSHOT_PAGE, giải mã danh sách máy trạm, trạng thái số liệu và sự cố đang mở.
3. Khi nhận bản tin SNAPSHOT_END, xác nhận toàn bộ ảnh chụp đã hoàn tất và nạp vào LocalCacheStore chính thức.
4. Đảm bảo hiển thị thanh tiến trình (Progress Bar) thông báo cho người dùng biết trạng thái đang tải dữ liệu.
5. Bắt đầu lắng nghe các sự kiện gia tăng (Incremental Events) tiếp theo.

> **📌 BẪY LỖI CẦN TRÁNH TRONG LG-C-005**
> Áp dụng các sự kiện cập nhật tức thời vào bộ đệm trước khi quá trình nhận Snapshot kết thúc, dẫn đến tình trạng dữ liệu mới bị đè bẹp bởi dữ liệu cũ trong Snapshot.

5. Tiêu chuẩn hoàn thành 11 mục (Definition of Done — DoD):
• [ ] 1. Hợp đồng giao thức & Wire Envelope chuẩn UTF-8 JSON
• [ ] 2. Lập trình bất đồng bộ Async/Await không chặn luồng mạng
• [ ] 3. Bắt ngoại lệ mạng an toàn (EOF, SocketException, Timeout)
• [ ] 4. Giải phóng tài nguyên sạch sẽ (Dispose / Using / CancellationToken)
• [ ] 5. An toàn dữ liệu (Khử trùng lặp lô / Khóa lạc quan OCC / Transaction)
• [ ] 6. Unit Test / Test Harness chạy đạt 100% không có cảnh báo
• [ ] 7. Tích hợp chéo thành công với module của đồng đội liên quan
• [ ] 8. Kiểm thử thực tế trên mạng LAN nhiều máy tính
• [ ] 9. Đồng bộ tài liệu thiết kế & mã nguồn khớp hoàn toàn
• [ ] 10. Thu thập đầy đủ hồ sơ minh chứng (Log, Screenshot, Pcap)
• [ ] 11. Sẵn sàng trả lời trôi chảy câu hỏi vấn đáp (Viva) liên quan

6. Kịch bản kiểm thử & Hồ sơ minh chứng:
• Mã bài test: TEST-C05
• Phương pháp kiểm thử: Mô phỏng nhận snapshot 50 thiết bị chia làm 5 trang; kiểm tra dữ liệu trong cache
• Tiêu chuẩn nghiệm thu đạt: Nạp đủ 50 thiết bị vào cache; chỉ kích hoạt hiển thị dữ liệu sau khi nhận

được bản tin SNAPSHOT_END
• Hồ sơ minh chứng lưu tại: Snapshot receipt timeline log, cache dump

7. Tình huống lỗi tiềm ẩn & Kế hoạch ứng phó: Hiển thị dữ liệu chắp vá khi đang nhận dở snapshot; cơ chế rào cản (Snapshot Barrier) ngăn chặn hiển thị trước khi hoàn tất

8. Câu hỏi vấn đáp (Viva) & Chiến lược trả lời:
• Chủ đề câu hỏi: Cơ chế khởi tạo dữ liệu bằng Snapshot phân trang (Snapshot Barrier)
• Câu trả lời kỳ vọng: Chia nhỏ dữ liệu lớn thành nhiều trang để không gây nghẽn kết nối mạng; khóa hiển thị cho tới khi nhận đủ toàn bộ các trang
• Câu hỏi mở rộng / Bẫy của giảng viên: Nếu một trang snapshot bị thất lạc trên đường truyền thì xử lý thế nào?

[HƯỚNG DẪN ĐỀ XUẤT]: *Tham khảo mã nguồn mẫu chuẩn và tài liệu kỹ thuật tại thư mục Storage. Khi hiện thực, luôn đặt an toàn luồng và xử lý hủy tác vụ (CancellationToken) lên hàng đầu.*

Nhiệm vụ 06: [LG-C-006] — Admin cache merge, reconnect và UI-thread marshal

Mã & Tên nhiệm vụ: [LG-C-006] Admin cache merge, reconnect và UI-thread marshal	**Tuần & Ưu tiên:** Tuần W2 (P0) | P0
Khối lượng công việc: Cơ bản: 3.0h | Dự phòng: 0.5h | Tổng: 3.5h	**Phân loại tải:** P0 Core
Trách nhiệm thực hiện: Phụ trách: SV C | Dự phòng: SV A | Duyệt: SV A	**Phân hệ & Thư mục:** Admin.Realtime (UI)
Sản phẩm bàn giao: CacheSynchronizer.cs & UiDispatcher.cs	**Hợp đồng giao diện:** Control.Invoke, Control.BeginInvoke, Control.InvokeAsync, eventSeq ordering

1. Mục đích kỹ thuật & Tại sao tồn tại: Trong WinForms, các gói tin mạng đến từ một luồng chạy ngầm (Background Thread). Nếu luồng này trực tiếp thay đổi thuộc tính của điều khiển giao diện (như Text hay BackColor), ứng dụng sẽ sập ngay lập tức với lỗi Cross-thread operation.

2. Kiến thức nền tảng cần nắm vững: Mô hình luồng đơn giao diện (STA - Single Thread Apartment), phương thức Control.InvokeRequired, Control.Invoke, Control.BeginInvoke, số thứ tự sự kiện (eventSeq).

3. Dữ liệu đầu vào & Đầu ra kỹ thuật: Đầu vào: Các gói tin sự kiện phát sinh từ máy chủ chạy trên luồng nền của Socket.
Đầu ra: UiDispatcher.cs điều phối dữ liệu an toàn về luồng UI và CacheSynchronizer.cs sắp xếp thứ tự cập nhật.

4. Các bước triển khai chi tiết:
1. Viết phương thức tiện ích RunOnUi(Action action) tự động kiểm tra control.InvokeRequired.
2. Nếu InvokeRequired là true, gọi control.BeginInvoke(action) để chuyển tác vụ cập nhật về luồng giao

diện chính.
3. Kiểm tra số thứ tự sự kiện (eventSeq): chỉ áp dụng các bản tin có eventSeq lớn hơn số hiện tại để loại bỏ các gói tin đến muộn.
4. Xử lý kịch bản mất mạng và kết nối lại: tự động dọn cache và yêu cầu Server gửi lại Snapshot mới.
5. Đảm bảo tốc độ làm mới màn hình mượt mà không gây giật lag khi có bão sự kiện.

> 📌 **BẪY LỖI CẦN TRÁNH TRONG LG-C-006**
> Dùng CheckForIllegalCrossThreadCalls = false để tắt cảnh báo luồng, gây hiện tượng vẽ sai giao diện, rác bộ nhớ và sập chương trình ngẫu nhiên không thể dò lỗi.

5. Tiêu chuẩn hoàn thành 11 mục (Definition of Done — DoD):
• [ ] 1. Hợp đồng giao thức & Wire Envelope chuẩn UTF-8 JSON
• [ ] 2. Lập trình bất đồng bộ Async/Await không chặn luồng mạng
• [ ] 3. Bắt ngoại lệ mạng an toàn (EOF, SocketException, Timeout)
• [ ] 4. Giải phóng tài nguyên sạch sẽ (Dispose / Using / CancellationToken)
• [ ] 5. An toàn dữ liệu (Khử trùng lặp lô / Khóa lạc quan OCC / Transaction)
• [ ] 6. Unit Test / Test Harness chạy đạt 100% không có cảnh báo
• [ ] 7. Tích hợp chéo thành công với module của đồng đội liên quan
• [ ] 8. Kiểm thử thực tế trên mạng LAN nhiều máy tính
• [ ] 9. Đồng bộ tài liệu thiết kế & mã nguồn khớp hoàn toàn
• [ ] 10. Thu thập đầy đủ hồ sơ minh chứng (Log, Screenshot, Pcap)
• [ ] 11. Sẵn sàng trả lời trôi chảy câu hỏi vấn đáp (Viva) liên quan

6. Kịch bản kiểm thử & Hồ sơ minh chứng:
• Mã bài test: TEST-C06
• Phương pháp kiểm thử: Bơm liên tục 100 event cập nhật từ background thread mạng vào giao diện
• Tiêu chuẩn nghiệm thu đạt: Giao diện cập nhật mượt mà; 0 ngoại lệ InvalidOperationException (Cross-thread operation not valid)
• Hồ sơ minh chứng lưu tại: UI thread safety test log, code review dispatcher

7. Tình huống lỗi tiềm ẩn & Kế hoạch ứng phó: Truy cập trực tiếp control từ luồng mạng gây crash ứng dụng; kiểm tra InvokeRequired và chuyển về UI thread

8. Câu hỏi vấn đáp (Viva) & Chiến lược trả lời:
• Chủ đề câu hỏi: Điều phối luồng trong Windows Forms (Thread Marshaling)
• Câu trả lời kỳ vọng: Control trong WinForms gắn liền với UI thread (STA); mọi cập nhật giao diện từ luồng mạng phải được chuyển qua Invoke hoặc BeginInvoke
• Câu hỏi mở rộng / Bẫy của giảng viên: Sự khác biệt giữa Control.Invoke (đồng bộ) và Control.BeginInvoke (bất đồng bộ)?

[HƯỚNG DẪN ĐỀ XUẤT]: *Tham khảo mã nguồn mẫu chuẩn và tài liệu kỹ thuật tại thư mục UI. Khi hiện thực, luôn đặt an toàn luồng và xử lý hủy tác vụ (CancellationToken) lên hàng đầu.*

Nhiệm vụ 07: [LG-C-007] — Dashboard, Lab View, Device Detail

Mã & Tên nhiệm vụ: [LG-C-007] Dashboard, Lab View, Device Detail	**Tuần & Ưu tiên:** Tuần W2 (P0) | P0
Khối lượng công việc: Cơ bản: 3.0h | Dự phòng: 0.5h | Tổng: 3.5h	**Phân loại tải:** P0 Core
Trách nhiệm thực hiện: Phụ trách: SV C | Dự phòng: SV B | Duyệt: SV B	**Phân hệ & Thư mục:** Admin.Views (UI)
Sản phẩm bàn giao: DashboardForm.cs, LabViewGridControl.cs, DeviceDetailDialog.cs	**Hợp đồng giao diện:** Grid layout phòng máy 20-40 máy trạm, trạng thái: Online (Xanh), Warning (Vàng), Offline (Xám)

1. Mục đích kỹ thuật & Tại sao tồn tại: Mục tiêu tối thượng của ứng dụng là giúp người quản lý nhìn thấy ngay lập tức tình trạng sức khỏe của phòng máy: máy nào đang mở, máy nào bị nghẽn CPU, máy nào đã sập. Cần hiển thị dạng lưới phòng máy (Lab View) sinh động và trực quan.

2. Kiến thức nền tảng cần nắm vững: Vẽ tùy biến điều khiển WinForms (Custom Control Painting), bảng màu cảnh báo chuẩn UX, hộp thoại chi tiết (Modal Dialog).

3. Dữ liệu đầu vào & Đầu ra kỹ thuật: Đầu vào: Dữ liệu trạng thái máy trạm và thông số đo lường thời gian thực từ LocalCacheStore.
Đầu ra: DashboardForm.cs tổng quan, LabViewGridControl.cs hiển thị sơ đồ phòng máy và DeviceDetailDialog.cs xem chi tiết.

4. Các bước triển khai chi tiết:
1. Thiết kế lưới phòng máy gồm các ô đại diện cho từng máy trạm (LAB-01, LAB-02...).
2. Áp dụng quy tắc màu sắc chuẩn: Xanh lá (ONLINE bình thường), Vàng (WARNING - CPU/RAM cao), Đỏ/Xám (OFFLINE / Sự cố nghiêm trọng).
3. Hiển thị thanh tiến trình mini đo phần trăm CPU/RAM trực tiếp trên từng thẻ máy trạm.
4. Bắt sự kiện Double-Click vào một máy trạm để mở DeviceDetailDialog hiển thị chi tiết tiến trình, địa chỉ IP, MAC và lịch sử cảnh báo.
5. Tự động cập nhật màu sắc tức thì khi nhận được sự kiện thay đổi từ luồng mạng.

> **📌 BẪY LỖI CẦN TRÁNH TRONG LG-C-007**
> Vẽ lại toàn bộ giao diện mỗi khi chỉ có 1 máy thay đổi trạng thái, làm màn hình bị nhấp nháy (flickering) liên tục.

5. Tiêu chuẩn hoàn thành 11 mục (Definition of Done — DoD):
- [ ] 1. Hợp đồng giao thức & Wire Envelope chuẩn UTF-8 JSON
- [ ] 2. Lập trình bất đồng bộ Async/Await không chặn luồng mạng
- [ ] 3. Bắt ngoại lệ mạng an toàn (EOF, SocketException, Timeout)
- [ ] 4. Giải phóng tài nguyên sạch sẽ (Dispose / Using / CancellationToken)
- [ ] 5. An toàn dữ liệu (Khử trùng lặp lô / Khóa lạc quan OCC / Transaction)

• [ ] 6. Unit Test / Test Harness chạy đạt 100% không có cảnh báo
• [ ] 7. Tích hợp chéo thành công với module của đồng đội liên quan
• [ ] 8. Kiểm thử thực tế trên mạng LAN nhiều máy tính
• [ ] 9. Đồng bộ tài liệu thiết kế & mã nguồn khớp hoàn toàn
• [ ] 10. Thu thập đầy đủ hồ sơ minh chứng (Log, Screenshot, Pcap)
• [ ] 11. Sẵn sàng trả lời trôi chảy câu hỏi vấn đáp (Viva) liên quan

6. Kịch bản kiểm thử & Hồ sơ minh chứng:
• Mã bài test: TEST-C07
• Phương pháp kiểm thử: Hiển thị 20 máy trạm; mô phỏng đổi trạng thái 1 máy sang Warning và 1 máy sang Offline
• Tiêu chuẩn nghiệm thu đạt: Màu sắc và icon cập nhật tức thì; không phụ thuộc duy nhất vào màu sắc (hỗ trợ người mù màu với nhãn trạng thái rõ ràng)
• Hồ sơ minh chứng lưu tại: Ảnh chụp màn hình Lab View 20 máy trạm ở các trạng thái khác nhau

7. Tình huống lỗi tiềm ẩn & Kế hoạch ứng phó: Người dùng không phân biệt được trạng thái do mù màu; kết hợp đồng thời màu nền, icon cảnh báo và văn bản trạng thái

8. Câu hỏi vấn đáp (Viva) & Chiến lược trả lời:
• Chủ đề câu hỏi: Thiết kế giao diện hỗ trợ tiếp cận (Accessibility) trong phần mềm quản trị
• Câu trả lời kỳ vọng: Không chỉ dùng màu sắc đơn thuần để chỉ báo trạng thái; luôn đi kèm biểu tượng hình học và nhãn chữ để đảm bảo khả năng tiếp cận
• Câu hỏi mở rộng / Bẫy của giảng viên: Làm thế nào để hiển thị mượt mà khi phòng máy có hàng trăm máy trạm?

[HƯỚNG DẪN ĐỀ XUẤT]: *Tham khảo mã nguồn mẫu chuẩn và tài liệu kỹ thuật tại thư mục UI. Khi hiện thực, luôn đặt an toàn luồng và xử lý hủy tác vụ (CancellationToken) lên hàng đầu.*

Nhiệm vụ 08: [LG-C-008] — Incident list/detail và action binding

Mã & Tên nhiệm vụ: [LG-C-008] Incident list/detail và action binding	**Tuần & Ưu tiên:** Tuần W3 (P0) | P0
Khối lượng công việc: Cơ bản: 3.0h | Dự phòng: 0.5h | Tổng: 3.5h	**Phân loại tải:** P0 Core
Trách nhiệm thực hiện: Phụ trách: SV C | Dự phòng: SV B | Duyệt: SV B	**Phân hệ & Thư mục:** Admin.Incident (UI)
Sản phẩm bàn giao: IncidentListView.cs & IncidentActionPanel.cs	**Hợp đồng giao diện:** INCIDENT_ACK, INCIDENT_START, INCIDENT_RESOLVE (incidentId, expectedRowVersion, commandId)

1. Mục đích kỹ thuật & Tại sao tồn tại: Khi sự cố xuất hiện, cán bộ vận hành cần tiếp nhận và xử lý nhanh chóng. Giao diện phải cung cấp danh sách sự cố rõ ràng, các nút bấm thao tác (ACK, START, RESOLVE) và gửi kèm thông tin phiên bản (expectedRowVersion) để bảo vệ tranh chấp.

2. Kiến thức nền tảng cần nắm vững: Danh sách dữ liệu (DataGridView / ListView), ràng buộc dữ liệu (Data Binding), cơ chế khóa lạc quan (OCC) trên giao diện.

3. Dữ liệu đầu vào & Đầu ra kỹ thuật: Đầu vào: Danh sách sự cố từ CSDL/Cache và các nút bấm tương tác của người vận hành.
Đầu ra: IncidentListView.cs hiển thị danh sách và IncidentActionPanel.cs xử lý các nút bấm hành động.

4. Các bước triển khai chi tiết:
1. Thiết kế bảng danh sách sự cố với các cột: Mã máy, Tên sự cố, Mức độ nghiêm trọng, Trạng thái, Thời gian phát sinh, Người phụ trách.
2. Tạo các nút hành động: Tiếp nhận (ACK), Bắt đầu xử lý (START), Giải quyết xong (RESOLVE).
3. Khi bấm nút, trích xuất incidentId và expectedRowVersion hiện tại, đóng gói vào bản tin tương ứng gửi lên Central Server.
4. Nếu nhận được phản hồi CONCURRENCY_CONFLICT từ Server, hiển thị thông báo: 'Sự cố đã được người khác cập nhật!' và tự động tải lại dữ liệu mới nhất.
5. Vô hiệu hóa nút bấm trong thời gian chờ phản hồi để tránh người dùng nhấn đúp (double-click).

> **📌 BẪY LỖI CẦN TRÁNH TRONG LG-C-008**
> Không gửi kèm expectedRowVersion, hoặc không xử lý phản hồi CONCURRENCY_CONFLICT, làm người dùng nghĩ rằng thao tác của mình đã thành công.

5. Tiêu chuẩn hoàn thành 11 mục (Definition of Done — DoD):
• [ ] 1. Hợp đồng giao thức & Wire Envelope chuẩn UTF-8 JSON
• [ ] 2. Lập trình bất đồng bộ Async/Await không chặn luồng mạng
• [ ] 3. Bắt ngoại lệ mạng an toàn (EOF, SocketException, Timeout)
• [ ] 4. Giải phóng tài nguyên sạch sẽ (Dispose / Using / CancellationToken)
• [ ] 5. An toàn dữ liệu (Khử trùng lặp lô / Khóa lạc quan OCC / Transaction)
• [ ] 6. Unit Test / Test Harness chạy đạt 100% không có cảnh báo
• [ ] 7. Tích hợp chéo thành công với module của đồng đội liên quan
• [ ] 8. Kiểm thử thực tế trên mạng LAN nhiều máy tính
• [ ] 9. Đồng bộ tài liệu thiết kế & mã nguồn khớp hoàn toàn
• [ ] 10. Thu thập đầy đủ hồ sơ minh chứng (Log, Screenshot, Pcap)
• [ ] 11. Sẵn sàng trả lời trôi chảy câu hỏi vấn đáp (Viva) liên quan

6. Kịch bản kiểm thử & Hồ sơ minh chứng:
• Mã bài test: TEST-C08
• Phương pháp kiểm thử: Bấm nút tiếp nhận (ACK) và bắt đầu xử lý (START); đối soát command gửi trên mạng
• Tiêu chuẩn nghiệm thu đạt: Gửi đúng format command với expectedRowVersion hiện tại; vô hiệu hóa nút trong khi chờ phản hồi để tránh double click
• Hồ sơ minh chứng lưu tại: Incident view screenshot, command request trace log

7. Tình huống lỗi tiềm ẩn & Kế hoạch ứng phó: Người dùng bấm đúp nút gửi 2 lệnh liên tiếp; vô hiệu hóa nút ngay khi bấm và sử dụng commandId duy nhất

8. Câu hỏi vấn đáp (Viva) & Chiến lược trả lời:
• Chủ đề câu hỏi: Gắn kết thao tác người dùng với cơ chế chống tranh chấp OCC
• Câu trả lời kỳ vọng: Lấy rowversion hiện có của dòng sự cố đưa vào bản tin lệnh; nếu bản ghi đã bị người khác sửa đổi thì server sẽ báo lỗi tranh chấp
• Câu hỏi mở rộng / Bẫy của giảng viên: Tại sao client phải xử lý trường hợp nhận mã lỗi 409 Conflict từ máy chủ?

[HƯỚNG DẪN ĐỀ XUẤT]: *Tham khảo mã nguồn mẫu chuẩn và tài liệu kỹ thuật tại thư mục UI. Khi hiện thực, luôn đặt an toàn luồng và xử lý hủy tác vụ (CancellationToken) lên hàng đầu.*

Nhiệm vụ 09: [LG-C-009] — RBAC UI UX và session lifecycle client-side

Mã & Tên nhiệm vụ: [LG-C-009] RBAC UI UX và session lifecycle client-side	**Tuần & Ưu tiên:** Tuần W3 (P0) | P0
Khối lượng công việc: Cơ bản: 2.0h | Dự phòng: 0.5h | Tổng: 2.5h	**Phân loại tải:** P0 Core
Trách nhiệm thực hiện: Phụ trách: SV C | Dự phòng: SV B | Duyệt: SV B	**Phân hệ & Thư mục:** Admin.RBAC (UI)
Sản phẩm bàn giao: RbacUiManager.cs	**Hợp đồng giao diện:** Role-based UI visibility: Viewer (chỉ xem), Operator (ACK, START, RESOLVE), Admin (toàn quyền kèm ASSIGN, User/Invite)

1. Mục đích kỹ thuật & Tại sao tồn tại: Mỗi người dùng đăng nhập có vai trò khác nhau. Giao diện phải tự động điều chỉnh: Viewer chỉ được xem, không thấy hoặc bị làm mờ nút bấm xử lý; Operator thấy nút xử lý sự cố; Admin thấy thêm nút phân công (ASSIGN) và quản lý người dùng.

2. Kiến thức nền tảng cần nắm vững: Quản lý quyền hạn trên UI (Role-Based Access Control UI), trạng thái kích hoạt điều khiển (Enabled / Visible), vòng đời phiên làm việc.

3. Dữ liệu đầu vào & Đầu ra kỹ thuật: Đầu vào: Vai trò người dùng (Role) nhận được sau khi đăng nhập thành công từ SessionStorage.
Đầu ra: RbacUiManager.cs điều khiển trạng thái ẩn/hiện và phân quyền trên toàn bộ các điều khiển giao diện.

4. Các bước triển khai chi tiết:
1. Xây dựng phương thức ApplyRbacPermissions(UserRole role) duyệt qua các thành phần trên màn hình.
2. Nếu vai trò là Viewer: Ẩn hoặc vô hiệu hóa các nút ACK, START, RESOLVE, ASSIGN và menu cấu hình.
3. Nếu vai trò là Operator: Kích hoạt các nút xử lý sự cố (ACK, START, RESOLVE), ẩn menu tạo tài khoản và cấu hình hệ thống.
4. Nếu vai trò là Admin: Hiển thị đầy đủ mọi chức năng bao gồm nút ASSIGN và quản lý mã mời (Invite Code).
5. Xử lý kịch bản hết hạn phiên (Session Expired): đưa người dùng về màn hình đăng nhập an toàn.

> 📌 **BẪY LỖI CẦN TRÁNH TRONG LG-C-009**
> Chỉ ẩn nút ở giao diện mà không xử lý phía máy chủ, hoặc để người dùng Viewer vẫn kích hoạt được phím tắt bàn phím để gửi lệnh.

5. Tiêu chuẩn hoàn thành 11 mục (Definition of Done — DoD):
• [ ] 1. Hợp đồng giao thức & Wire Envelope chuẩn UTF-8 JSON
• [ ] 2. Lập trình bất đồng bộ Async/Await không chặn luồng mạng
• [ ] 3. Bắt ngoại lệ mạng an toàn (EOF, SocketException, Timeout)
• [ ] 4. Giải phóng tài nguyên sạch sẽ (Dispose / Using / CancellationToken)
• [ ] 5. An toàn dữ liệu (Khử trùng lặp lô / Khóa lạc quan OCC / Transaction)
• [ ] 6. Unit Test / Test Harness chạy đạt 100% không có cảnh báo
• [ ] 7. Tích hợp chéo thành công với module của đồng đội liên quan
• [ ] 8. Kiểm thử thực tế trên mạng LAN nhiều máy tính
• [ ] 9. Đồng bộ tài liệu thiết kế & mã nguồn khớp hoàn toàn
• [ ] 10. Thu thập đầy đủ hồ sơ minh chứng (Log, Screenshot, Pcap)
• [ ] 11. Sẵn sàng trả lời trôi chảy câu hỏi vấn đáp (Viva) liên quan

6. Kịch bản kiểm thử & Hồ sơ minh chứng:
• Mã bài test: TEST-C09
• Phương pháp kiểm thử: Đăng nhập lần lượt bằng 3 tài khoản với 3 vai trò khác nhau; kiểm tra hiển thị nút
• Tiêu chuẩn nghiệm thu đạt: Viewer không nhìn thấy các nút thay đổi trạng thái; Operator thấy nút ACK/START/RESOLVE; Admin thấy thêm nút ASSIGN và Invite/User Management
• Hồ sơ minh chứng lưu tại: Ảnh chụp giao diện của 3 vai trò người dùng khác nhau

7. Tình huống lỗi tiềm ẩn & Kế hoạch ứng phó: Người dùng nhầm lẫn quyền trên UI; ghi rõ vai trò hiện tại trên thanh trạng thái của ứng dụng

8. Câu hỏi vấn đáp (Viva) & Chiến lược trả lời:
• Chủ đề câu hỏi: Phân định giữa UX Optimization trên Client và Server Authorization
• Câu trả lời kỳ vọng: Ẩn nút trên UI chỉ giúp người dùng không thao tác nhầm; việc xác thực quyền thực sự bắt buộc phải được thực thi tại Central Server
• Câu hỏi mở rộng / Bẫy của giảng viên: Nếu một người dùng sửa đổi client để gửi lệnh Resolve trái phép thì chuyện gì xảy ra?

[HƯỚNG DẪN ĐỀ XUẤT]: *Tham khảo mã nguồn mẫu chuẩn và tài liệu kỹ thuật tại thư mục UI. Khi hiện thực, luôn đặt an toàn luồng và xử lý hủy tác vụ (CancellationToken) lên hàng đầu.*

Nhiệm vụ 10: [LG-C-010] — History/Report/Health views

Mã & Tên nhiệm vụ: [LG-C-010] History/Report/Health views	**Tuần & Ưu tiên:** Tuần W3 (P0) | P0

Khối lượng công việc: Cơ bản: 2.0h | Dự phòng: 0.5h | Tổng: 2.5h	**Phân loại tải:** P0 Core
Trách nhiệm thực hiện: Phụ trách: SV C | Dự phòng: SV B | Duyệt: SV B	**Phân hệ & Thư mục:** Admin.Reports (UI)
Sản phẩm bàn giao: ReportViewerForm.cs & HealthViewDialog.cs	**Hợp đồng giao diện:** QUERY_INCIDENT_HISTORY, QUERY_METRICS_HISTORY, CSV Export utility

1. Mục đích kỹ thuật & Tại sao tồn tại: Giảng viên và người quản trị cần xem lại lịch sử các sự cố đã qua để đánh giá chất lượng phòng máy, đồng thời cần theo dõi tình trạng sức khỏe của chính máy chủ Central. Cần có các màn hình báo cáo và chức năng xuất dữ liệu ra file CSV.

2. Kiến thức nền tảng cần nắm vững: Truy vấn dữ liệu lịch sử, vẽ biểu đồ số liệu cơ bản, xử lý xuất tệp văn bản CSV chuẩn UTF-8, hộp thoại hiển thị trạng thái sức khỏe máy chủ.

3. Dữ liệu đầu vào & Đầu ra kỹ thuật: Đầu vào: Bản tin trả lời QUERY_INCIDENT_HISTORY, QUERY_METRICS_HISTORY và bản tin HEALTH_UPDATE.
Đầu ra: ReportViewerForm.cs hiển thị báo cáo kèm chức năng xuất CSV và HealthViewDialog.cs xem sức khỏe hệ thống.

4. Các bước triển khai chi tiết:
1. Thiết kế ReportViewerForm cho phép lọc sự cố theo khoảng ngày, theo máy trạm và theo trạng thái.
2. Gửi bản tin QUERY_INCIDENT_HISTORY lên máy chủ và hiển thị kết quả phân trang trên DataGridView.
3. Viết tính năng xuất danh sách sự cố ra file .csv với định dạng mã hóa UTF-8 with BOM (để mở bằng Excel không lỗi tiếng Việt).
4. Thiết kế HealthViewDialog hiển thị các thông số nhịp thở của Central Server: Số kết nối, Bộ đệm, Thời gian chạy liên tục (Uptime).
5. Cung cấp tính năng xem lại đồ thị CPU/RAM lịch sử của một máy trạm cụ thể.

> **📌 BẪY LỖI CẦN TRÁNH TRONG LG-C-010**
> Xuất file CSV mã hóa ASCII hoặc UTF-8 không có BOM, khiến người dùng mở file bằng Microsoft Excel bị lỗi font chữ tiếng Việt.

5. Tiêu chuẩn hoàn thành 11 mục (Definition of Done — DoD):
- [ ] 1. Hợp đồng giao thức & Wire Envelope chuẩn UTF-8 JSON
- [ ] 2. Lập trình bất đồng bộ Async/Await không chặn luồng mạng
- [ ] 3. Bắt ngoại lệ mạng an toàn (EOF, SocketException, Timeout)
- [ ] 4. Giải phóng tài nguyên sạch sẽ (Dispose / Using / CancellationToken)
- [ ] 5. An toàn dữ liệu (Khử trùng lặp lô / Khóa lạc quan OCC / Transaction)
- [ ] 6. Unit Test / Test Harness chạy đạt 100% không có cảnh báo
- [ ] 7. Tích hợp chéo thành công với module của đồng đội liên quan
- [ ] 8. Kiểm thử thực tế trên mạng LAN nhiều máy tính
- [ ] 9. Đồng bộ tài liệu thiết kế & mã nguồn khớp hoàn toàn

• [ ] 10. Thu thập đầy đủ hồ sơ minh chứng (Log, Screenshot, Pcap)
• [ ] 11. Sẵn sàng trả lời trôi chảy câu hỏi vấn đáp (Viva) liên quan

6. Kịch bản kiểm thử & Hồ sơ minh chứng:
• Mã bài test: TEST-C10
• Phương pháp kiểm thử: Thực hiện truy vấn sự cố 24h gần nhất và bấm xuất file CSV
• Tiêu chuẩn nghiệm thu đạt: Dữ liệu phân trang hiển thị đầy đủ; file CSV xuất ra mở được trên Excel với đúng bảng mã tiếng Việt UTF-8 BOM
• Hồ sơ minh chứng lưu tại: File CSV báo cáo mẫu, ảnh chụp màn hình form báo cáo

7. Tình huống lỗi tiềm ẩn & Kế hoạch ứng phó: Xuất file CSV bị lỗi font chữ tiếng Việt; ghi kèm UTF-8 BOM byte (0xEF, 0xBB, 0xBF) ở đầu tệp

8. Câu hỏi vấn đáp (Viva) & Chiến lược trả lời:
• Chủ đề câu hỏi: Kỹ thuật xuất báo cáo dữ liệu định dạng CSV hỗ trợ tiếng Việt
• Câu trả lời kỳ vọng: Bổ sung UTF-8 BOM để các ứng dụng văn phòng tự động nhận diện đúng bảng mã Unicode, tránh lỗi vỡ ký tự
• Câu hỏi mở rộng / Bẫy của giảng viên: Tại sao cần phân trang dữ liệu khi hiển thị lịch sử đo đạc?

[HƯỚNG DẪN ĐỀ XUẤT]: *Tham khảo mã nguồn mẫu chuẩn và tài liệu kỹ thuật tại thư mục UI. Khi hiện thực, luôn đặt an toàn luồng và xử lý hủy tác vụ (CancellationToken) lên hàng đầu.*

Nhiệm vụ 11: [LG-C-011] — MVP UI integration và accessibility

Mã & Tên nhiệm vụ: [LG-C-011] MVP UI integration và accessibility	**Tuần & Ưu tiên:** Tuần W3 (P0) | P0
Khối lượng công việc: Cơ bản: 3.5h | Dự phòng: 0.5h | Tổng: 4.0h	**Phân loại tải:** Tích hợp
Trách nhiệm thực hiện: Phụ trách: SV C | Dự phòng: SV A | Duyệt: SV A	**Phân hệ & Thư mục:** Admin.Integration (Integration)
Sản phẩm bàn giao: UiIntegrationReport.md	**Hợp đồng giao diện:** Toàn bộ giao diện người dùng WinForms MVP

1. Mục đích kỹ thuật & Tại sao tồn tại: Trước khi kết thúc tuần 3, toàn bộ các màn hình giao diện WinForms phải được ghép nối hoàn chỉnh vào một luồng trải nghiệm thống nhất, từ đăng nhập, theo dõi lưới phòng máy, nhận sự cố, bấm xử lý cho đến xuất báo cáo.

2. Kiến thức nền tảng cần nắm vững: Tích hợp giao diện người dùng (UI Integration), khả năng tiếp cận (Accessibility), kiểm tra phím tắt bàn phím và thứ tự Tab (TabIndex).

3. Dữ liệu đầu vào & Đầu ra kỹ thuật: Đầu vào: Toàn bộ các biểu mẫu và điều khiển WinForms đã hiện thực trong Tuần 1 đến Tuần 3.
Đầu ra: Báo cáo UiIntegrationReport.md ghi nhận toàn bộ luồng nghiệp vụ trên giao diện hoạt động trơn tru không lỗi.

4. Các bước triển khai chi tiết:
1. Rà soát thứ tự TabIndex trên tất cả các màn hình, đảm bảo người dùng có thể điều hướng bằng phím Tab thuận tiện.
2. Gắn các phím tắt tiện dụng (ví dụ F5 để làm mới, Enter để đăng nhập, Esc để đóng hộp thoại).
3. Kiểm tra độ tương phản màu sắc và kích cỡ font chữ, đảm bảo dễ đọc trong môi trường ánh sáng phòng máy.
4. Chạy thử luồng nghiệp vụ hoàn chỉnh: Đăng nhập -> Nhìn máy trạm đổi màu -> Bấm xử lý sự cố -> Xem báo cáo.
5. Lập báo cáo tích hợp giao diện làm hồ sơ bàn giao cho cổng xét duyệt chất lượng M4.

> 📌 **BẪY LỖI CẦN TRÁNH TRONG LG-C-011**
> Bỏ quên thuộc tính TabIndex khiến người dùng nhấn phím Tab con trỏ nhảy lung tung không theo thứ tự tự nhiên.

5. Tiêu chuẩn hoàn thành 11 mục (Definition of Done — DoD):
• [ ] 1. Hợp đồng giao thức & Wire Envelope chuẩn UTF-8 JSON
• [ ] 2. Lập trình bất đồng bộ Async/Await không chặn luồng mạng
• [ ] 3. Bắt ngoại lệ mạng an toàn (EOF, SocketException, Timeout)
• [ ] 4. Giải phóng tài nguyên sạch sẽ (Dispose / Using / CancellationToken)
• [ ] 5. An toàn dữ liệu (Khử trùng lặp lô / Khóa lạc quan OCC / Transaction)
• [ ] 6. Unit Test / Test Harness chạy đạt 100% không có cảnh báo
• [ ] 7. Tích hợp chéo thành công với module của đồng đội liên quan
• [ ] 8. Kiểm thử thực tế trên mạng LAN nhiều máy tính
• [ ] 9. Đồng bộ tài liệu thiết kế & mã nguồn khớp hoàn toàn
• [ ] 10. Thu thập đầy đủ hồ sơ minh chứng (Log, Screenshot, Pcap)
• [ ] 11. Sẵn sàng trả lời trôi chảy câu hỏi vấn đáp (Viva) liên quan

6. Kịch bản kiểm thử & Hồ sơ minh chứng:
• Mã bài test: TEST-C11
• Phương pháp kiểm thử: Điều hướng toàn bộ ứng dụng chỉ bằng bàn phím (Tab, Enter, Escape, phím mũi tên)
• Tiêu chuẩn nghiệm thu đạt: Thứ tự TabIndex tuần tự hợp lý; không có trường dữ liệu nào bị kẹt phím; các nút chính có phím nóng
• Hồ sơ minh chứng lưu tại: Checklist kiểm tra TabIndex và phím tắt, ảnh chụp giao diện

7. Tình huống lỗi tiềm ẩn & Kế hoạch ứng phó: Con trỏ nhảy lộn xộn khi bấm Tab; chuẩn hóa lại thuộc tính TabIndex trên từng control trong form designer

8. Câu hỏi vấn đáp (Viva) & Chiến lược trả lời:
• Chủ đề câu hỏi: Tiêu chuẩn điều hướng bàn phím trong ứng dụng Desktop chuyên nghiệp
• Câu trả lời kỳ vọng: Giúp người vận hành thao tác nhanh chóng trong phòng máy mà không cần phụ thuộc hoàn toàn vào chuột
• Câu hỏi mở rộng / Bẫy của giảng viên: Làm thế nào để xử lý các phím tắt toàn cục trong WinForms?

[HƯỚNG DẪN ĐỀ XUẤT]: *Tham khảo mã nguồn mẫu chuẩn và tài liệu kỹ thuật tại thư mục Integration. Khi hiện thực, luôn đặt an toàn luồng và xử lý hủy tác vụ (CancellationToken) lên hàng đầu.*

Nhiệm vụ 12: [LG-C-012] — MVP review và P1 GO/DEFER gate

Mã & Tên nhiệm vụ: [LG-C-012] MVP review và P1 GO/DEFER gate	**Tuần & Ưu tiên:** Tuần W3 (P0) | P0/Gate
Khối lượng công việc: Cơ bản: 2.5h | Dự phòng: 0.5h | Tổng: 3.0h	**Phân loại tải:** Tích hợp
Trách nhiệm thực hiện: Phụ trách: SV C | Dự phòng: SV B | Duyệt: SV B	**Phân hệ & Thư mục:** Admin.Gate (Integration)
Sản phẩm bàn giao: M4_Gate_Review.md	**Hợp đồng giao diện:** Tiêu chí nghiệm thu cổng chất lượng M4

1. Mục đích kỹ thuật & Tại sao tồn tại: Cổng chất lượng M4 ở cuối Tuần 3 là mốc sinh tử của đồ án: nhóm phải quyết định có tiếp tục làm tính năng mở rộng P1 (Collector) ở Tuần 4 hay cắt phạm vi (DEFER P1) để bảo toàn 100% điểm chất lượng cho phần MVP cốt lõi.

2. Kiến thức nền tảng cần nắm vững: Kỷ luật quản lý phạm vi đồ án (Scope Management), tiêu chuẩn nghiệm thu MVP, đánh giá rủi ro thời gian và khối lượng công việc.

3. Dữ liệu đầu vào & Đầu ra kỹ thuật: Đầu vào: Biên bản nghiệm thu MVP từ A (Agent), B (Server/DB) và C (WinForms UI) cùng quỹ thời gian còn lại của nhóm.
Đầu ra: Biên bản đánh giá cổng M4_Gate_Review.md ghi rõ quyết định GO hay DEFER đối với P1.

4. Các bước triển khai chi tiết:
1. Tổ chức cuộc họp đánh giá kỹ thuật 3 bên giữa A, B và C vào cuối Tuần 3.
2. Kiểm tra 3 tiêu chí bắt buộc để được phép làm P1: Không có lỗi P0 tồn đọng, kiểm thử đồng thời đạt 100%, mỗi thành viên còn ít nhất 15 giờ dự phòng.
3. Nếu không thỏa mãn cả 3 điều kiện: Kích hoạt ngay quyết định DEFER P1 theo đúng kế hoạch dự phòng đã cam kết.
4. Ghi nhận phân công tuần 4 chuyển sang tập trung vào: Tinh chỉnh hiệu năng, kiểm thử ổn định mạng LAN và chuẩn bị hồ sơ minh chứng.
5. Ký biên bản M4 Gate Review và lưu trữ vào thư mục hồ sơ nghiệm thu.

> **📌 BẪY LỖI CẦN TRÁNH TRONG LG-C-012**
> Cố chấp làm thêm tính năng P1 khi phần cốt lõi MVP vẫn còn lỗi chập chờn, dẫn đến tuần cuối không kịp ổn định hệ thống và bị đánh trượt.

5. Tiêu chuẩn hoàn thành 11 mục (Definition of Done — DoD):
• [ ] 1. Hợp đồng giao thức & Wire Envelope chuẩn UTF-8 JSON

• [ ] 2. Lập trình bất đồng bộ Async/Await không chặn luồng mạng
• [ ] 3. Bắt ngoại lệ mạng an toàn (EOF, SocketException, Timeout)
• [ ] 4. Giải phóng tài nguyên sạch sẽ (Dispose / Using / CancellationToken)
• [ ] 5. An toàn dữ liệu (Khử trùng lặp lô / Khóa lạc quan OCC / Transaction)
• [ ] 6. Unit Test / Test Harness chạy đạt 100% không có cảnh báo
• [ ] 7. Tích hợp chéo thành công với module của đồng đội liên quan
• [ ] 8. Kiểm thử thực tế trên mạng LAN nhiều máy tính
• [ ] 9. Đồng bộ tài liệu thiết kế & mã nguồn khớp hoàn toàn
• [ ] 10. Thu thập đầy đủ hồ sơ minh chứng (Log, Screenshot, Pcap)
• [ ] 11. Sẵn sàng trả lời trôi chảy câu hỏi vấn đáp (Viva) liên quan

6. Kịch bản kiểm thử & Hồ sơ minh chứng:
• Mã bài test: TEST-C12
• Phương pháp kiểm thử: Đối chiếu tình trạng thực tế của MVP với checklist M4 Gate
• Tiêu chuẩn nghiệm thu đạt: 0 lỗi Sev1/Sev2; cả 3 ứng dụng P0 chạy ổn định trên mạng LAN thật; có biên bản ký nhận thống nhất của cả 3 thành viên
• Hồ sơ minh chứng lưu tại: M4_Gate_Review.md có chữ ký xác nhận của 3 thành viên

7. Tình huống lỗi tiềm ẩn & Kế hoạch ứng phó: Còn lỗi Sev1 chưa giải quyết; tự động kích hoạt DEFER P1 để dành toàn bộ thời gian tuần 4 ổn định core

8. Câu hỏi vấn đáp (Viva) & Chiến lược trả lời:
• Chủ đề câu hỏi: Quy trình quản lý chất lượng phần mềm và cổng quyết định (Stage-Gate)
• Câu trả lời kỳ vọng: Thiết lập các mốc kiểm soát rõ ràng; chỉ mở rộng tính năng mới khi nền tảng cốt lõi đã đạt độ tin cậy tuyệt đối
• Câu hỏi mở rộng / Bẫy của giảng viên: Các tiêu chí then chốt để phân loại mức độ nghiêm trọng của lỗi (Sev1 so với Sev2)?

[HƯỚNG DẪN ĐỀ XUẤT]: *Tham khảo mã nguồn mẫu chuẩn và tài liệu kỹ thuật tại thư mục Integration. Khi hiện thực, luôn đặt an toàn luồng và xử lý hủy tác vụ (CancellationToken) lên hàng đầu.*

Nhiệm vụ 13: [LG-C-013] — Collector Agent-facing host (binary dùng chung)

Mã & Tên nhiệm vụ: [LG-C-013] Collector Agent-facing host (binary dùng chung)	**Tuần & Ưu tiên:** Tuần W4 (P1) | P1
Khối lượng công việc: Cơ bản: 6.0h | Dự phòng: 0.5h | Tổng: 6.5h	**Phân loại tải:** P1 Extension
Trách nhiệm thực hiện: Phụ trách: SV C | Dự phòng: SV A | Duyệt: SV A	**Phân hệ & Thư mục:** P1.Collector (Networking)
Sản phẩm bàn giao: LabGuard.Collector.exe & AgentListener.cs	**Hợp đồng giao diện:** Collector TCP server socket, MaxConcurrentConnections = 100

1. Mục đích kỹ thuật & Tại sao tồn tại: Nếu nhóm quyết định thực hiện tính năng mở rộng P1, C sẽ xây dựng ứng dụng máy chủ Collector (LabGuard.Collector.exe) đóng vai trò làm trạm trung chuyển gom dữ liệu cho từng phòng máy để giảm tải kết nối trực tiếp vào Central.

2. Kiến thức nền tảng cần nắm vững: Lập trình Console Application trên .NET, TcpListener lắng nghe Agent kết nối, cấu hình giới hạn kết nối đồng thời.

3. Dữ liệu đầu vào & Đầu ra kỹ thuật: Đầu vào: Cổng lắng nghe cục bộ (mặc định 5001) và cấu hình địa chỉ IP của Central Server.
Đầu ra: Tệp thực thi LabGuard.Collector.exe và lớp AgentListener.cs tiếp nhận kết nối từ các Agent trong phòng máy.

4. Các bước triển khai chi tiết:
1. Tạo dự án mới LabGuard.Collector dạng Console Service trên .NET 9.
2. Khởi tạo TcpListener lắng nghe kết nối từ các máy trạm trong dải mạng nội bộ phòng máy.
3. Giới hạn số lượng kết nối tối đa MaxConcurrentConnections = 100 để bảo vệ tài nguyên.
4. Bàn giao từng kết nối máy trạm cho bộ đọc FrameCodec để bóc tách gói tin an toàn.
5. Chuẩn bị cơ chế dự phòng tắt Collector và chuyển thẳng Agent về Central nếu hệ thống gặp lỗi.

> **📌 BẪY LỖI CẦN TRÁNH TRONG LG-C-013**
> Không giới hạn số kết nối khiến máy chủ Collector bị cạn kiệt socket handle khi các máy trạm kết nối lại đồng loạt.

5. Tiêu chuẩn hoàn thành 11 mục (Definition of Done — DoD):
• [ ] 1. Hợp đồng giao thức & Wire Envelope chuẩn UTF-8 JSON
• [ ] 2. Lập trình bất đồng bộ Async/Await không chặn luồng mạng
• [ ] 3. Bắt ngoại lệ mạng an toàn (EOF, SocketException, Timeout)
• [ ] 4. Giải phóng tài nguyên sạch sẽ (Dispose / Using / CancellationToken)
• [ ] 5. An toàn dữ liệu (Khử trùng lặp lô / Khóa lạc quan OCC / Transaction)
• [ ] 6. Unit Test / Test Harness chạy đạt 100% không có cảnh báo
• [ ] 7. Tích hợp chéo thành công với module của đồng đội liên quan
• [ ] 8. Kiểm thử thực tế trên mạng LAN nhiều máy tính
• [ ] 9. Đồng bộ tài liệu thiết kế & mã nguồn khớp hoàn toàn
• [ ] 10. Thu thập đầy đủ hồ sơ minh chứng (Log, Screenshot, Pcap)
• [ ] 11. Sẵn sàng trả lời trôi chảy câu hỏi vấn đáp (Viva) liên quan

6. Kịch bản kiểm thử & Hồ sơ minh chứng:
• Mã bài test: TEST-C13
• Phương pháp kiểm thử: Khởi chạy ứng dụng Collector trên máy trạm; cho Agent kết nối thử nghiệm
• Tiêu chuẩn nghiệm thu đạt: Chấp nhận kết nối từ Agent, đọc và phân khung FrameCodec đúng chuẩn giao thức
• Hồ sơ minh chứng lưu tại: Collector startup console log, cấu hình appsettings mẫu

7. Tình huống lỗi tiềm ẩn & Kế hoạch ứng phó: Quá tải số lượng kết nối; từ chối bớt kết nối khi vượt ngưỡng và báo NotReady về Gateway

8. Câu hỏi vấn đáp (Viva) & Chiến lược trả lời:
• Chủ đề câu hỏi: Kiến trúc ứng dụng Collector dùng chung cho nhiều phiên bản triển khai
• Câu trả lời kỳ vọng: Một file thực thi duy nhất nhận tham số cấu hình (Port, UpstreamHost, NodeId) qua command line hoặc appsettings để chạy thành nhiều instance độc lập
• Câu hỏi mở rộng / Bẫy của giảng viên: Tại sao không cần biên dịch 2 file mã nguồn riêng biệt cho 2 Collector?

[HƯỚNG DẪN ĐỀ XUẤT]: *Tham khảo mã nguồn mẫu chuẩn và tài liệu kỹ thuật tại thư mục Networking. Khi hiện thực, luôn đặt an toàn luồng và xử lý hủy tác vụ (CancellationToken) lên hàng đầu.*

Nhiệm vụ 14: [LG-C-014] — Collector upstream multiplex và reply router

Mã & Tên nhiệm vụ: [LG-C-014] Collector upstream multiplex và reply router	**Tuần & Ưu tiên:** Tuần W4 (P1) | P1
Khối lượng công việc: Cơ bản: 6.0h | Dự phòng: 0.5h | Tổng: 6.5h	**Phân loại tải:** P1 Extension
Trách nhiệm thực hiện: Phụ trách: SV C | Dự phòng: SV B | Duyệt: SV B	**Phân hệ & Thư mục:** P1.Collector (Networking)
Sản phẩm bàn giao: UpstreamMultiplexer.cs & ReplyRouter.cs	**Hợp đồng giao diện:** Single upstream TCP socket to Central, routing table clientConnectionId <-> socket

1. Mục đích kỹ thuật & Tại sao tồn tại: Collector phải gom luồng dữ liệu của hàng chục máy trạm để chuyển tiếp về Central qua một kết nối socket duy nhất (Multiplexing), đồng thời khi Central gửi phản hồi về, Collector phải biết gửi trả lại đúng máy trạm nào (Reply Routing).

2. Kiến thức nền tảng cần nắm vững: Kỹ thuật dồn kênh mạng (Stream Multiplexing), bảng định tuyến kết nối trong bộ nhớ, bản tin gom COLLECTOR_FORWARD.

3. Dữ liệu đầu vào & Đầu ra kỹ thuật: Đầu vào: Dòng gói tin từ nhiều máy trạm Agent và kết nối duy nhất đi tới Central Server.
Đầu ra: UpstreamMultiplexer.cs dồn kênh gửi lên Central và ReplyRouter.cs chuyển tiếp câu trả lời về đúng Agent.

4. Các bước triển khai chi tiết:
1. Duy trì kết nối TCP/TLS duy nhất từ Collector tới cổng nội bộ của Central Server.
2. Gán một mã định danh kết nối (clientConnectionId kiểu số nguyên) duy nhất cho mỗi máy trạm kết nối vào.
3. Đóng gói các gói tin của Agent vào bản tin gom COLLECTOR_FORWARD mang kèm clientConnectionId gửi lên Central.
4. Khi Central gửi phản hồi về qua kết nối upstream, đọc clientConnectionId và tra bảng ánh xạ để chuyển

tiếp đúng socket của Agent tương ứng.
5. Xóa bỏ ánh xạ trong bảng định tuyến khi máy trạm ngắt kết nối.

> 📌 **BẪY LỖI CẦN TRÁNH TRONG LG-C-014**
> Nhầm lẫn clientConnectionId giữa các máy trạm, làm câu trả lời của máy này bị chuyển nhầm sang màn hình của máy khác.

5. Tiêu chuẩn hoàn thành 11 mục (Definition of Done — DoD):
• [ ] 1. Hợp đồng giao thức & Wire Envelope chuẩn UTF-8 JSON
• [ ] 2. Lập trình bất đồng bộ Async/Await không chặn luồng mạng
• [ ] 3. Bắt ngoại lệ mạng an toàn (EOF, SocketException, Timeout)
• [ ] 4. Giải phóng tài nguyên sạch sẽ (Dispose / Using / CancellationToken)
• [ ] 5. An toàn dữ liệu (Khử trùng lặp lô / Khóa lạc quan OCC / Transaction)
• [ ] 6. Unit Test / Test Harness chạy đạt 100% không có cảnh báo
• [ ] 7. Tích hợp chéo thành công với module của đồng đội liên quan
• [ ] 8. Kiểm thử thực tế trên mạng LAN nhiều máy tính
• [ ] 9. Đồng bộ tài liệu thiết kế & mã nguồn khớp hoàn toàn
• [ ] 10. Thu thập đầy đủ hồ sơ minh chứng (Log, Screenshot, Pcap)
• [ ] 11. Sẵn sàng trả lời trôi chảy câu hỏi vấn đáp (Viva) liên quan

6. Kịch bản kiểm thử & Hồ sơ minh chứng:
• Mã bài test: TEST-C14
• Phương pháp kiểm thử: Kết nối 10 Agent vào Collector; kiểm tra luồng tin gửi về Central và ACK trả ngược lại
• Tiêu chuẩn nghiệm thu đạt: Dữ liệu được dồn kênh chính xác; 10 Agent nhận đúng bản tin ACK tương ứng mà không bị nhầm lẫn
• Hồ sơ minh chứng lưu tại: Multiplexing trace log, bảng định tuyến router

7. Tình huống lỗi tiềm ẩn & Kế hoạch ứng phó: Một Agent bị đứt kết nối trong khi ACK đang trên đường về; kiểm tra socket còn sống trước khi ghi trả ACK

8. Câu hỏi vấn đáp (Viva) & Chiến lược trả lời:
• Chủ đề câu hỏi: Kỹ thuật dồn luồng và bảng định tuyến phản hồi trong Proxy trung chuyển
• Câu trả lời kỳ vọng: Gắn nhãn định danh kết nối (ConnectionId) vào gói tin gửi upstream; khi nhận ACK sẽ tra bảng định tuyến để trả lời đúng socket client
• Câu hỏi mở rộng / Bẫy của giảng viên: Làm thế nào tránh rò rỉ bộ nhớ trong bảng định tuyến kết nối?

[HƯỚNG DẪN ĐỀ XUẤT]: *Tham khảo mã nguồn mẫu chuẩn và tài liệu kỹ thuật tại thư mục Networking. Khi hiện thực, luôn đặt an toàn luồng và xử lý hủy tác vụ (CancellationToken) lên hàng đầu.*

Nhiệm vụ 15: [LG-C-015] — Collector readiness/health và bounded buffers

Mã & Tên nhiệm vụ: [LG-C-015] Collector readiness/health và bounded buffers	**Tuần & Ưu tiên:** Tuần W4 (P1) | P1
Khối lượng công việc: Cơ bản: 4.0h | Dự phòng: 0.5h | Tổng: 4.5h	**Phân loại tải:** P1 Extension
Trách nhiệm thực hiện: Phụ trách: SV C | Dự phòng: SV A | Duyệt: SV A	**Phân hệ & Thư mục:** P1.Collector (Networking)
Sản phẩm bàn giao: CollectorHealthService.cs & BoundedForwardBuffer.cs	**Hợp đồng giao diện:** TCP Health Port cho Gateway probe, Bounded Channel buffer

1. Mục đích kỹ thuật & Tại sao tồn tại: Nếu đường truyền từ Collector về Central bị nghẽn, Collector không được để hàng đợi trong RAM phình to vô hạn làm sập máy. Cần có kênh đệm có giới hạn (Bounded Buffer) và mở cổng kiểm tra sức khỏe (Health Check Port) để theo dõi.

2. Kiến thức nền tảng cần nắm vững: Kênh hàng đợi có chặn (Bounded Channel), cổng kiểm tra sức khỏe TCP, giám sát lưu lượng chuyển tiếp.

3. Dữ liệu đầu vào & Đầu ra kỹ thuật: Đầu vào: Dòng gói tin cần chuyển tiếp và giới hạn dung lượng hàng đợi đệm (Capacity = 2000 gói tin).
Đầu ra: CollectorHealthService.cs báo cáo nhịp thở và BoundedForwardBuffer.cs bảo vệ an toàn bộ nhớ.

4. Các bước triển khai chi tiết:
1. Khởi tạo Bounded Channel với dung lượng 2000 phần tử để đệm các gói tin chờ gửi về Central.
2. Cấu hình chính sách khi đầy đệm: Chặn tạm thời luồng đọc của Agent (áp lực ngược - Backpressure) để Agent tự đệm ở local.
3. Mở một cổng TCP phụ phục vụ kiểm tra sức khỏe (Health Probe): trả về chuỗi 'OK' kèm số lượng kết nối hiện hữu.
4. Ghi nhận nhật ký cảnh báo khi hàng đợi vượt quá 80% dung lượng.
5. Đảm bảo ứng dụng giải phóng tài nguyên sạch sẽ khi nhận lệnh tắt tiến trình.

> **📌 BẪY LỖI CẦN TRÁNH TRONG LG-C-015**
> Dùng hàng đợi vô hạn khiến ứng dụng Collector bị tràn bộ nhớ OutOfMemory chỉ sau vài phút nếu đường mạng lên Central bị đứt.

5. Tiêu chuẩn hoàn thành 11 mục (Definition of Done — DoD):
- [ ] 1. Hợp đồng giao thức & Wire Envelope chuẩn UTF-8 JSON
- [ ] 2. Lập trình bất đồng bộ Async/Await không chặn luồng mạng
- [ ] 3. Bắt ngoại lệ mạng an toàn (EOF, SocketException, Timeout)
- [ ] 4. Giải phóng tài nguyên sạch sẽ (Dispose / Using / CancellationToken)
- [ ] 5. An toàn dữ liệu (Khử trùng lặp lô / Khóa lạc quan OCC / Transaction)
- [ ] 6. Unit Test / Test Harness chạy đạt 100% không có cảnh báo
- [ ] 7. Tích hợp chéo thành công với module của đồng đội liên quan

• [ ] 8. Kiểm thử thực tế trên mạng LAN nhiều máy tính
• [ ] 9. Đồng bộ tài liệu thiết kế & mã nguồn khớp hoàn toàn
• [ ] 10. Thu thập đầy đủ hồ sơ minh chứng (Log, Screenshot, Pcap)
• [ ] 11. Sẵn sàng trả lời trôi chảy câu hỏi vấn đáp (Viva) liên quan

6. Kịch bản kiểm thử & Hồ sơ minh chứng:
• Mã bài test: TEST-C15
• Phương pháp kiểm thử: Mô phỏng ngắt kết nối upstream tới Central; kiểm tra phản ứng của buffer và health probe
• Tiêu chuẩn nghiệm thu đạt: Hàng đợi buffer không vượt ngưỡng dung lượng; cổng health báo NotReady tới Gateway để tạm dừng tiếp nhận
• Hồ sơ minh chứng lưu tại: Collector health status log, buffer memory usage graph

7. Tình huống lỗi tiềm ẩn & Kế hoạch ứng phó: Nghẽn mạng upstream làm tràn RAM máy chủ trung chuyển; kích hoạt backpressure làm chậm tốc độ đọc từ Agent

8. Câu hỏi vấn đáp (Viva) & Chiến lược trả lời:
• Chủ đề câu hỏi: Cơ chế kiểm tra độ sẵn sàng (Readiness Probe) và điều tiết áp lực ngược (Backpressure)
• Câu trả lời kỳ vọng: Chỉ nhận tải khi downstream và upstream đều sẵn sàng; tự động kích hoạt hàng rào bảo vệ khi bộ đệm đạt ngưỡng giới hạn
• Câu hỏi mở rộng / Bẫy của giảng viên: Tại sao Readiness Probe khác với Liveness Probe?

[HƯỚNG DẪN ĐỀ XUẤT]: *Tham khảo mã nguồn mẫu chuẩn và tài liệu kỹ thuật tại thư mục Networking. Khi hiện thực, luôn đặt an toàn luồng và xử lý hủy tác vụ (CancellationToken) lên hàng đầu.*

Nhiệm vụ 16: [LG-C-016] — Multi-admin concurrency và race condition tests

Mã & Tên nhiệm vụ: [LG-C-016] Multi-admin concurrency và race condition tests	**Tuần & Ưu tiên:** Tuần W4 (P0) | P0/Stability
Khối lượng công việc: Cơ bản: 3.0h | Dự phòng: 0.5h | Tổng: 3.5h	**Phân loại tải:** Test/Harness
Trách nhiệm thực hiện: Phụ trách: SV C | Dự phòng: SV B | Duyệt: SV B	**Phân hệ & Thư mục:** Admin.Testing (Testing)
Sản phẩm bàn giao: MultiAdminTestReport.md	**Hợp đồng giao diện:** 2 WinForms Admin clients, cùng thao tác trên 1 Ticket sự cố

1. Mục đích kỹ thuật & Tại sao tồn tại: Cần kiểm chứng trên môi trường thực tế tình huống 2 cán bộ quản trị mở 2 cửa sổ WinForms khác nhau và cùng tranh chấp xử lý 1 sự cố tại cùng một giây, chứng minh giao diện thông báo xung đột rõ ràng và tự hồi phục mà không bị sập.

2. Kiến thức nền tảng cần nắm vững: Kiểm thử điều kiện tương tranh trên giao diện (UI Concurrency Testing), chụp ảnh màn hình minh chứng lỗi xung đột.

3. Dữ liệu đầu vào & Đầu ra kỹ thuật: Đầu vào: 2 máy tính chạy 2 phiên bản WinForms Admin khác nhau đăng nhập cùng lúc và 1 sự cố đang mở.
Đầu ra: Báo cáo MultiAdminTestReport.md đính kèm ảnh chụp màn hình thông báo CONCURRENCY_CONFLICT.

4. Các bước triển khai chi tiết:
1. Khởi chạy 2 ứng dụng WinForms Admin trên 2 máy tính trong mạng LAN.
2. Kích hoạt một sự cố cảnh báo trên hệ thống để cả 2 màn hình cùng nhìn thấy.
3. Cán bộ 1 bấm nút ACK để tiếp nhận sự cố (thao tác thành công).
4. Cán bộ 2 ngay sau đó bấm nút ACK trên màn hình chưa kịp làm mới của mình.
5. Xác nhận giao diện của Cán bộ 2 hiển thị hộp thoại thông báo xung đột rõ ràng, tự động tải lại trạng thái mới và không bị treo giao diện.

> 📌 **BẪY LỖI CẦN TRÁNH TRONG LG-C-016**
> Giao diện bị đứng đơ (freeze) khi nhận mã lỗi xung đột do xử lý ngoại lệ sai luồng giao diện.

5. Tiêu chuẩn hoàn thành 11 mục (Definition of Done — DoD):
• [ ] 1. Hợp đồng giao thức & Wire Envelope chuẩn UTF-8 JSON
• [ ] 2. Lập trình bất đồng bộ Async/Await không chặn luồng mạng
• [ ] 3. Bắt ngoại lệ mạng an toàn (EOF, SocketException, Timeout)
• [ ] 4. Giải phóng tài nguyên sạch sẽ (Dispose / Using / CancellationToken)
• [ ] 5. An toàn dữ liệu (Khử trùng lặp lô / Khóa lạc quan OCC / Transaction)
• [ ] 6. Unit Test / Test Harness chạy đạt 100% không có cảnh báo
• [ ] 7. Tích hợp chéo thành công với module của đồng đội liên quan
• [ ] 8. Kiểm thử thực tế trên mạng LAN nhiều máy tính
• [ ] 9. Đồng bộ tài liệu thiết kế & mã nguồn khớp hoàn toàn
• [ ] 10. Thu thập đầy đủ hồ sơ minh chứng (Log, Screenshot, Pcap)
• [ ] 11. Sẵn sàng trả lời trôi chảy câu hỏi vấn đáp (Viva) liên quan

6. Kịch bản kiểm thử & Hồ sơ minh chứng:
• Mã bài test: TEST-C16
• Phương pháp kiểm thử: 2 người cùng bấm nút Resolve trong cùng 1 giây
• Tiêu chuẩn nghiệm thu đạt: Người bấm trước thành công; người bấm sau nhận hộp thoại thông báo xung đột dữ liệu thân thiện và màn hình tự làm mới
• Hồ sơ minh chứng lưu tại: MultiAdminTestReport.md, video ghi lại thao tác xung đột

7. Tình huống lỗi tiềm ẩn & Kế hoạch ứng phó: Ứng dụng bị crash hoặc hiện mã lỗi thô; hiển thị thông báo nghiệp vụ rõ ràng hướng dẫn người dùng làm mới dữ liệu

8. Câu hỏi vấn đáp (Viva) & Chiến lược trả lời:
• Chủ đề câu hỏi: Trải nghiệm người dùng khi xảy ra xung đột tương tranh dữ liệu
• Câu trả lời kỳ vọng: Bắt mã lỗi xung đột OCC từ server và hiển thị hộp thoại thông báo dễ hiểu, tự động tải lại dữ liệu mới nhất cho người dùng

• Câu hỏi mở rộng / Bẫy của giảng viên: Làm thế nào để thông báo cho Admin thứ hai biết ngay khi Admin thứ nhất vừa thao tác?

[HƯỚNG DẪN ĐỀ XUẤT]: *Tham khảo mã nguồn mẫu chuẩn và tài liệu kỹ thuật tại thư mục Testing. Khi hiện thực, luôn đặt an toàn luồng và xử lý hủy tác vụ (CancellationToken) lên hàng đầu.*

Nhiệm vụ 17: [LG-C-017] — UI release regression và settings cleanup

Mã & Tên nhiệm vụ: [LG-C-017] UI release regression và settings cleanup	**Tuần & Ưu tiên:** Tuần W5 (P0) | P0/Release
Khối lượng công việc: Cơ bản: 3.0h | Dự phòng: 0.5h | Tổng: 3.5h	**Phân loại tải:** Test/Harness
Trách nhiệm thực hiện: Phụ trách: SV C | Dự phòng: SV A | Duyệt: SV A	**Phân hệ & Thư mục:** Release.UiRegression (Testing)
Sản phẩm bàn giao: UiRegressionChecklist.md	**Hợp đồng giao diện:** High-DPI Per-Monitor V2 support trong WinForms

1. Mục đích kỹ thuật & Tại sao tồn tại: Khi mang ứng dụng đi trình chiếu trên máy chiếu hội đồng hoặc màn hình máy tính của giảng viên (vốn có độ phân giải và tỷ lệ thu phóng Scale 125%, 150% khác nhau), giao diện không được bị mờ hoặc vỡ nét. Cần kiểm tra hồi quy toàn diện trước khi phát hành.

2. Kiến thức nền tảng cần nắm vững: Kiểm thử độ tương thích màn hình High-DPI trong Windows, dọn dẹp các tệp cấu hình tạm thời, kiểm tra hồi quy giao diện.

3. Dữ liệu đầu vào & Đầu ra kỹ thuật: Đầu vào: Ứng dụng WinForms đã hoàn thiện và các cấu hình hiển thị màn hình khác nhau (100%, 125%, 150%).
Đầu ra: Tài liệu UiRegressionChecklist.md xác nhận 100% màn hình hiển thị sắc nét, không lỗi font, không mất chữ.

4. Các bước triển khai chi tiết:
1. Kiểm tra cấu hình PerMonitorV2 trong tệp manifest và mã khởi tạo Program.cs.
2. Thử nghiệm thay đổi độ thu phóng màn hình Windows từ 100% lên 125% và 150%, kiểm tra các icon và chữ viết vẫn sắc nét.
3. Rà soát lại toàn bộ thông báo tiếng Việt trên giao diện, đảm bảo chuẩn chính tả và đồng nhất thuật ngữ.
4. Xóa bỏ toàn bộ các chuỗi kết nối cứng (hard-coded connection strings) hoặc tài khoản thử nghiệm trong mã nguồn.
5. Đóng gói bản chạy sạch sẽ vào thư mục phát hành chính thức.

> 📌 **BẪY LỖI CẦN TRÁNH TRONG LG-C-017**
> Giao diện hiển thị tốt ở máy lập trình (100% scale) nhưng khi chiếu lên máy chiếu hội đồng (150% scale) thì các nút bấm bị thụt mất chữ hoặc tràn ra ngoài cửa sổ.

5. Tiêu chuẩn hoàn thành 11 mục (Definition of Done — DoD):
• [ ] 1. Hợp đồng giao thức & Wire Envelope chuẩn UTF-8 JSON
• [ ] 2. Lập trình bất đồng bộ Async/Await không chặn luồng mạng
• [ ] 3. Bắt ngoại lệ mạng an toàn (EOF, SocketException, Timeout)
• [ ] 4. Giải phóng tài nguyên sạch sẽ (Dispose / Using / CancellationToken)
• [ ] 5. An toàn dữ liệu (Khử trùng lặp lô / Khóa lạc quan OCC / Transaction)
• [ ] 6. Unit Test / Test Harness chạy đạt 100% không có cảnh báo
• [ ] 7. Tích hợp chéo thành công với module của đồng đội liên quan
• [ ] 8. Kiểm thử thực tế trên mạng LAN nhiều máy tính
• [ ] 9. Đồng bộ tài liệu thiết kế & mã nguồn khớp hoàn toàn
• [ ] 10. Thu thập đầy đủ hồ sơ minh chứng (Log, Screenshot, Pcap)
• [ ] 11. Sẵn sàng trả lời trôi chảy câu hỏi vấn đáp (Viva) liên quan

6. Kịch bản kiểm thử & Hồ sơ minh chứng:
• Mã bài test: TEST-C17
• Phương pháp kiểm thử: Chạy WinForms trên màn hình Full HD ở các mức scale 100%, 125% và 150%
• Tiêu chuẩn nghiệm thu đạt: Giao diện không bị mờ nhạt (blurry), chữ không bị cắt cụt, các icon tự co giãn sắc nét
• Hồ sơ minh chứng lưu tại: Ảnh chụp màn hình đối chiếu ở 3 tỷ lệ DPI khác nhau

7. Tình huống lỗi tiềm ẩn & Kế hoạch ứng phó: Lỗi co giãn DPI làm vỡ form giao diện; bật cấu hình HighDpiMode = SystemAware hoặc PerMonitorV2 trong Program.cs

8. Câu hỏi vấn đáp (Viva) & Chiến lược trả lời:
• Chủ đề câu hỏi: Hỗ trợ màn hình độ phân giải cao (High DPI Scaling) trong WinForms
• Câu trả lời kỳ vọng: Cấu hình ứng dụng nhận biết DPI để hệ điều hành không tự động kéo dãn pixel làm mờ giao diện; thiết kế layout co giãn linh hoạt
• Câu hỏi mở rộng / Bẫy của giảng viên: Các thuộc tính Anchor và Dock trong WinForms hỗ trợ co giãn màn hình ra sao?

[HƯỚNG DẪN ĐỀ XUẤT]: *Tham khảo mã nguồn mẫu chuẩn và tài liệu kỹ thuật tại thư mục Testing. Khi hiện thực, luôn đặt an toàn luồng và xử lý hủy tác vụ (CancellationToken) lên hàng đầu.*

Nhiệm vụ 18: [LG-C-018] — Demo rehearsal và backup assets

Mã & Tên nhiệm vụ: [LG-C-018] Demo rehearsal và backup assets	**Tuần & Ưu tiên:** Tuần W5 (P0) | P0/Demo
Khối lượng công việc: Cơ bản: 1.0h | Dự phòng: 0.5h | Tổng: 1.5h	**Phân loại tải:** P0 Core
Trách nhiệm thực hiện: Phụ trách: SV C | Dự phòng: SV A | Duyệt: SV A	**Phân hệ & Thư mục:** Release.Demo (UI)

Sản phẩm bàn giao: DemoScript.md & DemoBackup.mp4	**Hợp đồng giao diện:** Kịch bản demo chuẩn gồm 6 bước nghiệp vụ xuyên suốt

1. Mục đích kỹ thuật & Tại sao tồn tại: Buổi bảo vệ đồ án trước hội đồng chỉ có 12 phút trình bày thực tế. Nhóm cần một kịch bản diễn tập chuẩn xác từng giây với 6 bước nghiệp vụ nối tiếp, đồng thời phải có video quay sẵn chất lượng cao để dự phòng rủi ro mạng hội đồng trục trặc.

2. Kiến thức nền tảng cần nắm vững: Kỹ năng trình diễn sản phẩm công nghệ (Tech Demo), kỹ năng phân vai thuyết trình, chuẩn bị phương án dự phòng sự cố sân khấu.

3. Dữ liệu đầu vào & Đầu ra kỹ thuật: Đầu vào: Hệ thống LabGuard hoàn chỉnh chạy trên 3 máy tính mạng LAN và kịch bản trình diễn 6 bước.
Đầu ra: Kịch bản DemoScript.md chi tiết từng lời thoại và tệp video quay sẵn DemoBackup.mp4 sắc nét.

4. Các bước triển khai chi tiết:
1. Soạn thảo kịch bản Demo 6 bước: Khởi động hệ thống -> Đăng nhập an toàn -> Giám sát phòng máy thời gian thực -> Giả lập sự cố quá tải CPU -> Tiếp nhận xử lý sự cố OCC -> Báo cáo thống kê & Khôi phục.
2. Phân vai rõ ràng: A phụ trách thao tác máy trạm và rút dây mạng; B giải thích cơ chế Server/CSDL; C điều khiển Dashboard và thuyết minh.
3. Quay video sắc nét toàn bộ quá trình demo từ đầu đến cuối, lưu vào USB và laptop cá nhân làm tài sản dự phòng.
4. Bấm giờ diễn tập thực tế đúng 12 phút, không để cháy giáo án.
5. Chuẩn bị sẵn dây mạng LAN và router Wi-Fi độc lập đề phòng mạng trường đại học bị chập chờn.

> 📌 **BẪY LỖI CẦN TRÁNH TRONG LG-C-018**
> Không chuẩn bị video dự phòng, khi mang lên hội đồng máy tính không nhận mạng hoặc phòng máy bị mất điện thì đứng nhìn và bị điểm liệt.

5. Tiêu chuẩn hoàn thành 11 mục (Definition of Done — DoD):
- [ ] 1. Hợp đồng giao thức & Wire Envelope chuẩn UTF-8 JSON
- [ ] 2. Lập trình bất đồng bộ Async/Await không chặn luồng mạng
- [ ] 3. Bắt ngoại lệ mạng an toàn (EOF, SocketException, Timeout)
- [ ] 4. Giải phóng tài nguyên sạch sẽ (Dispose / Using / CancellationToken)
- [ ] 5. An toàn dữ liệu (Khử trùng lặp lô / Khóa lạc quan OCC / Transaction)
- [ ] 6. Unit Test / Test Harness chạy đạt 100% không có cảnh báo
- [ ] 7. Tích hợp chéo thành công với module của đồng đội liên quan
- [ ] 8. Kiểm thử thực tế trên mạng LAN nhiều máy tính
- [ ] 9. Đồng bộ tài liệu thiết kế & mã nguồn khớp hoàn toàn
- [ ] 10. Thu thập đầy đủ hồ sơ minh chứng (Log, Screenshot, Pcap)
- [ ] 11. Sẵn sàng trả lời trôi chảy câu hỏi vấn đáp (Viva) liên quan

6. Kịch bản kiểm thử & Hồ sơ minh chứng:
• Mã bài test: TEST-C18
• Phương pháp kiểm thử: Chạy thử toàn bộ kịch bản demo theo đúng thời lượng quy định với 3 máy thật
• Tiêu chuẩn nghiệm thu đạt: Kịch bản demo trơn tru, làm nổi bật đầy đủ các yêu cầu môn học NT106; có video backup sẵn sàng
• Hồ sơ minh chứng lưu tại: DemoScript.md, video demo backup chất lượng 1080p

7. Tình huống lỗi tiềm ẩn & Kế hoạch ứng phó: Mạng phòng thi gặp sự cố trong buổi báo cáo; lập tức chuyển sang video backup chất lượng cao để trình bày không bị gián đoạn

8. Câu hỏi vấn đáp (Viva) & Chiến lược trả lời:
• Chủ đề câu hỏi: Kỹ năng lập kế hoạch demo và quản trị rủi ro khi báo cáo sản phẩm
• Câu trả lời kỳ vọng: Luôn chuẩn bị phương án dự phòng (video demo, dữ liệu mẫu, cấu hình offline) để đối phó với sự cố môi trường ngoài dự kiến
• Câu hỏi mở rộng / Bẫy của giảng viên: Làm thế nào minh chứng video demo được quay từ sản phẩm thật của nhóm?

[HƯỚNG DẪN ĐỀ XUẤT]: *Tham khảo mã nguồn mẫu chuẩn và tài liệu kỹ thuật tại thư mục UI. Khi hiện thực, luôn đặt an toàn luồng và xử lý hủy tác vụ (CancellationToken) lên hàng đầu.*

Nhiệm vụ 19: [LG-C-019] — Release review checklist và README

Mã & Tên nhiệm vụ: [LG-C-019] Release review checklist và README	**Tuần & Ưu tiên:** Tuần W5 (P0) | P0/Release
Khối lượng công việc: Cơ bản: 1.0h | Dự phòng: 0.0h | Tổng: 1.0h	**Phân loại tải:** P0 Core
Trách nhiệm thực hiện: Phụ trách: SV C | Dự phòng: SV B | Duyệt: SV B	**Phân hệ & Thư mục:** Release.Docs (System)
Sản phẩm bàn giao: README.md & RELEASE_CHECKLIST.md	**Hợp đồng giao diện:** Tài liệu hướng dẫn cài đặt chuẩn, yêu cầu môi trường .NET 9

1. Mục đích kỹ thuật & Tại sao tồn tại: Một sản phẩm phần mềm chuyên nghiệp phải có hướng dẫn cài đặt và vận hành rõ ràng để bất kỳ ai cũng có thể triển khai thành công chỉ sau 5 phút đọc tài liệu. Cần biên soạn tài liệu README và danh mục kiểm tra phát hành hoàn chỉnh.

2. Kiến thức nền tảng cần nắm vững: Kỹ năng viết tài liệu kỹ thuật (Technical Writing), định dạng Markdown, quy chuẩn đóng gói sản phẩm phần mềm.

3. Dữ liệu đầu vào & Đầu ra kỹ thuật: Đầu vào: Toàn bộ mã nguồn, tệp thực thi phát hành và yêu cầu môi trường (.NET 9 SDK, SQL Server 2022).
Đầu ra: Tài liệu README.md chuẩn mực và danh mục kiểm tra phát hành RELEASE_CHECKLIST.md.

4. Các bước triển khai chi tiết:
1. Soạn thảo README.md gồm: Tổng quan sản phẩm, Yêu cầu cấu hình tối thiểu, Hướng dẫn cài đặt

CSDL, Hướng dẫn chạy Central, Agent và Dashboard.
2. Đính kèm các lệnh chạy dòng lệnh mẫu cho cả Windows Terminal và PowerShell.
3. Kiểm tra danh mục phát hành: Mã nguồn không chứa tệp rác (.vs, bin, obj), các chứng chỉ mẫu và file cấu hình đã có sẵn.
4. Đóng gói bộ cài đặt hoặc tệp nén ZIP hoàn chỉnh theo đúng cấu trúc tiêu chuẩn UIT.
5. Bàn giao bộ hồ sơ phát hành cho cả nhóm kiểm tra chéo lần cuối.

> 📌 **BẪY LỖI CẦN TRÁNH TRONG LG-C-019**
> Viết hướng dẫn sơ sài, thiếu bước cài đặt runtime hoặc thiếu script tạo CSDL khiến người khác tải về không thể chạy được.

5. Tiêu chuẩn hoàn thành 11 mục (Definition of Done — DoD):
• [ ] 1. Hợp đồng giao thức & Wire Envelope chuẩn UTF-8 JSON
• [ ] 2. Lập trình bất đồng bộ Async/Await không chặn luồng mạng
• [ ] 3. Bắt ngoại lệ mạng an toàn (EOF, SocketException, Timeout)
• [ ] 4. Giải phóng tài nguyên sạch sẽ (Dispose / Using / CancellationToken)
• [ ] 5. An toàn dữ liệu (Khử trùng lặp lô / Khóa lạc quan OCC / Transaction)
• [ ] 6. Unit Test / Test Harness chạy đạt 100% không có cảnh báo
• [ ] 7. Tích hợp chéo thành công với module của đồng đội liên quan
• [ ] 8. Kiểm thử thực tế trên mạng LAN nhiều máy tính
• [ ] 9. Đồng bộ tài liệu thiết kế & mã nguồn khớp hoàn toàn
• [ ] 10. Thu thập đầy đủ hồ sơ minh chứng (Log, Screenshot, Pcap)
• [ ] 11. Sẵn sàng trả lời trôi chảy câu hỏi vấn đáp (Viva) liên quan

6. Kịch bản kiểm thử & Hồ sơ minh chứng:
• Mã bài test: TEST-C19
• Phương pháp kiểm thử: Người ngoài nhóm làm theo hướng dẫn trong README để cài đặt và chạy thử
• Tiêu chuẩn nghiệm thu đạt: Cài đặt thành công trên máy tính mới trong vòng 10 phút; tài liệu rõ ràng, dễ hiểu
• Hồ sơ minh chứng lưu tại: README.md, RELEASE_CHECKLIST.md

7. Tình huống lỗi tiềm ẩn & Kế hoạch ứng phó: Thiếu các bước cấu hình kết nối CSDL hoặc chứng chỉ; bổ sung chi tiết từng bước có ảnh chụp minh họa

8. Câu hỏi vấn đáp (Viva) & Chiến lược trả lời:
• Chủ đề câu hỏi: Chuẩn hóa tài liệu chuyển giao và hướng dẫn triển khai sản phẩm
• Câu trả lời kỳ vọng: Tài liệu rõ ràng giúp người chấm bài hoặc kỹ sư tiếp quản có thể dễ dàng dựng môi trường và kiểm chứng tính đúng đắn
• Câu hỏi mở rộng / Bẫy của giảng viên: Cần cung cấp những thông tin tối thiểu nào trong tài liệu hướng dẫn kỹ thuật?

[HƯỚNG DẪN ĐỀ XUẤT]: *Tham khảo mã nguồn mẫu chuẩn và tài liệu kỹ thuật tại thư mục System. Khi hiện thực, luôn đặt an toàn luồng và xử lý hủy tác vụ (CancellationToken) lên hàng đầu.*

Nhiệm vụ 20: [LG-C-020] — Viva C và bàn giao backup

Mã & Tên nhiệm vụ: [LG-C-020] Viva C và bàn giao backup	**Tuần & Ưu tiên:** Tuần W5 (P0) | P0/Viva
Khối lượng công việc: Cơ bản: 5.0h | Dự phòng: 0.0h | Tổng: 5.0h	**Phân loại tải:** Viva
Trách nhiệm thực hiện: Phụ trách: SV C | Dự phòng: SV B | Duyệt: SV B	**Phân hệ & Thư mục:** Viva (Viva)
Sản phẩm bàn giao: HANDBOOK_C.md & Viva rehearsal log	**Hợp đồng giao diện:** 30 câu hỏi viva chuyên sâu phần WinForms, Cache & Collector App

1. Mục đích kỹ thuật & Tại sao tồn tại: Buổi vấn đáp cá nhân quyết định trực tiếp điểm số của sinh viên C. Giảng viên sẽ hỏi sâu về luồng giao diện WinForms, cơ chế Invoke, điều phối gói tin RPC, khóa lạc quan và kiến trúc Collector. Cần diễn tập thuần thục 30 câu hỏi để đạt điểm tối đa.

2. Kiến thức nền tảng cần nắm vững: Toàn bộ kiến thức lý thuyết và thực hành của phần Dashboard WinForms, lập trình mạng phía Client, cơ chế dồn kênh Collector.

3. Dữ liệu đầu vào & Đầu ra kỹ thuật: Đầu vào: Bộ 30 câu hỏi vấn đáp chuyên sâu dành cho vai C trong sổ tay HANDBOOK_C.md.
Đầu ra: Hồ sơ chuẩn bị vấn đáp hoàn chỉnh kèm nhật ký diễn tập trả lời trôi chảy (Viva rehearsal log).

4. Các bước triển khai chi tiết:
1. Ôn tập kỹ nguyên lý luồng STA trong WinForms, giải thích tại sao phải dùng Control.InvokeRequired và hậu quả nếu vi phạm.
2. Nắm chắc cơ chế khớp cặp yêu cầu/phản hồi bằng TaskCompletionSource và ConcurrentDictionary.
3. Chuẩn bị đoạn code minh chứng cho việc bắt mã lỗi CONCURRENCY_CONFLICT và cập nhật lại giao diện người dùng.
4. Giải thích kiến trúc dồn kênh (Multiplexing) của Collector và cách định tuyến gói tin qua clientConnectionId.
5. Tự tin trả lời mạch lạc các câu hỏi tình huống của giảng viên và dẫn chứng chính xác dòng code trong đồ án.

> **📌 BẪY LỖI CẦN TRÁNH TRONG LG-C-020**
> Không trả lời được câu hỏi kinh điển về lỗi Cross-thread trong WinForms, khiến giảng viên đánh giá là không tự tay viết code giao diện.

5. Tiêu chuẩn hoàn thành 11 mục (Definition of Done — DoD):
• [ ] 1. Hợp đồng giao thức & Wire Envelope chuẩn UTF-8 JSON
• [ ] 2. Lập trình bất đồng bộ Async/Await không chặn luồng mạng
• [ ] 3. Bắt ngoại lệ mạng an toàn (EOF, SocketException, Timeout)

• [ ] 4. Giải phóng tài nguyên sạch sẽ (Dispose / Using / CancellationToken)
• [ ] 5. An toàn dữ liệu (Khử trùng lặp lô / Khóa lạc quan OCC / Transaction)
• [ ] 6. Unit Test / Test Harness chạy đạt 100% không có cảnh báo
• [ ] 7. Tích hợp chéo thành công với module của đồng đội liên quan
• [ ] 8. Kiểm thử thực tế trên mạng LAN nhiều máy tính
• [ ] 9. Đồng bộ tài liệu thiết kế & mã nguồn khớp hoàn toàn
• [ ] 10. Thu thập đầy đủ hồ sơ minh chứng (Log, Screenshot, Pcap)
• [ ] 11. Sẵn sàng trả lời trôi chảy câu hỏi vấn đáp (Viva) liên quan

6. Kịch bản kiểm thử & Hồ sơ minh chứng:
• Mã bài test: TEST-C20
• Phương pháp kiểm thử: Vấn đáp chéo giả định với thành viên A và B chấm điểm độc lập
• Tiêu chuẩn nghiệm thu đạt: Giải thích thành thạo các cơ chế UI thread, RPC correlation, snapshot cache và kiến trúc Collector
• Hồ sơ minh chứng lưu tại: HANDBOOK_C.md, biên bản diễn tập vấn đáp nội bộ

7. Tình huống lỗi tiềm ẩn & Kế hoạch ứng phó: Nhầm lẫn giữa luồng giao diện và luồng mạng; ôn tập kỹ sơ đồ marshaling trong handbook

8. Câu hỏi vấn đáp (Viva) & Chiến lược trả lời:
• Chủ đề câu hỏi: Tổng quan kiến trúc ứng dụng quản trị và điều phối giao diện của Lane C
• Câu trả lời kỳ vọng: Trình bày rõ ràng cấu trúc ứng dụng WinForms, cơ chế nhận snapshot và delta event để duy trì tính nhất quán thời gian thực
• Câu hỏi mở rộng / Bẫy của giảng viên: Bài học quan trọng nhất về việc thiết kế giao diện cho hệ thống giám sát sự cố?

[HƯỚNG DẪN ĐỀ XUẤT]: *Tham khảo mã nguồn mẫu chuẩn và tài liệu kỹ thuật tại thư mục Viva. Khi hiện thực, luôn đặt an toàn luồng và xử lý hủy tác vụ (CancellationToken) lên hàng đầu.*

PHẦN 8: MA TRẬN TÍCH HỢP & KỊCH BẢN KIỂM THỬ CHÉO

8.1. Ma trận tích hợp 3 bên (Integration Matrix): Trong hệ thống phân tán, các module không thể kiểm thử tách rời mãi được. Ma trận tích hợp quy định rõ các điểm giao thoa giữa 3 thành viên và 5 kịch bản tích hợp chéo end-to-end quan trọng nhất:

• Kịch bản 1: Bắt tay TLS & Nhịp tim sống (A <-> B):
Agent (máy A) kết nối tới Central (máy B), bắt tay chứng chỉ TLS thành công. Agent gửi HEARTBEAT mỗi 5 giây. Central cập nhật LastSeenUtc trong LivenessRegistry. Khi ngắt Agent, Central phát hiện quá hạn 20 giây và đánh dấu OFFLINE.

• Kịch bản 2: Đẩy lô số liệu & Khử trùng lặp (A <-> B):
Agent gửi gói tin METRIC_BATCH chứa 50 bản ghi. Central kiểm tra bảng ProcessedBatches bằng bộ khóa (DeviceId, BootId, BatchId). Central lưu vào CSDL và gửi BATCH_ACK. Agent nhận ACK và xóa đệm. Khi giả lập mạng lag làm Agent gửi lại, Central nhận diện trùng lặp, không ghi đè và gửi lại BATCH_ACK.

• Kịch bản 3: Đăng nhập & Đồng bộ Snapshot khởi tạo (B <-> C):
Ứng dụng WinForms (máy C) gửi LOGIN. Central (máy B) xác thực PBKDF2 và trả về LOGIN_RESULT kèm token. WinForms gửi SNAPSHOT_BEGIN, nhận các trang SNAPSHOT_PAGE nạp vào LocalCacheStore và kết thúc bằng SNAPSHOT_END. Toàn bộ 20 máy trạm hiển thị đầy đủ trên giao diện lưới.

• Kịch bản 4: Kích hoạt quy tắc & Khóa lạc quan OCC sự cố (A <-> B <-> C):
Agent A gửi CPU 95% liên tiếp 3 mẫu. Central B kích hoạt RuleEvaluator mở Incident mới, phát sóng INCIDENT_CREATED tới C. WinForms C hiển thị thẻ sự cố màu đỏ. Cán bộ 1 bấm ACK gửi expectedRowVersion=1 thành công. Cán bộ 2 bấm ACK trên màn hình chưa cập nhật, Central trả về CONCURRENCY_CONFLICT. Màn hình cán bộ 2 thông báo xung đột và tự làm mới.

• Kịch bản 5: Đứt cáp mạng & Tự động kết nối lại (Auto-Reconnect) (A, C -> B):
Rút dây mạng tại máy chủ Central. Cả Agent A và WinForms C nhận ngoại lệ EndOfStreamException, chuyển sang chế độ Reconnecting với thuật toán Exponential Backoff + Jitter. Khi cắm lại dây mạng, cả 2 bên tự động kết nối lại, bắt tay TLS thành công mà không cần người dùng can thiệp.

PHẦN 9: KỊCH BẢN KIỂM THỬ MẠNG LAN THỰC TẾ & HỒ SƠ MINH CHỨNG

9.1. Thiết lập môi trường mạng phòng Lab: Kiểm thử trên mạng LAN thật giữa 3 máy tính vật lý là yêu cầu bắt buộc của môn NT106 để chứng minh đồ án chạy thực sự qua hạ tầng mạng chứ không chỉ chạy ảo trên localhost.

Quy trình thiết lập IP LAN: 1. Chuẩn bị Router Wi-Fi cá nhân hoặc Switch 5-port có dây cáp Cat6 độc lập (đề phòng mạng trường học bị chặn cổng hoặc ngắt kết nối).
2. Máy B đóng vai trò Central Server đặt địa chỉ IP tĩnh (ví dụ: 192.168.1.100), mở cổng TCP 5000 và 5001 trên Windows Defender Firewall.
3. Máy A đóng vai trò Agent kết nối IP 192.168.1.100:5000.
4. Máy C đóng vai trò Admin Dashboard kết nối IP 192.168.1.100:5000.

9.2. Hồ sơ minh chứng thực nghiệm bắt buộc: Trong quá trình kiểm thử, nhóm phải thu thập đầy đủ 4 loại hồ sơ minh chứng kỹ thuật để đưa vào báo cáo và tệp nén bàn giao:
• Tệp bắt gói tin Wireshark (.pcapng): Bắt gói tin trên cổng 5000, chứng minh dữ liệu được bọc TLSv1.3 không thể đọc được nội dung nhạy cảm dạng bản rõ (plaintext).
• Ảnh chụp màn hình lỗi xung đột OCC: Bắt trọn khoảnh khắc hiển thị hộp thoại thông báo mã lỗi CONCURRENCY_CONFLICT khi 2 máy cùng bấm xử lý sự cố.
• Tệp nhật ký máy chủ (Server Console Log): Ghi nhận dòng log xác nhận nhận diện gói tin trùng lặp (Dedup Hit: Batch already processed, sending ACK).
• Báo cáo kiểm thử hồi quy (dotnet test report): Minh chứng 100% test cases tự động chạy đạt kết quả xanh sạch.

PHẦN 10: 90 CÂU HỎI VẤN ĐÁP (VIVA) & CHIẾN LƯỢC TRẢ LỜI ĐIỂM CAO

10.1. Chiến lược trả lời vấn đáp 3 bước vàng: Chiến lược trả lời vấn đáp của UIT để đạt điểm tối đa (9-10/10) tuân theo công thức 3 bước vàng:
• BƯỚC 1: Nêu bản chất lý thuyết cốt lõi ngắn gọn trong 1 câu (Intuition/Why).
• BƯỚC 2: Giải thích quyết định thiết kế của nhóm trong đồ án (Design Decision).
• BƯỚC 3: Mở Visual Studio và chỉ chính xác vị trí tệp, tên hàm và dòng code hiện thực (Code Evidence).

Dưới đây là tuyển tập 90 câu hỏi vấn đáp chuyên sâu được trích xuất trực tiếp từ sổ tay vấn đáp của 3 thành viên:

Bộ 30 câu hỏi chuyên sâu — SINH VIÊN A (AGENT & LẬP TRÌNH MẠNG)

Câu hỏi #01: Tại sao TCP được gọi là giao thức hướng dòng byte (Byte Stream)?

• **Trả lời cốt lõi:** TCP truyền một chuỗi byte liên tục và bảo đảm tin cậy, có thứ tự; tuy nhiên TCP hoàn toàn không giữ ranh giới thông điệp (Message Boundary) của tầng ứng dụng. Một lần gửi có thể bị chia nhỏ hoặc dính liền khi nhận.

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #02: Hiện tượng phân mảnh (Partial Read) và dính gói (Frame Coalescing) trong TCP là gì?

• **Trả lời cốt lõi:** Partial Read là hiện tượng hàm `ReadAsync` chỉ đọc được một phần thông điệp do gói tin bị phân mảnh trên mạng hoặc bộ đệm socket chưa đầy; Frame Coalescing là hiện tượng nhiều thông điệp được gửi liên tiếp bị dính chùm vào cùng một bộ đệm đọc.

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #03: Tại sao giải pháp phân khung (Framing) bằng Length-Prefix 4 byte lại tối ưu hơn ký tự kết thúc dòng (Newline Framing)?

• **Trả lời cốt lõi:** Vì dữ liệu trao đổi là JSON (RFC 8259) có thể chứa ký tự xuống dòng `\n` bên trong chuỗi string đã escape; Length-Prefix cho biết chính xác số byte cần đọc trước, giúp cấp phát bộ đệm chính xác và tránh duyệt quét từng ký tự tốn CPU.

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #04: Quy ước thứ tự byte Big-Endian (Network Byte Order) là gì và tại sao bắt buộc phải dùng?

• **Trả lời cốt lõi:** Big-Endian đặt byte có trọng số lớn nhất ở địa chỉ bộ nhớ thấp nhất (đầu tiên trên đường truyền). Vì các kiến trúc CPU khác nhau (x86 dùng Little-Endian, ARM có thể dùng Big/Little) biểu diễn số nguyên khác nhau, bắt buộc phải dùng trật tự chuẩn mạng để giải mã độ dài nhất quán.

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #05: Phương thức `ReadExactlyBytesAsync` hoạt động như thế nào khi gặp ngắt kết nối giữa chừng (EOF)?

• **Trả lời cốt lõi:** Phương thức đọc lặp trong vòng lặp cho đến khi đủ số byte yêu cầu. Nếu socket bị đóng sớm (`ReadAsync` trả về 0 byte trước khi đủ độ dài), phương thức ném `EndOfStreamException` và hủy stream ngay lập tức, ngăn ngừa việc chuyển mẩu byte rác sang bộ giải mã JSON.

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #06: Tại sao lại giới hạn kích thước khung truyền tối đa `MaxFrameSize = 256 KiB`?

• **Trả lời cốt lõi:** Để phòng chống tấn công từ chối dịch vụ (DoS / OOM attack). Nếu kẻ xấu gửi 4 byte độ dài giả mạo là 2 GB, server không bị lừa cấp phát mảng byte khổng lồ làm sập bộ nhớ.

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #07: Tại sao cờ `Socket.Connected` không phản ánh chính xác trạng thái kết nối mạng thời gian thực?

• **Trả lời cốt lõi:** Cờ `Connected` chỉ phản ánh trạng thái sau lần thao tác I/O gần nhất. Nếu client bị mất điện đột ngột hoặc rút dây mạng mà server chưa gọi lệnh đọc/ghi, cờ này vẫn trả về `true` cho đến khi TCP Keep-Alive timeout hoặc I/O thất bại.

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #08: Tại sao hệ thống phải cài đặt cơ chế Heartbeat ở tầng ứng dụng?

• **Trả lời cốt lõi:** Vì TCP Keep-Alive mặc định của hệ điều hành có thể mất tới 2 giờ để phát hiện sự cố; Heartbeat tầng ứng dụng chu kỳ 5s giúp Central Server xác định chính xác liveness của máy trạm và kích hoạt cảnh báo trong vòng 20 giây.

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #09: Tại sao việc đo lường liveness phải dùng đồng hồ đơn điệu (Monotonic Clock) tại server?

• **Trả lời cốt lõi:** Dùng `Stopwatch.GetTimestamp()` tại server để đo khoảng cách thời gian trôi qua độc lập với đồng hồ hệ thống (Wall-clock), loại trừ hoàn toàn nguy cơ sai lệch hoặc nhảy lùi thời gian do đồng bộ NTP của client.

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #10: Single Writer Pattern trên socket được cài đặt như thế nào và tại sao cần thiết?

• **Trả lời cốt lõi:** Tài liệu chính thức từ Microsoft ghi rõ: lớp `NetworkStream` trong .NET chỉ hỗ trợ an toàn cho 1 thao tác đọc và 1 thao tác ghi diễn ra đồng thời. `NetworkStream` KHÔNG hỗ trợ nhiều tác vụ ghi (`WriteAsync`) đồng thời; nếu nhiều luồng cùng gọi `WriteAsync` mà không tự đồng bộ sẽ gây đan xen byte và làm hỏng frame trên wire. Do đó LabGuard sử dụng `Channel<byte[]>` (`System.Threading.Channels`) làm hàng đợi ghi đơn luồng (Single-Writer Queue), bảo đảm chỉ có đúng một Task nền duy nhất đọc từ Channel và ghi tuần tự các frame vào socket.

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #11: Sự khác biệt giữa `Channel<T>` có giới hạn (Bounded) và không giới hạn (Unbounded)?

• **Trả lời cốt lõi:** Bounded Channel giới hạn số phần tử trong hàng đợi (ví dụ: 100 mẩu tin); khi đầy, luồng ghi sẽ tạm hoãn bất đồng bộ (Backpressure), ngăn chặn việc ứng dụng tiêu tốn RAM vô hạn khi mạng bị nghẽn.

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #12: Cơ chế Reconnect Exponential Backoff kết hợp Jitter hoạt động ra sao?

• **Trả lời cốt lõi:** Khi mất kết nối, Agent chờ một khoảng thời gian tăng theo cấp số nhân ($1\text{s}, 2\text{s}, 4\text{s}, 8\text{s}... \max 30\text{s}$) cộng thêm một khoảng ngẫu nhiên (jitter), tránh hiện tượng hàng trăm máy trạm kết nối lại cùng một tích tắc gây bão kết nối (reconnect storm).

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #13: Tại sao cờ `isReplay` được đặt bên trong `payload` thay vì ở envelope root?

• **Trả lời cốt lõi:** Vì cờ này chỉ có ý nghĩa nghiệp vụ đối với dữ liệu đo lường (`METRIC_BATCH`) và trạng thái dịch vụ (`SERVICE_STATUS`) khi phát lại từ bộ nhớ đệm; các bản tin điều khiển hoặc đăng nhập không bao giờ có ngữ cảnh replay.

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #14: Khóa khử trùng lặp (Deduplication Composite Key) của telemetry gồm những trường nào?

• **Trả lời cốt lõi:** Bộ ba `DeviceId + BootId + BatchId`. `BootId` (UUID) sinh mới mỗi lần máy khởi động để tránh trùng số thứ tự `BatchId` khi bộ đếm bắt đầu lại từ 1.

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #15: Phương thức thu thập chỉ số CPU trên Windows bằng `Pdh` hoặc `PerformanceCounter` cần chú ý điều gì?

• **Trả lời cốt lõi:** Việc đọc lần đầu tiên cần khởi tạo phiên và mẫu thứ hai sau một khoảng thời gian mới tính được phần trăm CPU; tác vụ này cần chạy trên luồng nền riêng để không làm trễ chu kỳ gửi nhịp tim mạng.

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #16: Cơ chế thăm dò dịch vụ cục bộ (Local TCP Service Probe) có thể gây blocking không?

• **Trả lời cốt lõi:** Có thể gây block nếu dùng phương thức đồng bộ hoặc timeout mặc định của hệ điều hành (20s). Do đó bắt buộc dùng `ConnectAsync` với `CancellationTokenSource` hủy sau đúng 2 giây.

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #17: Sự khác biệt cốt lõi giữa Agent thật và Agent Simulator là gì?

• **Trả lời cốt lõi:** Simulator thật 100% về socket TCP, mã hóa TLS, phân khung Length-Prefix và định dạng bản tin gửi đi; điểm giả lập duy nhất là chỉ số CPU/RAM được sinh ngẫu nhiên theo kịch bản thay vì đọc từ phần cứng thật.

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #18: Làm thế nào để một tiến trình Simulator tạo được 50 kết nối socket đồng thời mà không nghẽn tài nguyên?

• **Trả lời cốt lõi:** Sử dụng các phương thức bất đồng bộ `async/await` trên ThreadPool của .NET, các socket dùng chung luồng I/O thay vì tạo 50 Thread riêng của hệ điều hành.

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #19: Bộ cân bằng tải Gateway L4 khác gì so với Gateway L7 (Reverse Proxy)?

• **Trả lời cốt lõi:** Gateway L4 chỉ chuyển tiếp luồng byte nhị phân thô (TCP byte relay) dựa trên địa chỉ IP và port mà không giải mã TLS hay đọc nội dung JSON bên trong; Gateway L7 phải giải mã giao thức ứng dụng, tốn nhiều CPU hơn.

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #20: Thuật toán Least-Connections trong Gateway L4 được cài đặt như thế nào?

• **Trả lời cốt lõi:** Gateway duy trì một biến đếm số kết nối TCP đang mở (Active Connections) tới từng Collector; khi có client mới kết nối, Gateway chọn Collector có biến đếm nhỏ nhất và đang có trạng thái Health Check là `Up`.

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #21: Tại sao hệ thống không yêu cầu di chuyển socket đang mở (Socket Migration) khi Collector gặp sự cố?

• **Trả lời cốt lõi:** Việc di chuyển socket TCP đang mở giữa các server yêu cầu hạ tầng mạng chuyên dụng rất phức tạp và vượt quá phạm vi đồ án NT106; đồ án áp dụng mô hình chuẩn công nghiệp: cho phép Agent ngắt kết nối và tự động kết nối lại sang node khác.

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #22: Mục tiêu thiết kế thời gian failover p95 ≤ 45 giây được kiểm chứng như thế nào?

• **Trả lời cốt lõi:** Đo khoảng thời gian từ lúc tắt tiến trình Collector 01 cho đến khi Agent nhận ra lỗi kết nối, áp dụng backoff, kết nối lại thành công qua Gateway tới Collector 02 và gửi mẻ tin telemetry tiếp theo.

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #23: Tại sao log hệ thống phải được cài đặt cơ chế xoay vòng (Log Rotation)?

• **Trả lời cốt lõi:** Tránh việc file log phình to vô hạn làm cạn kiệt dung lượng ổ đĩa của máy trạm; hệ thống quy định mỗi file tối đa 10 MB và chỉ lưu giữ tối đa 5 file gần nhất.

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #24: Khi kết nối qua mạng Internet bằng Port Forwarding hoặc Tunnel, Agent cần cấu hình gì?

• **Trả lời cốt lõi:** Cần cấu hình tên miền hoặc địa chỉ IP công khai kèm cổng NAT; đồng thời thiết lập tên máy chủ (Server Name) trong `SslStream.AuthenticateAsClient` khớp với Common Name (CN) hoặc SAN của chứng thư số TLS.

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #25: Làm thế nào để phân biệt giữa lỗi mạng tạm thời (Transient Network Error) và máy trạm bị tắt nguồn thật sự?

• **Trả lời cốt lõi:** Lỗi mạng tạm thời sẽ khiến heartbeat bị trễ nhưng khi có mạng lại Agent sẽ gửi kèm mẻ tin replay; nếu quá 20s không có bất kỳ tín hiệu nào và probe TCP thất bại, Central mới xác định máy trạm Offline.

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #26: Vai trò của việc tạo mã băm SHA-256 cho gói bàn giao (Release Package) là gì?

• **Trả lời cốt lõi:** Bảo đảm tính toàn vẹn của mã nguồn và tệp thực thi, chứng minh tệp giao nộp cho giảng viên không bị chỉnh sửa, thiếu sót hoặc nhiễm mã độc sau khi hoàn tất kiểm thử.

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #27: Tại sao không nên sử dụng giao thức UDP cho việc truyền dữ liệu giám sát phòng máy?

• **Trả lời cốt lõi:** UDP không có cơ chế bảo đảm thứ tự và không tin cậy; dữ liệu sự cố và lệnh quản trị cần tính chính xác tuyệt đối, việc tự cài đặt cơ chế ACK và retransmit trên UDP sẽ làm phức tạp hóa đồ án không cần thiết.

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #28: Kỹ thuật Bounded Channel giải quyết vấn đề gì trong Writer Loop của Agent?

• **Trả lời cốt lõi:** Khi mạng bị chập chờn, các mẻ tin thu thập được đẩy vào Channel; nếu Channel đạt giới hạn 100 mẻ tin, tác vụ đẩy tin sẽ tạm dừng (Backpressure) để kích hoạt cơ chế gộp mẻ hoặc lưu bộ nhớ đệm phụ thay vì làm tràn RAM.

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #29: Nếu một máy trạm Agent bị đổi địa chỉ IP động qua DHCP, hệ thống nhận diện máy trạm ra sao?

• **Trả lời cốt lõi:** Hệ thống định danh thiết bị duy nhất dựa trên trường `deviceId` trong Envelope và Session Token sau khi xác thực, hoàn toàn độc lập với địa chỉ IP tầng mạng.

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #30: Khi demo trước hội đồng, nếu mạng Internet tại trường bị chập chờn, phương án dự phòng là gì?

• **Trả lời cốt lõi:** Đồ án chuẩn bị sẵn mạng LAN nội bộ phòng máy (hoặc phát qua Router Wi-Fi riêng / Switch cơ học) kết nối 3 laptop; kịch bản demo trên mạng LAN độc lập 100% với đường truyền Internet bên ngoài.

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Bộ 30 câu hỏi chuyên sâu — SINH VIÊN B (CENTRAL SERVER & CSDL)

Câu hỏi #01: Kiến trúc luồng (Threading Model) của Central Server được thiết kế như thế nào?

• **Trả lời cốt lõi:** Central Server sử dụng mô hình Async I/O dựa trên ThreadPool của .NET (`SocketAsyncEventArgs` / `ReadAsync`), không tạo 1 Thread riêng của hệ điều hành cho mỗi kết nối socket. Khi socket nhàn rỗi, luồng được hoàn trả cho ThreadPool để phục vụ tác vụ khác.

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #02: Tại sao không nên tạo một Thread riêng (`new Thread()`) cho mỗi client kết nối?

• **Trả lời cốt lõi:** Mỗi OS Thread trong Windows tiêu tốn mặc định khoảng 1 MB bộ nhớ stack và gây áp lực lớn cho CPU khi chuyển ngữ cảnh luồng (Context Switching). Với hàng trăm client, mô hình 1-thread-per-client sẽ làm cạn kiệt tài nguyên hệ thống.

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #03: Kỹ thuật Ingest Ring Buffer sử dụng `Channel<T>` có giới hạn nhằm giải quyết vấn đề gì?

• **Trả lời cốt lõi:** Đóng vai trò bộ đệm điều tiết tốc độ (Backpressure Buffer) giữa luồng nhận mạng cực nhanh và luồng ghi CSDL có độ trễ I/O đĩa. Khi bộ đệm đầy (ví dụ 500 mẻ tin), luồng đọc socket tạm hoãn để bảo vệ server không bị cạn kiệt RAM.

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #04: Tại sao Ingest Ring Buffer không được coi là nơi chống trùng lặp bền vững?

• **Trả lời cốt lõi:** Vì Ring Buffer nằm trên bộ nhớ RAM; nếu Central Server bị mất điện hoặc khởi động lại đột ngột, toàn bộ dữ liệu trong RAM sẽ bị mất. Nơi chống trùng bền vững bắt buộc phải nằm trên đĩa CSDL (bảng `ProcessedBatches`).

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #05: Cấu trúc bảng `ProcessedBatches` và chính sách dọn dẹp (TTL Policy) là gì?

• **Trả lời cốt lõi:** Bảng có khóa chính `(DeviceId, BootId, BatchId)` và cột `ReceivedUtc`. Dữ liệu được lưu vết trong 24 giờ để lọc trùng lặp khi Agent gửi lại; một job nền định kỳ xóa các bản ghi cũ hơn 24 giờ.

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #06: Khi phát hiện một mẻ tin đã tồn tại trong `ProcessedBatches`, server xử lý như thế nào?

• **Trả lời cốt lõi:** Server bỏ qua việc phân tích và ghi telemetry mới vào CSDL, nhưng **bắt buộc phải gửi lại `BATCH_ACK`** về cho Agent để Agent biết mẻ tin đã tới đích an toàn và xóa khỏi hàng đợi replay.

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #07: Cấu trúc bảng `ProcessedCommands` bảo đảm tính lũy thừa (Idempotency) của lệnh Admin ra sao?

• **Trả lời cốt lõi:** Bảng lưu khóa `(UserId, CommandId)` kèm mã băm `RequestDigest` (SHA-256) và kết quả đã trả `ResponsePayload`. Nếu Admin bấm lặp lại lệnh cùng `CommandId`, server trả ngay kết quả cũ đã commit mà không thực thi lại mutation.

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #08: Tại sao không dùng bảng `AuditLogs` hoặc `IncidentHistory` làm bảng chống trùng lệnh?

• **Trả lời cốt lõi:** `AuditLogs` và `IncidentHistory` phục vụ nhật ký lịch sử cho người quản trị theo dõi (mỗi lần bấm là một dòng log mới); bảng chống trùng lệnh (`ProcessedCommands`) có cấu trúc khóa duy nhất trên `CommandId` và TTL 7 ngày dành riêng cho logic máy.

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #09: Mã băm `RequestDigest` trong bảng `ProcessedCommands` ngăn chặn rủi ro gì?

• **Trả lời cốt lõi:** Ngăn chặn rủi ro tấn công hoặc nhầm lẫn tham số: nếu client gửi cùng một `CommandId` nhưng thay đổi nội dung lệnh (ví dụ đổi action từ ACK sang RESOLVE), server phát hiện mã băm khác nhau và ném lỗi `ERR_INVALID_COMMAND_DIGEST`.

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #10: Liveness Checker của Central Server phát hiện máy trạm Offline dựa trên nguyên lý nào?

• **Trả lời cốt lõi:** Duy trì bảng đăng ký nhịp tim trong bộ nhớ kèm mốc thời gian nhận gần nhất đo bằng `Stopwatch.GetTimestamp()`. Định kỳ mỗi giây, một tác vụ nền quét kiểm tra: nếu thời gian trôi qua vượt quá 20 giây, server tự động kích hoạt sự cố `DEVICE_OFFLINE`.

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #11: Quy tắc phát hiện lỗi CPU (`HIGH_CPU`) với Hysteresis và Streak được cài đặt như thế nào?

• **Trả lời cốt lõi:** Yêu cầu 3 mẫu liên tiếp có CPU $\ge 90\%$ (ở chu kỳ 5s, khoảng thời gian giữa mẫu 1 và 3 là đúng 10 giây). Khi đã mở sự cố, chỉ đóng sự cố khi nhận được 3 mẫu liên tiếp có CPU $\le 75\%$. Dải 15% Deadband ngăn hiện tượng flapping.

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #12: Quy tắc phát hiện dịch vụ chết (`SERVICE_DOWN`) quy định số lần thất bại và phục hồi ra sao?

• **Trả lời cốt lõi:** Cảnh báo kích hoạt khi probe TCP thất bại 2 lần liên tiếp (ở chu kỳ 5s); sự cố tự động chuyển trạng thái phục hồi kỹ thuật khi probe TCP thành công trở lại 2 lần liên tiếp.

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #13: Tại sao kiến trúc hệ thống phải phân tách thành `ConditionState` và `WorkflowState`?

• **Trả lời cốt lõi:** `ConditionState` (`ACTIVE`, `RECOVERED`, `UNKNOWN`) phản ánh hiện trạng kỹ thuật khách quan do Rule Engine đo lường từ Agent; `WorkflowState` (`OPEN`, `ACKNOWLEDGED`, `IN_PROGRESS`, `RECOVERED`, `RESOLVED`) quản lý tiến trình xử lý nghiệp vụ của con người.

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #14: Tình huống ``Tái phát sự cố'' (Relapse) xảy ra khi nào và state machine chuyển đổi ra sao?

• **Trả lời cốt lõi:** Khi sự cố đang ở `WorkflowState = RECOVERED` (lỗi kỹ thuật đã hết nhưng Admin chưa kịp đóng phiếu `RESOLVED`), nếu Agent lại báo lỗi trở lại (`ConditionState = ACTIVE`), sự cố sẽ tự động chuyển ngược về `WorkflowState = IN_PROGRESS` để tiếp tục theo dõi.

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #15: Ràng buộc duy nhất một Incident hoạt động (Unique Active Incident Constraint) được đảm bảo ra sao?

• **Trả lời cốt lõi:** Bằng Unique Filtered Index trên SQL Server: `CREATE UNIQUE INDEX UX_ActiveIncident ON Incidents(DeviceId, RuleType) WHERE WorkflowState <> 'RESOLVED'`, bảo đảm một máy không bao giờ bị mở nhiều phiếu cho cùng một lỗi.

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #16: Cột `rowversion` trong SQL Server là gì và hoạt động ra sao?

• **Trả lời cốt lõi:** `rowversion` là datatype riêng của SQL Server. Giá trị lưu trữ có kích thước 8 byte và do SQL Server tự sinh khi rowversion row được insert/update. Kiểu `binary(8)` thông thường KHÔNG có semantics tự cập nhật này. `rowversion` KHÔNG phải date/time. Nó đóng vai trò như một opaque version token phục vụ kiểm tra Optimistic Concurrency Control (OCC).

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #17: Optimistic Concurrency Control (OCC) giải quyết bài toán hai Admin cùng bấm ACK một sự cố như thế nào?

• **Trả lời cốt lõi:** Admin gửi lệnh kèm `expectedRowVersion` (coi như một opaque 8-byte token đặc thù của SQL Server). Lệnh cập nhật SQL kiểm tra: `UPDATE Incidents SET WorkflowState='ACKNOWLEDGED' WHERE IncidentId=@Id AND RowVersion=@expected`. Người đầu tiên cập nhật thành công và đổi `rowversion`; người thứ hai có điều kiện WHERE không khớp, số dòng bị ảnh hưởng là 0 $\rightarrow$ Server ném lỗi `409 Conflict`.

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #18: Tại sao nên dùng Optimistic Concurrency thay vì Pessimistic Locking (`SELECT FOR UPDATE`)?

• **Trả lời cốt lõi:** Pessimistic Locking giữ khóa bảng hoặc khóa dòng CSDL trong suốt thời gian người dùng mở form, làm nghẽn toàn bộ các kết nối khác. OCC không giữ khóa, cho phép đọc tự do và chỉ kiểm tra xung đột tại đúng thời điểm commit.

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #19: Giao dịch Incident được bao bọc trong Transaction Boundary như thế nào?

• **Trả lời cốt lõi:** Khi chuyển trạng thái sự cố, server thực thi đồng thời trong một giao dịch ACID: Cập nhật dòng bảng `Incidents`, thêm một dòng ghi nhật ký vào `IncidentHistory`, và cập nhật bảng chống trùng `ProcessedCommands`. Nếu có bất kỳ lỗi nào, toàn bộ giao dịch bị rollback.

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #20: Tại sao mật khẩu người dùng bắt buộc phải được băm kèm Salt bằng thuật toán PBKDF2?

• **Trả lời cốt lõi:** Salt (chuỗi ngẫu nhiên 16 byte / 128-bit) bảo đảm hai người dùng có cùng mật khẩu sẽ sinh ra hai chuỗi băm hoàn toàn khác nhau, vô hiệu hóa bảng tra cứu tính sẵn (Rainbow Table); kết hợp so sánh bằng `CryptographicOperations.FixedTimeEquals` để chống timing attack. Về mặt chuẩn mực: OWASP recommends Argon2id for new systems generally. If FIPS-140 compliance is required, OWASP recommends PBKDF2-HMAC-SHA256 with a work factor of 600,000 or more. LabGuard deliberately chooses PBKDF2 for BCL availability and course-scope simplicity; 600,000 is therefore the project baseline referenced from OWASP and must still be measured on the actual lab hardware.

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #21: Tại sao việc kiểm tra quyền hạn (Authorization / RBAC) bắt buộc phải thực hiện ở phía server?

• **Trả lời cốt lõi:** Ẩn nút trên giao diện WinForms chỉ là hỗ trợ trải nghiệm người dùng; kẻ xấu có thể can thiệp socket hoặc dùng tool gửi bản tin wire trực tiếp lên server. Server bắt buộc phải kiểm tra role trong Session Token trước khi thực thi lệnh.

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #22: Tại sao bản tin Snapshot gửi cho Admin phải được phân trang (`SNAPSHOT_PAGE`)?

• **Trả lời cốt lõi:** Nếu phòng máy có hàng trăm thiết bị và hàng nghìn lịch sử sự cố, gửi toàn bộ trong một bản tin JSON khổng lồ sẽ vượt trần `MaxFrameSize = 256 KiB` và làm nghẽn bộ đệm socket; phân trang giúp chia nhỏ dữ liệu và giải phóng luồng I/O.

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #23: Cơ chế Realtime Event Fanout được cài đặt như thế nào trong Central Server?

• **Trả lời cốt lõi:** Central Server duy trì danh sách các kết nối Admin đang mở (Subscriber Registry); khi có biến động từ Rule Engine, máy chủ đóng gói bản tin delta event và gửi tuần tự qua Channel Single Writer của từng Admin connection.

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #24: Kỹ thuật Slow-Client Isolation bảo vệ hệ thống như thế nào?

• **Trả lời cốt lõi:** Nếu một Admin Client có kết nối mạng quá chậm hoặc bị treo máy, hàng đợi gửi của Admin đó sẽ bị đầy; hệ thống kích hoạt chính sách drop event hoặc ngắt kết nối client đó sau một khoảng timeout, tuyệt đối không làm block các Admin Client khác.

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #25: Chính sách lưu trữ dữ liệu (Data Retention) được cài đặt và vận hành ra sao?

• **Trả lời cốt lõi:** Telemetry thô (`Metrics`) được lưu trong 48 giờ; một Background Worker định kỳ chạy vào ban đêm thực hiện câu lệnh xóa theo mẻ (Batch Delete) dựa trên cột `ReceivedUtc` để không làm khóa bảng lâu dài.

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #26: Kịch bản phục hồi thảm họa CSDL (Crash Recovery Proof) được kiểm chứng thế nào?

• **Trả lời cốt lõi:** Tiến hành sao lưu CSDL bằng lệnh `BACKUP DATABASE`, cố tình tắt đột ngột tiến trình SQL Server khi đang có tải ghi dồn dập, sau đó khởi động lại và chạy lệnh `RESTORE VERIFYONLY` để chứng minh CSDL tự phục hồi và dữ liệu không bị hỏng (corrupted).

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #27: Trong kiến trúc P1, cơ chế Fencing và Session Generation giải quyết vấn đề gì?

• **Trả lời cốt lõi:** Khi Agent chuyển từ Collector 01 sang Collector 02, Central Server tăng chỉ số `SessionGeneration`; nếu các gói tin chậm trễ từ Collector 01 cũ vẫn còn bay trên mạng tới sau, Central sẽ loại bỏ các gói tin này vì có chỉ số generation cũ hơn, ngăn chặn ghi đè dữ liệu cũ (split-brain).

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #28: Tại sao Central Server không lưu trạng thái phiên làm việc (Session) trong CSDL mà lưu trong RAM Registry?

• **Trả lời cốt lõi:** Kiểm tra token xác thực diễn ra ở mọi request với tần suất cực lớn; lưu Session Registry trong RAM (sử dụng `ConcurrentDictionary`) giúp truy xuất với độ trễ nano-giây, giảm tải triệt để cho CSDL SQL Server.

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #29: Tại sao kiểu dữ liệu `rowversion` không thể chuyển đổi tương đương sang SQLite?

• **Trả lời cốt lõi:** SQLite không có kiểu dữ liệu nhị phân 8 byte tự tăng nguyên bản ở mức engine như SQL Server; việc giả lập bằng trigger hoặc timestamp trong SQLite dễ bị sai lệch concurrency, do đó việc khóa chặt SQL Server 2022 Express là quyết định kiến trúc đúng đắn.

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #30: Khi Central Server bị khởi động lại, các Agent kết nối lại có làm sập server không?

• **Trả lời cốt lõi:** Nhờ thuật toán Exponential Backoff và Jitter phía Agent, các kết nối không ùa vào cùng một lúc; tại Central Server, hàng đợi Ingress Bounded Channel chỉ nhận số lượng kết nối tối đa cho phép, loại bỏ rủi ro nghẽn tài nguyên.

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Bộ 30 câu hỏi chuyên sâu — SINH VIÊN C (WINFORMS DASHBOARD & UI)

Câu hỏi #01: Kiến trúc luồng của Windows Forms được thiết kế theo nguyên lý nào?

• **Trả lời cốt lõi:** Windows Forms hoạt động theo mô hình Single-Threaded Apartment (STA), trong đó chỉ có đúng một luồng duy nhất (UI Thread) được sở hữu vòng lặp thông điệp Windows (Message Loop) và được phép truy cập, sửa đổi các thành phần giao diện Win32.

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #02: Tại sao việc thao tác control WinForms trực tiếp từ luồng mạng (Background Thread) lại gây lỗi?

• **Trả lời cốt lõi:** Vì vi phạm tính an toàn đa luồng của hệ thống cửa sổ Windows; truy cập đồng thời từ luồng khác sẽ gây xung đột bộ nhớ đồ họa GDI, làm sai lệch trạng thái hiển thị và ném ngoại lệ `InvalidOperationException: Cross-thread operation not valid`.

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #03: Sự khác biệt kỹ thuật giữa `Control.Invoke` và `Control.BeginInvoke` là gì?

• **Trả lời cốt lõi:** `Control.Invoke` là cuộc gọi đồng bộ, luồng mạng sẽ bị block chờ cho đến khi UI Thread thực thi xong delegate; `Control.BeginInvoke` là cuộc gọi bất đồng bộ, luồng mạng đẩy delegate vào Message Queue của UI rồi tiếp tục chạy ngay mà không chờ đợi.

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #04: Khi nào nên dùng `Control.BeginInvoke` thay vì `Control.Invoke` trong ứng dụng realtime?

• **Trả lời cốt lõi:** Khi nhận các sự kiện delta thời gian thực liên tục từ mạng (`DEVICE_UPDATE`, `INCIDENT_UPDATE`), nên dùng `BeginInvoke` để luồng mạng không bị chặn nếu UI Thread đang bận vẽ đồ thị, tránh làm nghẽn bộ đệm socket.

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #05: Tại sao thuộc tính `CheckForIllegalCrossThreadCalls = false` bị nghiêm cấm trong sản phẩm chuẩn?

• **Trả lời cốt lõi:** Vì câu lệnh này chỉ tắt cờ kiểm tra cảnh báo của .NET Framework chứ hoàn toàn không làm cho mã nguồn an toàn đa luồng; ứng dụng vẫn tiềm ẩn nguy cơ race condition nghiêm trọng, vẽ lỗi form hoặc tự sập ngẫu nhiên.

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #06: Phương thức `InvokeAsync` trong .NET 9+ WinForms mang lại lợi ích gì?

• **Trả lời cốt lõi:** Cho phép await trực tiếp một thao tác điều phối giao diện từ bất kỳ Task nền nào mà không cần tạo delegate thủ công, giúp mã nguồn bất đồng bộ sạch sẽ và tận dụng triệt để `IProgress<T>` hoặc `SynchronizationContext`.

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #07: Cơ chế RPC bất đồng bộ trên một kết nối TCP duy nhất được cài đặt như thế nào?

• **Trả lời cốt lõi:** Khi gửi một request, client tạo một `TaskCompletionSource<TResponse>`, lưu vào `ConcurrentDictionary` với key là `requestId` (UUID v4) rồi gửi frame. Luồng đọc socket khi nhận được response sẽ bóc tách `requestId`, tìm `TaskCompletionSource` tương ứng và gọi `SetResult()` để hoàn tất Task.

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #08: Làm thế nào để xử lý Timeout cho các yêu cầu RPC khi mạng bị ngắt hoặc server không phản hồi?

• **Trả lời cốt lõi:** Sử dụng `CancellationTokenSource` với thời gian chờ (ví dụ 5 giây): `cts.CancelAfter(5000)`. Đăng ký callback khi token bị hủy sẽ gọi `TrySetCanceled()` trên `TaskCompletionSource` và gỡ bản ghi khỏi Dictionary.

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #09: Tại sao ứng dụng Admin không gửi truy vấn định kỳ (Polling) mà sử dụng Snapshot + Realtime Delta?

• **Trả lời cốt lõi:** Polling liên tục tạo tải CPU và băng thông mạng rất lớn cho Central Server, đồng thời gây trễ thông tin. Snapshot + Delta chỉ nạp dữ liệu đầy đủ 1 lần lúc mở máy, sau đó server chủ động đẩy đúng các thay đổi nhỏ, giảm 90% lưu lượng mạng.

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #10: Quá trình nạp dữ liệu ban đầu bằng Snapshot phân trang (`SNAPSHOT_BEGIN/PAGE/END`) diễn ra ra sao?

• **Trả lời cốt lõi:** Client nhận `SNAPSHOT_BEGIN` để biết tổng số thiết bị và khởi tạo bộ nhớ tạm; nhận tuần tự các trang `SNAPSHOT_PAGE` để nạp danh sách thiết bị; khi nhận `SNAPSHOT_END`, client khóa bộ nhớ đệm `LocalCache`, cập nhật một lần lên UI và bắt đầu tiêu thụ các delta event.

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #11: Nếu một delta event đến trong khi ứng dụng đang nạp Snapshot phân trang, client xử lý thế nào?

• **Trả lời cốt lõi:** Client đưa delta event vào một hàng đợi chờ (Pending Event Queue); sau khi xử lý xong `SNAPSHOT_END` và xác định chỉ số `initialSequence`, client mới tuần tự áp dụng các delta event có sequence lớn hơn để không làm sai lệch trạng thái.

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #12: Màn hình sơ đồ phòng máy Lab View được thiết kế trực quan như thế nào?

• **Trả lời cốt lõi:** Thiết kế dạng lưới các ô máy tính (Grid Layout). Mỗi ô hiển thị tên máy (ví dụ `PC-07`), địa chỉ IP, trạng thái dạng chữ (`ONLINE`, `OFFLINE`, `WARNING`), kèm icon sinh động và viền màu tương ứng; không bao giờ chỉ dựa vào một màu nền đơn điệu.

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #13: Tại sao trạng thái trên Lab View phải xuất phát từ sự kiện mạng thật thay vì timer giả lập?

• **Trả lời cốt lõi:** Để bảo đảm tính trung thực của một đồ án mạng máy tính. Khi máy tính phòng máy bị rút dây mạng, ô hiển thị chuyển sang `OFFLINE` là do Central phát sự kiện `DEVICE_UPDATE` sau khi hết deadline heartbeat 20s, không phải do hàm timer trên WinForms tự đếm giờ.

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #14: Optimistic Concurrency Control (OCC) được thể hiện trên giao diện Admin như thế nào khi gặp lỗi 409 Conflict?

• **Trả lời cốt lõi:** Khi Admin bấm nút Resolve nhưng sự cố đã bị người khác thao tác trước đó, server trả về `status = "CONFLICT"`. UI bắt mã lỗi này, hiển thị thông báo: "Sự cố đã được Admin khác cập nhật", đồng thời tự động cập nhật lại thông tin mới nhất trên form mà không làm mất dữ liệu của người dùng.

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #15: Tại sao client phải gửi kèm `expectedRowVersion` trong các lệnh thao tác sự cố (`INCIDENT_ACK`, `INCIDENT_RESOLVE`)?

• **Trả lời cốt lõi:** Để Central Server so sánh với giá trị `rowversion` hiện tại trong CSDL. Nếu hai giá trị khớp nhau, chứng tỏ dữ liệu Admin đang nhìn thấy là mới nhất; nếu khác nhau, chứng tỏ đã có người khác sửa đổi trước đó.

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #16: Phân quyền người dùng (RBAC) được thể hiện ở giao diện WinForms ra sao?

• **Trả lời cốt lõi:** Sau khi đăng nhập, căn cứ vào role trong `SessionToken`: Nếu là `Viewer` (chế độ chỉ xem), toàn bộ các nút mutation (ACK, Start, Resolve, Assign, Invite) bị ẩn hoặc disable; nếu là `Operator`, được hiển thị và thao tác các nút xử lý sự cố trực ban (ACK, Start, Resolve); nếu là `Admin`, có toàn quyền Operator kèm theo nút phân công (Assign) và quản trị tài khoản / lời mời (Invite / User Management). Mọi thao tác đều được Central Server kiểm tra quyền độc lập.

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #17: Tại sao việc ẩn nút trên WinForms không thay thế được việc kiểm tra bảo mật phía server?

• **Trả lời cốt lõi:** Vì kẻ xấu có thể can thiệp bộ nhớ ứng dụng hoặc viết một script TCP độc lập để gửi lệnh trực tiếp vào socket. Giao diện chỉ mang tính chất hỗ trợ trải nghiệm người dùng (UX), quyền lực bảo mật tối thượng bắt buộc thuộc về server.

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #18: Cơ chế Reconnect Backoff phía Admin Client hoạt động như thế nào khi mất kết nối tới Central?

• **Trả lời cốt lõi:** Client chuyển giao diện sang trạng thái vô hiệu hóa các nút bấm, hiện thanh thông báo màu vàng "Mất kết nối, đang thử lại sau Xs...". Thời gian thử lại tăng dần (1s, 2s, 4s... max 30s) và tự động yêu cầu Snapshot mới khi kết nối lại thành công.

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #19: Tại sao sau khi Reconnect thành công, Admin Client bắt buộc phải yêu cầu Snapshot mới thay vì dùng tiếp cache cũ?

• **Trả lời cốt lõi:** Trong khoảng thời gian mất mạng, client đã bỏ lỡ nhiều delta events; việc tiếp tục dùng cache cũ sẽ dẫn đến hiển thị sai lệch hoặc gửi sai `expectedRowVersion` gây lỗi xung đột liên tục.

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #20: Bộ nhớ đệm cục bộ `LocalCache` trong WinForms được quản lý bằng cấu trúc dữ liệu nào?

• **Trả lời cốt lõi:** Sử dụng `ConcurrentDictionary<string, DeviceViewModel>` và `ConcurrentDictionary<string, IncidentViewModel>`, bảo đảm các thao tác đọc từ UI Thread và ghi từ luồng mạng diễn ra an toàn mà không gây deadlock.

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #21: Kỹ thuật Virtual Mode trên `DataGridView` hoặc `ListView` giải quyết vấn đề gì khi hiển thị hàng nghìn bản ghi?

• **Trả lời cốt lõi:** Không nạp sẵn hàng nghìn object đồ họa vào bộ nhớ; điều khiển chỉ yêu cầu dữ liệu cho đúng số dòng đang hiển thị trên màn hình qua sự kiện `CellValueNeeded`, giúp ứng dụng mở tức thì và tiêu tốn cực ít RAM.

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #22: Làm thế nào để xuất dữ liệu báo cáo lịch sử sự cố ra file CSV chuẩn UTF-8?

• **Trả lời cốt lõi:** Sử dụng `StreamWriter` với encoding `new UTF8Encoding(true)` (có ký tự BOM - Byte Order Mark) để Microsoft Excel trên Windows có thể mở file CSV và hiển thị Tiếng Việt có dấu chính xác mà không bị lỗi font.

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #23: Tại sao Sinh viên C chỉ phát triển một ứng dụng Collector dùng chung cho cả 2 instance ở P1?

• **Trả lời cốt lõi:** Để bảo đảm tính nhất quán của giao thức và mã nguồn. Nếu hai thành viên viết hai bản Collector riêng sẽ dễ dẫn đến lệch pha xử lý khung truyền; việc viết một bản chuẩn và chạy thành 2 instance là mô hình chuẩn mực công nghiệp.

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #24: Ứng dụng Collector trung gian (`LabGuard.Collector.exe`) thực hiện nhiệm vụ gì?

• **Trả lời cốt lõi:** Đóng vai trò proxy gom mẻ: Lắng nghe kết nối TCP từ Agent, phân khung Length-Prefix, xác thực token ban đầu, sau đó ghép kênh (Multiplexing) chuyển tiếp dữ liệu lên Central Server qua một kết nối socket duy nhất.

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #25: Cơ chế Upstream Multiplexer trong Collector hoạt động ra sao?

• **Trả lời cốt lõi:** Hàng chục Agent gửi dữ liệu vào Collector; Collector gom các mẻ tin này vào một hàng đợi Channel duy nhất và gửi tuần tự lên Central Server, kèm theo cơ chế định tuyến (Reply Router) để trả đúng ACK về cho từng Agent tương ứng.

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #26: Collector báo cáo trạng thái sẵn sàng (Readiness / Health Check) tới Gateway L4 như thế nào?

• **Trả lời cốt lõi:** Collector mở một cổng riêng hoặc lắng nghe tín hiệu thăm dò từ Gateway; nếu bộ đệm Ingress hoặc kết nối Upstream tới Central gặp sự cố, Collector trả về trạng thái không sẵn sàng để Gateway không điều phối thêm kết nối mới.

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #27: Làm thế nào để kiểm thử kịch bản hai Admin cùng xử lý một sự cố (Multi-Admin Concurrency)?

• **Trả lời cốt lõi:** Khởi chạy đồng thời 2 tiến trình `LabGuard.Admin.exe` trên 2 màn hình, cùng mở một phiếu sự cố `INC-001`. Cả 2 người cùng bấm nút "Acknowledge" trong cùng 1 giây. Kết quả: 1 bên thành công, 1 bên hiện hộp thoại Conflict và tự làm mới dữ liệu.

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #28: Các biện pháp chống tràn giao diện (UI Responsiveness) khi có bão sự kiện gửi về là gì?

• **Trả lời cốt lõi:** Sử dụng kỹ thuật gom mẻ cập nhật (Batch UI Updates) hoặc Debounce/Throttle: Thay vì mỗi event gọi một lần `Invoke`, client gom các event trong khoảng 100ms và gọi một lần `BeginInvoke` để cập nhật toàn bộ bảng.

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #29: Tại sao cần chuẩn bị video demo dự phòng (Demo Backup Video) trước ngày vấn đáp?

• **Trả lời cốt lõi:** Phòng ngừa các rủi ro bất khả kháng tại phòng thi (mất điện, đứt cáp mạng LAN, lỗi xung đột IP phòng máy của trường); video quay sẵn rõ nét từng kịch bản giúp chứng minh sản phẩm đã chạy thực tế hoàn hảo.

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

Câu hỏi #30: Quy trình đóng gói và tài liệu hướng dẫn `README.md` cần cung cấp những thông tin gì?

• **Trả lời cốt lõi:** Cung cấp cấu hình phần cứng yêu cầu, chuỗi kết nối CSDL mẫu, tài khoản Admin/Operator mặc định, thứ tự khởi động các ứng dụng (Central Server trước $\rightarrow$ Agent $\rightarrow$ Admin Client) và danh mục các kịch bản demo mẫu.

• **Câu hỏi mở rộng / Tình huống bẫy:** Nếu gặp trường hợp thực tế khi mạng chập chờn hoặc tải tăng vọt, em sẽ đo lường và xử lý tình huống này ở dòng code nào?

• **Vị trí code minh chứng:**

PHẦN 11: SỔ TAY XỬ LÝ SỰ CỐ & 14 RỦI RO THỰC TẾ

11.1. Bảng 14 rủi ro kỹ thuật & vận hành (14 Risks Registry): Hệ thống phân tán luôn tiềm ẩn các sự cố bất ngờ. Dưới đây là bảng 14 rủi ro thực tế (RSK-01 đến RSK-14) kèm dấu hiệu sớm, biện pháp phòng ngừa và kế hoạch dự phòng khẩn cấp:

Rủi ro [RSK-01]: Chệch hướng phạm vi (Scope Creep) / Làm Collector P1 quá sớm

• Chủ sở hữu theo dõi: SV C | Xác suất: Trung bình | Ảnh hưởng: Cao
• Dấu hiệu cảnh báo sớm: Thành viên bắt đầu viết code Collector trước khi vượt qua cổng chất lượng M4 ở Tuần 3.
• Biện pháp phòng ngừa (Mitigation): Tuân thủ nghiêm ngặt nguyên tắc: Chỉ kích hoạt P1 nếu M4 đạt 100% và còn dư ít nhất 15h dự phòng.
• Kế hoạch ứng phó khẩn cấp (Fallback): Kích hoạt quyết định DEFER P1 ngay lập tức, cắt giảm 47.0h, dồn toàn lực vào ổn định và kiểm thử MVP.
• Trạng thái rà soát: Đang theo dõi (Tuần 3 (Cổng M4))

Rủi ro [RSK-02]: Lệch pha giao thức mạng (Protocol Drift) giữa 3 thành viên A, B, C

• Chủ sở hữu theo dõi: SV A | Xác suất: Trung bình | Ảnh hưởng: Cao
• Dấu hiệu cảnh báo sớm: Serialization ném lỗi JSON, thiếu trường dữ liệu hoặc một bên tự ý đổi tên trường trong envelope.
• Biện pháp phòng ngừa (Mitigation): Dùng chung hợp đồng CONTRACT_MATRIX.md, 16 fixture JSON chuẩn và kiểm thử chéo bằng test-frame-codec.
• Kế hoạch ứng phó khẩn cấp (Fallback): Dừng code ngay lập tức, họp khẩn 30 phút đối chiếu theo 03_PROTOCOL_SPEC.md để chốt lại định dạng.
• Trạng thái rà soát: Đang theo dõi (Tuần 1 (Cổng M1))

Rủi ro [RSK-03]: Lỗi phân mảnh gói tin TCP (Partial Read / Frame Coalescing)

• Chủ sở hữu theo dõi: SV A | Xác suất: Trung bình | Ảnh hưởng: Cao
• Dấu hiệu cảnh báo sớm: Server đọc thiếu byte JSON payload, ứng dụng văng ngoại lệ EndOfStreamException hoặc treo luồng.
• Biện pháp phòng ngừa (Mitigation): Hiện thực FrameCodec chuẩn với 4 byte Length Big-Endian, vòng lặp ReadExactlyBytesAsync đọc tích lũy đủ byte.
• Kế hoạch ứng phó khẩn cấp (Fallback): Chạy bộ test-frame-codec với 10,000 khung ngẫu nhiên để cô lập lỗi giải mã trước khi tích hợp vào server.
• Trạng thái rà soát: Đang theo dõi (Tuần 1 (Cổng M2))

Rủi ro [RSK-04]: Central Server bị nghẽn hàng đợi / Tràn bộ nhớ RAM (OOM)

• Chủ sở hữu theo dõi: SV B | Xác suất: Trung bình | Ảnh hưởng: Cao
• Dấu hiệu cảnh báo sớm: Hàng đợi IngressChannel đạt trên 80% dung lượng (800/1000 phần tử), độ trễ nạp tăng cao.
• Biện pháp phòng ngừa (Mitigation): Dùng BoundedChannel giới hạn 1000 phần tử, áp lực ngược (Backpressure) và nạp số liệu theo lô bất đồng bộ.
• Kế hoạch ứng phó khẩn cấp (Fallback): Tạm dừng tiếp nhận kết nối mới, ghi log cảnh báo và điều chỉnh chu kỳ gửi số liệu của Agent lên 10 giây.
• Trạng thái rà soát: Đang theo dõi (Tuần 2 (Cổng M3))

Rủi ro [RSK-05]: Bão kết nối lại đồng loạt (Reconnect Storm) làm sập Central

• Chủ sở hữu theo dõi: SV A | Xác suất: Trung bình | Ảnh hưởng: Trung bình
• Dấu hiệu cảnh báo sớm: Sau khi Central khởi động lại, 40 Agent cùng gửi bắt tay TLS tại đúng mili-giây thứ 0.
• Biện pháp phòng ngừa (Mitigation): Cài đặt thuật toán Exponential Backoff kèm độ trễ ngẫu nhiên (Full Jitter 1-5s) trên tất cả Agent.
• Kế hoạch ứng phó khẩn cấp (Fallback): Cấu hình Server Socket Backlog = 100 và giới hạn tốc độ tiếp nhận kết nối mới trên giây.
• Trạng thái rà soát: Đang theo dõi (Tuần 2)

Rủi ro [RSK-06]: Xung đột tranh chấp (Race Condition) khi 2 Admin cùng thao tác sự cố

• Chủ sở hữu theo dõi: SV B/C | Xác suất: Trung bình | Ảnh hưởng: Cao
• Dấu hiệu cảnh báo sớm: Thao tác của cán bộ vận hành 1 bị đè bẹp bởi cán bộ vận hành 2 (Lost Update).
• Biện pháp phòng ngừa (Mitigation): Dùng Optimistic Concurrency Control (OCC) với cột rowversion trong SQL Server và gửi expectedRowVersion từ UI.
• Kế hoạch ứng phó khẩn cấp (Fallback): Server trả về lỗi CONCURRENCY_CONFLICT; WinForms tự động thông báo và tải lại trạng thái mới nhất.
• Trạng thái rà soát: Đang theo dõi (Tuần 3 (Cổng M4))

Rủi ro [RSK-07]: Cơ sở dữ liệu SQL Server gặp sự cố / Khởi động lại đột ngột

• Chủ sở hữu theo dõi: SV B | Xác suất: Trung bình | Ảnh hưởng: Cao
• Dấu hiệu cảnh báo sớm: SqlException báo mất kết nối hoặc timeout khi thực thi lệnh INSERT/UPDATE.
• Biện pháp phòng ngừa (Mitigation): Bọc câu lệnh trong cơ chế Retry có backoff, duy trì cờ DbDegraded và bảo vệ hàng đợi đệm trong RAM.
• Kế hoạch ứng phó khẩn cấp (Fallback): Giữ kết nối máy trạm, tiếp tục nhận số liệu vào bộ đệm bộ nhớ; tự động ghi nhận bù khi DB online lại.
• Trạng thái rà soát: Đang theo dõi (Tuần 3)

Rủi ro [RSK-08]: Đơ luồng giao diện WinForms (UI Freeze) do gọi mạng hoặc vẽ lại

• Chủ sở hữu theo dõi: SV C | Xác suất: Trung bình | Ảnh hưởng: Trung bình
• Dấu hiệu cảnh báo sớm: Giao diện hiện thông báo '(Not Responding)' khi nhận bão số liệu đo lường từ 40 máy.
• Biện pháp phòng ngừa (Mitigation): Mọi tác vụ mạng chạy trên Task nền độc lập; dùng Control.Invoke/BeginInvoke marshal an toàn về luồng UI.
• Kế hoạch ứng phó khẩn cấp (Fallback): Chỉ cập nhật ô dữ liệu thay đổi trên DataGridView/Grid thay vì gọi Invalidate() toàn bộ form.
• Trạng thái rà soát: Đang theo dõi (Tuần 2)

Rủi ro [RSK-09]: Lỗi chứng chỉ số TLS / Bắt tay thất bại trên mạng LAN thật

• Chủ sở hữu theo dõi: SV A/B | Xác suất: Trung bình | Ảnh hưởng: Cao
• Dấu hiệu cảnh báo sớm: AuthenticationException: The remote certificate is invalid according to the validation procedure.
• Biện pháp phòng ngừa (Mitigation): Tạo chứng chỉ phát triển tự ký với SAN chứa đúng địa chỉ IP máy chủ LAN; cài đặt CA vào máy kiểm thử.
• Kế hoạch ứng phó khẩn cấp (Fallback): Có sẵn cờ cấu hình DevAllowUntrustedRoot trong môi trường nội bộ để gỡ rối nhanh khi cần.
• Trạng thái rà soát: Đang theo dõi (Tuần 4 (Cổng M5))

Rủi ro [RSK-10]: Mạng Wi-Fi / LAN của trường đại học bị chặn cổng hoặc ngắt kết nối

• Chủ sở hữu theo dõi: SV A | Xác suất: Cao | Ảnh hưởng: Trung bình
• Dấu hiệu cảnh báo sớm: Không thể ping thấy nhau hoặc cổng 5000 bị chặn bởi chính sách Firewall/Isolation của trường.
• Biện pháp phòng ngừa (Mitigation): Mang theo Router Wi-Fi cá nhân độc lập và dây cáp mạng LAN chuẩn Cat6 để tự thiết lập mạng riêng.
• Kế hoạch ứng phó khẩn cấp (Fallback): Chuyển toàn bộ 3 khối về chạy trên localhost (127.0.0.1) trên cùng 1 máy tính xách tay cấu hình cao.
• Trạng thái rà soát: Đang theo dõi (Tuần 5 (Diễn tập Demo))

Rủi ro [RSK-11]: Một thành viên bị kẹt nhiệm vụ (Blocker) quá 1 ngày làm việc

• Chủ sở hữu theo dõi: SV A/B/C | Xác suất: Trung bình | Ảnh hưởng: Cao
• Dấu hiệu cảnh báo sớm: Báo cáo Standup hàng ngày ghi nhận nhiệm vụ giữ nguyên trạng thái BLOCKED trên 24 giờ.
• Biện pháp phòng ngừa (Mitigation): Kích hoạt vai trò Backup Owner tương ứng nhảy vào pair programming hỗ trợ ngay trong ngày.

• Kế hoạch ứng phó khẩn cấp (Fallback): Tổ chức họp khẩn, giảm độ phức tạp của giải pháp hoặc tách nhỏ phần việc để không cản trở 2 bạn còn lại.
• Trạng thái rà soát: Đang theo dõi (Hàng ngày (Daily Standup))

Rủi ro [RSK-12]: Sử dụng AI sinh code nhưng thành viên không hiểu bản chất

• Chủ sở hữu theo dõi: SV A/B/C | Xác suất: Trung bình | Ảnh hưởng: Cao
• Dấu hiệu cảnh báo sớm: Thành viên ấp úng, không giải thích được lý do chọn giải pháp hoặc luồng đi của dữ liệu trong hàm.
• Biện pháp phòng ngừa (Mitigation): Buộc phải trả lời đạt bộ 30 câu hỏi Viva cá nhân; cam kết không commit bất kỳ dòng code nào chưa hiểu rõ.
• Kế hoạch ứng phó khẩn cấp (Fallback): Diễn tập phản biện nội bộ hàng tuần, truy hỏi từng hàm cho đến khi thành viên tự tin giải thích mạch lạc.
• Trạng thái rà soát: Đang theo dõi (Hàng tuần)

Rủi ro [RSK-13]: Giao diện bị vỡ hạt / Mất nút bấm khi chiếu lên máy chiếu hội đồng (High-DPI)

• Chủ sở hữu theo dõi: SV C | Xác suất: Cao | Ảnh hưởng: Trung bình
• Dấu hiệu cảnh báo sớm: Nút bấm bị cắt chữ, chữ bị mờ hoặc form bị tràn khi cắm máy chiếu có tỉ lệ phóng 125%/150%.
• Biện pháp phòng ngừa (Mitigation): Khai báo PerMonitorV2 trong tệp manifest của WinForms và dùng layout TableLayoutPanel linh hoạt.
• Kế hoạch ứng phó khẩn cấp (Fallback): Cài đặt độ phân giải màn hình về chuẩn 1920x1080 Scale 100% trước khi kết nối dây HDMI máy chiếu.
• Trạng thái rà soát: Đang theo dõi (Tuần 5 (Cổng M6))

Rủi ro [RSK-14]: Tràn nhật ký giao dịch SQL Server (LDF Full) do kiểm thử số liệu kéo dài

• Chủ sở hữu theo dõi: SV B | Xác suất: Thấp | Ảnh hưởng: Cao
• Dấu hiệu cảnh báo sớm: Ổ cứng báo sắp hết dung lượng, SQL Server ném lỗi 'The transaction log for database is full'.
• Biện pháp phòng ngừa (Mitigation): Cấu hình CSDL ở chế độ Simple Recovery Model cho môi trường thử nghiệm và chạy lệnh xóa lô nhỏ.
• Kế hoạch ứng phó khẩn cấp (Fallback): Thực hiện lệnh DBCC SHRINKFILE để thu hẹp tệp log và chạy tác vụ RetentionWorker xóa dữ liệu cũ.
• Trạng thái rà soát: Đang theo dõi (Tuần 4)

11.2. Cẩm nang tra cứu & khắc phục 5 lỗi lập trình kinh điển:

• Lỗi 1: InvalidOperationException (Cross-thread operation not valid):
Nguyên nhân: Luồng mạng nền cố thay đổi thuộc tính Text hoặc BackColor của Control trong WinForms.
Khắc phục: Dùng if (control.InvokeRequired) control.Invoke(new Action(() => { ... }));

• Lỗi 2: EndOfStreamException (Đứt kết nối đột ngột giữa chừng):
Nguyên nhân: Hàm stream.ReadAsync trả về 0 byte khi đối phương ngắt socket.
Khắc phục: Bắt ngoại lệ EndOfStreamException, đóng socket sạch sẽ, chuyển sang trạng thái Reconnecting.

• Lỗi 3: ConcurrencyException / CONCURRENCY_CONFLICT:
Nguyên nhân: Người khác đã bấm ACK/RESOLVE sự cố trước làm thay đổi giá trị rowversion trong CSDL.
Khắc phục: Hiển thị thông báo thân thiện 'Sự cố đã được xử lý bởi người khác', tự động nạp lại dữ liệu.

• Lỗi 4: AuthenticationException (Lỗi chứng chỉ số TLS trên mạng LAN):
Nguyên nhân: Chứng chỉ tự ký chưa được cài đặt vào máy kiểm thử hoặc tên máy không khớp IP LAN.
Khắc phục: Cấu hình SAN (Subject Alternative Name) chứa đúng IP 192.168.x.x của máy chủ Central.

• Lỗi 5: SqlException: The transaction log for database is full:
Nguyên nhân: Chạy thử tải nạp hàng triệu bản ghi làm đầy tệp nhật ký LDF của SQL Server.
Khắc phục: Đổi CSDL sang Simple Recovery Model và chạy lệnh DBCC SHRINKFILE.

PHẦN 12: KỊCH BẢN DIỄN TẬP DEMO 12 PHÚT TRƯỚC HỘI ĐỒNG

12.1. Kịch bản 6 bước Demo chuẩn xác: Thời gian trình diễn sản phẩm trước hội đồng chấm thi UIT chỉ kéo dài đúng 12 phút. Nhóm phải diễn tập bấm giờ chính xác từng giây theo 6 bước nghiệp vụ chuẩn sau:

- **Phút 00:00 - 01:30 | BƯỚC 1: Khởi động hệ thống & Bắt tay an toàn:**
Thao tác: Bật Central Server, khởi động 2 Agent trên máy A và máy trạm phụ, mở WinForms trên máy C.
Thuyết minh: Giới thiệu kiến trúc 3 khối, giải thích cơ chế bắt tay TLS và xác thực danh tính.

- **Phút 01:30 - 03:30 | BƯỚC 2: Giám sát phòng máy thời gian thực:**
Thao tác: Máy C hiển thị lưới phòng máy 20 máy trạm đang ở trạng thái Xanh lá (ONLINE).
Thuyết minh: Giải thích cơ chế gửi nhịp tim định kỳ 5s và đường ống nạp số liệu BoundedChannel.

- **Phút 03:30 - 05:30 | BƯỚC 3: Giả lập sự cố quá tải tài nguyên:**
Thao tác: Chạy script kích tải CPU 100% trên máy A.
Thuyết minh: Sau 3 chu kỳ liên tiếp (Hysteresis), máy A chuyển sang màu Vàng (Cảnh báo), Server tạo Ticket sự cố mới màu Đỏ trên màn hình C.

- **Phút 05:30 - 08:00 | BƯỚC 4: Tiếp nhận & Thao tác sự cố có khóa lạc quan (OCC):**
Thao tác: Mở 2 cửa sổ Admin cùng lúc, cùng bấm nút ACK trên 1 sự cố để chứng minh bẫy CONCURRENCY_CONFLICT thành công.
Thuyết minh: Giải thích thuật toán khóa lạc quan bằng cột rowversion trong SQL Server.

- **Phút 08:00 - 10:00 | BƯỚC 5: Thử thách rút dây mạng (Fault Tolerance):**
Thao tác: Rút trực tiếp dây mạng của máy A trước mặt giảng viên.
Thuyết minh: Đồng hồ đếm đúng 20 giây (3 chu kỳ 15s + 5s grace), Dashboard tự động chuyển máy A sang màu Xám (OFFLINE). Cắm dây lại, máy tự hồi phục.

- **Phút 10:00 - 12:00 | BƯỚC 6: Báo cáo thống kê & Tổng kết:**
Thao tác: Mở màn hình Báo cáo lịch sử, bấm nút Xuất CSV và xem đồ thị sức khỏe máy chủ.
Thuyết minh: Tóm tắt kết quả đạt được, sẵn sàng chuyển sang phần vấn đáp cá nhân.

> 📌 **KẾ HOẠCH DỰ PHÒNG RỦI RO SÂN KHẤU (STAGE FALLBACK PLAN)**
> Nếu phòng thi bị mất điện, rớt mạng Wi-Fi trường hoặc máy tính bị đơ bất ngờ: Nhóm lập tức mở tệp video DemoBackup.mp4 quay sẵn chất lượng cao 1080p và tiếp tục thuyết minh bình thường. Giảng viên vẫn đánh giá rất cao sự chuyên nghiệp và khả năng chuẩn bị dự phòng rủi ro của nhóm!

PHẦN 13: HƯỚNG DẪN SỬ DỤNG FILE TASK TRACKER V2

13.1. Quy trình vận hành bảng tính hàng ngày: Tệp bảng tính LabGuard_Team_Task_Tracker_v2.xlsx là Trung tâm chỉ huy (Project Control Center) của toàn đội. Mỗi thành viên cần thực hiện đúng quy trình hàng ngày sau:

4 bước vận hành hàng ngày: 1. Đầu giờ làm việc: Mở file Excel, sheet đầu tiên nhìn thấy luôn là '00_Hom_Nay_Lam_Gi'. Đọc kỹ nhiệm vụ active của mình và 4 cảnh báo nóng.
2. Đọc thẻ nhiệm vụ: Vào sheet cá nhân ('03_Person_A', '04_Person_B', '05_Person_C') để đọc chi tiết 18 trường của nhiệm vụ.
3. Kiểm tra tiêu chuẩn DoD: Trong lúc viết code, đối chiếu 11 tiêu chuẩn Definition of Done.
4. Cuối ngày trước 23:00: Cập nhật trạng thái nhiệm vụ trong '02_Master_Tasks' và ghi chú vào '11_Daily_Standup'.

PHỤ LỤC: BẢNG TRA CỨU NHANH MÃ LỖI, MẪU GÓI TIN & LỆNH THÔNG DỤNG

Phụ lục A: Bảng danh mục mã lỗi chuẩn của hệ thống LabGuard

• **ERROR_RESPONSE:** Lỗi chung khi xử lý bản tin không xác định hoặc dữ liệu không đúng định dạng.

• **CONCURRENCY_CONFLICT:** Xung đột khóa lạc quan OCC: RowVersion không khớp, sự cố đã bị sửa bởi phiên khác.

• **AUTH_FAILED:** Xác thực thất bại: Sai tên đăng nhập hoặc mật khẩu băm PBKDF2 không khớp.

• **SESSION_EXPIRED:** Phiên làm việc token đã hết hạn hoặc bị thu hồi bởi quản trị viên.

• **INVALID_INVITE_CODE:** Mã mời đăng ký không tồn tại, đã hết hạn hoặc đã được sử dụng trước đó.

• **RATE_LIMIT_EXCEEDED:** Tốc độ gửi bản tin từ thiết bị vượt quá ngưỡng cho phép của hàng đợi Server.

• **PAYLOAD_TOO_LARGE:** Kích thước gói tin vượt quá giới hạn an toàn 256 KiB của FrameCodec.

Phụ lục B: Các câu lệnh dòng lệnh thông dụng cho dự án

• **Biên dịch sạch toàn bộ solution:** *dotnet build --no-incremental*

• **Chạy toàn bộ bài kiểm thử đơn vị:** *dotnet test --verbosity normal*

• **Kiểm tra thông tin runtime .NET:** *dotnet --info*

• **Sao lưu CSDL SQL Server:** *BACKUP DATABASE LabGuard TO DISK = 'C:\Backup\LabGuard.bak' WITH INIT, COMPRESSION*

• **Kiểm tra toàn vẹn vật lý CSDL:** *DBCC CHECKDB ('LabGuard') WITH NO_INFOMSGS*

• **Phục hồi CSDL từ bản sao lưu:** *RESTORE DATABASE LabGuard FROM DISK = 'C:\Backup\LabGuard.bak' WITH REPLACE*